국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

보안 과제( ), 일반 과제(√) / 공개(√), 비공개( ), 발간등록번호( )

정보통신·방송 연구개발 사업 제3차년도 최종 보고서

과제번호 2018-0-00251

스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술 개발 최종보고서

2021 . 2. 15.

주관기관 / 충남대학교 산학협력단
참여기관 / 수원대학교 산학협력단
한국과학기술원
㈜파이랩

과학기술정보통신부
정보통신기획평가원

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 출 문

과학기술정보통신부 장관 귀하

"스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술 개발"(총 수행기간 : 2018.4.1. ~ 2020 12.31)과제의 최종보고서 를 제출합니다.

2021 . 2 . 15 .

주 관 기 관 명 : 충남대학교 산학협력단 손영이 (인)

총 괄 책 임 자 : 류 재 철

1. 동 보고서상의 기재 내용이 사실임을 확약하며, 만약, 사실이 아닐 경우 과학기술기본법 제 11조의2 및 정보통신·방송 연구개발사업 관리 규정 제 11조에 따라 참여제한·환수 등의 어떠한 불이익도 감수하겠습니다.
2. 정보통신·방송연구개발 관리규정 제36조에 따른 보고서 공개·활용에 동의합니다.

※ (인)은 1장에 찍는 것이 원칙이나, 불가피한 경우 개별적으로 가능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 출 문

과학기술정보통신부 장관 귀하

"스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술 개발"(총 수행기간 : 2018.4.1. ~ 2020.12.31)과제의 최종보고서를 제출합니다.

2020. 2. 15.

공동연구기관명 : 수원대학교 산학협력단 임 교 빈 (인)

공동연구기관책임자 : 교수 김 명 선 (인)

1. 동 보고서상의 기재 내용이 사실임을 확약하며, 만약, 사실이 아닐 경우 과학기술기본법 제 11조의2 및 정보통신·방송 연구개발사업 관리 규정 제 11조에 따라 참여제한·환수 등의 어떠한 불이익도 감수하겠습니다.
2. 정보통신.방송연구개발 관리규정 제36조에 따른 보고서 공개·활용에 동의합니다.

※ *(인)은 1장에 찍는 것이 원칙이나, 불가피한 경우 개별적으로 가능*

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 출 문

과학기술정보통신부 장관 귀하

"스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술 개발"(총 수행기간 : 2018.4.1. ~ 2020.12.31)과제의 최종보고서를 제출합니다.

2020 . 2 . 15 .

공동연구기관명 : 한국과학기술원 신 성 철 (인)

공동연구기관책임자 : 한국과학기술원 교수 류 석 영 (인)

1. 동 보고서상의 기재 내용이 사실임을 확약하며, 만약, 사실이 아닐 경우 과학기술기본법 제 11조의2 및 정보통신·방송 연구개발사업 관리 규정 제 11조에 따라 참여제한·환수 등의 어떠한 불이익도 감수하겠습니다.
2. 정보통신.방송연구개발 관리규정 제36조에 따른 보고서 공개·활용에 동의합니다.

※ (인)은 1장에 찍는 것이 원칙이나, 불가피한 경우 개별적으로 가능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

제 출 문

과학기술정보통신부 장관 귀하

"스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술 개발"(총 수행기간 : 2018.4.1. ~ 2020.12.31)과제의 최종보고서를 제출합니다.

2020 . 2 . 15 .

공동연구기관명 : ㈜파이랩테크놀로지 박 도 현 (인)

공동연구기관책임자 : 이 종 협 (인)

1. 동 보고서상의 기재 내용이 사실임을 확약하며, 만약, 사실이 아닐 경우 과학기술기본법 제 11조의2 및 정보통신·방송 연구개발사업 관리 규정 제 11조에 따라 참여제한·환수 등의 어떠한 불이익도 감수하겠습니다.
2. 정보통신.방송연구개발 관리규정 제36조에 따른 보고서 공개·활용에 동의합니다.

※ *(인)은 1장에 찍는 것이 원칙이나, 불가피한 경우 개별적으로 가능*

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

보고서 요약서

과제번호	2018-0-00251	총 수행기간	2018.04.01 ~ 2020.12.31	단계구분	기초[ ], 응용[√], 개발[ ]
과제특성				기술성숙도 (TRL)	과제시작 시점(2) 과제종료 시점(5)
세부사업명	정보보호핵심원천기술개발 사업				
연구과제명	총괄과제명				
	과제명	스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술개발			
총괄책임자	류재철	총 연구기간 참여 연구원 수	총: 36명 내부: 36명 외부: 0 명	총 연구비	정부: 1,400,000 천원 민간: 246,423 천원 계: 1,646,423 천원
주관기관명 및 소속 부서명	충남대학교 산학협력단			공동연구기관명 수원대학교 산학협력단, 한국과학기술원, ㈜파이랩	
국제공동연구	상대국명:			상대국 연구기관명:	
위탁연구	연구기관명:			연구책임자:	

※ 국내·외의 기술개발 현황은 연구개발계획서에 기재한 내용으로 갈음

연구개발성과의 보안등급 및 사유	일반

9대 성과 등록·기탁번호

구분	논문	특허	보고서 원문	연구시설·장비	기술요약 정보	소프트웨어	화합물	생명자원		신품종	
								생명정보	생물자원	정보	실물
등록·기탁번호	4	9									

'연구성과 관리·유통 전담기관 지정 고시' 참조

국가과학기술종합정보시스템에 등록한 연구시설·장비 현황

구입기관	연구시설·장비명	규격 (모델명)	수량	구입연월일	구입가격 (천원)	구입처 (전화)	비고 (설치장소)	NTIS 등록번호

국가과학기술종합정보시스템에 등록한 연구시설·장비 현황은 해당과제 연구개발 시작부터 최종 연구 종료 시점까지 현금으로 구입한 1개(건)당 3천만원(부가세 포함) 이상의 연구시설·장비 및 3천만원 미만이라도 공동 활용이 가능한 연구시설·장비로 국가과학기술종합정보시스템에 등록한 시설·장비를 기재 – 제출시 삭제

요약(연구개발성과를 중심으로 개조식으로 작성하되, 500자 이내로 작성합니다)	보고서 면수 133

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

요약문

연구의 목적 및 내용	최종 목표	**o 목표** **- 암호와 프로그래밍 언어 기술을 이용한 신뢰 가능한 스마트 컨트랙트 환경 구축** **o End Product** - 익명화 및 암호화를 이용한 스마트 컨트랙트 프라이버시 보호 모듈(SW) - 자동 역공학이 가능한 스마트 컨트랙트 취약점검증 도구(SW)
	전체 내용	**o 연구의 목적** - 블록체인의 특성 상 스마트 컨트랙트의 모든 내용이 노출되기 때문에 사용자의 프라이 버시를 보호하기 위한 도구 개발 및 스마트 컨트랙트 코드의 취약점을 분석하여 안전한 환경을 제공해주는 도구 개발 **o 연구의 내용** - **스마트 컨트랙트 프라이버시 보호 기술 :** 트랜잭션의 익명화 및 암호화로 송수신자 정보와 소유 금액 정보가 블록 체인 상에서 조회가 되지 않도록 영지식 증명 처리하여 특정 검증자만이 컨트랙트의 정보를 관리함으로써 정보 노출을 방지 - **스마트 컨트랙트 취약점 분석 기술 :** 다양한 컨트랙트 프로그래밍 언어에 적용 가능하도록 중간언어로 디컴파일하는 기능을 지원하며 자동 역공학을 통한 스마트 컨트랙트 개발 단계 분석 환경 제공 **■ 1차년도** **o 트랜잭션 익명화/암호화 적용을 위한 블록체인 구조 연구** - 트랜잭션 구조에 의한 연결 공격 및 거래 추적 위협 분석 - NIZK 기반의 알고리즘 및 SNARK 기법 분석 - 영지식 증명을 이용한 증거 생성 및 검증 기법 개발 - 스마트 컨트랙트 생성 및 실행 환경 분석 - 익명화된 트랜잭션 기반 합의를 위한 트랜잭션 정의 - 공개키 기반의 암호화된 트랜잭션 검증 프로토콜 개발 **o 스마트 컨트랙트 작성 언어 및 분석 플랫폼의 기반 기술 구성** - 개발자의 의도와 다르게 동작하는 Solidity 기능 조사 - Solidity 대상의 바이트코드 변환기 연구 - 대표적인 이더리움 스마트 컨트랙트 공격 코드 수집 - EVM 바이트코드 요약 실행을 위한 의미 구조 정리 - 스마트 컨트랙트 중간 언어 복원 시스템 개발 - 스마트 컨트랙트 중간 언어 검증 시스템 개발 **■ 2차년도** **o 컨트랙트 실행 환경의 위변조 방지를 위한 프로토콜 개발** - 트랜잭션 재전송 공격에 의한 중복성 검증(이중 지불 방지) 메커니즘 개발 - 영지식 증명을 위한 SNARK 알고리즘 구현 및 실험

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 영지식 증명에 대한 안전성 분석
- 컨트랙트와 저장소의 상태 관리를 위한 해시 트리 설계
- 해시 트리 기반의 노드간 상태 위변조 검증 프로토콜 개발

o 스마트 컨트랙트 자동 역공학 도구 및 바이트코드 분석기 개발

- 스마트 컨트랙트 자동 역공학 기술 연구
- 스마트 컨트랙트 정합성 검증 기술 연구
- 요약 실행을 위한 도메인 정의 및 구현
- EVM 내에서 동작 가능한 요약 실행 프레임워크 개발
- 호출되는 함수를 기반으로 컨트랙트의 행위 도출 기술 개발
- EVM에서 동작 가능한 런타임 스택 인스펙션 도구 개발

■ 3차년도

o 블록체인 플랫폼을 활용한 영지식 증명 기반 가상 머신 모듈 개발

- 블록체인 헤더 파라미터 정의
- 실행 환경에 따른 영지식 증명을 위한 SNARK 프로토콜 최적화
- 익명 트랜잭션 처리를 위한 가상 머신 모듈 개발
- 스마트 컨트랙트 생성 및 검증 노드의 프라이버시 보호 기능 검증 수행

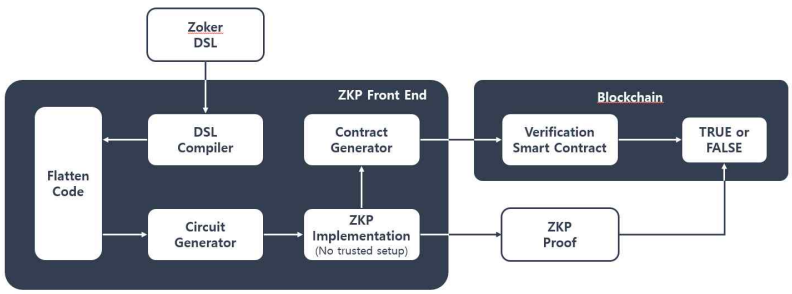
o 스마트 컨트랙트 공격 검출 기능 및 프로토타입 개발

- 컨트랙트 행위 기반의 공격 검출 기술 개발
- 공격 검출을 위한 요약 실행 의미 상세 분석 기법 개발
- Solidity 취약점 패치 안전성 검증 및 컨트랙트 보안 수준 검증 기술 개발
- 다중 블록체인 플랫폼에서 적용 가능한 자동 역공학 도구

연구개발성과

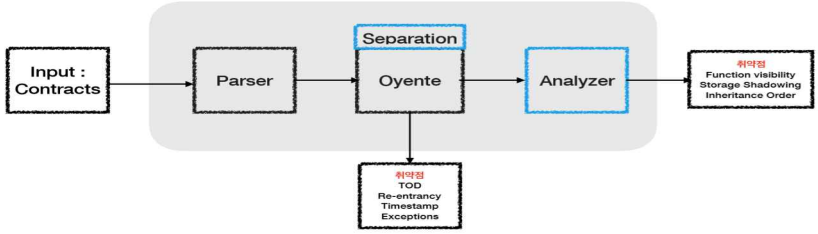
o 개발한 도구

- 사용자의 편의성 증대 및 영지식 증명을 검증 컨트랙트를 생성해주는 도구(Zoker)



[그림 5] 영지식 증명 검증 컨트랙트 생성 도구(Zoker)

- 안전한 코드 환경을 위한 스마트 컨트랙트 취약점 분석기 개발 도구(Oyente+)



[그림 2] 스마트 컨트랙트 취약점 분석 개발 도구(Oyente+)

o 성과 달성표

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

구분		목표	실적
특허 출원	국내	9	9
	국외	-	-
	계	9	9
특허 등록	국내	4	4
	국외	-	-
	계	4	4
국제 표준화	기고서 제출	1	1
	기고서 채택	-	-
	표준안 채택	-	-
국내 표준화	기고서 제출	2	3
	기고서 채택	3	3
	표준안 채택	-	-
기술 이전	계약 건수	1	1
	계약액	30(백만원)	35(백만원)
등록·기탁 대상 성과물	소프트웨어	4	5
인력 양성	학사	00	00
	석사	00	00
	박사	00	00
기타	시제품	2	2
	성과 홍보	2	2

연구개발성과의 활용계획 (기대효과)

■ **스마트 컨트랙트 프라이버시 보호 기술**

- 프라이버시 보호 기술로 프라이빗 블록체인 플랫폼에서 특정 검증자에게만 개인정보가 공개되므로 금융, 공공서비스, 행정, 의료 등 민감한 정보를 다루는 분야에도 활용 가능
- 국내에서 활발하게 연구되지 않았던 영지식 증명 알고리즘과 같은 기술을 연구함으로써 다른 보안 분야에도 적용하여 연구범위 확대 가능

■ **스마트 컨트랙트 취약점 분석 기술**

- 새로운 언어인 Solidity 및 컨트랙트 언어에 관한 역공학 기술을 확보하여 민간 기업뿐만 아니라 블록체인 도입을 고려하는 정부 및 공공 기관의 테스트 도구로 활용하여 발생 가능한 취약점을 사전 탐지함으로써 추후 발생 가능한 금전적 손해를 미연에 방지
- 보안 전문가가 아닌 개발자들이 작성하는 스마트 컨트랙트에 보조 도구로 활용하여 실수에 의한 보안 사고를 예방하고 도구의 자동화를 통해 분석 비용에 드는 비용 최소화

국문핵심어 (5개 이내)	스마트 컨트랙트	블록체인	영지식 증명	중간 언어	요약 실행
영문핵심어 (5개 이내)	Smart Contract	Blockchain	Zero Knowledge Proof	Inter Language	Abstract Interpretation

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 목 차 〉

1. 연구개발과제의 개요 ····· 10

2. 연구 수행 내용 및 성과 ····· 25

3. 목표 달성도 및 관련 분야 기여도 ····· 104

4. 연구개발성과의 기여 및 활용 계획 등 ····· 114

붙임 1. 사업화대상기술별 기술명세서 ····· 116
붙임 2. 참고 문헌 ····· 126
붙임 3. 자체 보안관리 진단표 ····· 128
붙임 5. 연구데이터 관리계획 ····· 129

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1. 연구개발과제의 개요

1-1. 연구개발 목적

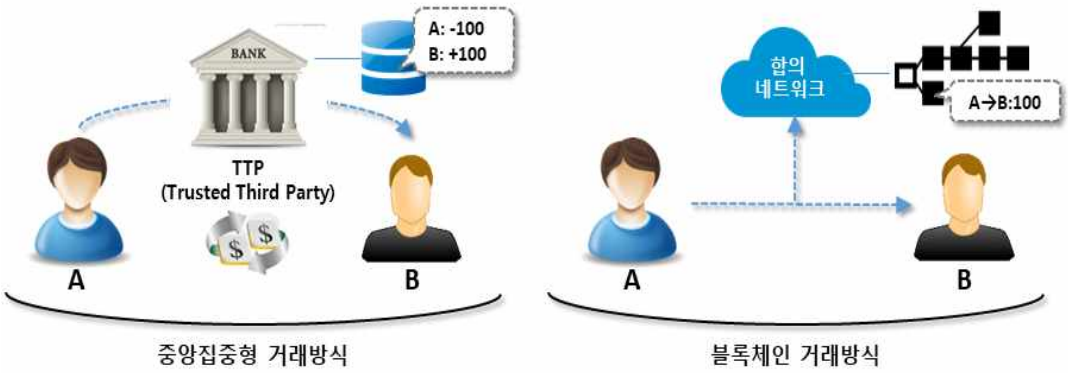
□ **블록체인의 발전과 전망**

○ **4차 산업혁명 시대에 접어들면서 2016년부터 국내에 본격적으로 주목받기 시작한 블록체인 기술은 제2의 인터넷으로 기대를 받고 있으며, 특히 암호 화폐 중심으로 기술이 활발하게 발전, 확대되고 있음**

- 블록체인은 2009년 사토시 나카모토(Satoshi Nakamoto)라는 익명의 개발자(혹은 그룹)에 의해 고안된 비트코인(Bitcoin)에서 등장한 기술임

- 비트코인은 P2P 기반의 네트워크에서 신뢰할 수 있는 제 3자(Trusted Third Party, TTP) 없이도 거래가 가능한 환경을 구축하기 위해 거래내역의 위변조가 불가능한 블록체인 구조를 도입한 최초 응용 사례임

  · [그림 1]과 같이 블록체인은 TTP 없이도 참여자의 합의를 통하여 블록체인에 거래내역을 저장함으로써 안전성 및 신뢰성을 확보



[그림 1] 중앙집중형 거래방식과 블록체인 거래방식의 비교

- 블록체인 구조는 위변조 방지와 탈중앙화 환경을 구성하는데 적합하며, 암호 화폐를 탑재하여 기존의 텍스트, 이미지 등 정보 컨텐츠 외에 돈, 자산 등과 같은 가치 있는 데이터를 전송 가능하여 미래형 인터넷으로도 불림

○ **2014년 등장한 이더리움(Ethereum)이 1997년 닉 사보(Nick Szabo)에 의해 처음 제안된 스마트 컨트랙트 개념을 블록체인 플랫폼으로 개발함으로써, 화폐 기능을 넘어 계약이 가능한 ’ 블록체인 2.0 ‘ 시대가 시작됨[그림 2]**



[그림 2] 블록체인 발전 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 스마트 컨트랙트란 당사자간에 스스로 계약을 이행할 수 있는 프로토콜로써 신뢰할 수 있는 제3자 없이도 실행을 보장함을 의미
- 비탈릭 부테린(Vitalik Vutelin)이 개발한 이더리움은 스마트 컨트랙트를 프로그래밍 가능한 코드로 구현하고 위변조가 불가능한 블록체인에 저장함으로써 강제 이행성을 가지도록 구현

○ **한편, 참여자가 제한 없이 블록체인의 구성원이 될 수 있는 암호 화폐와는 달리 참여자에 제한을 두고 운영하는 프라이빗 블록체인(Private Blockchain) 개념도 등장함**

- 세계 금융권 연합 R3가 형성되면서 은행권간에 거래 내역을 공유하는 프라이빗 블록체인을 통해 글로벌 거래를 적은 시간과 수수료로 통합 할 수 있을 것으로 기대받고 있음(R3 CEV 주도)
- 프라이빗 블록체인은 중앙 관리자 주체가 존재하여 탈중앙화는 아니지만 참여자에 한해서 모두가 접근 가능한 DB를 구축할 수 있어 유통, 운수, 무역 분야 등에서 혁신적으로 업무를 개선(리눅스 재단, IBM 주도)

□ **스마트 컨트랙트의 프라이버시와 취약점 이슈**

○ **블록체인은 비트코인 등장부터 지속적으로 발전하며 순기능을 하고 있으나, 기술적으로 해결되어야 할 대표적인 문제가 있음**

- **(확장성)** 확장성의 문제는 퍼블릭 블록체인은 초당 거래내역수가 제한되어 있어 비트코인의 경우 초당 3~7건 정도의 거래만 가능하며, 프라이빗 블록체인의 경우 합의에 참여하는 노드가 20~50개 정도만 가능한 기술적 한계가 있음
- **(프라이버시)** 이더리움 창립자인 비탈릭 부테린에 의하면 확장성과 함께 가장 어려운 이슈가 프라이버시 문제로 블록체인이 모든 거래내역을 공유함으로써 안전성과 신뢰성을 확보하기 때문에 상반된 이슈라 할 수 있음
- **(해킹)** 2014년부터 거래소의 해킹은 계속 발생하고 있으나 이는 블록체인 자체의 문제라고 보기는 어려운 반면, 스마트 컨트랙트 해킹으로 인해 발생한 The DAO 해킹[1]과 Parity Wallet Bug[2]와 같은 취약점에 대한 대응책은 향후 블록체인의 고도화를 위해서 반드시 확보되어야 할 기술

○ **블록체인 등장 초기부터 제기 되어왔던 확장성과 달리, 프라이버시와 취약점 문제는 스마트 컨트랙트가 활용되는 블록체인 2.0에서 더욱 부각되고 있어 관련 문제 해결을 통해 신뢰 가능한 블록체인 환경을 구축하고자 함 [그림 3]**



[그림 3] 스마트 컨트랙트의 프라이버시 보호 및 취약점 분석 기술 목표

1) The DAO 해킹 : 최초의 스마트 컨트랙트 해킹 사건으로 600억원 상당의 화폐가 해커의 계좌로 이체, 2016.6

2) Parity Wallet Bug : 스마트 컨트랙트 기반의 지갑 서비스 회사인 패리티사의 제품이 코드 결함으로 인해 3000억원 가량의 화폐가 동결, 2017.7

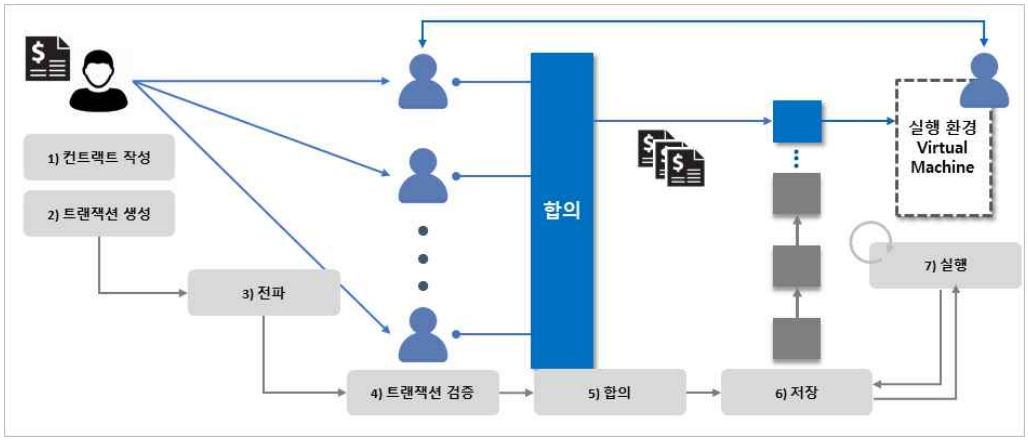
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1-2. 연구개발의 필요성

□ 스마트 컨트랙트 프라이버시 보호 기술

○ 스마트 컨트랙트는 사용자에 의해 작성되어 실행되기까지 [그림 4]와 같은 과정의 라이프 사이클을 가짐

- 블록체인의 참여자는 크게 트랜잭션(혹은 거래내역)을 생성하는 사용자와, 트랜잭션을 검증하는 검증자로 구성되며, 검증자는 스마트 컨트랙트가 실행되는 가상 머신(Virtual Machine)을 가지고 있음
- 한번 저장된 컨트랙트는 위변조 되지 않으며 코드로 작성된 컨트랙트에서 선언된 변수는 가상 머신에서 별도의 상태를 가지고 관리됨
- 컨트랙트는 저장 후에 컨트랙트를 호출하는 트랜잭션이 블록에 저장될 때마다 반복적으로 실행되며 출력으로 생성하는 트랜잭션 또한 블록체인에 저장됨



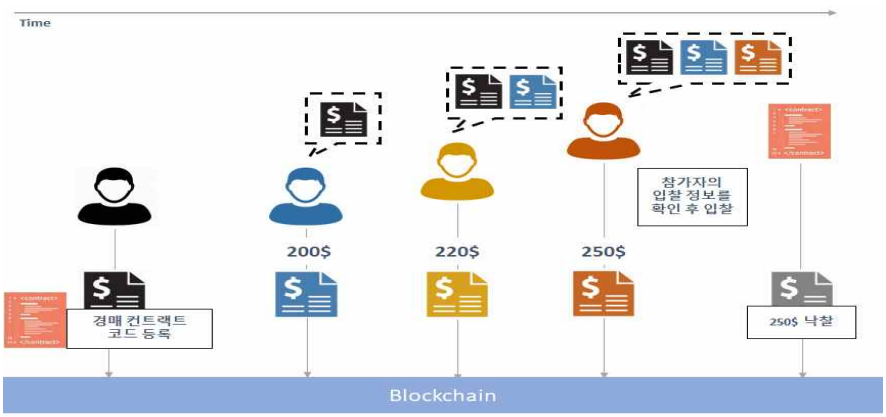
[그림 4] 블록체인 환경의 스마트 컨트랙트 라이프 사이클

○ 스마트 컨트랙트의 프라이버시로 인한 문제는 크게 2가지로 분류

- (문제 1) 트랜잭션 및 컨트랙트 코드의 노출로 인한 제한된 응용 환경

· 퍼블릭 블록체인 환경에서 발생하는 문제로 순차적으로 생성되는 블록체인의 트랜잭션으로 동작하는 스마트 컨트랙트가 실행되기 때문에 특정 응용에서는 뒤늦은 참가자가 이전의 모든 정보를 확인하고 계약에 참여하기 때문에 발생

· [그림 5]에서 보는 것과 같이 경매와 같은 계약을 스마트 컨트랙트로 구현하였을 경우 참가자의 입찰 정보가 모두 공개되기 때문에 마지막에 참여하는 사람이 가장 유리하므로 제대로 된 응용 환경을 구축할 수 없음

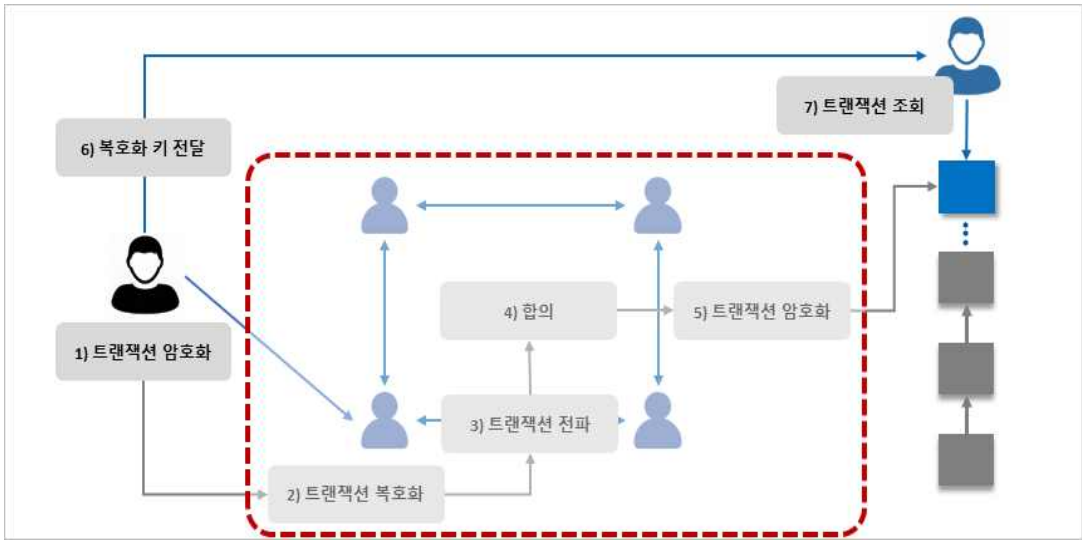


[그림 5] 트랜잭션 및 컨트랙트 코드의 노출로 인한 제한된 응용 환경

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- **(문제 2) 검증자는 컨트랙트를 포함한 모든 트랜잭션 정보를 확인 가능**

· 프라이빗 블록체인의 검증자는 [그림 6]과 같이 참여자가 제한된 환경에서 트랜잭션을 암호화 하지만 합의를 위해서 모든 트랜잭션을 복호화 할 수 있음



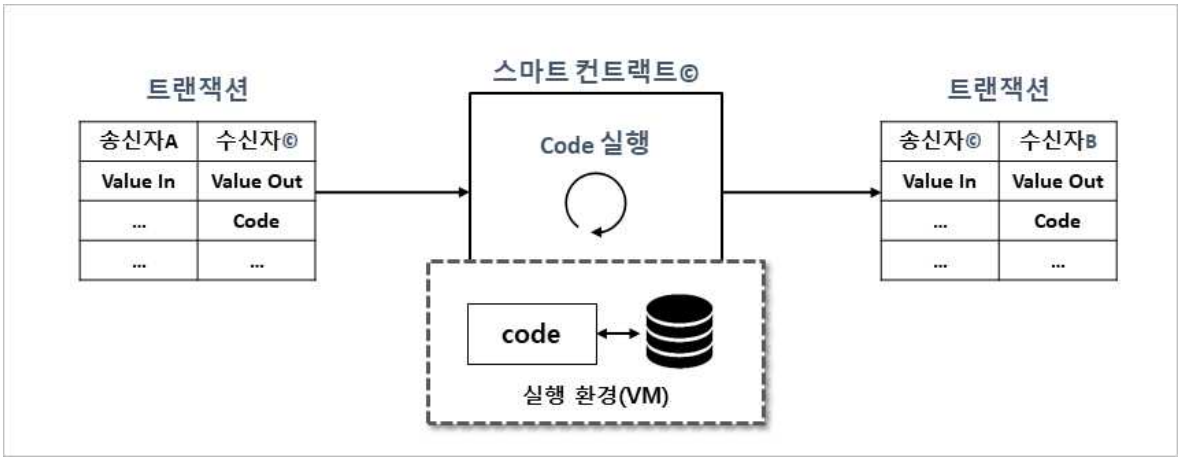
[그림 6] 검증자는 컨트랙트를 포함한 모든 트랜잭션 정보를 확인 가능

· 예를 들어, 은행들이 검증자가 되는 금융권 블록체인 환경은 거래 은행만이 알던 이체 정보가 모든 은행에게 공개되는 현상이 발생

· 병원이 검증자가 되는 블록체인 환경에서는 주치의가 있는 병원뿐만 아니라 컨소시움을 구성하는 모든 병원이 개인의 의료 기록에 대해 확인 가능

○ **이와 같은 문제를 해결하기 위해서는 트랜잭션과 스마트 컨트랙트가 실행되는 환경인 가상머신과 함께 블록체인 프로토콜을 수정하여 프라이버시 보호를 수행할 수 있음**

- [그림 7]과 같이 스마트 컨트랙트는 실행 환경에서 동작하는 코드와 저장 공간을 가지고 있으며 컨트랙트를 수신자로 하는 트랜잭션이 발생할 때마다 실행하여 결과를 트랜잭션 형태로 출력함



[그림 7] 트랜잭션을 입출력하는 스마트 컨트랙트와 실행 환경

- 첫 번째 프라이버시 문제는 트랜잭션의 송수신자 정보를 드러내지 않도록 함으로써 해결이 가능함

· 트랜잭션 정보를 드러내지 않으면서 금액(Value)과 같은 정보에 대해 발생하는 재전송 공격(혹은 이중 지불 공격)을 방지하기 위해서 **영지식 증명3) 기반의 트랜잭션으로 해결**

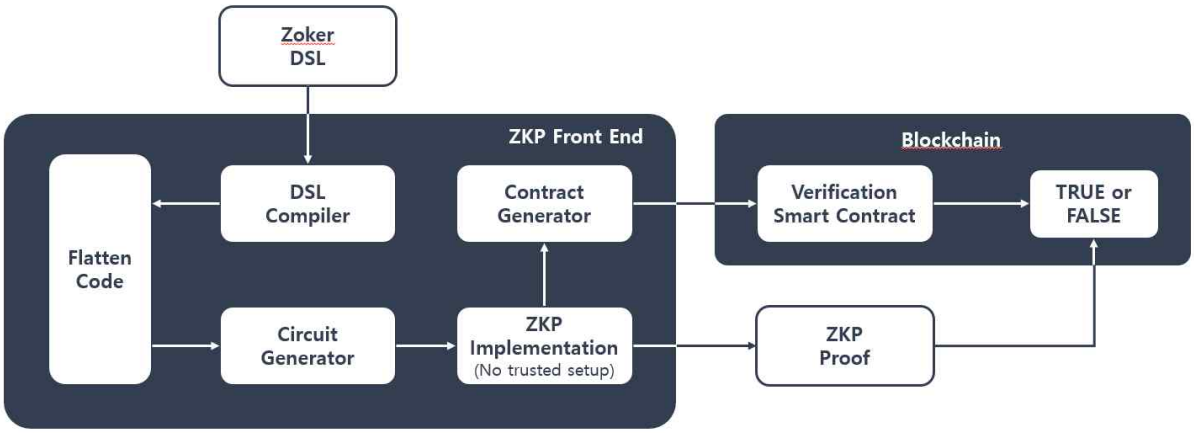
3) 영지식 증명(Zero Knowledge Proof) : 암호 키와 같은 비밀 정보를 노출시키지 않은 채 상대방에게 비밀을 알고 있다는

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 두 번째 프라이버시 문제는 모든 검증 자들이 트랜잭션을 복호화하여 합의를 수행하는 것이 아니라 암호화된 상태에서 합의를 수행하여 정보가 노출되지 않도록 함으로써 해결 가능

· 암호화된 트랜잭션으로 합의를 수행하기 위해서는 **암호키 관리를 위한 실행 환경과 검증자 별로 다른 저장소를 가지는 형태의 블록체인을 구현**

· **이에, 아래 [그림 8]과 같이 사용자의 편의성 증대 및 영지식 증명을 검증 컨트랙트를 생성해주는 도구 개발을 최종 목표로 함.**



[그림 8] Zoker의 구조

□ **스마트 컨트랙트 취약점 분석 기술**

○ **프로그래밍 언어로 작성 되는 스마트 컨트랙트는 블록체인 환경에서 동작 하기 때문에 일반적인 프로그램과 차이점을 가짐**

- 대표적으로 스마트 컨트랙트 지원 플랫폼인 이더리움의 경우, 컨트랙트를 실행 하기 위한 비용인 가스(GAS)[4]를 지불해야하고, 코드의 길이가 평균 수백 정도로 다른 프로그래밍언어로 작성된 프로그램에 비해 매우 짧음

- 또한, 블록체인에 저장되기 때문에 코드 자체의 위변조는 불가능하며, 컨트랙트를 실행하기 위한 가상 머신이 별도로 존재

○ **스마트 컨트랙트는 코드이기 때문에 취약점을 내포하고 있을 가능성이 존재하며, 이더리움에서 가장 많이 활용되는 프로그래밍 언어인 Solidity와 가상 머신인 EVM(Ethereum Virtual Machine)은 <표 1>과 같은 취약점들이 존재**

<표 1> Solidity와 EVM이 가지는 주요 취약점

Level	취약점 종류	설명
Solidity	Call to Unknown	- 정의되지 않은 함수 호출로 인해 폴백함수 실행
	Exception disorder	- 예외 발생시 트랜잭션의 행동을 모두 되돌리는 rollback (revert) 과정이 호출 환경마다 다르게 발생
	Reentrancy	- 하나의 함수가 개발자의 의도치 않게 반복적으로 호출될 수 있는 취약점
	Unprotected functions	- 실행과정의 실제 수행 조건과 결과가 중요한 하는 핵심 함수(e.g. send)들이 적절한 확인 없이 실행
	Integer Overflow/Underflow	- 과도한 연산이나 올바르지 못한 값이 사용되어 정수의 범위를 벗어나는 경우

사실을 증명시킬 수 있는 암호 기법

4) 가스(GAS) : 블록체인 특성상 마이너가 코드를 검증 및 실행을 하기 때문에 악의적으로 네트워크 마비 코드 실행을 방지하기 위한 코드 실행 수수료

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	Unauthorized ownership takeover	- 권한 확인이 중요한 컨트랙트에서 비정상적인 소유자 정보 변경이 가능
	Detect tautologies	- 결과가 명확한 코드가 무의미하게 실행되는 경우
	Complex fallback function	- 너무 복잡한 폴백함수를 구성하여 다른 계약의 호출이 불가
	uninitialized storage pointer usage	- 초기화되지 않은 스토리지 포인터 사용
	access control violation vulnerability	- 접근 제어 위반
EVM	Call to untrusted contract	- 신뢰할 수 없는 컨트랙트가 호출될 수 있는 경우
	DoS with gas	- 부정확한 gas cost의 할당으로 인하여 사용되는 자원과 비용의 mismatch
	Transaction disorder	- Miner에 의해서 실제 적용되는 transaction의 순서가 변경 가능
	Keeping secrets	- 컨트랙트 내의 변수를 트랜잭션의 내용을 통해 추론 가능

○ **대표적인 스마트 컨트랙트 해킹 사건으로는 The DAO 해킹 사건과 Parity Wallet Bug가 있으며 취약점을 이용하여 실제 해킹이 발생 할 수 있음을 입증**

- 이더리움 스마트 컨트랙트 언어인 Solidity의 취약점 중 하나인 <표 1>의 Reentrancy와 Call to UnKnown을 이용하여, 인출 기능을 가진 함수가 반복 수행됨으로써 암호 화폐인 이더(Ether)가 대량으로 유출

  · 이더를 전송할 때 사용되는 value()와 send()함수는 송신자의 폴백(fallback[5])함수를 호출할 수 있는데, [그림 9]와 같이 공격자는 함수 1회 호출을 반복적으로 호출될 수 있게 조작하는 것이 가능

- 대부분의 Solidity 개발 도구 환경에서는 Reentrancy취약점을 막기 위하여 개발 단계에서 주의하도록 권고하고 있지만, 개발자가 직접 변경하지 않을 경우 취약점이 있는 컨트랙트가 그대로 등록될 수 있음



[그림 9] Fallback 함수를 이용한 컨트랙트 공격

- Parity Wallet Bug는 <표 1>의 Call to Unknown과 논리 에러(Logic error)를 이용하여 공격자가 지갑을 탈취해 이더를 훔쳐간 사례

  · 개발자가 함수의 가시성 설정을 누락하는 경우 공격자가 호출되어서는 안 되는 함수를 호출하면서 지갑의 소유주가 바뀐 사례로 볼 수 있음

  · Wallet 컨트랙트가 간접 호출인 delegateCall을 이용하여 WalletLibrary의 함수를 호출하도록 되어있었고, 이를 이용해 해커는 [그림 10]처럼 지갑의 소유주를 바꾸고 이더를 이체해 감

  · 이러한 부분은 개발 단계에서 발견할 수 있는 것이 아닌 컨트랙트 코드 실행 직전이나 실행 중 단계에서 발견하는 것이 더 용이함

5) 폴백 : 호출된 함수가 해당 스마트 컨트랙트에 없거나 기타 예외상황에서 불리는, 이름 없는 함수

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 10] Parity Wallet Bug로 소유주가 바뀌면서 이더리움을 도난당하는 과정

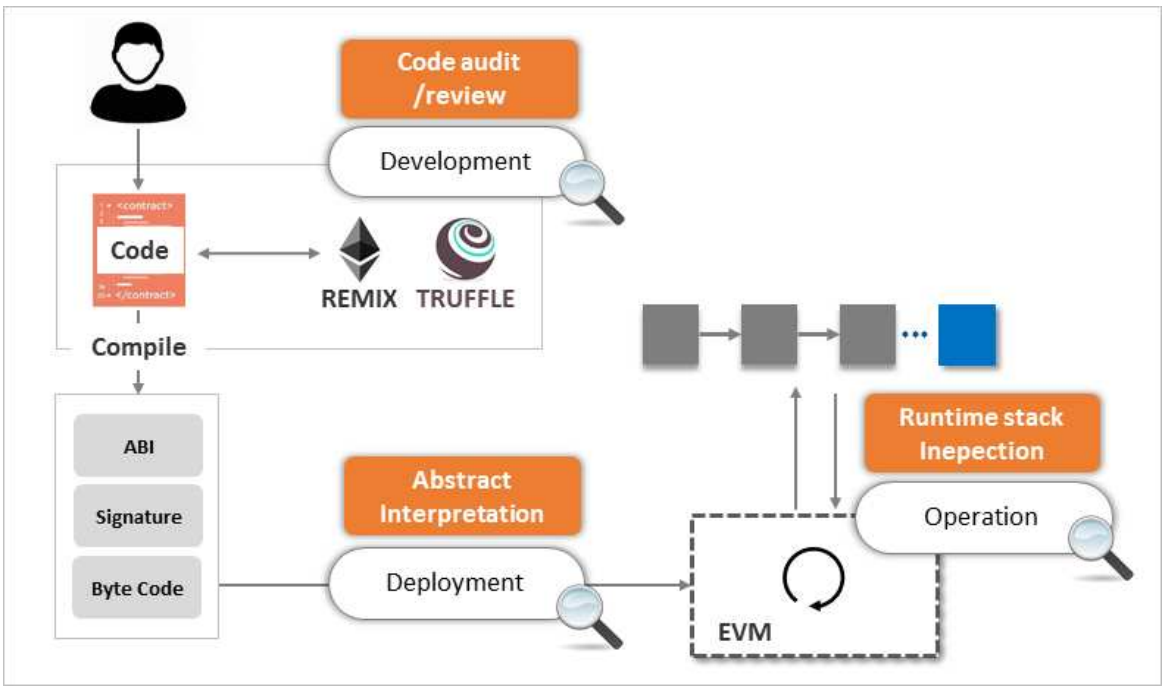
○ 이외에도 블록체인 환경의 스마트 컨트랙트에서 발생 가능한 공격으로 바이트 코드로 변환되고 실행될 때, 각 연산실행에 필요한 가스가 적합하게 설정되지 않은 점을 이용하여 이더리움 네트워크를 공격하는 것이 가능

- 2016년 11월에 비교적 저렴한 가스 비용을 지닌 연산부호 "EXITCODESIZE" 를 다량으로 발생시켜 이더리움의 네트워크가 마비되는 사태 발생
- 이와 같은 문제는 컨트랙트 작성이 아닌 실행 단계에서 취약점을 분석해 낼 수 있어야 함

○ **해킹 사건을 통하여 스마트 컨트랙트는 단순히 작성 단계에서 보조 하는 것으로는 해결할 수 없으며, [그림 11]과 같이 개발(Development) 단계, 트랜잭션으로 전송되는 배포(Deployment) 단계, 가상 머신에서의 실행(Operation) 단계로 나누어 분석이 진행되어야 함**

- 그러나 현재로써는 취약점을 찾기 위해 여러 도구들을 이용 해야하고 스마트 컨트랙트 배포, 실행 단계에서 취약점을 막는 사례가 없으며 대상 언어도 Solidity에 한정되어 있음
- 현재는 웹 기반 개발환경인 Remix와 개발용 블록체인 환경인 Truffle과 같은 툴에서 간단한 분석으로 안전하게 작성되어 있는지 확인 후 컴파일 되어 트랜잭션 상태로 업로드되고 EVM에서 실행됨

○ **본 연구에서는 스마트 컨트랙트의 여러 단계에서 취약점을 분석하기 위하여 [그림 101과 같이 개발 시에는 코드 정합성 검증(Code audit), 컨트랙트 배포 시에는 요약 실행[6](Abstract interpretation), 실행 시 런타임 스택 인스펙션(Runtime stack Inspection) 기법을 적용**



[그림 11] 스마트 컨트랙트가 실행되기까지의 과정

6)요약실행(Abstract Interpertation) : 요약 값을 사용하여 실행함으로써 프로그램의 동작에 대한 정보를 계산

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **본 연구에서 제안하는 방법이 탐지할 수 있는 취약점은 <표 2>와 같음**

<표 2> 제안하는 연구 결과로 탐지 가능한 취약점

Level	취약점 원인	탐지여부
Solidity	Call to Unknown	O
	Exception disorder	O
	Reentrancy	O
	Unprotected functions	O
	Integer Overflow/Underflow	O
	Unauthorized ownership takeover	O
	Detect tautologies	O
	Complex fallback function	O
	Call to untrusted contract	O
	uninitialized storage pointer usage	O
	access control violation vulnerability	O
EVM	DoS with gas	O
	Transaction disorder	O
	Keeping secrets	X

- Keeping secrets는 프라이버시의 문제이기 때문에 스마트 컨트랙트의 코드상에서 해결할 수 있는 문제가 아님

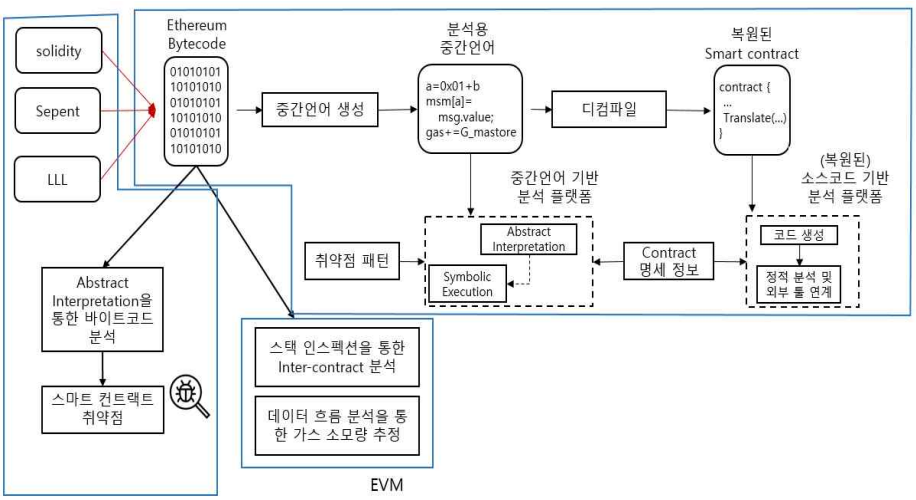
○ **제안하고 있는 스마트 컨트랙트의 각 단계별 코드 분석 도구는 아래와 같음**

- **(개발 단계)** 개발되는 코드의 안전성 강화를 위한 도구는 자동 역공학을 통하여 인터 컨트랙트(Inter-Contract) 수준의 분석을 함

· Solidity 소스코드에서는 다른 컨트랙트의 바이트코드를 소스코드로 변환한 후, 이것을 개발 중인 컨트랙트와 함께 분석하는 인터 컨트랙트 수준의 분석을 함

· 소스코드로 변환을 위해 스마트 컨트랙트의 특성을 잘 반영한 중간언어를 정의하고 이를 거쳐 변환함으로써 Solidity 외에 다양한 스마트 컨트랙트 프로그래밍 언어를 지원할 수 있도록 설계함

· [그림 12]와 같이 취약점 패턴과 컨트랙트 명세 정보를 이용하여 중간언어 기반 분석 플랫폼에서 취약점 분석이 이루어짐



[그림 12] 스마트 컨트랙트 취약점 분석과정

· 위와 같이 디컴파일되어 복원된 소스코드는 소스코드 기반 분석 플랫폼에서 컨트랙트 명세와 일치하는지 정합성 검증이 이루어짐

- **(배포 단계)** 배포 단계의 바이트 코드 분석은 EVM에서 실행되기 직전에 분석이 이루어짐

· 이더리움 스마트 컨트랙트를 작성하는 프로그래밍언어는 Solidity, Sepent, LLL등 다양하고 이들 모두 컴파일되어 바이트코드로 생성되어 실행되기 때문에 EVM 바이트코드 수준에서의 분석은 반드시 필요함

· 요약 실행을 이용한 정적 분석을 통하여 스마트 컨트랙트가 취약점을 가지고 있는지에 대해 분석함

· 요약 실행의 특성상 반복문이 존재하더라도, 실행 경로에 대한 풀커버러지를 달성하는 것이 가능함

- **(실행 단계)** 런타임 스택 인스펙터(Runtime Stack Inspector)를 통해 callee-caller 주체(Credential)간의 관계를 확인하여 코드를 실행하게 함

· 바이트코드는 EVM에서 실행될 때 분석이 이루어짐

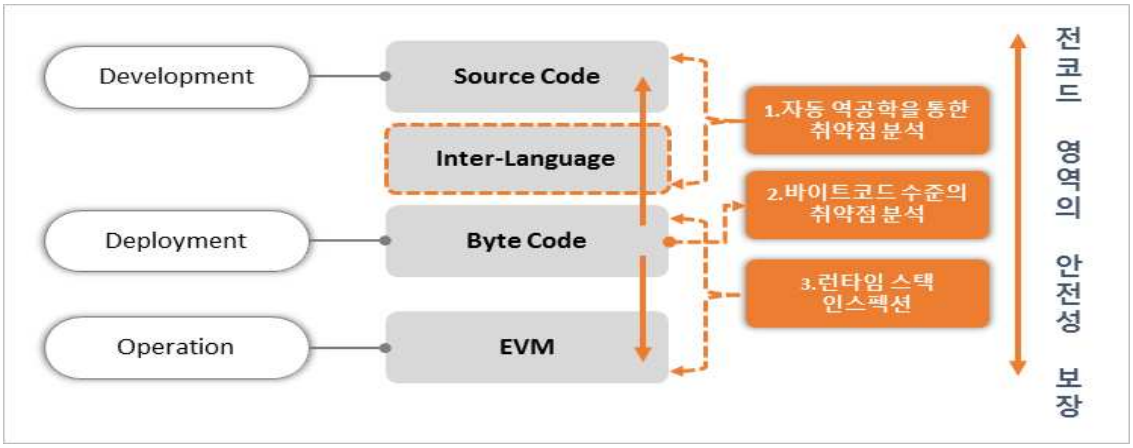
· 런타임 함수 호출 스택(Call stack)검사를 통해 callee-caller를 파악하고 실행 주체를 파악한 뒤, 인터 컨트랙트 간의 관계에서 컨트랙트가 하고자 하는 행위 검사의 타당성 분석

· Reentrancy로 인해 생성된 반복문은 호출 체인(Call chain) 검사를 통하여 반복된 실행을 방지하는 것이 가능

· 민감한 개인 정보 및 이더, 가스, 토큰과 같은 주요 자원 데이터에 대한 흐름을 파악하는 분석기를 설계하여, 이를 바탕으로 코드의 자원 소모량을 추정

· 반복문으로 인한 Dos with gas는 자원 소모량 추정을 통해 사전에 가스 소모량을 추정하여 과도하게 가스가 소모될 경우 실행을 사전에 차단함

○ **본 연구에서 제안하는 방법은 [그림 13]와 같이 이더리움 실행을 위한 전체 코드 영역인 소스코드, 바이트코드에 대해 안전성을 보장할 수 있음**



[그림 13] 제안하는 취약점 분석코드가 보장할 수 있는 영역

- **(개발 단계)** Solidity 소스코드에서는 다른 컨트랙트의 바이트코드를 자동 역공학하여 중간 언어를 거쳐 소스코드로 변환하여 개발 단계에서 보다 넓은 범위의 안전성을 보장할 수 있음

- **(배포 단계)** 바이트코드에서의 분석은 Solidity뿐만 아니라 다양한 스마트 컨트랙트 언어로 작성된 프로그램에 대한 취약점 분석을 가능하게 하여 안전성을 보장함

- **(실행 단계)** 바이트코드를 EVM에서 실행할 때 분석은 허가되지 않은 함수 호출을 방지하고 데이터 흐름 분석을 이용한 가스 소모량 추정을 통해 DDos공격을 예방

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1-3. 연구개발 범위

□ 연도별 연구개발 범위 및 연구 수행결과는 아래의 표와 같음

구분(연도)	세부연구명	세부 연구 목표	연구개발 수행 내용	연구 결과
1차 연도 (2018)	트랜잭션 익명화/암호화 적용을 위한 블록체인 구조 연구	스마트 컨트랙트 생성 및 실행 환경 분석	○ 이더리움 EVM 및 GAS 수수료 체계 분석 ○ EOS,하이퍼레져 등 스마트 컨트랙트 플랫폼 분석	○ 이더리움 등 스마트 컨트랙트 생성 및 관리 체계 비교 및 분석
		트랜잭션 구조에 의한 연결 공격 및 거래 추적 위협 분석	○ 비트코인 등 UTXO 구조의 트랜잭션과 Account 방식의 트랜잭션 비교 분석	○ 주소 체계 및 연결성에 의한 위협 분석
		익명화된 트랜잭션 기반 합의를 위한 프로토콜 분석	○ 이더리움, 하이퍼레져 등 연결성 방지를 위한 기밀성 적용 프로토콜 분석	○ 스마트 컨트랙트 플랫폼 별 합의 알고리즘 비교
		공개키 기반의 암호화된 트랜잭션 검증 프로토콜 개발	○ 트랜잭션 기밀성을 위한 블록체인 암복호화 구조	○ 스마트 컨트랙트를 활용한 IPFS 파일 공유 시스템 구현 ○ Raiden Network를 활용한 상태 채널 구현
		NIZK 기반의 알고리즘 및 SNARK 기법 분석	○ SNARK기법과 Setup 특성 및 연산량, 통신량 분석	○ Setup 특성 분석 내용 및 기존 SNARK 기법의 복잡도 분석
		영지식 증명을 이용한 증거 생성 및 검증 기법 개발	○ 주어진 값이 공개된 범위에 포함됨을 증명하는 증거 생성 및 검증 알고리즘 개발 ○ 증거생성 및 검증알고리즘의 안전성을 분석한 안전성 분석	○ 증거 생성 및 검증을 확인하기 위한 알고리즘 코드 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	스마트 컨트랙트 작성 언어 및 분석 플랫폼의 기반 기술 구성	개발자의 의도와 다르게 동작하는 Solidity 기능 조사	○ 개발자가 fallback 함수 호출을 원치 않을 때, 사용 가능한 함수 정의	○ nonfallback 등을 Solidity에 추가하는 전처리기 개발
		Solidity 대상의 바이트코드 변환기 연구	○ 원본 EVM 코드를 수정하여 새로 정의된 안전한 EVM 바이트코드를 만들어내는 EVM 바이트코드 재작성기	○ 바이트코드 변환기 설계서, 기능 확인용 EVM 확장 프로토타입 개발
		대표적인 이더리움 스마트 컨트랙트 공격 코드 수집	○ 신규 스마트 컨트랙트 공격 발굴 ○ 공격 코드 수집 EVM 바이트코드 요약 실행을 위한 의미 구조 정리	○ Solidity 언어 설계 분석
		스마트 컨트랙트 중간언어 시스템 개발 연구	○ 스마트 컨트랙트 분석 시스템의 검증을 위한 test case 생성 기법 개발	○ 스마트 컨트랙트 Test case 자동 생성 시스템
		스마트 컨트랙트 중간언어 복원 기술 연구	○ 이더리움 시스템 기반의 스마트 컨트랙트 분석 기술 연구 ○ 이더리움 스마트 컨트랙트 중간언어 복원 시스템 개발	○ 이더리움 스마트 컨트랙트를 위한 중간언어 복원 시스템 (Emil) ○ 스마트 컨트랙트 자동 속성 분석 기술 개발
		스마트 컨트랙트 중간언어 복원 기술 연구	- 이더리움 시스템 기반의 스마트 컨트랙트 분석 기술 연구 - 이더리움 스마트 컨트랙트 중간언어 복원 시스템 개발	- 이더리움 스마트 컨트랙트를 위한 중간언어 복원 시스템 (Emil) - 스마트 컨트랙트 자동 속성 분석 기술 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

구분(연도)	세부연구명	세부 연구 목표	연구개발 수행 내용	연구 결과
2차 연도 (2019)	SNARK 알고리즘 구현과 실험 및 영지식 증명 적용을 위한 스마트 컨트랙트 생성 프로그램 설계	Σ-Protocol (ZKP) 설계	-FS 변환기법을Σ-Protocol에 적용하여 Random Oracle 모델에서 안전한 SNARK 설 계 -Unruh 변환기법을 적용하여 Quantum RO 모델에 안전성 검증	· 양자안전한 Σ-Protocol 기법을 활용하여 양자안전한 SNARK을 생성
		Proof 크기의 개선을 및 안전성 검증 연구	- Parameter를 변경하여 Proof 사이즈 감소 - ZKBOO의 안전성 검증 연구	· Proof 사이즈 2배 개선 · Circuit 확인 및 Honest-Verifer Zk 검증
		영지식 증명 생성 (Zokrates) 및 재작성 (Zkay) 프로그램 분석	- 영지식 증명 검증이 스마트 컨트랙트에서 이루어질 수 있도록 하는 도구 분석 - 영지식 증명 컨트랙트를 재작성 해주는 도구 분석	· Private Type 지정 · DSL 특징 분석
		새로운 영지식 증명을 이용한 생성 (Zoker) 프로그램 설계	- 구현한 SNARK 알고리즘을 이용하여 영지식 증명 생성 프로그램 구조 분석	· Zoker의 구조 설계 · Trusted setup의 필요 없는 알고리즘 분석
		On-chain 프로토콜 비용감소를 위한 SVZK 설계	- 영지식 증명의 Scalability를 개선하기 위한 방법 분석	· SVZK 영지식 증명 검증 비용 비교 및 구조 설계

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	스마트 컨트랙트 자동 역공학 도구 및 바이트코드 분석기 개발	EVM에서 동작 가능한 런타임 스택 인스펙션도구 프로토타입 설계	- 런타임 스택 인스펙션을 위한 실행 주체와 관련된 행위 집합 연구 - Call chain 식별을 위한 매커니즘 설계 진행	· EVM 바이트 코드의 callee-caller 주체 간의 관계를 확인하는 런타임 스택 인스펙션 도구 프로토 타입 개발
		정책을 이용한 런타임 스택 인스펙션도구 프로토타입 개발	- 정책의 형태로 한 자료구조에 기술하고 정책을 위배 했는지에 대한 여부 파악	· json을 선정하고 go언어에서 json을 사용할 수 있도록 연구
		바이트 코드 식별 모델 구현을 위한 머신러닝 모델 구축	- 속성 레퍼런스 선정 - 특징 추출 기법 정의 및 모델 학습	· aeCNN모델을 사용하여 높은 성능을 확인, 케이스 스터디를 통한 DApp의 속성 파악
		스마트 컨트랙트 역공학 지원 언어 확장	- 고수준의 제어 구조를 복원하기 위하여 구조 분석 기술 개발 - Control flow graph를 복원하여 패턴 복원	· 복원된 Control flow 정보를 고수준 언어인 Solidity의 Statement로 자동 변환 가능한 툴 개발
		스마트 컨트랙트 취약점 분석기 개발을 위한 취약점 탐구	- Storage Layout 문제점 분석 - high-level 언어와 low-level 언어의 storage 탐지 방법 비교	· Storage Layout 분리 기법을 제안하여 문제점 해결 방법 제안
		Oyente의 확장 개발	- Storage layout 분리 기법을 Oyente에 적용	· 스마트 컨트랙트 신규 취약점 발견 및 Oyente와 연동되는 분석기 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

구분(연도)	세부연구명	세부 연구 목표	연구개발 수행 내용	연구 결과
3차 연도 (2020)	실행 환경 및 전용 SNARK 알고리즘 최적화 작업 및 블록체인 플랫폼(이더리움)에 맞는 영지식 증명을 지원하는 기반 도구 개발	실행 환경에 따른 영지식 증명을 위한 SNARK 프로토콜 최적화	- 사용자 실행환경에 특화된 트랜잭션 코드 집함을 위한 전용 SNARK 개발	·증거의 크기가 2단계에서 약 50% 수준으로 개선할 기법 제안
		전용 SNARK 프로토콜의 시범구현 및 구현 수준 최적화	- 대칭키 암호기법을 적용하였으나, 크기를 줄일 수 있는 방법 제안	·Rust를 활용하여 언어 구현 및 라이브러리화
		Bulletproofs+ 기법의 증거크기 개선	- 가중치 내적을 활용하여 증거크기 개선	·수백 KBytes에 이르는 증거의 크기를 수십 Bytes로 개선
		이더리움 플랫폼에서 영지식 증명을 지원하는 도구 개발	- Solidity 코드 재작성	·lexer, paser 등을 포함하여 컨트랙트를 재작성해주는 Compiler의 구현
			- Zoker 구현	·재작성 및 generator를 활용하여 영지식 증명이 적용되는 도구 구현

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	스마트 컨트랙트 공격 검출 기능 및 프로토타입, 취약점 분석기 개발	EVM+ 개발을 위한 매커니즘 개발	- Call chain 식별을 위한 매커니즘으로, c함수의 호출 경로를 파악할 수 있음	·함수 selector를 활용하여 런타임 인스펙션에 적용 방법 고안
		EVM에서 동작 가능한 런타임 스택 인스펙션 도구 개발	- EVM 바이트코드의 callee-caller 주체 간의 관계를 확인하는 런타임 스택 인스펙션 도구 개발	·새로운 키워드(ForInit)과 (ForINITSTART) 를 활용하여 call stack 도구 개발 완료
		스마트 컨트랙트 취약점 탐색 테스트를 위한 테스트셋 생성 기술 개발	- 공격가능한 버그를 조사하여 테스트셋을 구축	·Testset 코드 생성 완료
		이종 역공학 프로그래래밍 언어 지원	- 바이트 코드 상태의 스마트 컨트랙트를 역공학하여 중간어를 생성	·중간언어를 통해 가장 많이 사용되고 있는 언어인 Solidity와 Vyper를 지원
		취약점 분석을 위한 Formal verification 과정 분석	- test scenario나 test case 없이 smart contract 안에서 취약점이 발생 가능한 모든 부분을 수학적인 방법으로 찾아낼수 있는 방법 분석	·VeriSmart 형태의 formal verifiaction 방법의 발전 방향 제시
		스마트 컨트랙트 취약점 분석기 개발	- 현재의 스마트 컨트랙트 분석기 수준 파악 및 미래 연구 주제를 파악하기 위해 Survey 진행	·oyente의 확장된 버전을 개발하였으며 기존의 취약점에 비해 나은 성능을 가진 개발 도구 개발 완료

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구수행내용 및 성과

2-1. 1차년도

1) 스마트 컨트랙트 프라이버시 보호 기술 연구

□ NIZK 기반의 알고리즘 및 SNARK 기법 분석

○ 기존에 제안된 SNARK 기법에 대한 종합적 분석을 위한 조사 및 분류

- Doubly Efficient Proofs 계열. Proof for Muggle 또는 GKR 기법 [GKR08]으로 알려져 있는 기법으로 증명자와 검증자 양쪽 모두에 대하여 다항식 시간안에 임의의 연산의 정당성 증명

- Probabilistic Checkable Proofs (PCP) 계열. Kilian [Kil92]과 Micali [Mic94]에 의하여 제안된 기법으로 PCP 자체는 영지식성을 요구하지 않으나 약간의 연산량과 전송량을 추가로 부담하여 ZNARK으로 변환

- Linear PCP 계열. Ishai 등의 연구 그룹과 Groth [Gro16]에 의하여 개별적으로 연구된 기법으로 임의의 함수를 선형의 벡터의 결합으로 표현한 후, 주어진 벡터를 적절한 인코딩 기법을 사용하여 정상적으로 연산이 이루어진 경우 효율적으로 검증할 수 방법

- IKOS 계열. Ishai-Kushilevitz-Ostrovsky-Sahai (IKOS) [IKOS07]가 임의의 Multi-party Computation (MPC) 기법을 Zero-Knowledge Proof (ZKP) 기법으로 변환하는 방법

○ 기존 알려된 SNARK 기법의 Setup 과정에서 신뢰기관의 필요성에 대한 분석

○ 기존 알려진 SNARK 기법의 증거생성 및 검증과정에서 증거생성 알고리즘과 증거검증 알고리즘의 연산량 및 전송량을 분석

○ 기존 알려진 SNARK 기법을 기반 암호기법에 따라 분류한 후, 각각의 장단점 분석(<표 3>)

<표 3> SNARK 장단점 비교

	신뢰기관 요구	증명의 길이	증명 알고리즘 수행시간	검증 알고리즘 수행시간
GKR 계열	아니오	○	○	○
PCP 계열	예	○	×	○
Linear PCP 계열	아니오	○	×	◎
IKOS 계열	아니오	×	◎	○

◎: 좋음, ○: 보통, ×: 나쁨

○ 기존 SNARK 기법의 분석을 바탕으로 새로운 SNARK 기법의 설계를 위한 방향 설정

□ 영지식 증명을 이용한 증거 생성 및 검증 기법 개발

○ 영지식 증명기법을 사용하여 암호화되어 공개된 암호문의 평문값이 임의의 폐구간에 속하는 값임을 증명하는 기법 설계

- 영지식 증명기법을 사용하여 암호화되어 공개된 암호문의 평문값이 임의의 폐구간에 속하는 값임을 증명하는 기법 설계

○ 주어진 값이 임의의 정수이므로 Pedersen Commitment 기법대신 Fujisaki-Okamoto Commitment 기법을 적용

○ Commitment 기법을 사용하여 증명하려는 값을 공개, 이때 Commitment 기법의 안전성에 의하여 공개된 암호문의 평문값은 알 수 없음

○ 증명자와 검증자가 공통으로 사용하는 파라미터를 생성하는 설정 알고리즘 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 설정 알고리즘은 먼저 Fujiski-Okamoto (FO) Commitment 기법을 수행하기 위한 파라미터를 생성

○ 공개된 암호문에 포함된 평문 정수값이 증명자와 검증자에게 모두 알려진 구간에 포함된다는 증명을 생성하는 효율적 알고리즘 개발

○ 주어진 증명값을 사용하여 암호문에 숨겨진 평문값이 알려진 구간에 포함된다는 주장을 검증하는 알고리즘 개발

- 공개된 값 a,b와 구간 $I=[a..b]$에 대하여, x의 Commit된 값으로부터 $x-a$와 $b-x$를 계산하게 하고 증명자가 먼저 계산하여 양수라고 주장하는 값 m의 Commitment 값과 같게 됨을 연속된 값들을 통하여 증명

○ 설정/증명/검증 알고리즘이 안정성 조건을 만족함을 엄밀히 분석

○ 설정/증명/검증 알고리즘 구현 및 성능 실험(<표 4>)

<표 4> 영지식 구간증명 기법의 구현 결과 및 성능 비교

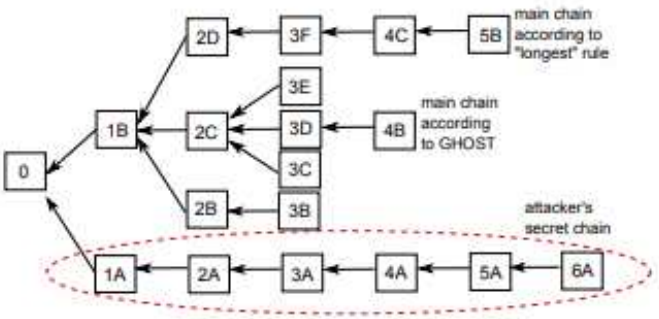
	1200 비트		1300 비트		1400 비트	
	증명	검증	증명	검증	증명	검증
새로운 기법	224.2msec	158.0msec	235.3msec	165.5msec	247.1msec	173.5msec
	257.4msec	180.4msec	268.4msec	188.0msec	279.9msec	195.4msec

□ 익명화된 트랜잭션 기반 합의를 위한 프로토콜 분석

○ 블록 생성 시간 단축에 따른 고스트 프로토콜 기반 합의

- 이더리움의 블록 생성 시간은 평균 10~15초 이내로 비트코인의 평균 10분에 비해 매우 빠른 블록 생성 시간을 가지기 때문에 동기화 문제와 더불어 블록 생성자의 보상을 위해 고스트 프로토콜(Ghost Protocol)을 도입

- 고스트 프로토콜은 [그림 14]와 같이 주체인을 선택할 때 최대 7세대까지 포함하여 동시에 생성된 분기의 블록을 인정하여 보상을 해주는 체제로 자원 낭비를 줄이고 블록체인의 51% 공격에 의한 안전성을 강화



[그림 14] 고스트 프로토콜에서의 체인

○ PoW와 PoS 하이브리드 합의를 위한 Casper 프로젝트

- 현재 이더리움에 적용된 합의 알고리즘은 비트코인의 PoW(Proof of Work)의 ASIC 저항성을 개선한 메모리 의존적인 작업 증명 방식인 Ethash 알고리즘을 적용한 PoW를 사용

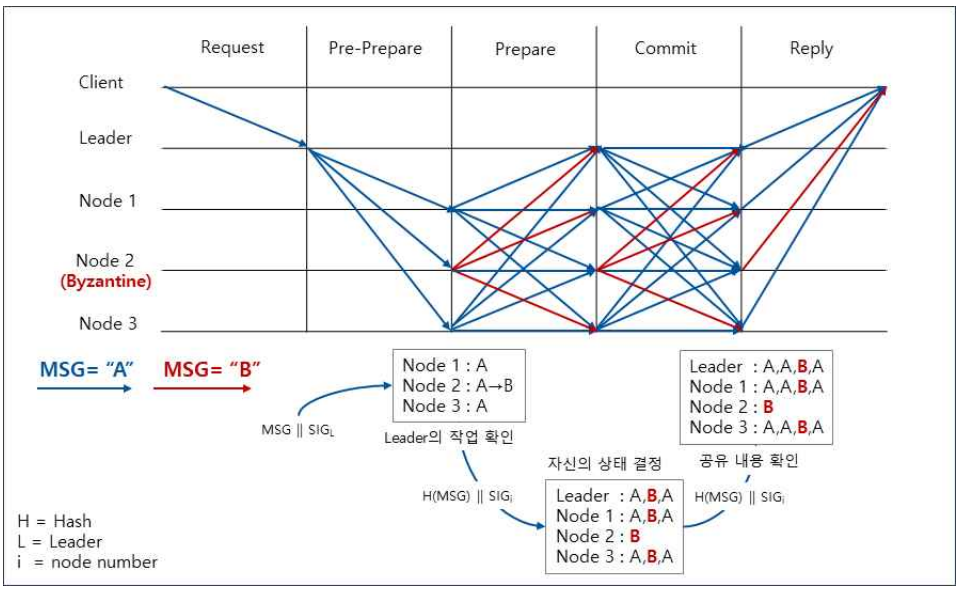
- 하지만, PoW 계열 자체가 가지는 에너지 낭비 문제와 51% 공격에 대한 최종성 문제를 해결하기 위하여 PoS(Proof of Stake) 방식의 합의를 도입할 예정으로 Casper 프로젝트를 진행 중

- PoW와 PoS의 하이브리드 방식인 Casper는 블록을 생성하는 Miner와 주체인을 결정하는 검증자를 따로 두며, 검증자의 3/2이상이 하나의 블록을 지정할 경우 해당 블록은 51%공격이 발생하더라도 최종성을 가지므로 변경 불가능한 블록으로 합의를 이루는 방식

- 현재 50블록 마다 투표 시기(epoch)를 정하여 도입 예정이며 점차 줄여 1블록 이내로 줄어들면 이더리움 내에 Miner 없이 검증자로만 블록을 생성해나가는 방식으로 변경 될 예정

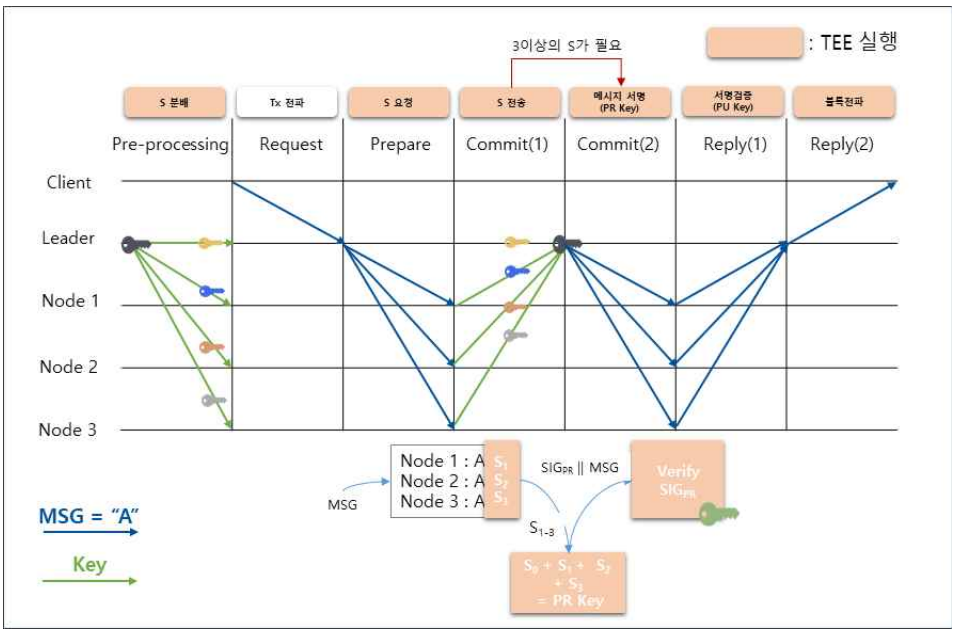
○ Hyperledger의 PBFT를 이용한 합의 과정

- 스마트 컨트랙트를 지원하는 프라이빗 블록체인 중 가장 많이 사용되는 Hyperledger는 0.6버전에서 PBFT를 기반으로 합의 과정을 도입함
- PBFT는 [그림 15]와 같이 비잔틴 노드 f에 대하여 3f+1을 만족하는 노드 구성에서 장애 허용을 만족함
- 정해진 소수의 노드에 의해 합의를 수행하기 때문에 이더리움에 비해 매우 빠른 1000tps 이상의 속도를 가지지만, 소수의 노드를 관리해야 하는 키 관리 문제가 따르므로 탈중앙화 환경에는 부적합



[그림 15] PBFT가 적용된 블록체인의 합의 과정

- 또한 참여 노드 수 n에 대해 노드간 합의를 위한 메시지 수가 $O(n^2)$으로 증가 하기 때문에 다수의 노드를 운영하기에 한계가 존재
- 이 같은 문제를 해결하기 위하여 TEE(Trust Execution Environment)를 도입할 경우 [그림 16]과 같이 Secret Sharing 기반으로 노드수에 따른 메시지 처리량을 줄이는 것이 가능



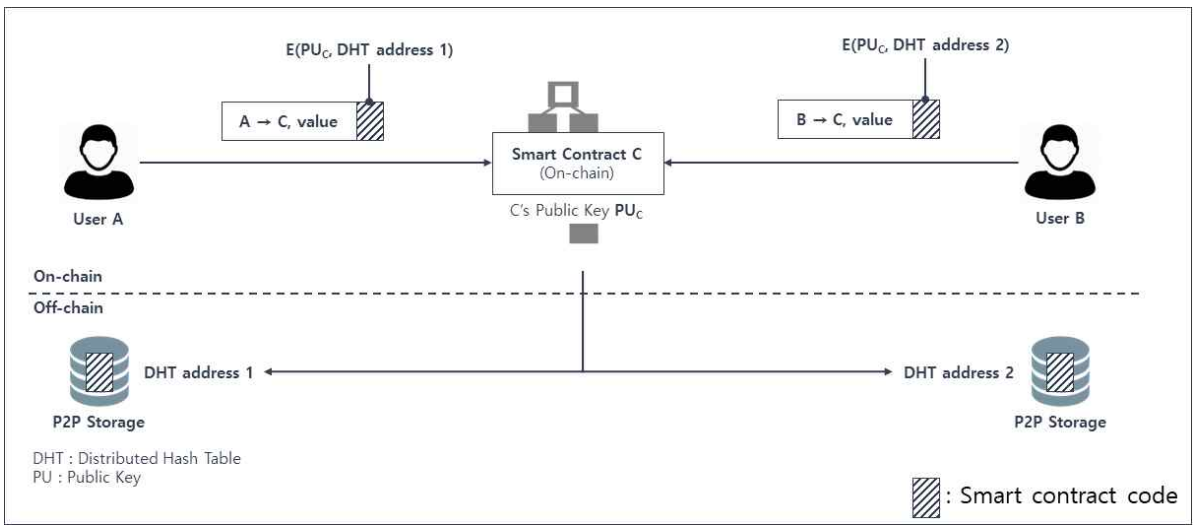
[그림 16] Secret Sharing based Fast BFT

○ EOS.IO의 BFT-DPOS 합의 과정

- 2018년 6월 메인넷을 오픈한 EOS는 스마트 컨트랙트를 지원하는 플랫폼으로 21명의 BP(Block Producer)를 투표로 선출하여 BFT합의를 수행하도록 블록을 생성해나가는 DPOS(Delegated Proof of Stake) 방식을 사용
- 21명의 BP들은 6개의 블록을 생성 후 다음 BP에게 블록 생성 권한을 넘기며, 21명이 라운드 로빈 방식으로 블록을 생성하여 모든 차례가 종료하면 새로운 21명의 투표를 선출하는 과정을 반복
- 이더리움은 스마트 컨트랙트를 실행하는 트랜잭션을 생성한 외부소유계정의 금액에서 차감하는 방식의 GAS 제도에 반해 EOS는 스마트 컨트랙트를 개발하여 등록한 개발자에게 실행 비용을 부과하는 방식으로 댑 개발 시 일반 사용자의 부담을 줄이고 개발자의 비용을 추가하는 사업 방식을 채택

□ 공개키 기반의 암호화된 트랜잭션 검증 프로토콜

- 트랜잭션의 기밀성을 위해서는 블록체인 네트워크에서 암호화된 트랜잭션 상태에서 합의를 수행할 필요가 있으며 이를 위한 인프라가 필요
- 트랜잭션의 서명 작성 및 검증에 사용되는 공개키외에 추가적인 암복호화를 위한 공개키 쌍을 도입하고 트랜잭션 암호화에 활용하며 암호화된 상태의 금액 소유에 대한 증명은 영지식 증명을 이용함
- 트랜잭션의 서명 작성 및 검증에 사용되는 공개키외에 추가적인 암복호화를 위한 공개키 쌍을 도입하고 트랜잭션 암호화에 활용
- 본 연구에서는 트랜잭션에 포함되는 스마트 컨트랙트는 [그림 17]과 같이 컨트랙트 C의 공개키로 암호화되며, 컨트랙트 호출 시 기밀성을 위하여 C의 공개키로 트랜잭션을 암호화하며 실행 될 때 개인키로 복호화하여 결과를 실행하는 프라이빗 컨트랙트(Private Contract) 구조를 도입



[그림 17] Private Contract 개념도

- 이 외의 추가 데이터의 전송을 위해 컨트랙트를 기반으로 하는 P2P Storage를 통해 보다 많은 데이터를 전송할 수 있는 프레임워크를 DHT Table 기반으로 제공

□ 프라이버시 보호를 위한 상태 채널 연구

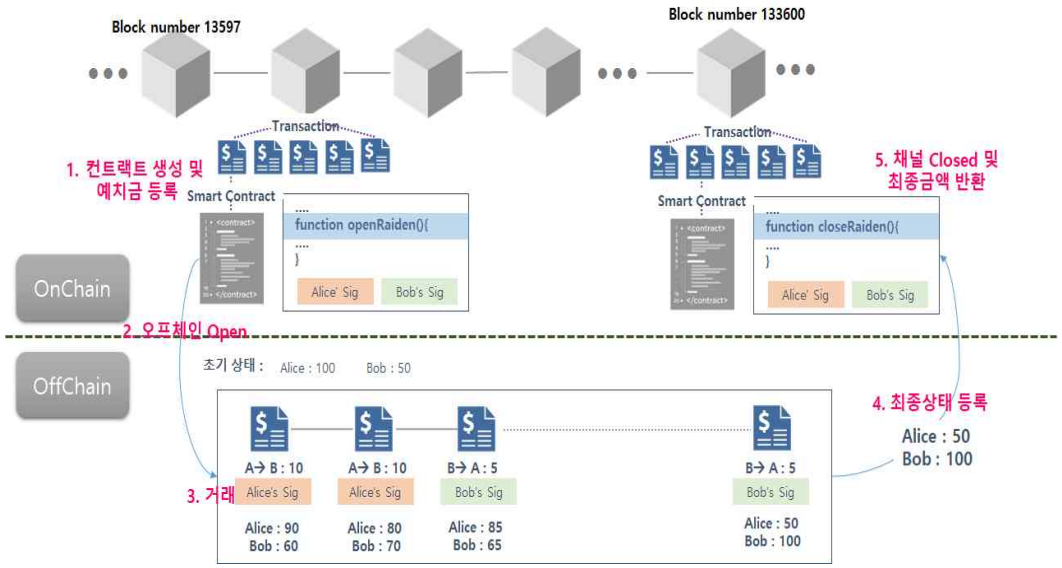
- 21명의 BP를 정하여 소수 합의를 이끌어내는 EOS와 다르게 탈중앙성을 유지하기 위한 이더리움 블록체인은 확장성, 보안성, 탈중앙성의 트렐레마(Trilemma)라고 하는 난제를 해결하기 위해 연구를 진행
- 이에 본 연구에서도 영지식 증명을 적용한 가운데 확장성을 높이기 위한 연구도 함께 진

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

행될 필요가 있으며 기본적으로 블록체인에 저장되지 않는 off-chain 기술로 프라이버시를 보장하는 프로토콜인 상태 채널(State Channel)에 대한 연구를 병행

○ 스마트 컨트랙트 지원 플랫폼의 상태 채널 기술

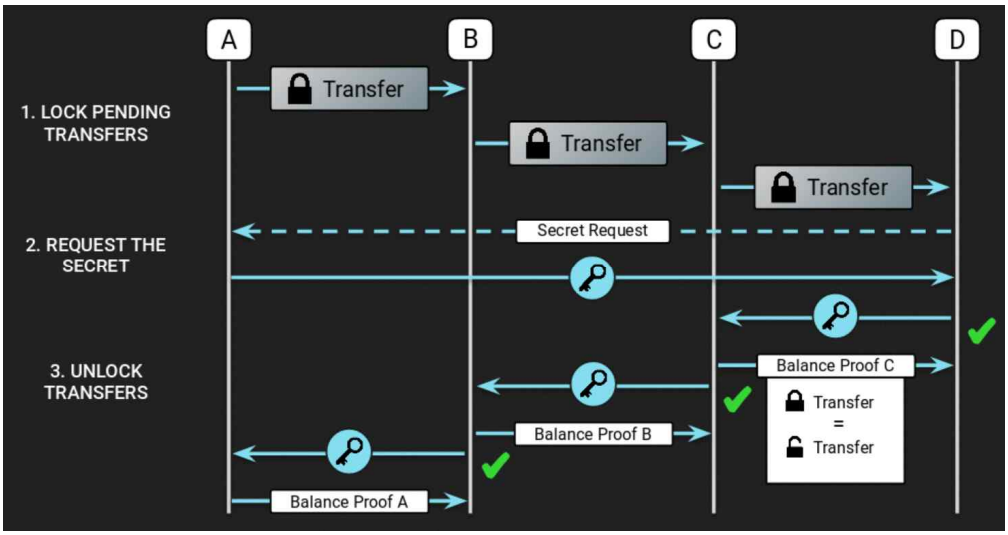
- 상태 채널은 비트코인의 확장성 문제를 해결하기 위한 방법 중 하나로 라이트닝 네트워크(lightning Network)에서 시작되었으며, 현재 이더리움에서도 활발히 연구되고 있는 기술임
- 비트코인의 라이트닝 네트워크는 다중 서명(Multi-Signature) 기반의 주소를 생성하여 양자간의 서명이 있어야만 UTXO를 사용할 수 있는 조건을 이용하여 최초에 다중 서명 구조로 금액을 이체하면 채널이 오픈된 상태가 됨
- 이후 이체 한도 내에서 양자 간의 거래를 블록체인 합의와 상관없이 off-chain 트랜잭션으로 거래가 가능하며, 채널을 닫을 때(UTXO를 사용하는 시기) off-chain 트랜잭션 결과를 모두 제출하면 최종 정산하여 잔액을 분배하는 방식으로 구성
- 이더리움은 아직 상태 채널에 대한 연구가 진행중이며, 비트코인이 다중 서명 기반과 스크립트를 사용하는 것과 다르게 [그림 18]과 같이 스마트 컨트랙트를 기반으로 라이덴 네트워크 채널을 형성
- 라이덴 네트워크를 오픈하기 위해서는 라이덴 컨트랙트 내에 존재하는 채널을 열어주는 함수를 호출해야 함
- 오픈함수 호출 다음에는 사용자가 각자 스마트 컨트랙트에 예치한 금액만큼 오프체인을 통해서 거래가 가능하게 되고, 채널을 클로즈 할 경우에는 각자 자신이 오프체인을 통해 받았던 트랜잭션에 대한 증명을 통해 각자가 받아야 할 토큰의 양을 정할수 있음



[그림 18] onChain에서 거래를 등록하고 OffChain을 이용하여 거래

○ 멀티 홉 트랜잭션의 라우팅 프로토콜 기술 개발

- 양자간의 거래 뿐만 아니라 채널 간 연결을 통해 [그림 19]와 같이 멀티 홉 거래(Multi-hop transfer) 기능을 지원 가능
- 현재 상태 채널에서는 중간 경로에 있는 노드들의 가용 금액을 확인할 수 없어 경로 상의 자금이 부족하여 전송이 지연되는 형상의 문제, 노드 무응답에 대한 대처 방법, 중간의 경로에서 노드 간의 off-chain 거래를 유추할 수 있는 가능성에 대한 문제를 해결되어야 할 필요가 있음
- 이를 해결하기 위하여 스마트 컨트랙트에 등록된 상태 채널의 경로를 모두 색출하여 해당 경로에 사전 요청을 통해 경로를 확보하여 최단 거리에 기반하여 경로를 설정하는 기술을 개발함



[그림 19] Raiden network's Locked Transfer

- 본 기술에서 경로를 제공하는 노드에 대해 수수료에 대한 보상이 존재하므로 동기를 제시하고 금액의 가능 유무만 응답하므로 프로토콜 내에서의 금액 유추는 불가능하여 기밀성을 유지

2) 스마트 컨트랙트 취약점 분석 기술 연구

□ 스마트 컨트랙트 공격 분석과 스마트 컨트랙트 분석 플랫폼의 기반 기술 구성

○ 개발자의 의도와 다르게 동작하는 Solidity 기능 조사

- Fallback 함수가 의도치 않게 호출되어 공격이 일어나는 예가 다수 있었고, 분산 처리가 되는 환경이므로 함수 호출 권한에 대한 명확한 정의가 없어 방어하지 못하는 예도 다수 있음
- 그 중 Fallback함수는 Solidity에서 함수를 호출할 때 인자 또는 타입이 맞지 않는 등의 예외가 발행한 경우에 호출되나 예외가 발생해서 Fallback함수가 호출될 때 별도의 알림이 없기 때문에 개발자는 이러한 사실을 알 수 없음
- 예를 들어 [그림 20]과 같이 C가 실제로 contract Alice 의 주소인지가 확인이 되는 경우라고 하더라도, Bob에 정의된 인터페이스가 실제로 Alice의 인터페이스와 일치하지 않을 수 있으므로 c.ping은 잘못된 참조일 가능성이 있을 경우 Alice의 fallback함수가 자동적으로 실행됨
- 이 때 Alice의 fallback함수에는 악성행위가 주입될 수 있으나, Bob의 계약은 Alice의 fallback 함수 호출 여부나 fallback 함수 동작을 알 수 없게 되므로 현재 이런 문제를 방지하기 위한 현재 Solidity에서 제공되는 기능은 없음

```
contract Alice { function ping(uint) returns (uint) }
contract Bob   { function pong(Alice c){ c.ping(42); } }
                                          Alice의 address
```

[그림 20] 취약한 fallback 함수 호출 예

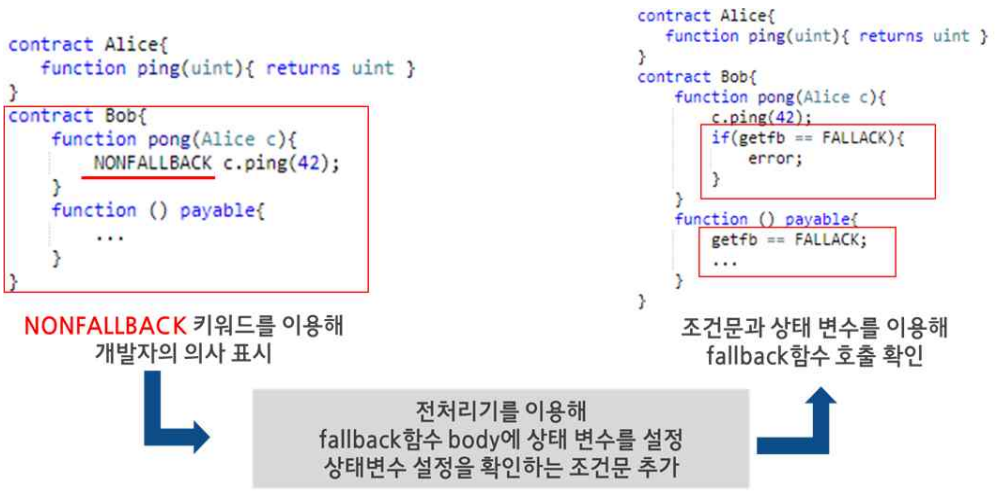
○ 개발자의 의도와 다르게 동작하는 Solidity 기능에 대한 방어 기법 연구

① Solidity 언어의 전처리기 연구

- 사용자에게 새로운 키워드 (NonFallback) 를 제공하여 Fallback함수의 호출에 대한 의도를 표현할 수 있도록 유도, 이러한 언어를 Sol+로 명명
- 사전 검증용으로 사용자가 키워드를 사용하면 Fallback함수가 불렸는지 확인할 수 있게 기능을 덧붙여 안전한 코드로 재작성해주는 전처리기를 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

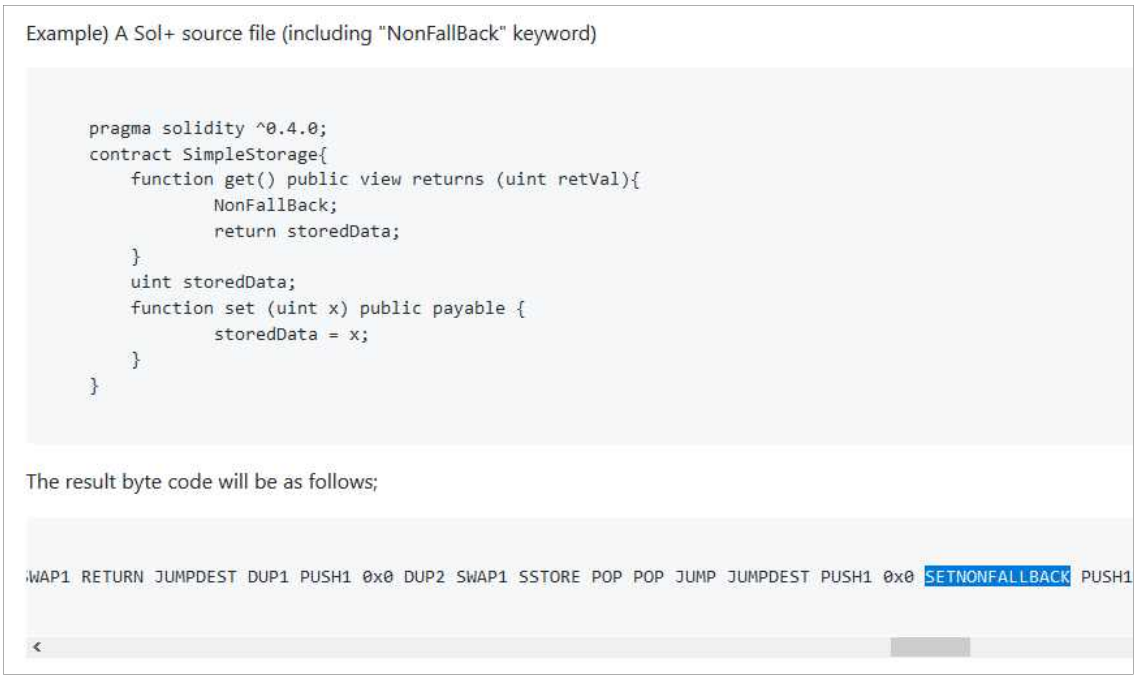
- 전처리기는 키워드를 인식해서 코드에 라이브러리를 추가하고 라이브러리를 이용해서 Fallback함수가 불렸는지 확인하는 작업을 진행



[그림 21] Solidity 전처리기의 역할

② Solidity 컴파일러에 확장에 대한 연구

- Sol+언어를 지원하기 위한 또다른 방법으로 기존 Solidity 컴파일러를 수정할 수 있음: 이러한 컴파일러 이름을 Solc+로 명명
- 기존의 Solidity 컴파일러가 새로운 키워드 (NonFallback)를 받을 수 있도록 Solidity 컴파일러를 수정, 즉 [그림 21]과 같이 새로운 키워드를 처리할 수 있도록 하는 새로운 Ethereum VM 바이트 코드 (예 SETNONFALLBACK, 0x25) 를 생성하도록 Solidity 컴파일러를 수정
- [그림 22]에서는 고급언어인 Sol+ 코드 상에 NonFallBack 키워드가 컴파일러를 Sol+를 거친 후 EVM의 신규 바이트코드인 SETNONFALLBACK으로 변환됨을 확인할 수 있음



[그림 22] Sol+ 파일의 동작 결과

③ Ethereum VM 확장에 대한 연구

- 새로운 Ethereum 바이트 코드를 처리할 수 있도록 Ethereum 가상머신을 수정: 이러한 가상머신을 EthVM으로 명명

- 가장 많이 사용되는 go-ethereum의 가상머신을 수정, [그림 23]과 같은 test가 가능

```
escho@ubuntu:~/developments/go/src/github.com/ethereum/go-ethereum$ $GOPATH/bin/evm --debug --code 25  run
0x
#### TRACE ####
SETNONFALLBACK  pc=00000000 gas=10000000000 cost=0

STOP            pc=00000001 gas=10000000000 cost=0

#### LOGS ####
```

[그림 23] EthVM+에서 0x25 (SETNONFALLBACK) 명령을 처리한 후 결과

- 현재는 SETNONFALLBACK 명령에 대해 내부 전역 flag를 변경하는 수준이지만, 이 정보로 함수 호출에서 사전 검사를 트리거 하는 로직으로 확장 할 계획
- 결과적으로 [그림 24]와 같이 NonFallBack과 같은 새로운 키워드를 포함하는 Sol+코드가 컴파일러인 Solc+에 의해 변환되어 SETNONFALLBACK과 같은 새로운 바이트코드를 포함하는 VM 코드로 변환되고 이 코드는 EthVM+에서 수행됨



[그림 24] 컴파일러 EthVM+의 상호 관계

④ Solidity 대상의 바이트코드 변환기 연구

- <표 5>와 같이 사용자가 키워드를 사용하면 Fallback함수가 불리지 않도록 바이트 코드를 재작성 하는 바이트 코드 변환기 설계 진행
- 사용자가 삽입을 원하는 코드를 입력하면 원하는 부분에서 사용자가 입력한 코드가 실행되도록 바이트 코드를 바꿔주는 수준까지 고려
- 사용자가 원하는 부분에 분기문을 추가하고 입력 코드를 수행한 후 되돌아오도록 하는 것으로 설계, 코드의 포맷과 후면의 해시정보 및 전체 코드 크기 등을 고려

<표 5> 제안하는 취약성 완화 기법들의 종류

<table>
<tr><th></th><th colspan="3">종류</th></tr>
<tr><td>기존 VM 기능 유지</td><td colspan="3">Solidity to Solidity 변환기
(Solidity 전처리기)</td></tr>
<tr><td rowspan="3">새로운 VM 기능 추가</td><td rowspan="2">새로운 바이트코드
명령 생성</td><td>동일한 Solidity 언어</td><td>Bytecode to bytecode
재작성기</td></tr>
<tr><td>새로운 Solidity 언어</td><td>NewSolc 컴파일러
(Solc+)</td></tr>
<tr><td colspan="3">새로운 VM (EthVM+)</td></tr>
</table>

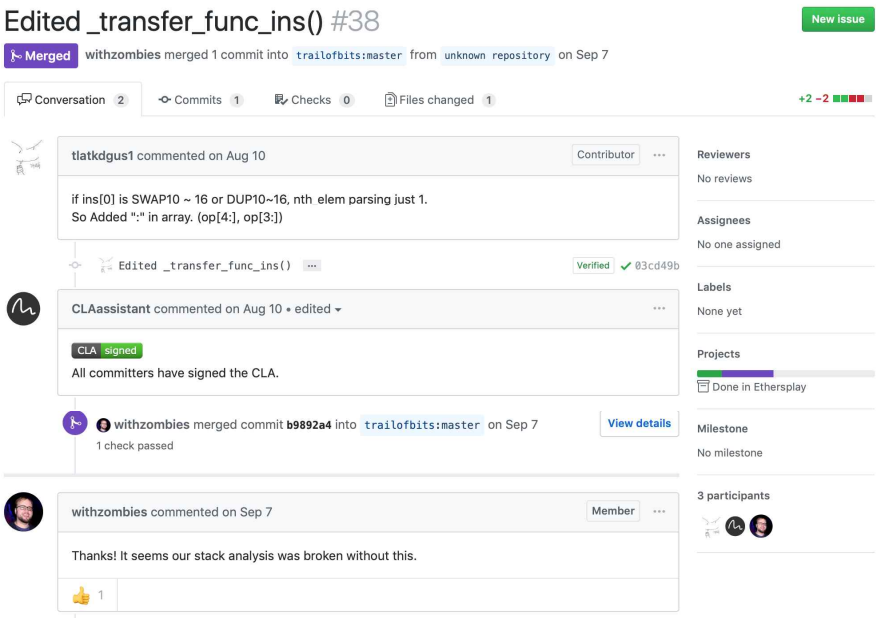
□ 이더리움 스마트 컨트랙트 분석 기술 및 취약점 연구

○ 바이트 코드 상태의 스마트 컨트랙트에 대한 보안성 분석과 활용을 위한 역공학의 필요성 증가

- 최근 이슈화 되고 있는 스마트 컨트랙트의 보안에 대한 문제와 함께, 스마트 컨트랙트의 악의성에 대한 대책의 필요성이 요구되고 있음, 따라서 실질적인 스마트 컨트랙트의 행동 특성이 들어 있는 바이트 코드에 대한 효과적인 분석 방법이 필요함

○ 바이트 코드 스마트 컨트랙트 분석을 위한 역공학 단계별 기술 분석 수행

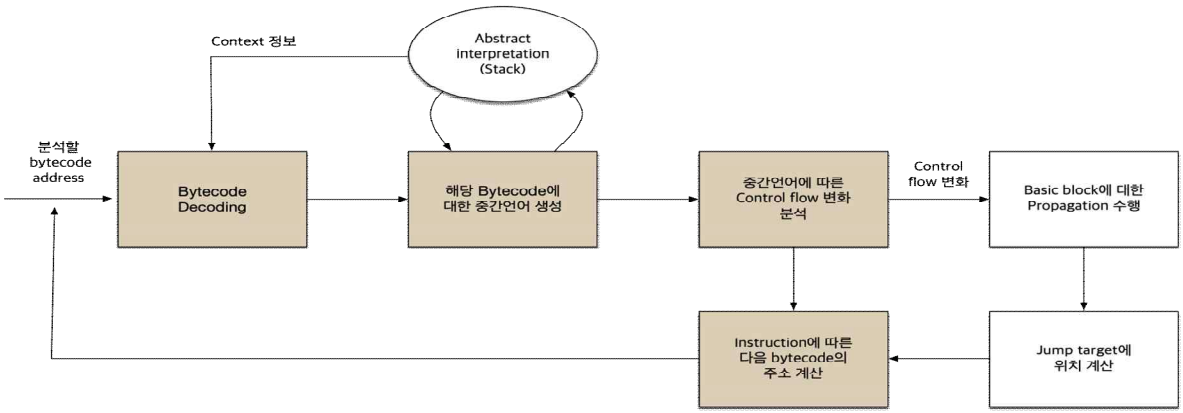
- 현재 활발하게 이루어지고 있는 이더리움 스마트 컨트랙트 분석 시스템에 대한 심층적인 분석 수행
- [그림 25]와 같이 사용 목적에 따른 분석 시스템의 분류 및 개별 기술 분석 (바이트코드 컨트롤 분석, 취약점 탐지, 디컴파일러 등)



[그림 25] Ethersplay에 적용된 분석 내용

○ 기존의 스마트 컨트랙트 분석 시스템의 한계 및 문제점 분석

- 제한된 샘플에서만 수행되는 시스템의 한계 파악
- Control flow의 부정확한 파악에 위한 분석 결과의 부정확 확인
- [그림 26]과 같이 기존 시스템(Trail of Bits의 Ethersplay)의 바이트코드 디코딩 및 스택 분석과정 에서의 부정확한 동작을 발견하여 제보(pull request) 후 적용



[그림 26] 개발된 이더리움 스마트 컨트랙트에서의 재귀적 디스어셈블리 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 스마트 컨트랙트 중간언어 복원 시스템 기술 개발

○ 악의적인 이상 변형을 대비한 이더리움 바이트코드 디스어셈블러 기술 개발

- 현재의 일반적인 스마트 컨트랙트 분석시스템은 이더리움 바이트코드를 분석하기 위해 EVM assembly 코드로 변경하는 과정에서 전체의 코드를 순차적으로 읽어들이는 linear sweep 기법을 적용하고 있음.
- 스마트 컨트랙트의 바이트코드 레벨 변형을 통한 악의적인 행동 변화를 탐지하기 위해서는 스마트 컨트랙트의 control flow를 따라가는 디스어셈블러가 필요.
- 하지만 Stack을 이용하는 EVM의 특성상 jump (또는 branch)가 stack을 통해 간접적으로 대상 주소를 얻는 방식을 적용하고 있어 디스어셈블 과정에서 control flow의 정보를 얻기에 한계를 가지고 있음
- 본 과제에서는 EVM 바이트코드의 재귀적(recursive) 디스어셈블 기능을 구현하기 위하여 바이트코드 디코딩, abstract interpretation을 이용하는 중간언어 리프팅, (expression) propagation을 반복적으로 수행하는 구조를 설계하여 적용
- 개발된 기술을 통하여 기존의 linear sweap 기반의 기법들 보다 더 정확하고 악의적으로 변형된 스마트 컨트랙트의 바이트 코드에 대한 분석 기능 지원

○ 이더리움 어셈블리 대상의 중간언어 변환 기술 개발

- 본 과제에서 개발된 디스어셈블리의 결과를 바탕으로 EVM 어셈블리 코드를 중간언어로 변환하는 기술 개발
- 복합적인 의미를 가지는 명령어의 실제 실행 과정을 중간언어로 변환
- 중간언어에 스마트 컨트랙트 및 이더리움의 EVM의 동작을 위해서 필요한 암묵적인 정보들을 명시적으로 함께 표현하여 효과적 분석이 가능 (e.g., GAS 및 environment 정보 이용 코드를 자동 생성)
- 연구자 및 개발자에게 생소한 EVM 시스템의 execution model 및 환경을 잘 정의된 중간언어 문법으로 함께 포함
- Stack을 이용하여 연산자를 구성하는 특성을 고려하여, 사용하는 중간 정보를 구별하기 위한 Static Single Assignment (SSA) 방식 적용

<표 6> 중간언어의 생성과정

바이트코드	0x6001600201
디스어셈블리결과	PUSH(1) w/ [0x01] PUSH(1) w/ [0x02] ADD
중간언어	v_1 = 0x01 v_gas = v_gas + G_PUSH1 v_3 = 0x02 v_gas = v_gas + G_PUSH1 v_5 = v_3 + v_1 v_gas = v_gas + G_ADD

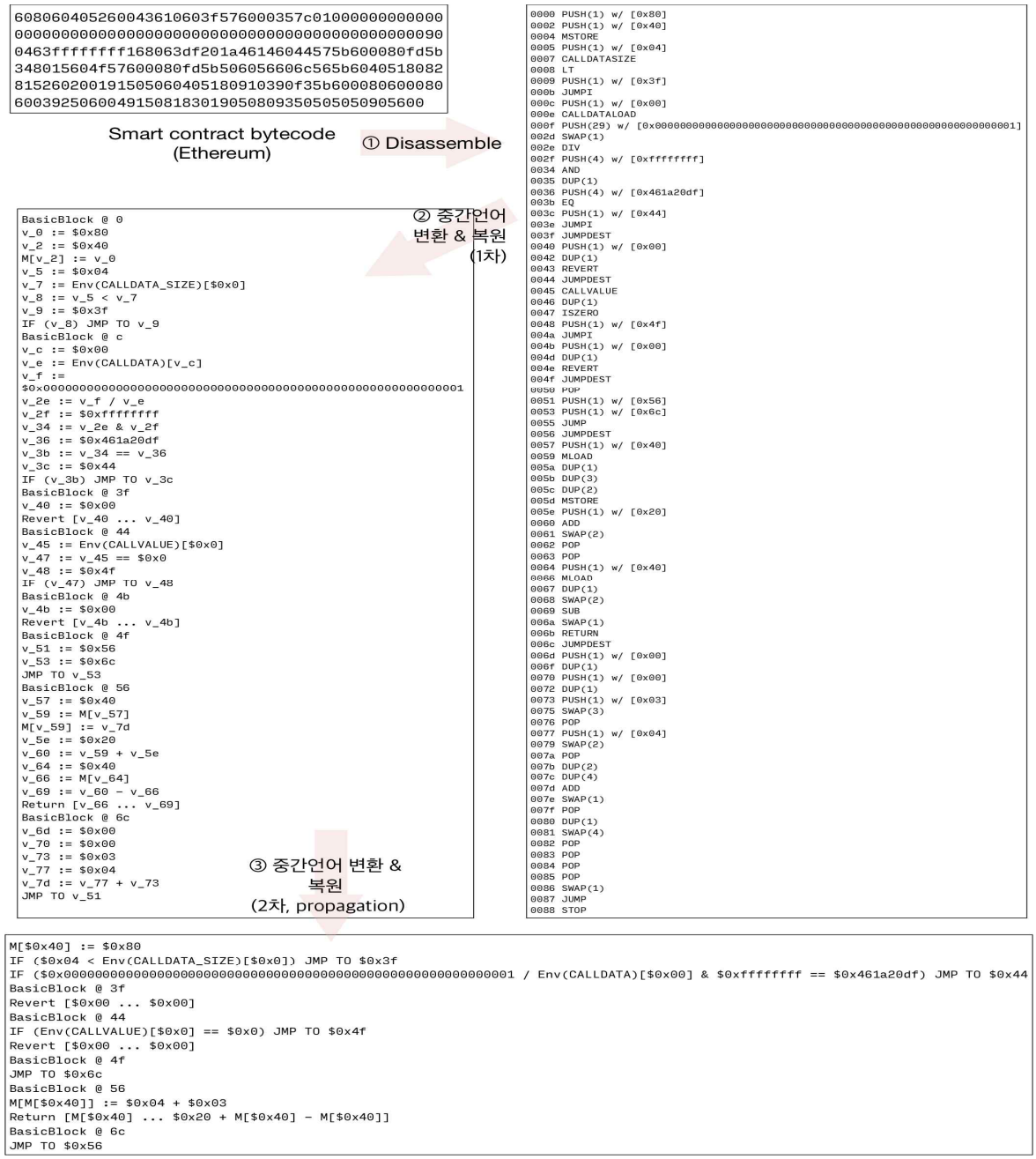
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ 분석 기술을 통합한 중간언어 변환 시스템 개발

- 개발된 분석 기술들을 통합하여 자동으로 이더리움 바이트코드를 분석이 용이한 중간언어로 변환하는 시스템을 [그림 27]과 같이 개발
- 재귀적 디스어셈블 과정과 중간언어 생성과정을 통합하여 이더리움 스마트 컨트랙트의 동작과정을 직접 분석 가능
- 생성된 중간언어 스마트 컨트랙트의 실행 과정에서 발생할 수 있는 Gas depletion 등의 문제들을 분석하기 위한 기반 기능 제공
- 이더리움 비잔티움 버전까지의 모든 바이트 코드 변환 기능을 지원
- 가독성 및 추가적인 역공학 과정을 고려한 2단계 중간언어 지원

  1단계: 함축적인 실행과정을 배재하고 모든 실행 과정을 중간언어에 표현

  2단계: 실행과정을 실제 유효한 액션단위로 종합하여 가독성 및 명료성을 지원
- 차후의 역공학 연구를 지원하기 위한 확장가능 구조

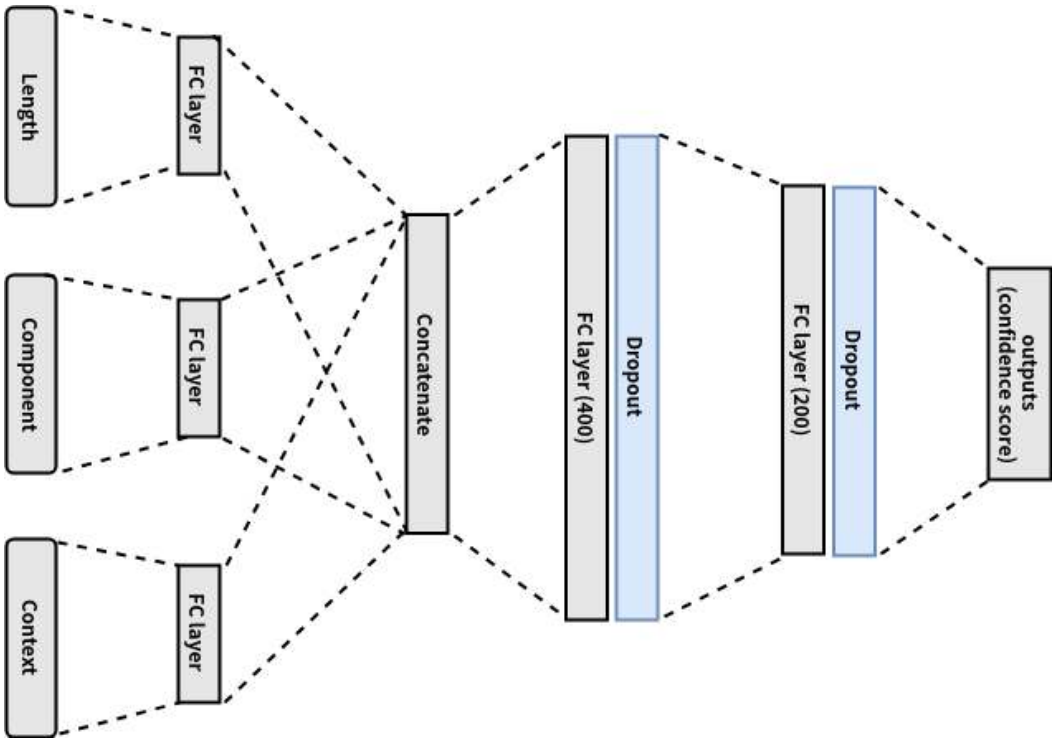


[그림 27] 개발된 중간언어 분석 시스템의 진행 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **스마트 컨트랙트 자동 속성 분석 기술 개발**

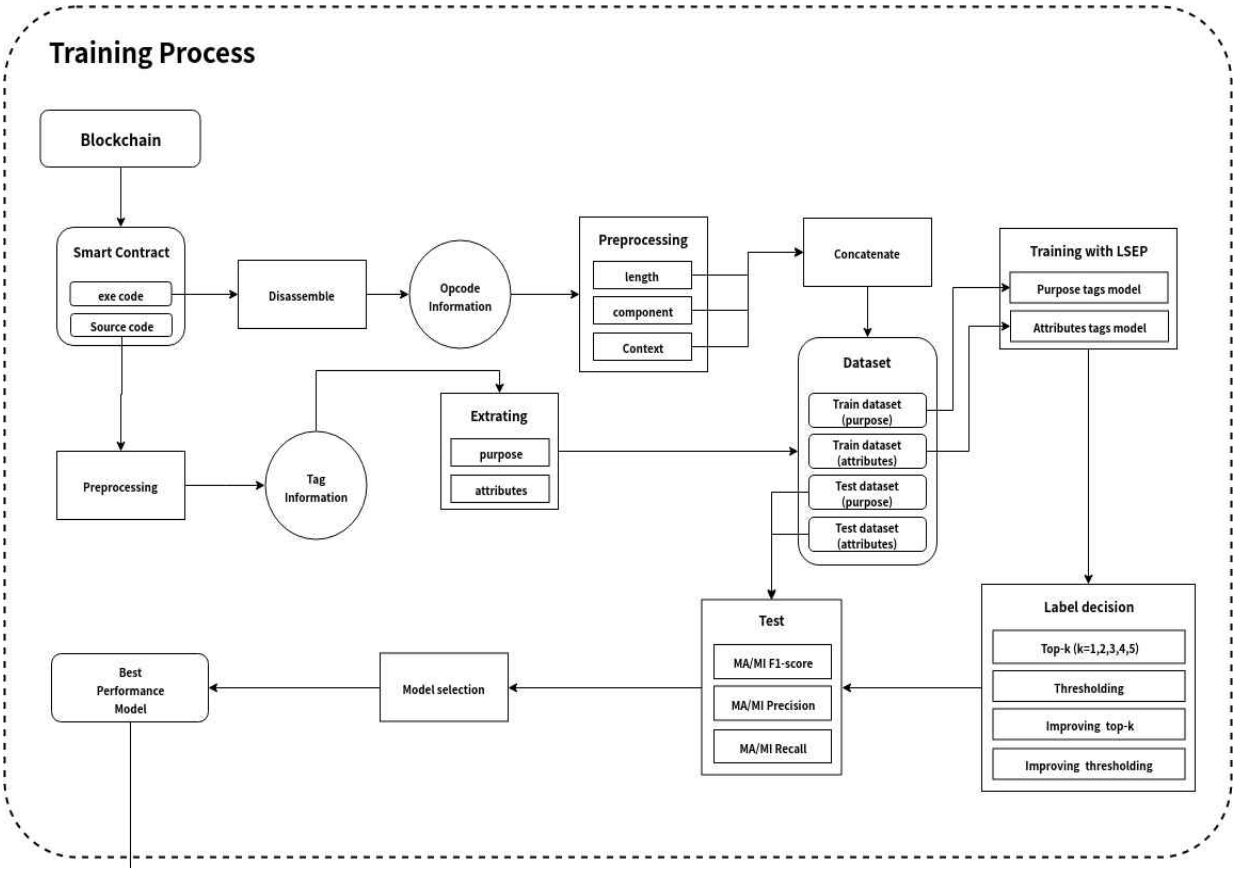
- 바이트 코드 상태의 스마트 컨트랙트를 효과적으로 분석하기 위한 자동 속성 분석 기술 개발
- 머신러닝 기법을 이용하여 기존의 스마트 컨트랙트 바이트코드와 소스코드를 학습하여, 주어진 바이트코드의 특성을 설명하는 키워드를 생성
- 생성된 tag는 해당 스마트 컨트랙트의 실제 행동에 대한 정보를 개략적으로 알려줌으로써, 스마트 컨트랙트의 취약점 및 특성 분석
- 학습된 시스템을 통하여 바이트코드 상태의 스마트 컨트랙트의 확장성 있는 분류 시스템으로도 활용 가능



[그림 28] 분석을 위해서 사용하는 특성(Feature)와 network 구조

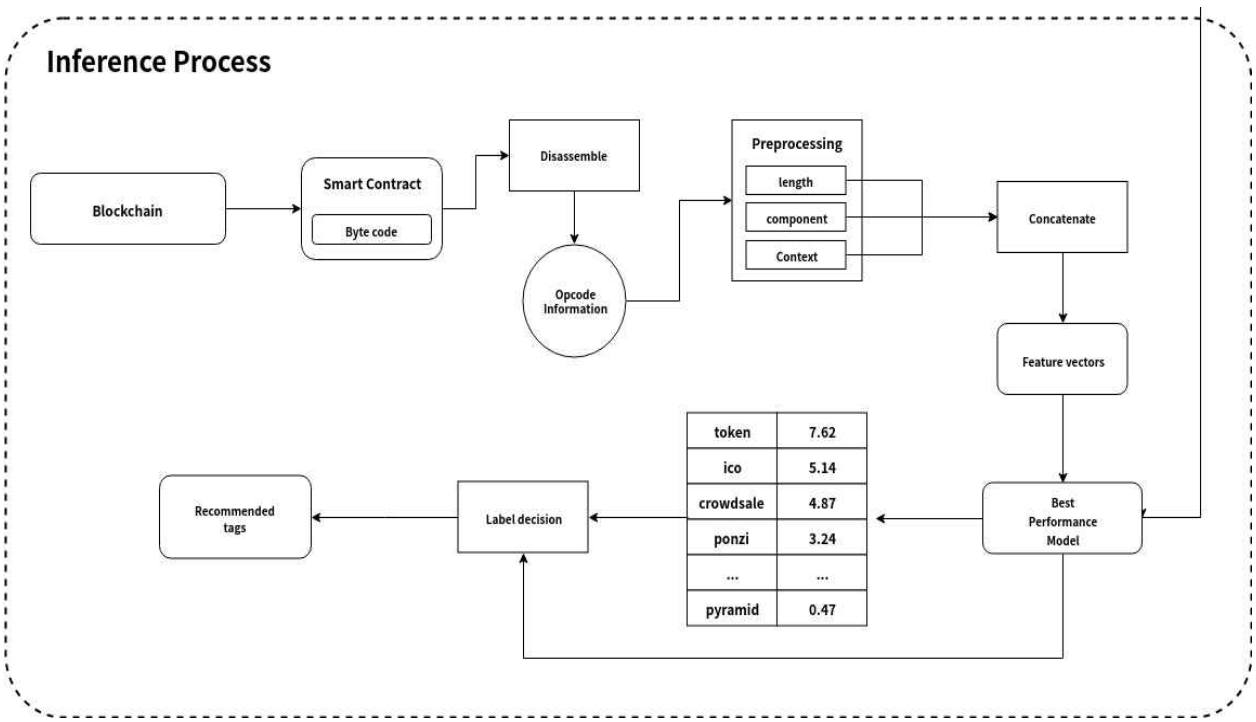
- [그림 28]과 같이 스마트 컨트랙트를 분석하기 위한 dataset을 구성하고, 구성된 dataset의 각 스마트 컨트랙트에서 스마트 컨트랙트의 길이, component, context 정보를 추출하여 학습 수행
- 스마트 컨트랙트의 바이트코드 정보를 체계적으로 추출하기 위하여 중간언어 분석 기법 적용
- 자동화 시스템으로의 확장을 위해 추천 시스템의 접근 방법 이용
- **(스마트 컨트랙트 정보 학습)** [그림 29]와 같이 스마트 컨트랙트의 바이트코드를 대상으로 중간언어로 분석한 코드 정보와 길이, 문맥 등을 추출
- 학습 과정은 스마트 컨트랙트의 소스코드와 바이트보드에 대하여 다음과 같은 분석을 수행
  1. 스마트 컨트랙트 소스코드 분석: 스마트 컨트랙트의 소스코드에 포함되어 있는 함수명과 내용을 대표할 수 있는 단어들을 추출. 유의미한 정보를 얻기 위하여 불필요한 특수문자를 제거하고 정규표현식을 통하여 단어 단위의 정보를 확인
  2. 스마트 컨트랙트 바이트코드 분석: 스마트 컨트랙트의 바이트코드에서 개발된 시스템을 통해 변환된 중간언어에서 핵심 실행 코드를 추출. 또한, 코드의 실행 과정에 따라 달라질 수 있는 흐름을 반영하기 위하여 문서 임베딩 기법을 도입하여 문맥 정보도 함께 생성하여 반영
- 수집한 스마트 컨트랙트의 태그 정보를 키워드 생성 시스템으로 발전시키기 위하여 1대 1 매칭이 아닌 1대다(one-to-many) 매칭을 지원하는 추천 시스템 알고리즘을 적용

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 29] 스마트 컨트랙트 dataset을 기반한 분석 및 학습 과정

- **(스마트 컨트랙트 키워드 생성)** [그림 30]과 같이 키워드 생성과정에서는 학습과정에서 다양한 알고리즘으로 구현된 모델 중에서 가장 높은 성능을 보이는 모델을 적용.
- 학습과 동일한 방법으로 스마트 컨트랙트의 바이트코드에서 정보를 추출하여 학습된 내용을 바탕으로, 각 대상 tag들에 대한 rating을 산출
- 산출된 정보를 바탕으로 각 스마트 컨트랙트에 합당한 키워드를 생성[그림 31]



[그림 30] 학습된 내용을 이용한 스마트 컨트랙트 키워드 생성 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

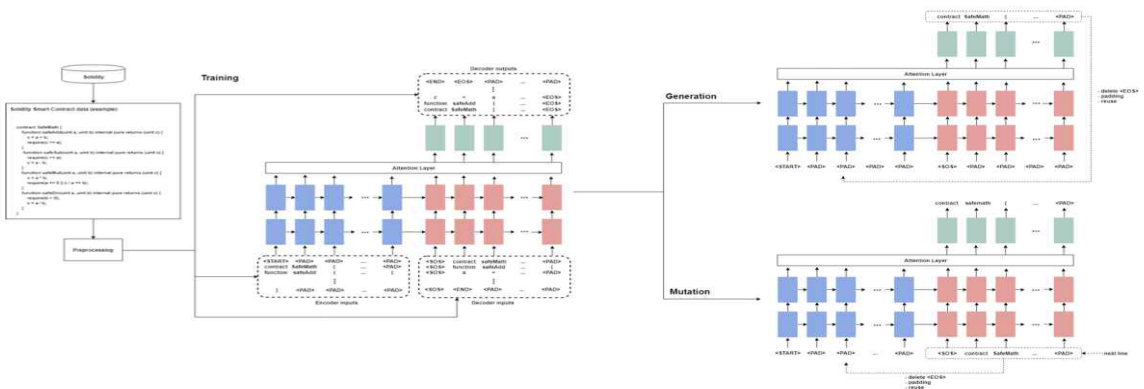
Title	Contract & Library	Function	Predictedtags
EtherCashPay	SafeMath Token StandardToken ReserveToken AccountLevels AccountLevelsTest **EtherCashPay**	EtherCashPay, changeAdmin, changeAccountLevelsAddr, changeFeeAccount, changeFeeMake, changeFeeTake, changeFeeRebate, deposit, withdraw, depositToken, withdrawToken, balanceOf, order, trade, tradeBalances, testTrade, availableVolume, amountFilled, cancelOrder	**Purpose(10):** token, payment **Purpose(15):** token, trading, payment **Purpose(20):** token, trading, payment **Attributes:** ownable, withdrawable, transferable, receivable, standard, address, balance, approve, safemath
JustCoin	**JustCoin**	JustCoin, balanceOf, withdraw, reinvestDividends sellMyTokens, getMeOutOfHere, fund, buyPrice sellPrice, dividends, withdrawOld, balance buy, kek, sell, reserve, getTokensForEther calculateDividendTokens, getEtherForTokens fixedLog, fixedExp, mul, div, sub, add	**Purpose(10):** token, crowdfunding, pricing **Purpose(15):** token, crowdfunding, pricing **Purpose(20):** token, crowdfunding, pricing, scam **Attributes:** ownable, transferable, mintable, standard, balance, safemath
AccessoryData	AccessControl SafeMath Enums IAccessoryData **AccessoryData**	createAccessorySeries, setAccessory addAccessoryIdMapping, transferAccessory ownerAccessoryTransfer, getAccessorySeries getAccessory, getOwnerAccessoryCount getAccessoryByIndex, getTotalAccessorySeries getTotalAccessories	**Purpose(10):** gambling **Purpose(15):** payment, gambling **Purpose(20):** token, gambling, data **Attributes:** withdrawable, transferable, standard, safemath

[그림 31] 스마트 컨트랙트 바이트코드에서 생성된 키워드의 예

□ 스마트 컨트랙트 중간언어 시스템의 검증 기술 개발

○ 스마트 컨트랙트의 코드 자동 생성을 통한 테스트 기술 개발

- 스마트 컨트랙트 관련된 시스템의 검증을 위해서는 제한적으로 존재하는 스마트 컨트랙트의 한계를 극복하기 위한 기법이 필요
- 기존의 스마트 컨트랙트를 기반으로 테스트를 위한 다양한 스마트 컨트랙트를 자동으로 생성하는 기술을 개발
- 머신 러닝의 일종인 sequence to sequence 모델을 기반하여 기존의 스마트 컨트랙트를 변형(mutation)하거나 새롭게 생성(generation)하는 기술을 개발
- [그림 32]와 같이 이더리움의 Solidity 언어를 기반으로 하여 sequence to sequence 모델을 학습 시키고, 학습된 모델을 기반으로 새로운 solidity 코드를 생성



[그림 32] 스마트 컨트랙트 시스템 및 중간언어 검증을 위한 테스트 코드 자동 생성 시스템의 실행 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<표 7> 스마트 컨트랙트 시스템 검증을 위해 자동 생성된 테스트 코드의 예

```
contract StandardToken is Token {
  function transfer ( address _to , uint256 _value ) returns ( bool success )
{
    if ( balances [ msg . sender ] >= _value && _value > <VALUE> ) {
      balances [ msg . sender ] -= _value ;
      balances [ _to ] += _value ;
      Transfer ( msg . sender , _to , _value ) ;
      return true ;
      } else {
        return false ;
        }
      }
...
```

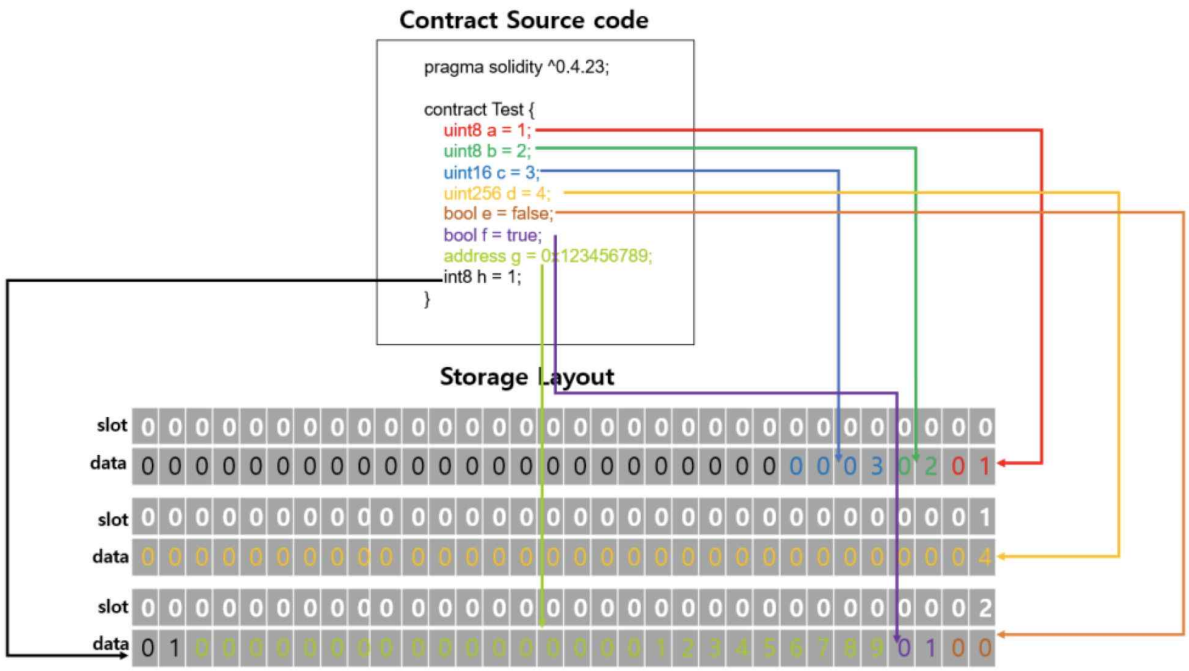
- RNN(Recurrent Neural Networks)와 같은 뉴럴 네트워크를 기반으로 구축된 인코더(encoder)와 디코더(decoder)로 구성되어 있으며, RNN외에 LSTM(Long-Short Term Memory) 또는 GRU(Gated Recurrent Unit)들도 적용 가능
- 생성된 solidity 코드를 <표 7>과 같이 후처리 과정을 거쳐 실제 바이트코드로 생성하여 중간언어 검증 시스템으로 적용

□ 취약점 분석 대상 파악을 위한 스마트 컨트랙트 개발 언어 분석

○ Solidity 언어 설계 취약점 분석

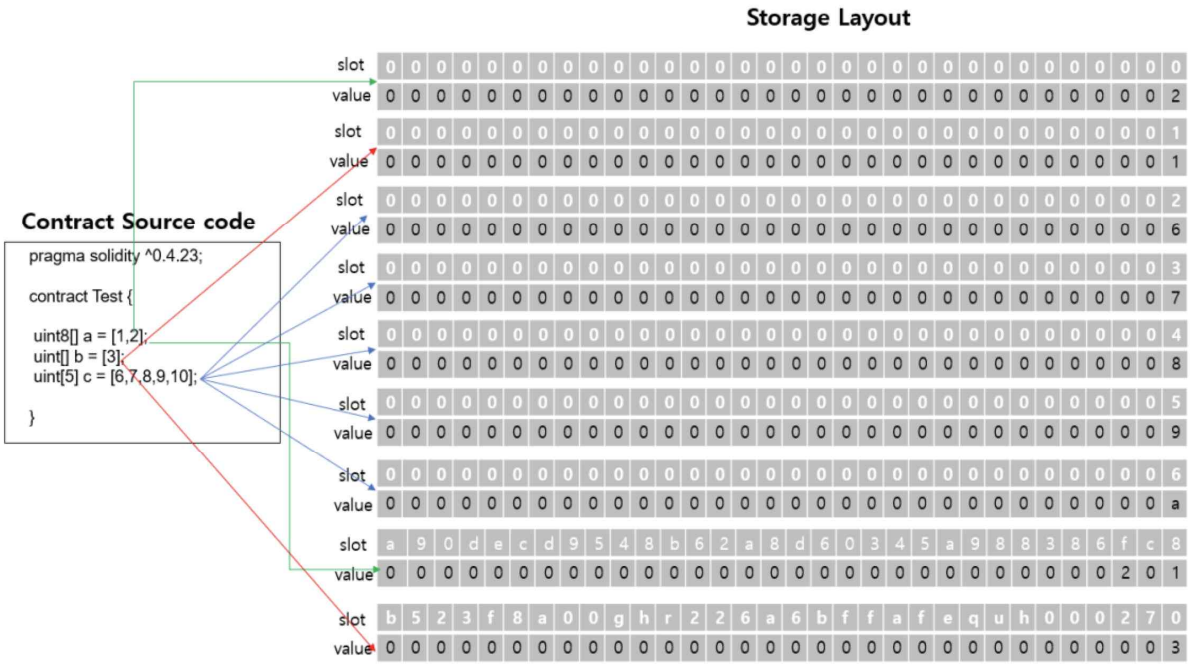
- 스마트 컨트랙트에서 데이터를 영구적으로 저장할 수 있는 공간은 스토리지가 유일함.
- 스토리지는 영구적으로 저장 가능한 유일한 공간으로 해커의 타겟이 될 수 있음.
- 스토리지 설계 상에 대한 보안 이슈 분석 필요.

○ Solidity 스토리지 설계 및 동작 방식 분석



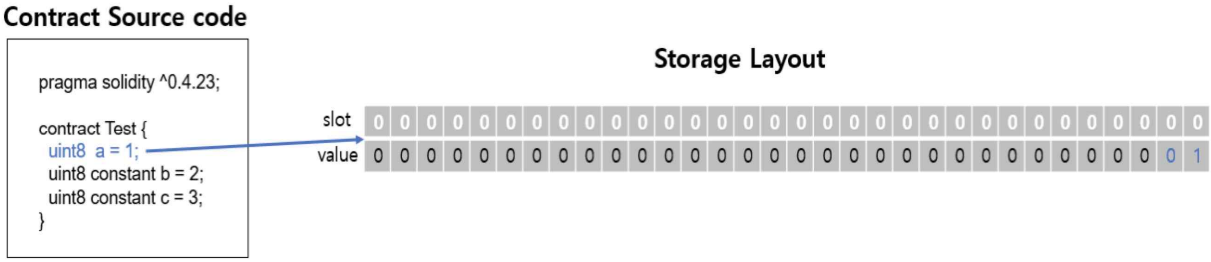
[그림 33] 기본형 타입의 스토리지 저장 결과

- 스토리지는 [그림 33]과 같이 slot과 data로 이루어진 map 자료구조로 생성
- 하나의 slot은 256 바이트의 크기를 저장할 수 있음
- 하나의 slot에 다수의 데이터가 packing되어 저장될 수 있는 구조
- 기본형의 타입에 대해서는 [그림 34]와 같이 스토리지 레이아웃이 생성됨
- 예를 들어 기본형인 uint 타입의 a, b 변수 및 uint16 타입의 c 변수는 총 32 바이트로 slot 0번에 packing되어 저장



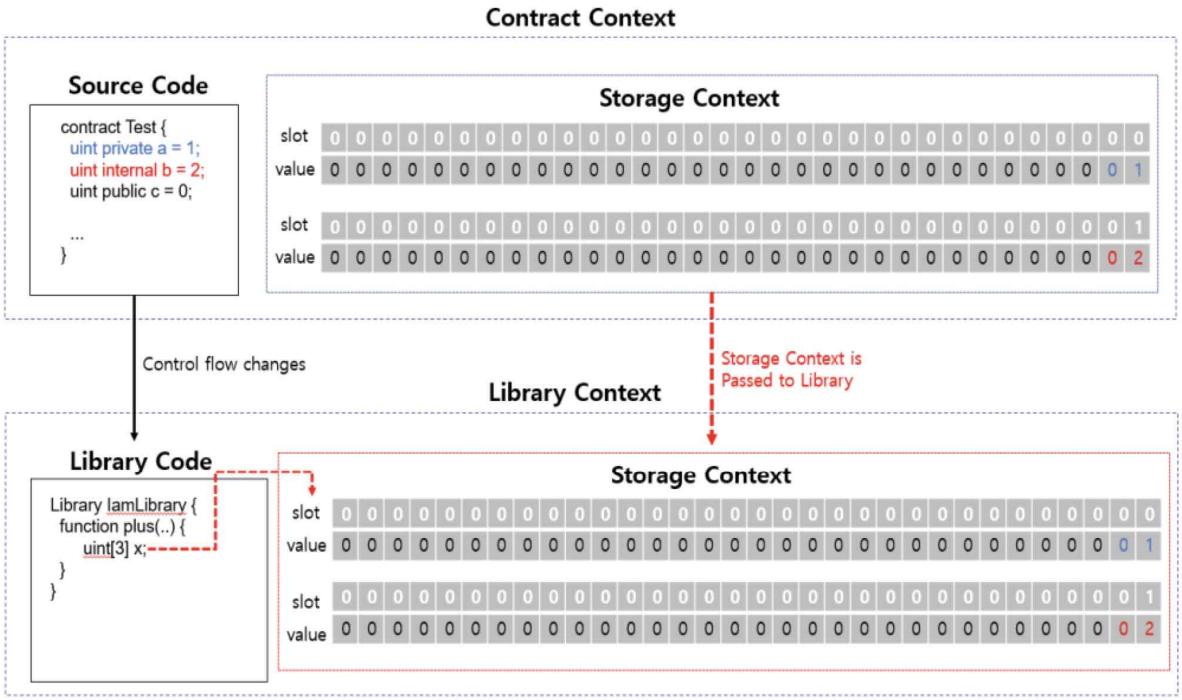
[그림 34] 참조형 타입의 스토리지 저장 결과

- 참조형의 타입에 대해서는 [그림 35]와 같이 스토리지에 저장됨
- 동적 배열의 경우, 하나의 slot 전체를 사용하여 배열의 길이를 저장함
- 정적 배열의 경우, 길이가 고정되어 동적 배열과 다르게 따로 길이를 스토리지에 저장하지 않음.
- 배열의 인덱스는 해쉬 함수를 통해 생성됨



[그림 35] constant 변수의 스토리지 저장 결과

- constant의 변수는 [그림 36]과 같이 스토리지에 저장되지 않음
- constant는 바이트코드의 코드섹션에 상수값으로 저장됨
- 컨트랙트가 라이브러리와 연동되는 경우, 라이브러리를 사용하는 컨트랙트의 스토리지 환경이 해당 라이브러리로 전달됨



[그림 36] 라이브러리를 사용하는 컨트랙트의 스토리지 저장 결과

- 호출한 컨트랙트의 스토리지 환경(Context)이 라이브러리로 넘어가기 때문에 악성 라이브러리의 경우 보안 이슈 발생 가능.

○ 스토리지 레이아웃 공격 분석

- 하나의 변수에서 오버플로우(overflow) 발생 시 스토리지 레이아웃 붕괴(corruption) 가능성 분석
- Solidity 컴파일러 0.4.3 이전 버전의 경우 변수 오버플로우로 인해 스토리지 레이아웃 붕괴 가능

```
uint8 a = 0;
uint16 b = 2;

function setPackData2() public{
    a--;
}
```

```
Before
key: 0x0000000000000000000000000000000000000000000000000000000000000000
value: 0x0200
a: 0 uint8, b: 2 uint16

After setPackData2
key: 0x0000000000000000000000000000000000000000000000000000000000000000
value: 0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
a: 255 uint8, b: 65535 uint16
```

[그림 37] 근접한 변수의 오버플로우로 인한 스토리지 값의 변화

- [그림 37]과 같이 setPackData2() 함수를 호출할 경우, uint8 타입의 a 변수에서 overflow가 발생하며, 스토리지에 저장되어 있는 b 변수의 값 조작 가능
- setPackData2() 호출 시, b 변수 값이 2에서 65535로 변경됨
- Solidity 컴파일러 0.4.3 이후 버전에서는 스토리지에 값을 저장하기 전에 PUSH1 ff 및 AND 연산을 통해 오버플로우 비트를 제거함
- [그림 38]은 새로운 버전과 이전 버전 Solidity 컴파일러의 차이를 보여줌

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
uint8 data1 =0xff;
Uint8 data2 = 0;

function setPackData() public {
   data1 = data + 1;
}
```

New version

```
087 PUSH1 01
089 PUSH1 00
091 DUP1
092 SWAP1
093 SLOAD
094 SWAP1
095 PUSH2 0100
098 EXP
099 SWAP1
100 DIV
101 PUSH1 ff
103 AND
104 ADD
105 PUSH1 00
107 DUP1
108 PUSH2 0100
111 EXP
112 DUP2
113 SLOAD
114 DUP2
115 PUSH1 ff
117 MUL
118 NOT
119 AND
120 SWAP1
121 DUP4
difference
122 PUSH1 ff
124 AND
Difference
125 MUL
126 OR
127 SWAP1
128 SSTORE
```

OLD Version

```
075 PUSH1 01
077 PUSH1 00
079 PUSH1 00
081 SWAP1
082 SLOAD
083 SWAP1
084 PUSH2 0100
087 EXP
088 SWAP1
089 DIV
090 PUSH1 ff
092 AND
093 ADD
094 PUSH1 00
096 PUSH1 00
098 PUSH2 0100
101 EXP
102 DUP2
103 SLOAD
104 DUP2
105 PUSH1 ff
107 MUL
108 NOT
109 AND
110 SWAP1
111 DUP4
112 MUL
113 OR
114 SWAP1
115 SSTORE
```

[그림 38] 새로운 버전의 컴파일러(왼쪽)와 이전 버전의 컴파일러(오른쪽)의 차이점

- **초기화되지 않은 스토리지 포인터의 Semantic을 활용한 공격 분석**
  - 초기화되지 않은 스토리지 포인터의 경우 디폴트로 0번째 스토리지를 가리킴.
  - [그림 39]와 같이 test 함수에서 a의 스토리지 포인터를 1로 변경할 경우, balances 스토리지 변수 값이 1로 변경

```
contract StoragePointer {

    uint balances = 0;

   function test(uint total) public {
       uint[3] a;
       a[0] = 1;
   }
}
```

[그림 39] 초기화 되지 않은 스토리지 포인터의 Semantic

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 초기화 되지 않은 스토리지 포인터를 활용하여 악성 라이브러리 개발 가능.
- 라이브러리의 경우, caller의 storage에 직접적으로 접근 불가능하지만, 초기화되지 않은 스토리지 포인터를 활용할 경우, caller의 모든 스토리지에 접근 가능.
- 악성 라이브러리에서 caller의 private, internal 변수에 접근 가능한 것을 확인.

○ **함수 Visibility Semantic에 의한 취약점 분석**

- Solidity는 함수 visibility에 public, private, internal, external이 올 수 있음.
- public visibility의 경우, 모든 사용자가 접근할 수 있는 함수로 생성됨.
- 함수의 visibility를 지정하지 않을 경우 디폴트로 public 함수가 됨.
- 이는 보안을 고려하지 않은 취약한 설계로, 개발자가 실수할 경우 보안 이슈 발생 가능.
- 개발자 실제로 실수하는지 확인 필요하며, 어떤 보안 문제가 발생할 수 있는지 분석 필요
- 함수 visibility로 인한 취약한 컨트렉트를 자동으로 탐지하는 분석기 개발.
- [그림 40]는 분석기에서 자동으로 탐지한 실제 컨트렉트의 취약한 함수를 표시함.

```
function transferFrom(address _from, address _to, uint256 _value)
    public returns (bool success)
{
    // sanity check
    require(_to != 0x0 && _from != 0x0);
    require(_from != _to && _to != address(this));

    // check for overflows
    // require(_value > 0 &&
    //   balances[_from] >= _value &&
    //   allowed[_from][_to] <= _value &&
    //   balances[_to] + _value < balances[_to]);

    // update public balance
    allowed[_from][_to] = allowed[_from][_to].sub(_value);
    balances[_from] = balances[_from].sub(_value);
    balances[_to] = balances[_to].add(_value);

    // emit transfer event
    Transfer(_from, _to, _value);
    return true;
}
```

[그림 40] 분석기로 탐지한 실제 컨트랙트의 취약한 함수

□ **Solidity 언어 분석기 개발**

○ Solidity Parser 개발

- Etherscan으로부터 수집한 22,649개의 스마트 컨트랙트를 모두 정상적으로 파싱 가능한 parser 개발

○ Solidity 체커 개발

- Operational semantics을 사용하여 초기화 되지 않은 스토리지 포인터의 동작을 정형화 하여 정리
- [그림 41]은 스토리지 포인터의 동작 방식을 정형화하여 정리한 내용을 나타냄.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

$$
\begin{array}{lll}
s ::= d\ (\texttt{=}\ e)^? \mid \texttt{var}\ x\ \texttt{=}\ e & & \text{statement} \\
d ::= \tau\ l^?\ x & & \text{typed variable} \\
e ::= x\ (\texttt{=}\ e)^? & & \text{expression} \\
\tau ::= \iota \mid \tau[n^?] \mid x(.x)^? & & \text{type} \\
l ::= \texttt{storage} \mid \texttt{memory} \mid \texttt{storptr} & & \text{location} \\
x & & \text{variable}
\end{array}
$$

μ uninitialized storage pointer
β boolean
ϵ empty list

$\boxed{\Sigma \vdash_{\Gamma,\mathcal{A}} s \Rightarrow \langle \bar{\mu}, \Sigma \rangle}$

$$\frac{\Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta), \Sigma' \rangle \qquad \beta \vdash_{\Gamma,\mathcal{A}} d \Rightarrow (l',\beta')}{\Sigma \vdash_{\Gamma,\mathcal{A}} \tau\ l^?\ x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, \Sigma'[x \mapsto (l',\beta')] \rangle}$$

$$\frac{\texttt{false} \vdash_{\Gamma,\mathcal{A}} d \Rightarrow (l,\beta)}{\Sigma \vdash_{\Gamma,\mathcal{A}} \tau\ l^?\ x \Rightarrow \langle \epsilon, \Sigma[x \mapsto (l,\beta)] \rangle}$$

$$\frac{\Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta), \Sigma' \rangle}{\Sigma \vdash_{\Gamma,\mathcal{A}} \texttt{var}\ x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, \Sigma'[x \mapsto (l,\beta)] \rangle}$$

$\boxed{\beta \vdash_{\Gamma,\mathcal{A}} d \Rightarrow (l,\beta)}$

$$\beta \vdash_{\Gamma,\mathcal{A}} \tau\ \texttt{storage}\ x \Rightarrow (\texttt{storptr}, \beta)$$

$$\beta \vdash_{\Gamma,\mathcal{A}} \tau\ \texttt{memory}\ x \Rightarrow (\texttt{memory}, \texttt{true})$$

$$\frac{getSL_{\Gamma,\mathcal{A}}(\tau) = \texttt{storptr}}{\beta \vdash_{\Gamma,\mathcal{A}} \tau\ x \Rightarrow (\texttt{storptr}, \beta)}$$

$$\frac{getSL_{\Gamma,\mathcal{A}}(\tau) = \texttt{memory}}{\beta \vdash_{\Gamma,\mathcal{A}} \tau\ x \Rightarrow (\texttt{memory}, \texttt{true})}$$

$\boxed{\Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta), \Sigma \rangle}$

$$\frac{x \in \mathrm{Dom}(\Sigma) \qquad \Sigma(x) = (\texttt{storptr}, \beta) \qquad \Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta'), \Sigma' \rangle \qquad l \neq \texttt{storptr}}{\Sigma \vdash_{\Gamma,\mathcal{A}} x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, (\texttt{storptr}, \texttt{true}), \Sigma'[x \mapsto (\texttt{storptr}, \texttt{true})] \rangle}$$

$$\frac{x \in \mathrm{Dom}(\Sigma) \qquad \Sigma(x) = (\texttt{storptr}, \beta) \qquad \Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (\texttt{storptr}, \beta'), \Sigma' \rangle}{\Sigma \vdash_{\Gamma,\mathcal{A}} x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, (\texttt{storptr}, \beta'), \Sigma'[x \mapsto (\texttt{storptr}, \beta')] \rangle}$$

$$\frac{x \in \mathrm{Dom}(\Sigma) \qquad \Sigma(x) = (\texttt{memory}, \texttt{true}) \qquad \Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta), \Sigma' \rangle}{\Sigma \vdash_{\Gamma,\mathcal{A}} x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, (\texttt{memory}, \texttt{true}), \Sigma' \rangle}$$

$$\frac{x \notin \mathrm{Dom}(\Sigma) \qquad \Gamma(\texttt{this}) = (\Sigma_{\mathrm{g}}, \bar{x}_{\mathrm{s}}) \qquad x \in \mathrm{Dom}(\Sigma_{\mathrm{g}}) \qquad \Sigma \vdash_{\Gamma,\mathcal{A}} e \Rightarrow \langle \bar{\mu}, (l,\beta), \Sigma' \rangle}{\Sigma \vdash_{\Gamma,\mathcal{A}} x\ \texttt{=}\ e \Rightarrow \langle \bar{\mu}, (\texttt{storage}, \texttt{true}), \Sigma' \rangle}$$

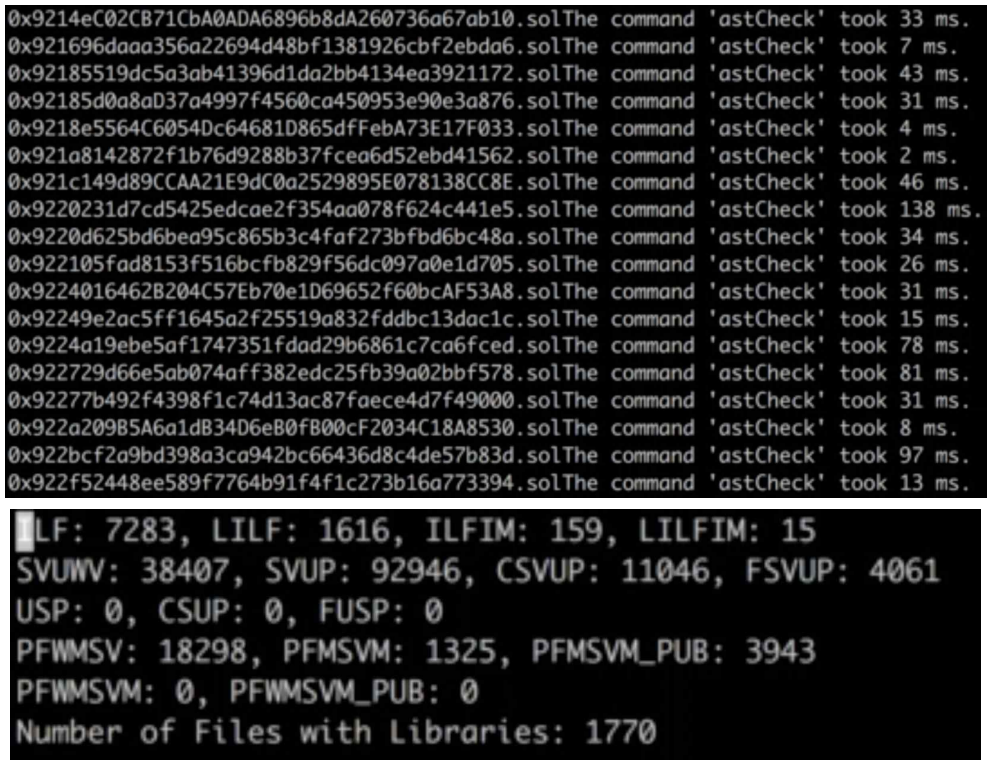
$$\frac{x \in \mathrm{Dom}(\Sigma) \qquad \Sigma(x) = (\texttt{storptr}, \texttt{false})}{\Sigma \vdash_{\Gamma,\mathcal{A}} x \Rightarrow \langle x, (\texttt{storptr}, \texttt{false}), \Sigma \rangle}$$

$$\frac{x \in \mathrm{Dom}(\Sigma) \qquad \Sigma(x) = (l,\beta) \qquad ((l = \texttt{storptr} \ \wedge\ \beta = \texttt{true}) \ \vee\ l \neq \texttt{storptr})}{\Sigma \vdash_{\Gamma,\mathcal{A}} x \Rightarrow \langle \epsilon, (l,\beta), \Sigma \rangle}$$

$$\frac{x \notin \mathrm{Dom}(\Sigma) \qquad \Gamma(\texttt{this}) = (\Sigma_{\mathrm{g}}, \bar{x}_{\mathrm{s}}) \qquad x \in \mathrm{Dom}(\Sigma_{\mathrm{g}})}{\Sigma \vdash_{\Gamma,\mathcal{A}} x \Rightarrow \langle \epsilon, (\texttt{storage}, \texttt{true}), \Sigma \rangle}$$

[그림 41] 정형화한 스토리지 포인터의 동작 방식

- 정형화된 수식을 기반으로 스토리지 포인터 취약점 탐지 가능한 체커 개발
- 정형화된 수식을 기반으로 함수 visibility 취약점 탐지 가능한 체커 개발
- [그림 42]는 parser 및 체커를 실행하는 화면과 결과 내용을 나타냄



[그림 42] 정형화한 스토리지 포인터의 동작 방식

○ 취약한 스마트 컨트랙트 수집

- Etherscan은 이더리움 블록체인에 배포된 스마트 컨트랙트를 확인할 수 있는 Browsing 기능이 존재함
- Etherscan으로부터 Solidity 언어로 작성된 스마트 컨트랙트를 수집하는 크롤러 개발
- 총 22,647개의 컨트랙트를 수집함
- 수집한 컨트랙트를 대상으로 자동화 도구 및 매뉴얼 분석을 통해 취약한 컨트랙트 분류
- [그림 43]은 Etherscan으로부터 수집한 22,647 컨트랙트를 나타냄


0x3f2825959643490226af75d88a2983e86eccc51f.sol
0x3f2cc0a71c1d6362c8bcd4853296558fab354fccb.sol
0x3f2ef511aa6e75231e4deafc7a3d2ecab3741de2.sol
0x3f30b7eedb2bedf4a8600b7ee5bb2b3162ef13d8.sol
0x3f3195776dcf995bdd7efd702d07d5818d8a24ba6.sol
0x3f31e357e24ad90d6ef7e71bfe8499f504d8e054.sol
0x3f35f8a62b67da6464e25895bff896004cbcc7a8.sol
0x3f3a3b42e4564b20948307de9bb5b66398c325fa.sol
0x3f407351f74b1340c3f507dc334dfc8cfb2b7e97.sol
0x3f418d108772412095c2959767ab7282971df516.sol
0x3f44759ffda157f32f04b31cdfcd93fef4a74722.sol
0x3f44b083973d5cDdAE571d61921d8e8f0248df43.sol
0x3f44bcc032276782914207082f78b59faea772b6.sol
0x3f45d2d5feb6b4b800d2d3b84442eeddf54a735a.sol
0x3f4b726668da46f5e0e75aa5d478acec9f38210f.sol
0x3f4dd010fbbc9a9b6d95f1f53837d7e9f3befac8.sol
0x3f53c5193d46d0101c43ce560ad1426c797ee462.sol
0x3f580f24a468f9048d41f80fb491b1a072033ec3.sol
0x3f593a15eb60672687c32492b62ed3e10e149ec6.sol
0x3f5b4887bf8d9ef0d4eb5d84ca1335583e6bfc9a.sol
0x3f5d00617ffb9947f7cf1fadee2cc93eab526.sol
0x3f5d8e4b7a68f30735de6349c37cee1fec9f383d.sol
0x3f5e60c4e294ab62ea706f3858adf42454b0d3ba.sol
0x3f60651308846a417362377910a2a8c5caede024.sol
0x3f60a98202cf7ac8be9afc077e6c28f2638009f4.sol
0x3f61bb128b60b6d063fd/63b880da4a8ed4ceeee.sol
0x3f666500c624d423063ee3a15df6f49cd5098d927.sol
0x3f6e75dea1a997bf6bb59334c425cf7f0803f3c7.sol
0x3f73ea71ebaa1f2622ae191eeef2184e10c1ce05.sol
0x3f75b099c7344D1A8fA8279e03e10d0E873C5FB.sol
0x3f78c83f85fe41621e3cf81c38ccf917e3852a0e.sol
0x3f796139c44bb79ea4744a1ca43b9ae91fdf62eb.sol
0x3f7e197c9d3a2a3e41ec64d1994bbcec3f31b574.sol
0x3f81a0f47133247da4777b6a004b422e38bd48d0.sol
0x3f864c77e4361baaf676ace3830228aed0f85bf.sol
0x3f874330cbd26f5122fa9f63f8d3d44fe6c776de.sol
0x3f8f74033e9aab9f2e8b782fdaca886b17b61f5a.sol
0x3f8fc6758e0ad014ee426565a6383969b22d9e37.sol
0x3f9093d041e8c9e4ea58e4379a9220b10329dddf.sol
0x3f953f2a343df5d48e22d216f9a23dbb954fc208.sol
0x3f96c9e80b8245bc1a96a3b7d2297f5bdf4965.sol
0x3f98c788830d173295352744 6beba94a72b7d34.sol
0x3f9ad22a9c2a52bda2a0811d1080fc9cd23c6c46.sol
0x3f9b6264f60cd50b456ed69475862db1c8c62c38.sol
0x3f9b89dc8190f8fac46def0bdd5eacf826c50408.sol
0x3f9c0a5773817ffaa5a1061e1e33c1d9cB888dff.sol
0x3f9ef884433bdcbe10e6477e58be727bc4e6731a.sol
0x3fa067c75d5161a787ca543cb867f2042bce4f82.sol
0x3fa9f26b2c0ebc692503568938f34784aa9c7bd9.sol
0x3fab1c4216e401de80cef4ac5387452d216314c6e.sol
0x3fb0ed756276d4c97b226ce8daef23cc8058653e.sol
0x3fbf67bab8461d59e49b1568a637596Bf3c3a9a6.sol
0x3fc39666ba6e199514ce2a66abf45b8ab4a83599.sol
0x3fc607a93c296f1d4ea511b1e6b455f4f3190999.sol
0x3fc8b73ed26c59605671a5620492014a45fcc046.sol
0x3fca60965bb0035d38b95cb528ef231ed8905911.sol
0x3fccb426c33b1ae0671153905 34b968592368d05.sol
0x3fcee23add6e86dde3c4d395cbce1cae7f16d06d.sol
0x3fcf78f7646f85ceb9344c5ba5a05a0e31a462af.sol
0x3fd5219b7280291dd83beb8baff96d15f22aab49.sol
0x3fe2cf5f86883d4b6b5adb4c7c4d3ef714fad5a9.sol
0x3fe4f5286883d406Bb6adb4c7c4d3ef714fad5a9.sol
0x3fe5f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fe6f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fe7f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fe8f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fe9f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fefcccdcaa140b28a2ae9331f0b2805ce0831a9.sol
0x3ff0a0f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff1f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff2f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff3f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff4f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff5f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff6f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff7f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff8f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ff9f5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ffaf5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ffbf5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ffcf5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ffdf5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3ffef5286883d406Bb6adbk4c7c4d3ef714fad5a9.sol
0x3fff3212bdda5e2a80d401559965419e4643b96a8.sol
0x3fff3216f86a723f2c23b03b5cd1f622eb1a204159.sol
0x3fff3519eeeea3e76f1f699ccce5e23ee0bdda41ac.sol
0x3fff3b832855a9193b224ce0522da53ca8aecad813.sol
0x3fff4385c58cc0aa2adb56e05b50fc9f7e7b213511.sol
0x3fff5b4d0ec8e7207339b83d6ac17e075db3b85d8a.sol
0x3fff6a5b33d65b0d9b575150e09df1d4e40a4f8c.sol
0x3fff6a7c57def8667afadb4e85b64a928047e1c55c.sol
0x3fff6e7b9e6a026bfcd541c65fedbe355d3b203641.sol
0x3fff7e5cc3d9cf61b6553340672a4e8cbab0b7483d.sol
0x3fff7f5cc3d9cf61b6553340672a4e8cbab0b7483d.sol
0x3fff8e4c69edf1b340fcad64fbd6b37a7b9e265.sol
0x3fff8f5f93458f6793.sol
0x3fffa4a5a685efc53f445bf50f3dab45ded1b028134.sol
0x3fffa6f1c1d9fe3a6a345a86f7dcfa5bf71adc13d.sol
0x3fffa8190bc259e8ded544d935780aa4709ec3e64.sol
0x3fffac2f230dbea513bb42d60ecffca5d44db1b8e.sol
0x3fffad1cb392e36bdf5004450ab76fab23a5337c05.sol
0x3fffb4a5a685efc53f445bf50f3dab45ded1b028134.sol
0x3fffb8393bc6c430741b3c5ad483391a5e21220bb.sol
0x3fffd3d3b591e792eb3d9373227b4b786db37ba2087.sol
0x3fffe2dc587badbd10b4fe17f0f5f2934588f6793.sol
0x3fffeed655f36233fd58fdda74720b4eb85fddec.sol
0x3fffeee1

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ EVM 바이트 코드의 LLVM 변환기 개발

○ 재귀적 탐색을 통한 EVM 역어셈블러 개발

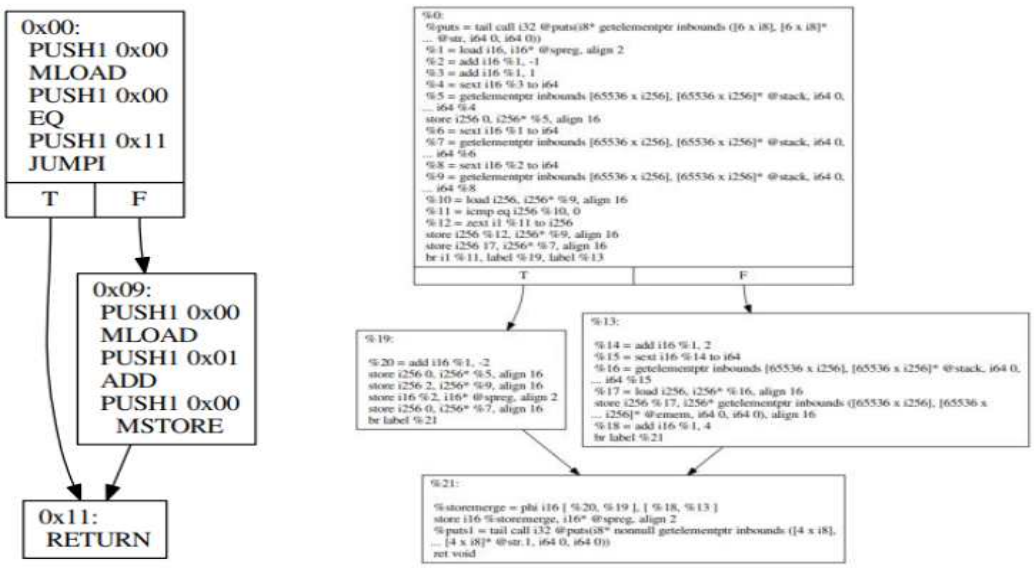
- 블록체인에 배포되어 있는 컨트랙트는 EVM 바이트코드로 기록되어 있음
- EVM 바이트코드에서 이더리움 어셀블리 언어로 변환하는 작업 필요
- 재귀적 탐색을 통한 역어셈블 개발하며, 주소 0에 있는 바이트 값을 어셈블리 언어로 변환한 뒤 이후에 탐색해야 할 주소를 큐에 추가하여 재귀적으로 변환 작업 진행
- 스택에 존재하는 JUMP 위치를 추출하여 변환작업 수행함
- 수집한 컨트랙트 대부분의 경우 역어셈블 가능한 것을 확인

○ 주소 의존성 명령어 제거

- 효과적인 제어 흐름 그래프 생성을 위해 주소 의존성 명령어를 제거함
- EVM의 PC 명령어는 상수로 치환하며, JUMP 및 JUMPI와 같은 명령어는 제어 흐름 그래프의 간선을 추가하는 것으로 대체

○ 기본 블록 및 제어 흐름 그래프 생성

- 제어 흐름 그래프에서 모든 명령어는 자신이 실행된 이후의 명령어를 가리키는 간선을 가지며, 이후의 명령어와 합쳐질수 있는 경우 간선을 제거한 뒤 같은 블록으로 병합 진행
- 위의 과정을 반복하여 모든 기본 블록을 생성



[그림 44] EVM 바이트코드의 제어 흐름 그래프(왼쪽), LLVM 중간 언어 그래프(오른쪽)

○ LLVM 변환기 개발

- 제어 흐름 그래프를 기반으로 LLVM 중간언어 프로그램 생성
- 함수 프롤로그/에필로그 생성, 기본 블록 단위의 프롤로그/에필로그 생성, 명령어 단위 변환 단계로 수행
- 함수 프롤로그/에필로그 생성 단계에서는 LLVM 내의 main 함수에서 스택, 메모리, 스토리지 초기화 진행
- 기본 블록 프롤로그/에필로그 생성 단계에서는 블록과 연결된 간선이 1개인 경우 조건 없는 br을 추가하여 다음 블록 라벨로 점프하고, 간선이 2개인 경우 조건 변수를 사용하여 점프 명령어를 추가
- EVM 명령어 단위 변환 단계에서는 EVM 명령어를 LLVM IR로 변경함
- [그림 44]의 왼쪽은 EVM 바이트코드에서 추출된 제어 흐름 그래프를 나타내며, 오른쪽은 이를 LLVM 중간 언어 그래프로 변환한 것을 나타냄
- EVM 바이트코드를 LLVM으로 변환하면서 LLVM 언어에서 동작하는 요약 실행 기술을 활용 가능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-2. 2차년도

1) 스마트 컨트랙트 프라이버시 보호 기술 연구

□ 특수한 산술회로를 위한 영지식 증명을 위한 SNARK 알고리즘 구현 및 실험

○ 1차년도의 결과에서 사용하는 개별 연산별 연산의 검증기법 대신 2009년 Ishai 등에 의하여 제안된 MPC-in-the-head 기법을 연산검증 기법에 적용을 적용하여 Giacomelli 등이 Usenix 2016에서 제안한 ZKBoo 기법 [GMO16]의 증명자와 검증자의 연산 성능의 개선하여 구현

- 해시함수나 대칭키 암호기법과 같이 추가의 복잡한 암호학적 가정을 요구하지 않는 암호기법만 적용하여 양자안전성을 보장하는 SNARK의 설계
- Σ-Protocol이 핵심 도구로서 FS 변환기법을 적용하여 설계된 Σ-Protocol을 Non-interactive하게 변환하여 SNARK

○ ZKBoo에서 제안하는 기법에서 생성되는 증명의 크기를 80-bit 안전성을 기준으로 48% 개선

- Common.h: 다른 함수나 Class에서 사용하는 Data Structure와 Literal 정의
- OsxAdapter.cc: Thread 실행시 특정 Core에 Bind할 수 있도록 Mac OS X에 지정하기 위한 코드
- BigInteger.cc/BigInteger.h: 유한체 연산을 지원하기 위한 것으로 고속 연산을 지원하기 위해 특정 Field를 고정하여 사용. 특히 Field를 소수체와 이진체로 분리하여 특정 Prime이나 이진체의 Reduction Polynomial의 Degree를 미리 지정하고 개별 Prime이나 Polynomial의 연산을 최적화하여 사용.
- CircuitHandler.cc/CircuitHandler.h: 임의의 연산 f에 대한 검증연산을 지원하기 위하여 입력으로 주어진 연산 f의 Circuit을 만들기 위한 Class와 이것의 구현이다. 2차년도에 구현한 것은 MiMC에 제시된 연산 f를 대상으로 하였고, 확장 가능하게 구현하였다.
- ZBKooContainer.cc/ZKBooContainer.h: ZKBoo를 정의하는 Class와 이것의 구현을 포함한다. 앞에서 제시한 성능개선 방법을 적용하여 구현함.

□ 영지식 증명의 안전성 분석

○ ZKBoo 기반으로 구현한 SNARK 알고리즘의 완전성 (Completeness), 건전성 (Soundness)와 영지식성 (Zero-knowledgeness) 증명

○ 암호학적 가정에 의존하는 다른 SNARK 기법과 달리 대칭키 암호기법만을 적용하여 Quantum Random Oracle에서 Post-quantum 안전성 달성

- 구현된 기법의 안전성은 ZKBoo의 안전성만 확인하면 되므로 ZKBoo 기법의 안전성

□ 영지식 증명 적용을 위한 스마트 컨트랙트 생성 프로그램 Zoker

○ 필요성 (블록체인의 프라이버시)

- 블록체인은 모든 정보를 공개하기 때문에 개인정보를 다루는 작업을 블록체인에서 작업하기 어렵고 블록체인에 올라간 정보는 쉽게 삭제하거나 수정할 수 없기 때문에 민감한 데이터를 취급하는 것은 더욱 위험함
- 때문에 블록체인을 이용하는 서비스에서는 프라이버시를 지키기 위해서 블록체인에 정보를 올리지 않고 작업을 처리하는 off-chain 작업 처리를 하거나 암호화하여 저장하는 방법, 영지식 증명의 방법이 사용될 수 있음
- off-chain 연산은 블록체인에 올리지 않고 작업을 처리하기 때문에 블록체인의 scalability를 향상시킬 수 있고 또한 정보를 올리지 않기 때문에 privacy를 개선할 수도 있지만 블록체인에 올리지 않는 off-chain 연산은 블록체인에서 실행하는 on-chain 연

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

산에 비해 분산화된 상황에서 신뢰가 필요 없이 올바른 실행을 보장하는 스마트 컨트랙트에 비해 신뢰성이 떨어짐

- 또한 암호화해서 저장하면 on-chain에서 값을 계산하기 불가능하기 때문에 암호화 방식 중 동형 암호화 방식이 사용될 수 있지만 수학적 연산이 아닌 경우에는 적용하기 어려움

○ 영지식 증명

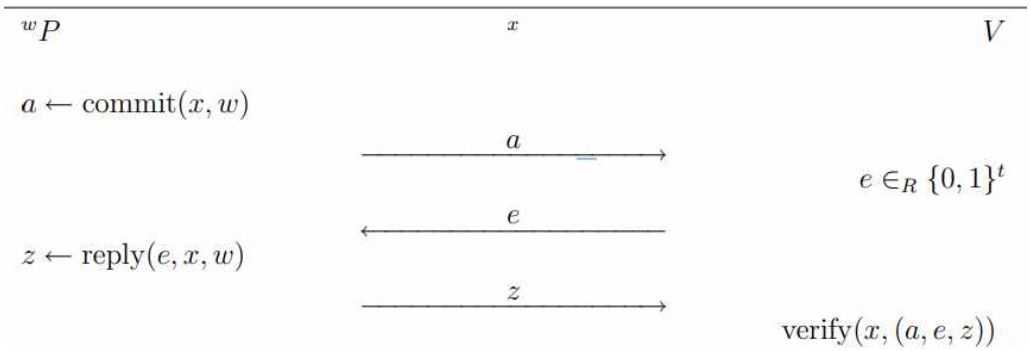
- 영지식 증명은 블록체인에서 개인정보 보호를 위해 사용하는 암호학적 방법으로 자신의 비밀 값을 숨기고 비밀 값을 통해 만들어낸 증명 값을 제출하여 자신의 작업이 옳았음을 증명하는 방법임
- 하지만 단점으로는 속도가 느리고 작업이 크기 때문에 블록체인의 scalability를 감소시킬 수 있고, 블록체인은 통신을 여러 회 진행하기 어렵기 때문에 Interactive Zero knowledge를 사용하기도 어려움

○ Interactive Zero knowledge

- Interactive Zero knowledge는 서로 통신을 하며 비밀 값을 알려주지 않고 결과를 확인하는 방법인데, 값을 드러내지 않고 증명하고자 하는 증명자는 자신의 값을 미리 고정시켜 둔 후 검증하려는 검증자가 random한 값을 통해 challenge를 시도하고 일정 회수 이상 challenge를 통과하면 값을 확인하지 않더라도 올바르게 계산하였음을 검증할 수 있음

○ Sigma Protocol

- Sigma protocol은 영지식 증명을 설계할 때 어떤 흐름으로 값이 전달되는 지를 증명하는 방법으로 [그림 45]에서 볼 수 있듯이 Commitment, Challenge, Response의 세 단계로 이루어짐
  - Commitment는 증명자가 자신의 값을 고정시켜 이후에 response를 할 때에 부정하거나 값을 변조하지 못하게 하기 위한 단계이다. 이 단계에서 증명하고자 하는 값을 Hash나 지수 연산과 같은 방법을 이용해 암호화해 저장함
  - Challenge 단계는 검증자가 증명자의 비밀 값을 모르는 상태로 증명자가 옳은 연산을 했는지 확인하기 위해 랜덤한 값을 증명자에게 전송해 그 값을 통해 연산을 하게 만듬



[그림 45] Basic Sigma Protocol

  - Response 단계에서는 증명자의 초기 Commitment 값과 Challenge의 random값을 통해 영지식 증명의 증명 값을 만들어 검증자가 검증할 수 있도록 만듬
- Sigma protocol에서 이 세 단계를 만족하면 영지식 증명을 만족함을 확인

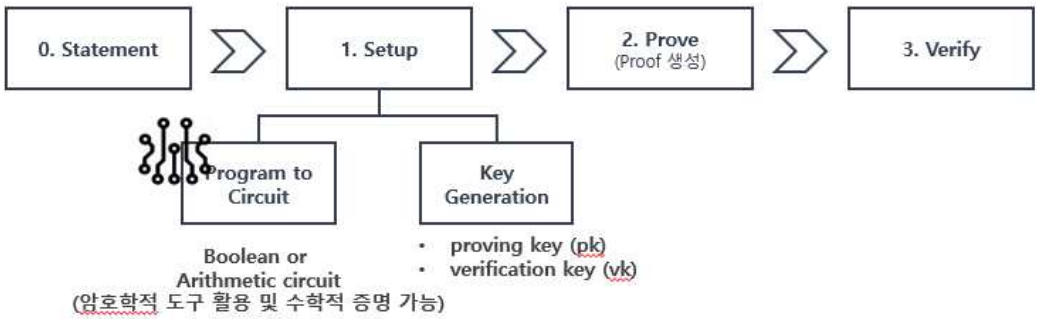
○ Non-Interactive Zero knowledge

- 블록체인에서는 통신을 통해 계속 값을 주고받기 어려우므로 Interactive Zero-knowledge를 사용하기 어렵기 때문에 Interactive zero knowledge에 비해 안전성이 조금 떨어질 수 있지만 통신이 필요 없는 Non-Interactive Zero knowledge가 사용됨
- Non-Interactive Zero knowledge proofs에서는 Prover에게 Challenge를 할 수 없지만 Prover가 임의의 값을 선택할 수 없으면 Challenge의 역할을 할 수 있으므로 Hash나 지수 연산과 같이 역으로 계

산하기 어려운 값을 통해 challenge의 랜덤 값을 대신하여 Sigma protocol의 단계를 Commitment와 Response만의 증명자의 작업만으로 해결할 수 있게 만듬

○ Zk-SNARK

- Zk-SNARK은 Zero knowledge Succinct Non Interactive ARgument Knowledge의 줄임말로 Non Interactive Zero knowledge의 방법 중 proof의 크기와 검증 비용이 O(1)으로 비용이 작은 영지식 증명의 Implementation임



[그림 46] zk-SNARK process

- zk-SNARK은 [그림 46]에서 보이는 것처럼 Computation(statement)에 대해 Circuit으로 만드는 circuit 정의, R1CS로 변환 단계가 있고 proof의 크기나 검증 시간을 줄이기 위한 QAP, ECC단계를 거쳐 Proof을 생성하는 순서로 이루어짐
- 연산은 어떤 값을 숨기는 지 정의하여 Circuit으로 만들어 낸다. Circuit을 input과 output을 정의하는 R1CS라는 Rank 1 Constraint System을 통해 연산 회로로 변경하지만 R1CS는 계산에서의 복잡도가 높고 검증하기도 복잡하므로 이 상태로 사용하기 어려움
- 때문에 QAP이라는 Quadratic Arithmetic Program이라는 프로그램을 이용해 문제를 다항식의 문제로 변형시켜 검증을 더욱 간결하게 만들어내므로 QAP을 이용하면 이전의 계산문제보다 복잡도가 줄어들기는 하지만 O(1)으로 줄일 수는 없음
- 이를 해결하기 위해 EC(Elliptic Curve)를 이용해서 다항식의 한 점을 고르고 그 한 점에서의 값이 같은 것을 증명하여 성능을 개선할 수 있지만 이 때, 한 지점을 고르는 부분에서 성능을 개선할 수 있고 보안상 특정 지점을 알고 있다면 fake proof을 생성할 수 있기 때문에 위험하기 때문에 zk-SNARK에서는 Trusted third party의 Trusted setup이 필요

○ Trusted Setup

- Trusted setup은 신뢰할 수 없는 환경을 가정하는 블록체인 환경에서 신뢰가 필요한 제3자가 필요하므로 탈중앙화에 악영향을 줄 수 있으며 Setup을 진행한 사람은 Fake Proof을 만들 수 있기 때문에 Trusted setup이 필요한 시스템에서는 신뢰성이 떨어질 수 있음
- 이를 막기 위해 MPC(Multi-party Computation)을 이용하여 랜덤한 특정 지점을 선택하여 Fake proof을 생성하지 못하게 만들 수 있지만 MPC를 이용하는 것은 단 한번만 일어나긴 하지만 비용이 크기 때문에 비용을 높이게 되는 문제가 생길 수 있음

○ Zk-STARK, Bulletproofs

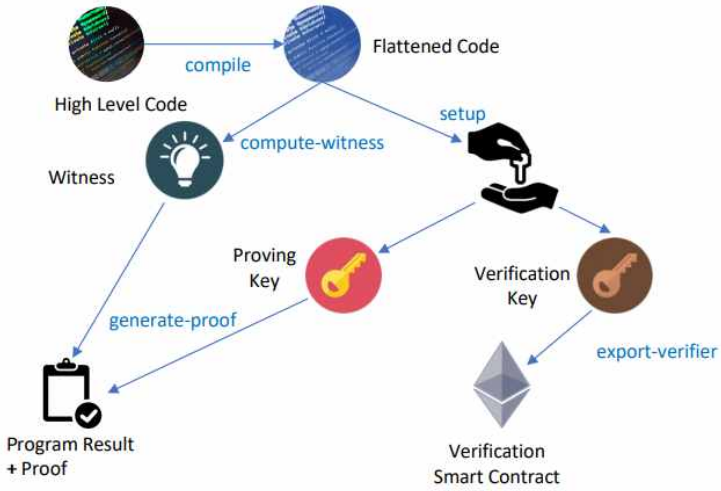
- 영지식 증명의 Implementation은 zk-SNARKs 많이 있는 것이 아닌 zk-STARK(Zero knowledge Scalable Transparent ARguments of Knowledge), Bulletproofs와 같은 영지식 증명이 사용될 수 있음
  - zk-STARK는 proofs의 크기와 증명 시간은 크지만 검증 시간은 짧은 implementation임
- ZK-STARK는 증명 값을 Merkle Tree와 Hash를 이용해 검증하기 때문에 검증 시간 자체는 짧지만 proofs의 크기가 커진다. 하지만 다른 영지식 증명 Implementation과 다르게 양자 컴퓨팅에 견딜 수 있다고 알려져 있기 때문에 양자컴퓨터가 발전하고 있는 지

금 보안상 안전한 Implementation임

- Bulletproofs는 range proofs, inner product proofs, circuit 증명과 같은 여러 방법에 대한 증명을 지원하지만 검증 시간이 오래 걸리기 때문에 range proofs 검증으로 주로 사용되지만 range proofs의 검증 비용은 약 700만이고 이더리움에서의 블록 가스 제한이 현재 1,000만이기 때문에 한 블록 내에 한 개의 증명값의 검증만을 실행할 수 있어 여러 검증이 실행되기 어려움

○ Zokrates

- 영지식 증명을 사용하기 위해서는 위와 같은 암호학적 지식을 알고 있어야 함
- 또한 아직 영지식 증명의 표준이 나오지 않았기 때문에 쉽게 적용하기 어렵기 때문에 블록체인에서 아직 영지식 증명을 이용한 스마트 컨트랙트가 많이 나오지 않았음



[그림 47] Zokrates 프로세스

- 이러한 점을 해결하기 위해 Zokrates라는 프로그램을 지원함.
  - zokratres의 프로세스는 [그림 47]과 같으며 Zokrates에서는 프로그램이 검증하려는 logic을 Zokrates에서 지원하는 DSL(Domain Specific Language)로 작성하면 해당 로직을 검증하는 Verification Contract를 생성해 줌
  - 또한 입력을 넣어주면 그에 맞는 proof을 생성해주기 때문에 영지식 증명을 개발자가 깊게 알지 못하더라도 블록체인 환경에서 비교적 쉽게 적용할 수 있게 만들어 줌
- 이러한 Zokrates는 High level language를 zk-SNARK을 지원하기 위한 library로 만들어진 libSNARK의 입력으로 넣어주기 위해 flatten code로 변환하는 parser를 이용
- 그 후 parser에서 하나하나의 정의문으로 만들어 준 flatten code를 한줄 씩 읽어 libSNARK의 입력으로 넣어 R1CS constraint를 생성하여 circuit을 생성하고 증명 값을 생성하기 위한 proving key와 증명 값을 검증하기 위한 verification key를 각각 생성
- 만들어진 flatten code는 시스템을 이용하는 사람들이 공유하여 서로 같은 circuit과 proving key를 서로 공유
- 영지식 증명의 검증을 블록체인에서 실행하고자 하는 검증자는 Zokrates에서 verification key를 이용해 Verification Contract를 생성
  - 생성한 Verification Contract를 이더리움 컨트랙트로 deploy하고 이 deploy된 컨트랙트를 이용해 탈중앙화된 환경에서 비밀정보를 숨긴 채 값을 검증할 수 있음
- 값을 증명하려는 증명자는 공유된 proving key를 이용하여 Zokrates에 비밀 값의 입력을 해 Verification Contract에서 사용될 증명 값을 생성
  - 값이 옳았다면 Verification Contract에서 값을 검증하였을 때 참이 나오고 값이 옳지

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

않다면 거짓이 나옴

- 이 때 영지식 증명에서 proof의 생성비용은 비싸고 on-chain에서 굳이 실행할 필요가 없기 때문에 off-chain에서 proof을 생성하고 검증만을 on-chain에서 실행
- 하지만 Zokrates에서도 문제가 남아있는데 SNARK을 사용하여 Trusted Setup이 필요하고 verification key를 공유하기 어렵다는 문제, verification contract와 verification contract의 검증하려는 로직이 연결이 없기 때문에 로직 파악이 어렵다는 문제, verification contract가 사용되는 스마트 컨트랙트는 off-chain과 on-chain에서 실행되는 로직이 각각 나뉘어 있기 때문에 실행이 언제 일어나는 것인지 알기 어렵다는 문제와 같이 아직 개선될 사항이 많이 남아있음.

○ zkay

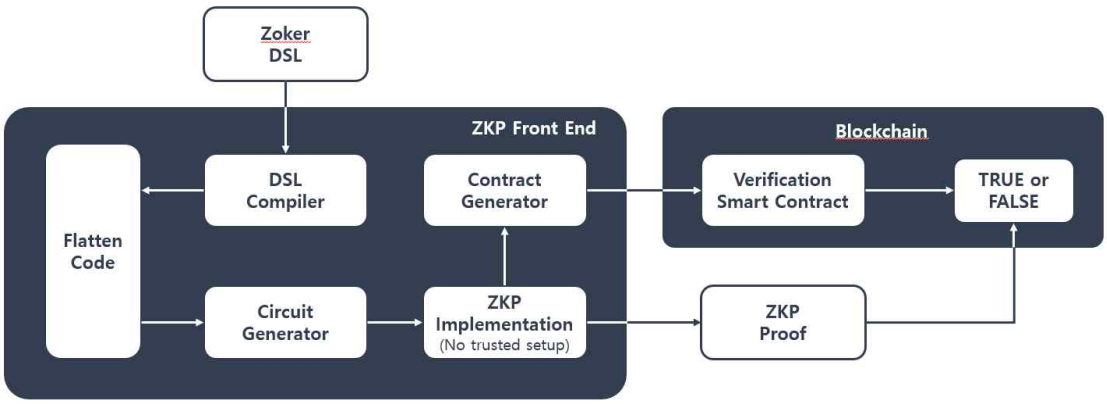
- zkay는 Zokrates에서 Zero knowledge proofs을 깊게 알지 못하더라도 verification contract를 생성하는 것에서 넘어가 Zero knowledge를 알지 못하더라도 쉽게 프라이버시를 보장하는 smart contract를 작성할 수 있게 도와주는 프로그램이다. Zokrates를 사용할 경우 off-chain과 on-chian의 기능이 나뉘어 있지만 개발자가 이를 직접 처리하기 어렵다는 단점이 있음
- 또한 zero knowledge에 대한 지식이 부족한 개발자가 개발하는 도중 드러내지 않아야 하는 개인정보가 드러나게 개발할 수도 있는데, 이 경우 zero knowledge proof을 사용하는 의미가 없어질 수 있고 오히려 오버헤드만 늘어나게 됨
- 또한 스마트 컨트랙트는 블록체인에서의 정보를 다루기 때문에 개인정보가 드러나는 것과 잘못된 로직을 작성하는 것은 기존 프로그래밍 언어보다 더욱 민감한 문제임
- 이러한 문제를 막기 위해 zkay에서는 solidity의 언어에서 일부 keyword를 추가하고 privacy type이라는 새로운 타입을 지원하여 위와 같은 문제를 개발자들의 입장에서 쉽게 해결할 수 있도록 도움
- privacy 타입은 파이썬이나 자바에서 사용되고 있는 annotation과 비슷한 방법으로 변수의 타입에 어떠한 address의 Public key로 관리하는 지를 지정하는 타입
  - @me라고 적을 경우 msg.sender의 public key로 암호화되어 트랜잭션을 보낸 사람만이 정보를 접근할 수 있고 solidity 내에서 적혀있는 address 변수의 이름으로 지정할 경우 해당 address의 유저만이 접근할 수 있게 만들어 줌
  - 또한 verify, reveal이라는 새로운 키워드의 내장 함수를 지원하여 Verification Contract를 개발자들이 직접 만들 필요 없이 zkay 시스템에서 직접 연결하고 만들어 줌으로 인해 개발자들의 입장에서는 on-chain, off-chain에서의 작업을 직접 처리할 필요 없이 zkay시스템에 맡겨 영지식 증명에 대한 깊은 이해 없이 쉽게 프라이버시를 보장하는 스마트 컨트랙트를 작성할 수 있음
- 이러한 zkay의 구현은 solidity의 문법에 추가적인 keyword를 추가한 lexer를 통해 파싱한 후 parser에서 solidity에 없는 새로운 키워드를 이용한 문법에 대해 미리 작업을 처리
- 예시로 privacy type이 사용된 변수를 사용하는 함수는 privacy type의 값은 트랜잭션을 보낸 사람의 public key로 암호화되어 저장될 것이므로 type을 bin이라는 zkay에서 임의로 지정한 암호화된 타입인 bin structure type으로 변경
  - 또한 함수에서 사용되는 로직은 Verification Contract로 작성하여 해당 작업은 public key로 암호화된 값을 해당 유저의 private key로 값을 복호화한 후 값을 처리한 후 값의 결과가 맞는지 확인하는 zero knowledge proofs을 생성
  - 때문에 모든 데이터는 사용자들의 public key로 암호화되어 있는채로 저장되고 암호화된 데이터를 처리할 때 zero knowledge proofs을 이용해 verify 함수와 연결하여 처리할 수 있게 도와줌

- 그 후 off-chain에서 처리해야 하는 작업을 zkay javascript 프로그램으로 지원하여 입력만 기존에 사용하던 식으로 넣어주면 proof 생성도 자동으로 처리한다. 때문에 개발자는 on-chain, off-chain에 대해서 기존에 solidity를 통해 개발하는 것과 같은 방식으로 프라이버시를 보장하는 컨트랙트를 작성 가능

- 하지만 zkay에서도 Zokrates를 이용해 mapping을 했기 때문에 libSNARK을 사용하였기 때문에 Trusted setup문제를 해결하지 못하였고, address를 기반으로 PKI 시스템을 이용하도록 만들었기 때문에 Threshold 시스템과 같은 Multi party Computation을 지원하기 어렵고 모든 사람이 투표하고 투표가 끝난 후 개표해야 하는 투표시스템과 같은 시스템에서 적용하기 어려움

○ Zoker

- Zoker는 수원대에서 개발하는 영지식 증명의 Implementation인 Backend를 개발자들이 쉽게 적용할 수 있게 만드는 zero knowledge의 frontend로 Python과 비슷한 High level Language를 Domain Specific Language로 지원하여 개발자들이 쉽게 검증 logic을 작성할 수 있게 만든다. zoker의 구조는 [그림 48]와 같은 흐름으로 진행



[그림 48] Zoker의 구조

- 이 때 DSL을 입력을 받으면 DSL Compiler에서 DSL을 읽어 수원대에서 개발한 Implementation의 입력으로 넣기 위한 형태인 flatten code로 재구성함

  · 재구성된 데이터를 통해 Zoker에서는 implementation을 이용하여 Domain Specific Language로 작성한 로직과 같은 로직을 가지는 수학적 circuit을 만듦

- Circuit을 생성하면 로직을 공유할 수 있기 때문에 Verification Contract와 zero knowledge proof을 생성할 수 있음

  · Verification Contract는 ZKP Implementation의 검증 로직을 가져오는 것이기 때문에 ZKP Implementation을 Ethereum의 스마트 컨트랙트를 위해 사용되는 언어인 Solidity에 맞게 포팅하여 이더리움에서 검증할 수 있게 만들 수 있음

- 하지만 Solidity는 storage, memory 문제와 같이 단순히 포팅할 경우에 실행 시간이 오래 걸리게 포팅할 수 있으므로 실행 시간이 오래 걸리면 단순히 시간이 오래 걸리는 것만이 아니라 비용이 커질 수 있고 Scalability에 악영향을 줄수 있기 때문에 이를 최적화하여 비용을 최소화하는 방향으로 언어를 포팅해야 함

- Verification Contract는 검증 로직을 포팅하여 만들 수 있고 이 입력으로 넣을 zero knowledge proofs은 Circuit이 생성되면 Implementation을 이용해 zero knowledge proof를 생성

  · 생성된 proofs은 Verification Contract를 통해 on-chain에서 검증

○ Zoker와 Zokrates의 차이점

- Zoker는 기존의 영지식 증명을 사용하는 것을 도와주는 Zokrates의 개선된 버전을 지향하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

는 프로그램으로 Zokrates에서 지원하는 library는 libSNARK으로 Trusted setup을 필요로 하는 시스템이었지만 zoker는 trusted setup이 필요하지 않은 영지식 증명을 사용하여 금융 시스템이나 실거래, Decentralized Identity와 같은 fake proof에 민감한 분야에서도 쉽게 적용할 수 있는 스마트 컨트랙트를 작성할 수 있음

- Implementation은 수원대에서 개발하는 ZKBOO에서 개선된 버전의 영지식 증명을 사용하여 proof size가 작고 proving 시간도 짧은 편인 영지식 증명으로 사용자가 증명 값을 생성하는데 시간이 너무 오래 걸리지 않게 만들어 사용자 경험을 개선할 수 있음
  - 또한 Trusted Setup이 필요 없기 때문에 신뢰기관이 필요하거나 Multi party computation이 필요하지 않아 오버헤드가 적고 개발자 입장에서 쉽게 영지식 증명을 적용할 수 있음
- Zoker의 Zero knowledge proofs Implementation은 proving time이 짧고 proof size가 작기 때문에 이후 언급할 proof을 저장하고 선택적으로 검증하는 SVZK(Selective Verification Zero Knowled proofs)와 함께 사용하게 될 경우 저장 비용이 적어 더 효율적인 시스템을 가질 수 있음

□ On-chain 프로토콜 비용감소를 위한 SVZK

○ SVZK (Selective Verification Zero knowledge)의 필요성

- 이러한 블록체인에서의 영지식 증명의 사용 문제 중 Scalability를 해치는 성능 문제를 개선하기 위한 방법으로 SVZK(Selective Verification Zero knowledge)라는 시스템을 제안

<표 8> Signature 비용 비교

	실행 시 Gas 비용 (Gas)
ECDSA	25,881
Schnorr Signature	122,725
BLS Signature	364,977
zk-SNARK	736,696

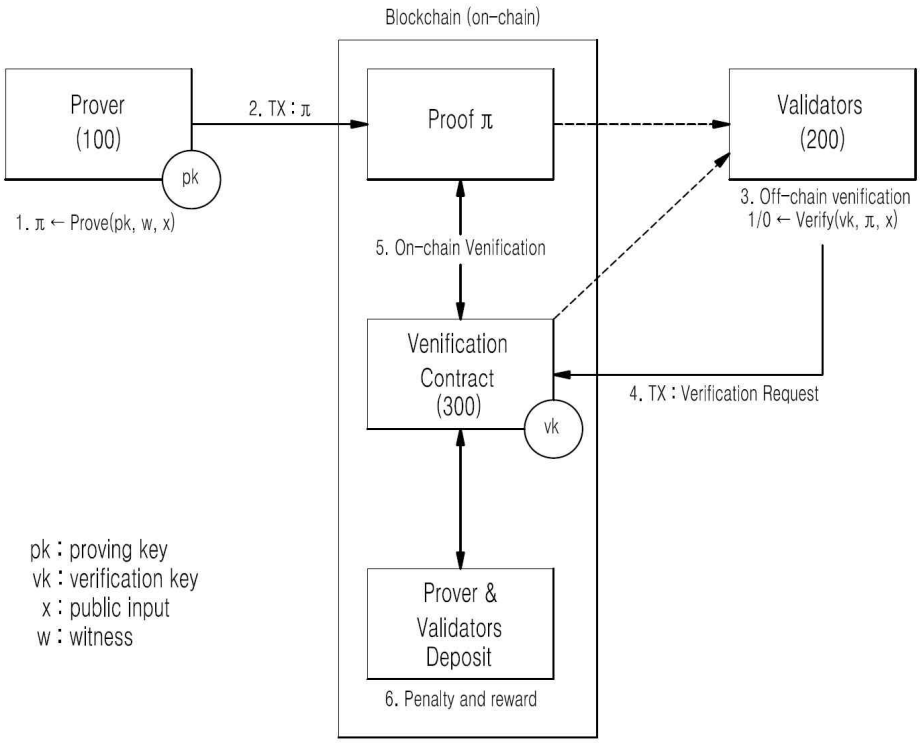
- <표 8>에서는 스마트 컨트랙트로 작성한 Signature, zk-SNARK의 성능 비교를 확인할 수 있음
  - 위의 표에서 볼 수 있듯이 영지식 증명의 검증은 증명 값을 만들어내는 작업에 비해 비용이 저렴할 수 있지만 여전히 블록체인에서 사용하기에는 일반적인 트랜잭션과 비교했을 때 비용이 크기 때문에 이러한 검증을 모든 증명 값에 대해 실행하지 않고 off-chain에서 검증해본 후 증명 값이 옳지 않은 증명에 대해서만 on-chain에서 실행해 옳지 않음을 드러내는 방법을 제안

○ SVZK의 구조

- [그림 49]에서 볼 수 있듯이 SVZK 시스템에서는 영지식 증명의 증명을 적은 비용으로 증명하려는 prover, 다른 사람들이 자신의 시스템에서 적은 비용으로 영지식 증명을 이용하게 하는 setup, 다른 사람의 증명 값이 옳은 지 확인하는 challenger(validators)로 구성되어 있음
- setup 단계에서는 SVZK 시스템에서 증명하고자 하는 값을 설정하는 과정으로 증명 값을 검증하는 Verification Contract를 SVZK 시스템에 등록하고 이 작업을 통해 하나의 proof만을 검증하는 것이 아닌 여러 contract를 검증하는 시스템을 가짐
- Verification Contract를 등록하면 prover는 자신의 증명 값을 SVZK의 Storage에 올린 후 일정 기간 동안 아무런 Challenge도 받지 않으면 증명 값이 옳음을 확정 지음
  - 이를 위해 prover가 증명하고자 하는 contract, 증명 값을 지정해야 하며 증명 값을 틀리게 올릴 수 있기 때문에 이를 막기 위한 예치금도 제출해야 함
  - 만약 prover의 증명 값이 틀리지 않았다면 예치금은 다시 환급받음
- Challenger는 모든 prover가 옳은 증명 값을 올리지 않을 것이기 때문에 SVZK에 올라오는 모든 증명 값을 검증해 본 후 검증 결과 옳지 않은 증명 값인 경우 증명 값을 on-chain에

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

서 실행시켜 값이 틀림을 증명하고 prover가 제출한 증명 값을 폐기시키며 prover가 제출한 예치금을 challenger가 받음



[그림 49] SVZK의 구조

- 이로 인해 영지식 증명을 이용해 값을 증명하는 증명자는 기존보다 더욱 저렴한 비용으로 값을 증명할 수 있음

<표 9> SVZK와 영지식 증명 검증의 비용 비교

	constraint	Verification (gas)	Storage (gas)
If 1	765	736,696	486,781
If 100	765,00	736,696	471,781
public 0 private16	494	727,083	451,356
public 16 private 0	494	880,705	777,809

- <표 9>에서는 if 의 개수에 따른 성능 비교, public으로 드러내는 값의 개수에 따른 성능 비교를 보임
  - 이에서 볼 수 있듯이 기존의 영지식 증명을 on-chain에서 검증하는 방법을 이용하게 되면 ethereum에서 Zokrates의 Verification Contract를 이용할 경우 736,696의 Gas가 소모되었는데 저장하는 작업만 실행할 경우 471,781의 Gas만이 소모되어 약 1.5배의 Scalability의 개선
- 하지만 기존의 영지식 증명 검증과는 다르게 Storage에 저장하는 비용과 신고할 때에 검증하는 비용이 들어 두 번의 비용이 들 경우 기존 방식보다 더욱 많은 Gas를 사용하여 비용이 클 수 있으나 대부분의 경우 옳은 증명 값을 올릴 것이기 때문에 이러한 문제는 크지 않을 것으로 보임
- 또 다른 SVZK의 단점으로는 블록체인 내에서 모든 증명 값이 검증되지 않기 때문에 신뢰성을 감소시킬 수 있다. 이러한 문제는 게임이론을 적용하여 challenger가 더욱 유도될 수 있도록 만들거나 hash 값만을 on-chain에 올리고 off-chain을 통해 증명 값을 올림으로서 on-chain의 storage의 줄여 성능을 개선할 수 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

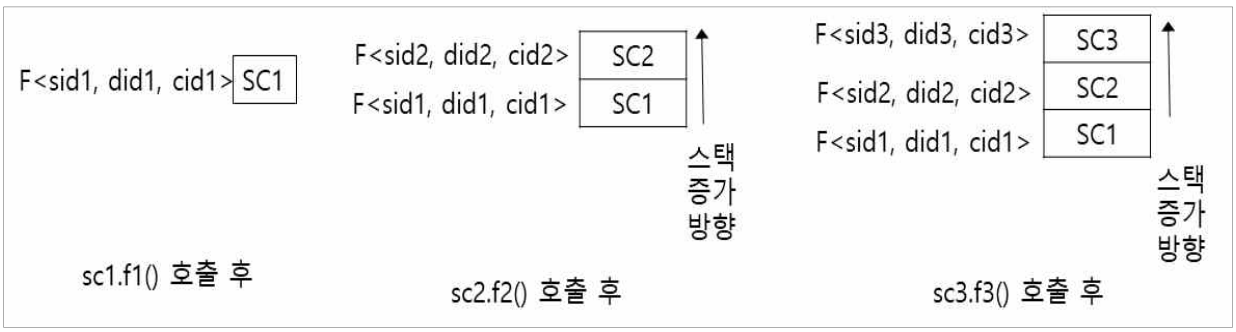
2) 스마트 컨트랙트 취약점 분석 기술 연구

□ EVM에서 동작 가능한 런타임 스택 인스펙션도구 프로토타입 설계

- EVM 바이트코드의 callee-caller 주체 간의 관계에 기반한 분석을 가능하게 하는 런타임 스택 인스펙션 도구 프로토타입 설계
- 실행 주체를 확인하고 의도하는 행위의 실행 가능 여부를 판단할 수 있는 도구 제시
- Go-Ethereum에서 동작하는 EVM에 적용할 수 있도록 Go언어로 작성된 call stack 프로토타입 설계

○ 런타임 스택 인스펙션을 위한 실행 주체(credential)와 관련 행위 집합 연구

- [그림 50]에서 스마트 컨트랙트 sc1의 함수 f1()에서 sc2의 f2()가 불리고, sc2의 f2()에서 다시 sc3의 f3()가 호출될 때
  - sc1~sc3는 서로 다르거나 동일할 수 있음
  - sc1~sc3의 계정 id를 sid1.. sid3이라고 함
  - sc1~sc3을 deploy한 사람의 id를 did1.. did3이라고 함
  - sc1~sc3을 생성한 사람(컨트랙트)의 id를 cid1.. cid3이라고 함
- sc1.f1() 호출 후에는 credential = F<sid1, did1, cid1>, 이어 sc2.f2()를 호출 후에는 실행 주체(credential) =G(F<sid1, did1, cid1), F<sid2, did2, cid2>), 이후 sc3.f3() 호출 후에는 실행 주체(credential) = G(F<sid1, did1, cid1), F<sid2, did2, cid2>, F<sid3, did3, cid3>) 으로 정의



[그림 50] 함수 호출에 따른 credential의 변화

- 이와 같은 실행 주체에 대해 각 취약성마다 적절한 G와 F를 찾는 것에 대한 연구를 진행
- 즉, 보안 정책은 호출 시점의 실행 주체를 기준으로 결정되는데, F는 3가지 종류의 id를 적절히 조합을 한 값, G는 스택 트레이스상에 나타나는 call chain 상에 나타나는 일련의 Credential fragment 들의 리스트에 대해 효율을 위해 적절히 가공한 값을 의미함

○ Call chain 식별을 위한 매커니즘 설계 진행 중

- 실행 주체에 대한 정보를 모아 런타임에 스마트컨트랙트의 함수가 어떤 경로로 호출되었는지 파악하고 제어하기 위해 Solidity 언어에서 쓰이는 함수 call 매커니즘 분석을 진행
- 스마트 컨트랙트에서 함수 호출 및 수행을 위한 프로그램은, 파일로 존재하여 정적분석하는 경우와 실제로 런타임에 구동되는 코드에 차이가 있음. 특히 스마트컨트랙트 코드는 creation부분과 runtime부분으로 나뉘어 각각 처음 생성할 때에만 존재하거나 일반적인 런타임에 존재하게 됨
- 이런 부분에 의해 함수 call의 종류에 따라 call stack에 영향을 줄 수 있기 때문에 call 종류별로 컴파일된 어셈블리 파일을 분석
- 이러한 특성을 반영해서 함수의 실행 주체를 찾기 위한 매커니즘을 연구 중

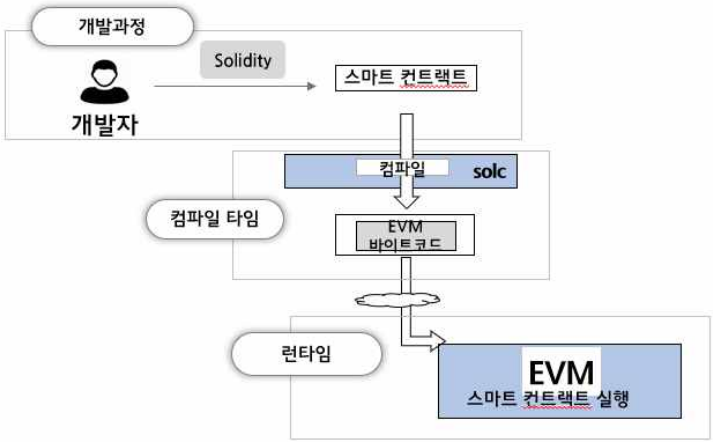
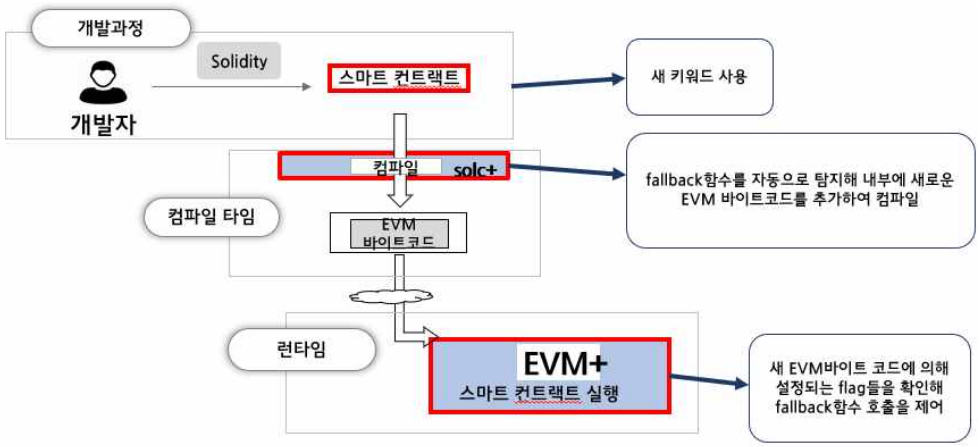
○ **EVM에서 동작 가능한 런타임 스택 인스펙션도구 프로토타입 개발**

- EVM 바이트코드의 callee-caller 주체 간의 관계를 확인하는 런타임 스택 인스펙션 도구 프로토타입 개발
- 실행 주체를 확인하고 의도하는 행위의 실행 가능 여부를 판단할 수 있는 도구 제시
- Go-Ethereum에서 동작하는 EVM에 적용할 수 있도록 Go언어로 작성된 call stack 프로토타입 개발 중

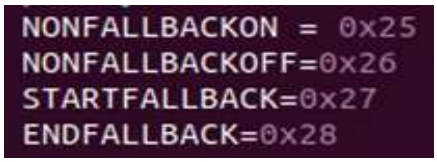
○ **Fallback함수 호출 제어 기능을 할 수 있는 런타임 스택 인스펙션**

- 궁극적으로는 런타임 스택 인스펙션을 위해 함수 이름과 주소 등을 가지는 구조체를 기본단위로 하는 스택을 활용하며 fallback 함수에 대해 앞서 F와 G에 해당하는 검출 기법 고안 중
- 1차년도에 진행했던 fallback함수 호출 제어 연구를 확장함.
- 현재는 수정된 EVM (EVM+) 개발하고, Solidity 컴파일러를 확장하여 런타임에 EVM+에 정보를 생성함으로써 제어 기능을 수행
- Go-Ethereum에서 동작하는 EVM에서 fallback함수는 함수 이름, 인자, 리턴 값이 없이 개발자가 임의로 작성하는 함수로, 함수 실행 시 오류가 발생하거나 ether를 전송받아야 하는 경우에 실행됨
- 개발자가 임의로 작성하는 함수이기 때문에, 함수 내부에서의 동작 내용을 알 수 없어서 위험에 노출될 수 있어 개발자의 의도대로 제어가 필요함
- fallback함수는 EVM내에서 따로 키워드가 있거나 식별 가능 하도록 구분되는 특징이 없이 일반 함수처럼 보임
- 따라서 fallback함수를 식별해서 제어할 수 있는 기술적인 방법이 필요함을 인지하고 그에 따라 Solidity 컴파일러와 EVM을 수정함 (<표 10>의 붉은 부분)

<표 10> 기존의 스마트 컨트랙트 실행환경과 수정된 스마트 컨트랙트 실행환경

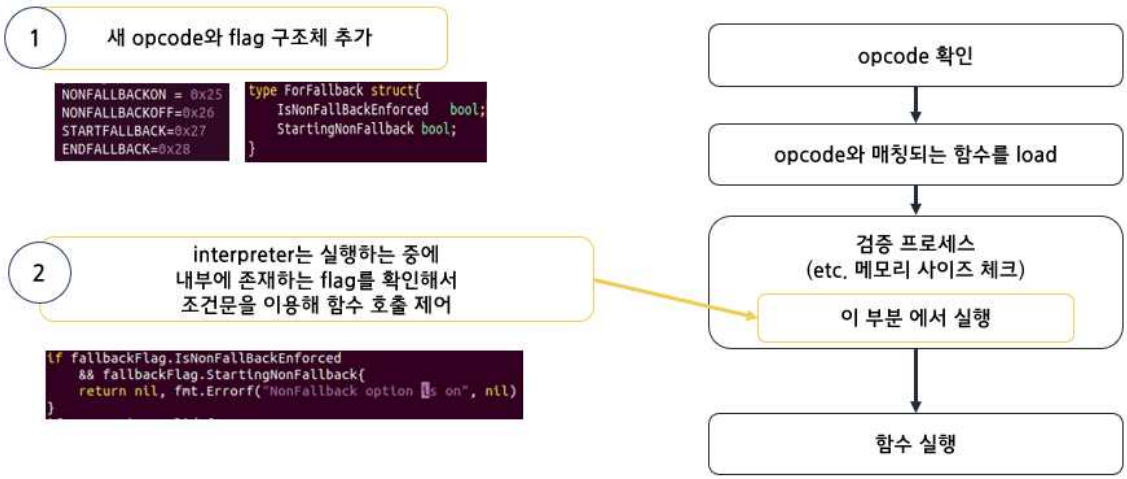
기존의 실행환경	
수정된 실행환경	

- 개발자는 [그림 51]과 같이 개발과정에서 새롭게 추가된 키워드 NONFALLBACKON, NONFALLBACKOFF를 이용해서 자신의 의도대로 fallback함수 호출의 제한 범위를 지정하여 의도적으로 안전성을 높임
- 수정된 Solidity 컴파일러는 컴파일 과정에서 간단한 분석을 이용해서 자동으로 함수가 fallback함수 인지 확인함. 아래와 같이 STARTFALLBACK, ENDFALLBACK이라는 새로운 바이트코드를 도입하고 fallback 함수 정의마다 생성



[그림 51] 추가된 EVM바이트 코드

- 따라서 EVM은 크게 2가지, 각 2개씩의 새 opcode들과 이를 확인 및 처리할 수 있는 flag가 추가되는 형태로 확장됨. 즉, opcode 의 interpreter 단계에서 이러한 flag를 확인하는 조건문이 추가됨



[그림 52] 수정된 EVM

- 컴파일러가 추가한 EVM바이트코드를 확인해서 EVM 내부에 존재하는 flag를 설정
- [그림 52]와 같이 수정된 EVM의 interpreter는 실행하는 중에 내부에 존재하는 flag를 확인해서 조건문을 이용해 함수 호출 제어할 수 있게 됨
- 조건문을 이용해서 함수 호출을 제어하는 부분은 EVM에서의 함수 실행 과정 중에 검증 프로세스 단계에서 동작하게 됨
- 위의 수정된 Solidity 컴파일러와 수정된 EVM의 코드는 Github에 공개됨

○ 기타 제어 기능을 할 수 있는 런타임 스택 인스펙션

- 신뢰할 수 없는 callback 함수 검출 (객체 o가 다른 객체 o'를 호출한 후 리턴하기 전에 o를 호출하는 과정에서 호출자 함수의 상태를 변경하는 경우), owner 남용 검출 등 주요 이슈들에 대해 런타임 스택 인스펙션을 실행할 수 있도록 앞서 F와 G에 해당하는 기술을 찾고 검출하는 기법 고안 중

○ Solidity에 존재하는 owner 취약성을 런타임 인스펙션을 위한 기법 고안

- 생성자에서만 호출되어야 하는 함수가 생성자가 아닌 다른 함수에 의해 호출될 경우 Fallback 함수가 호출될 수 있음

○ 수정된 EVM 테스팅을 위한 실행환경 구성

- EVM에서 실행 시에 실제로 동작하는지 확인하기 위해 private network를 구성하여 테스팅 환경을 구성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 스마트 컨트랙트 취약점 탐색을 위한 자동 역공학 기술 개발

○ 바이트 코드 식별 모델 구현을 위한 데이터베이스 구축

- 바이트 코드에 포함된 속성(attribute)에 대한 레퍼런스가 필요하기 때문에, 바이트 코드와 소스코드가 매칭되어 있는 데이터 셋을 구축하기 위하여 etherscan.io에 등록된 Verified Smart Contract 데이터를 크롤링 기법을 이용하여 수집
- 메인 모델에서 바이트 코드의 패턴을 보다 정확히 인식할 수 있도록, 입력 데이터를 처리하는 모델을 구축하기 위하여 대량의 바이트 코드 데이터를 수집하고자, Ethereum의 fullnode 데이터를 기반으로 구축된 Google Bigquery에서 데이터 셋을 구축
- 구축된 데이터 셋의 현황은 <표 11>과 같음

<표 11> 바이트 코드 식별 모델을 위한 구축한 데이터 베이스 셋

Source	Number of record	Field
Etherscan	54,209	- Address - DApp title - Solidity version - Bytecode - Sourcecode
Bigquery	2,227,842	- Address - Bytecode

○ 속성 레퍼런스 구축

- 기존에 정립된 레퍼런스가 부족하기 때문에, 바이트 코드에서 인식할 속성 항목에 대한 새로운 레퍼런스를 구축
- 각 바이트 코드에 해당하는 ABI를 기반으로 contract, function 등의 구성요소의 이름 정보를 추출. 추출된 정보는 해당 구성요소의 핵심 키워드를 포함하고 있어 이를 활용해 레퍼런스에 사용될 토큰을 생성
- 전체 18,557개의 후보 토큰에서 중복된 정보를 내재하고 있는 토큰을 병합하고자 다음과 같은 처리 과정을 거침
  - 동일한 어근을 가지는 토큰에 대한 병합.
  - 토큰 빈도가 평균 값(50.564개) 보다 적은 토큰 제거.
  - 불용어 제거. ex) of, to, contract, smart ...
- 필터링된 773개의 토큰 중, 토큰의 희소성을 배제하기 위하여 상위 25%에 해당하는 193개의 토큰에서, <표 12>와 같이 레퍼런스로 사용할 수 있는 16개의 토큰을 선정
- 레퍼런스 토큰은 스마트 컨트랙트의 주요 객체 (token, account, contract)에 관련되며, 널리 사용되는 용어를 기준으로 선정하였음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<표 12> 속성 레퍼런스 토큰

Attributes	Occurrence	Frequency	Description
transfer	30,028	0.9197	토큰 전송 기능
own	20,805	0.6372	오너십 관리 기능
destroy	8,953	0.2742	selfdestruct 기능
burn	7,757	0.2376	토큰 소각 기능
trade	6,745	0.2066	토큰 구매 및 판매 등의 거래 관련 기능
mint	5,756	0.1763	토큰 발행 기능
withdraw	5,665	0.1735	Ether 출금 기능
upgrade	5,658	0.1733	업그레이드가 가능한 스마트 컨트랙트 구조
lock	5,175	0.1585	일정 기간동안 특정 기능에 대한 잠금 기능
crowdsale	5,012	0.1535	ICO를 목적으로 작성된 스마트 컨트랙트
whitelist	2,939	0.0900	화이트 리스트 기능
pause	2,821	0.0864	일정 기간 스마트 컨트랙트 사용 제한
remove	2,792	0.0855	특정 데이터 삭제 기능
freeze	2,400	0.0735	특정 주소의 이용을 제한하는 기능
game	2,194	0.0672	게임을 목적으로 작성된 스마트 컨트랙트
airdrop	1,277	0.0391	토큰 분배 기능

○ **특징 추출 기법 정의**

- 효율적으로 바이트 코드의 패턴을 인식하기 위해, 바이트 코드로부터 특징을 추출하여 머신러닝 모델 구축에 활용
- 기존의 연구에서는 스마트 컨트랙트 간의 유사도 측정을 위해 fuzzy hashing 기법을 이용하여 특징을 정의[7]하였음
- 해당 기법은 개별 속성에 대한 패턴을 인식한다는 관점에서 부합하지 않아, 본 연구에서는 두 가지의 접근 방법인 1) Handcrafted features, 2) Raw bytecode features를 정의하고 이를 비교하여 최적의 특징 추출 기법을 탐색
- Handcrafted features는 스마트 컨트랙트의 opcode에서 추출할 수 있는 세 가지 정보(길이, 구성요소, 문맥)를 이용하는 추출 기법임
- 여기서 opcode는 불필요한 정보를 최소화하기 위해, 오퍼랜드를 통일하기 위해 'OPERAND'로 변환하였으며, 'PUSH1', 'PUSH2' 등으로 구분된 opcode를 'PUSH'로 병합하여 사용됨
- 길이는 opcode의 개수, bytecode의 문자열 길이, 고유한 opcode의 개수로 정량적인 척도에 의해 산출할 수 있는 특징을 의미함
- 구성요소는 스마트 컨트랙트를 구성하는 opcode의 종류를 One-Hot encoding 기법으로 표현한 특징을 나타냄

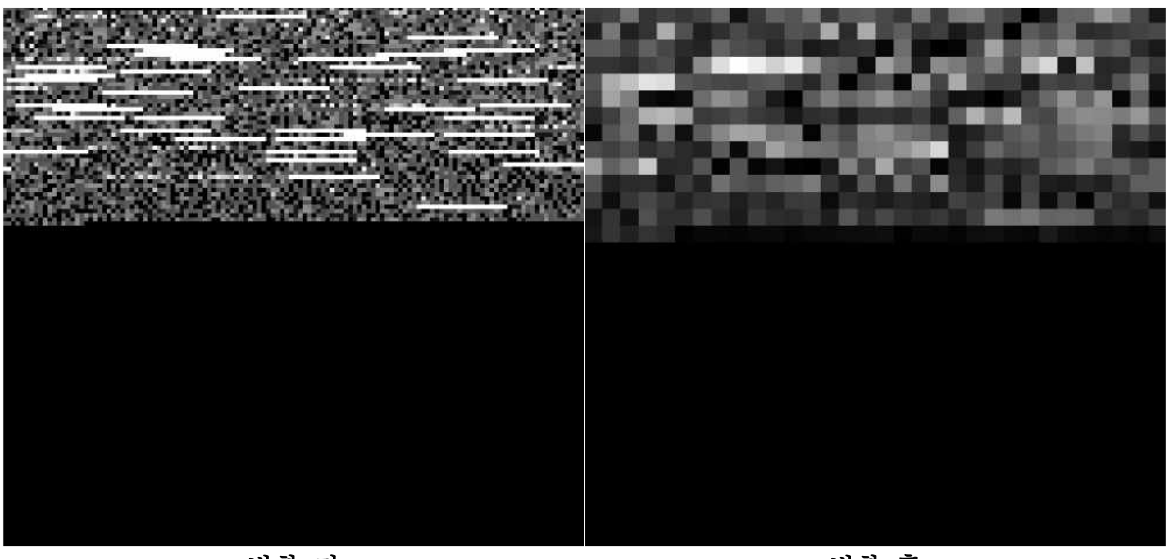
7) R. Norvill, B. B. F. Pontiveros, R. State, I. Awan, and A. Cullen, "Automated labeling of unknown contracts in Ethereum," in Proc. 26th Int. Conf. Comput. Commun. Netw. (ICCCN), Jul./Aug. 2017, pp. 1–6.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 문맥은 opcode set으로 학습된 Doc2Vec 모델을 구축하여, 스마트 컨트랙트의 내용을 나타낼 수 있는 수치 값으로 재표현하여 산출
- Raw bytecode features는 normalized opcode를 2차원의 데이터 구조로 reshape하여 생성된 특징으로, opcode의 위치와 순서 정보를 유지할 것을 기대
- 각 특징 추출 기법으로 생성된 특징 값은 속성 레퍼런스에 해당하는 데이터와 매칭되어 모델 구축에 활용

○ **모델 정의**

- 두 가지 특징 추출 기법인 handcrafted features와 raw bytecode features는 각각 1차원과 2차원 데이터로 활용 가능한 뉴럴 네트워크가 구분돼 1차원 데이터 처리가 가능한 Feed Forward Neural Networks (FFNNs)와 2차원 데이터 처리에 적절한 Convolutional Neural Networks (CNNs)를 사용하였음
- 각 뉴럴 네트워크의 구조는 기존의 연구에서 널리 사용되는 ResNet-18 [8]을 참고하여 FFNNs는 dense layer를 이용해 구축되었으며, CNNs는 convolutional layer를 이용하여 각각 **FFNN**, **CNN**로 칭하는 모델을 구축
- 또, 각 모델을 확장하고자 **FFNN**은 파라미터 수를 1,420,625에서 22,335,761로 15.72배 증가시킨 **wideFFNN**을 구축하였으며, **CNN** 모델에 입력되는 2차원 데이터의 크기를 128x128에서 32x32로 압축하여 주요 정보만 포함할 수 있도록 재표현하는 모델인 AutoEncoder를 함께 사용하는 **aeCNN** 모델을 구축
- 여기서 AutoEncoder에 의해 변환되는 데이터의 전후 과정은 다음의 [그림 53]과 같음

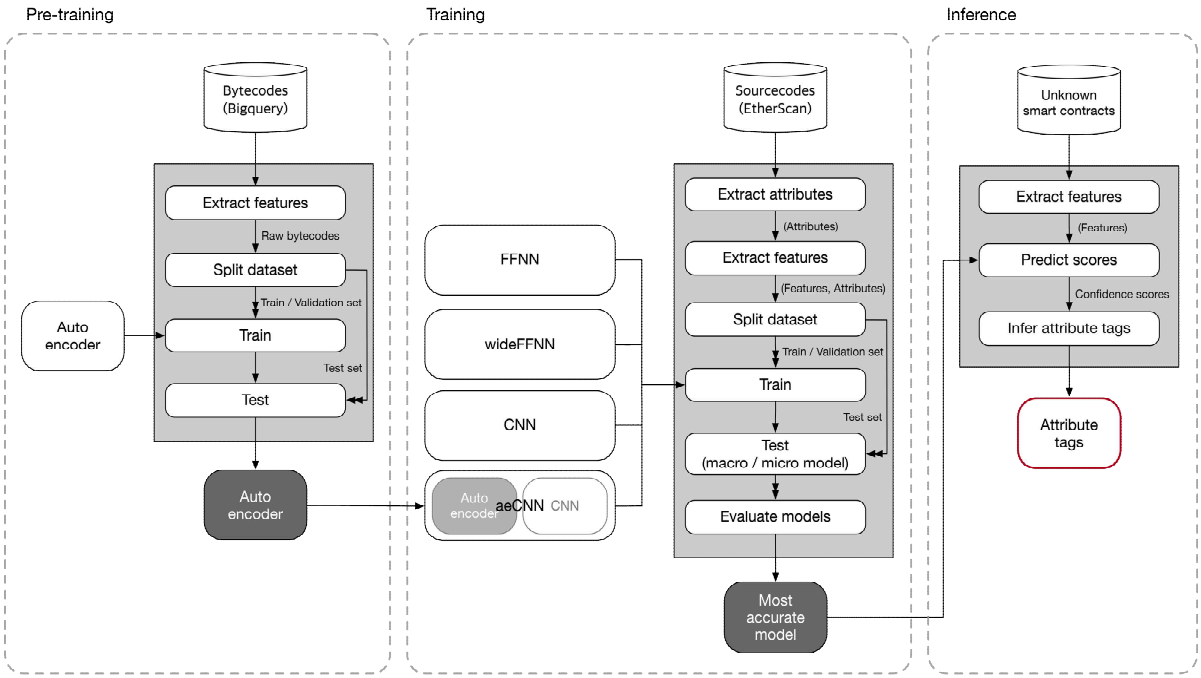


변환 전 **변환 후**

[그림 53] AutoEncoder에 의해 변환된 데이터

○ 전체 프로세스 정의

8) K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Jun. 2016, pp. 770–778.



[그림 54] 실험의 전반적인 프로세스

- [그림 54]와 같이 전체 프로세스는 pre-training, training, inference stage로 구분
- Pre-training stage는 Google Bigquery에서 수집한 데이터 셋을 이용하여, AutoEncoder 모델을 학습하는 단계임
  - 해당 단계에서는 raw bytecode features 기법만 적용하여, 속성에 대한 레퍼런스 없이 모델 학습에 사용하는 unsupervised learning으로 구성된다.
  - Training stage에서는 메인 모델을 Etherscan 데이터 셋과 레퍼런스 셋을 생성하여 supervised learning이 진행된다.
- **FFNN**, **wideFFNN**, **CNN**, **aeCNN**에 대한 실험이 진행되며, 각 모델에 대한 성능은 사전에 구분된 테스트 데이터에 의해 정량적인 성능 평가 척도를 산출해 비교하여 최적의 모델을 선택함
  - 여기서 사용되는 정량적인 성능 평가 척도는 다음과 같음
    1. Macro precision
    2. Macro recall
    3. Macro F1-score
    4. Micro precision
    5. Micro recall
    6. Micro F1-score
- 해결하고자 하는 문제가 동시에 복수의 속성에 대해 예측해야 하는 multi-label 문제이기 때문에, 기존의 accuracy와 같은 척도는 적합하지가 않고, 레퍼런스의 분포가 불균형하기 때문에 이를 보완하기 위해 precision, recall, f1-score를 사용
- Macro와 Micro 모델에 의해 평가되어 선정된 모델은 다음의 inference stage에서 사용
- Inference stage는 알려지지 않은 스마트 컨트랙트의 바이트 코드를 입력으로 받아 모델

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

에 의해 사용자가 이해할 수 있는 속성 정보를 태그 형태로 제공

○ 실험 환경 정의

- 각 모델은 동일한 조건 하에서 실험할 수 있도록 하이퍼 파라미터를 동일하게 적용하여 실험을 진행
- 적용된 하이퍼 파라미터 정보는 다음과 같음
  - Epoch: 1000
  - Batch size: 32
  - Learning rate: 5E-5
  - Decay rate: 5E-5
  - Decay method: cosine decay
  - Activation function: ReLU function
  - Weight initializer: xavier normal initializer
  - Optimizer: Adam optimizer

○ 실험 결과 산출

- 학습 과정에서 소요된 시간은 에폭 당 CNN 3.10분, aeCNN 0.40분, FFNN 0.09분, 그리고 wideFFNN 0.12분이 소요됨
- FFNN 모델은 CNN 모델보다 평균적으로 34.44배 빠른 학습 속도를 보임
- 또한, AutoEncoder의 적용으로 인해 압축된 데이터는 aeCNN의 학습속도를 7.77배 빠르게 하도록 영향을 미침

<표 14> 성능 평가 척도에 의한 모델의 성능

Model	Macro model			Micro model		
	Precision	Recall	F1-score	Precision	Recall	F1-score
CNN	0.6254	0.6003	0.6111	0.8316	0.7987	0.8139
aeCNN	0.6628	0.6370	0.6482	0.8094	0.8047	0.8062
FFNN	0.5669	0.4918	0.5255	0.7677	0.7153	0.7399
wideFFNN	0.6037	0.5742	0.5863	0.7126	0.7646	0.7367

- <표 14>와 같이 성능 평가 척도에 의하면 micro 모델과 macro 모델의 f1-score가 각각 0.8062, 0.6492로 전반적으로 높은 값을 보이는 **aeCNN** 모델이 가장 좋은 성능을 보이는 것을 알 수 있음
  - CNN의 모델의 경우, micro 모델에서의 f1-score가 0.8139로 aeCNN보다 근소하게 높은 수치를 보이는 반면에, macro 모델에서는 0.6111로 현저하게 낮은 수치를 보여, 최적의 모델로 판단하기에 부족
  - Micro 모델에서의 성능과 macro 모델에서의 성능 차이는 전반적인 예측 성능이 높은 수치를 보임에도 특정 속성에 대한 예측에 대해 낮은 성능을 보인다는 해석 가능
- 따라서, 전반적으로 두 모델에서 높은 성능을 보이는 aeCNN이 최적의 모델이라 판단

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- FFNN의 모델의 경우, 정확도가 micro 모델 기준으로 52.55%에서 파라미터의 수를 늘린 wideFFNN에서 58.63%로 성능이 향상되었으나, CNN 모델의 성능보다 낮은 것으로 확인
- 하지만, FFNN 모델에 사용된 handcrafted features는 raw bytecode features와 달리 바이트코드의 크기에 제약이 없으며, 1차원 데이터라는 점을 활용해, 기존의 머신러닝 기법에 적용가능하다는 점에 있어 다양한 활용이 가능하다는 장점이 있음

○ 케이스 스터디

- 구축한 모델의 활용 가능성을 확인하기 위해, 실제로 운용되고 있는 DApp을 대상으로 모델을 적용하여 그 결과를 확인
- DApp 데이터는 dappradar.com에서 제공하는 6개의 카테고리 (Games, Exchanges, Collectibles, Marketplace, Gambling, HighRisk) 중에서 가장 활성화된 5개의 DApp을 각각 선정하여 <표 15>과 같이 총 30개의 데이터를 수집하였음

<표 15> 수집된 DApp의 목록

Category	DApp
Games	BlockchainCuties, CryptoKitties, 0xUniverse, MyCryptoHeroes Etheremon
Exchanges	IDEX, EasyTrade, LocalEthereum, EthfinexTrustless, ForkDelta
Collectibles	CryptoStrikers, CryptoFlowers, CryptoAtoms, CryptoPunks, Panda Earth
Marketplace	OpenSea, Emoon, DecentralandMarketplaces, WyvernExchange, Auctionity
Gambling	Etheroll, Fomo3D, Dice2Win, Play0x, Edgeless
HighRisk	SnailTree, SnailThrone, Gandhiji.io, PoWH3D, Dividends3D

- <표 16>은 카테고리 내에 포함된 DApp이 보유하고 있는 속성을 종합하여 80% 이상의 DApp에서 사용되고 있는 속성을 나타냄

<표 16> Dapp이 보유하고 있는 속성

Category	Attributes
Games	upgrade, crowdsale, mint, trade, transfer, withdraw, destroy, kill, pause, own (100 %)
Exchanges	transfer, withdraw (100%) own (80%)
Collectibles	transfer, upgrade, own (100%) destroy, kill, mint, trade, withdraw (80%)
Marketplace	trade, transfer, withdraw, own (100%) upgrade, crowdsale, destroy (80%)
Gambling	game, withdraw, own (100%) upgrade, transfer, destroy (80%)
HighRisk	withdraw, trade (100%) transfer (80%)

- 카테고리 단위로 비교했을 때, HighRisk를 제외한 모든 카테고리에서 공통적으로 transfer, withdraw, own 속성이 높은 비중을 차지하고 있음
- 반면, HighRisk 카테고리에서는 own 속성이 거의 나타나지 않는 것을 알 수 있는데,

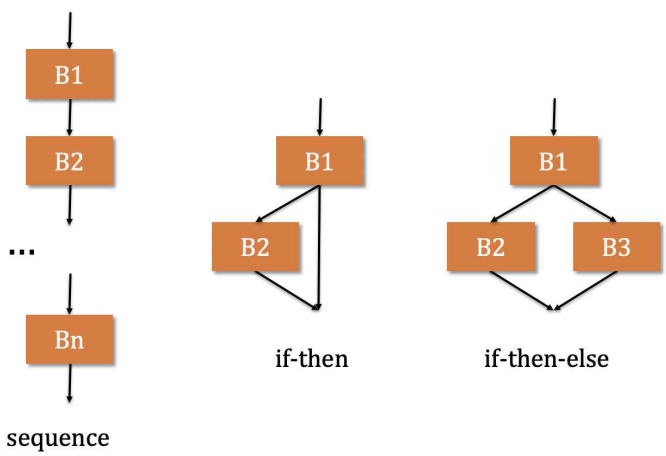
DappRadar에서는 pyramid나 ponzi scheme의 위험이 있는 DApp을 해당 카테고리로 분류하는 것을 고려하면, 위험성이 높은 DApp에서는 오너십 관리를 위한 함수가 구현되지 않거나, 개별적으로 구현된 형태의 코드 패턴을 보인다는 것을 확인할 수 있음

- 개별적인 스마트 컨트랙트 단위로 분석할 경우에도 모델의 바이트 코드 분석 성능에 대해 확인
- 구축된 모델은 바이트 코드를 대상으로 분석하는 모델이기 때문에, ABI에서 확인할 수 없는 internal function인 _mint나 _burn에 대해 인식하고 속성 값을 부여하는 결과를 볼 수 있음 (CryptoStrikers/StrikersCore.sol)
- 스마트 컨트랙트를 업그레이드할 수 있도록 하기 위해, 데이터와 로직 파트로 구분하여 관리하는 upgradability의 경우, 전반적인 코드에 대한 맥락을 분석할 수 있어야 한다. 구축한 모델은 로직 컨트랙트에 해당하는 바이트 코드에 대해 upgrade 속성을 부여하여, 복잡한 맥락에 대해서도 학습이 가능함 (CryptoAtoms/CryptoAtomLogic.sol)
- Mapping 변수를 활용하여 접근 가능한 주소 값을 관리하는 화이트 리스트에 대해서도 인식하여 whitelist 속성을 부여하는 것을 확인 (WyvernExchange/WyvernExchange.sol)
- delete 명령어를 이용해 특정 값을 제거하는 경우에도, mapping 변수 뿐만 아니라, array의 원소 값에 대한 케이스에서도 인식하여 remove 속성을 부여하는 것을 확인할 수 있었다. (Fomo3D/PlayerBook.sol)

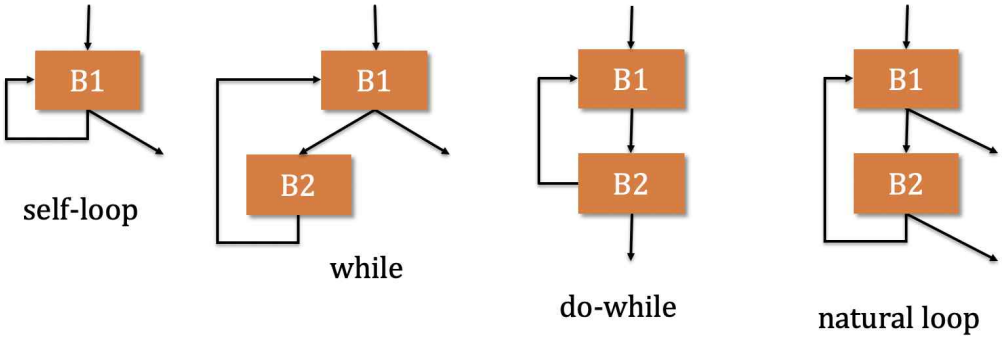
□ 스마트 컨트랙트 역공학 지원 언어 확장

○ 고수준 스마트 컨트랙트의 역공학 지원

- 분석을 위한 중간언어 역공학을 바탕으로 하여 고수준의 스마트 컨트랙트 역공학을 위한 추가 기술을 개발
- 고수준의 제어 구조를 복원하기 위하여 구조 분석 기술 개발 및 구현
- 중간언어 코드를 바탕으로 [그림 55]와 같이 control flow graph(CFG)를 복원하고, 제어 구조 패턴에 맞추어서 구조를 복원
- Cycle이 있는 제어 구조와 cycle이 없는 제어 구조 (acyclic)에 대하여 아래와 같은 패턴을 바탕으로 구조를 자동으로 파악하여 고수준 문법을 적용

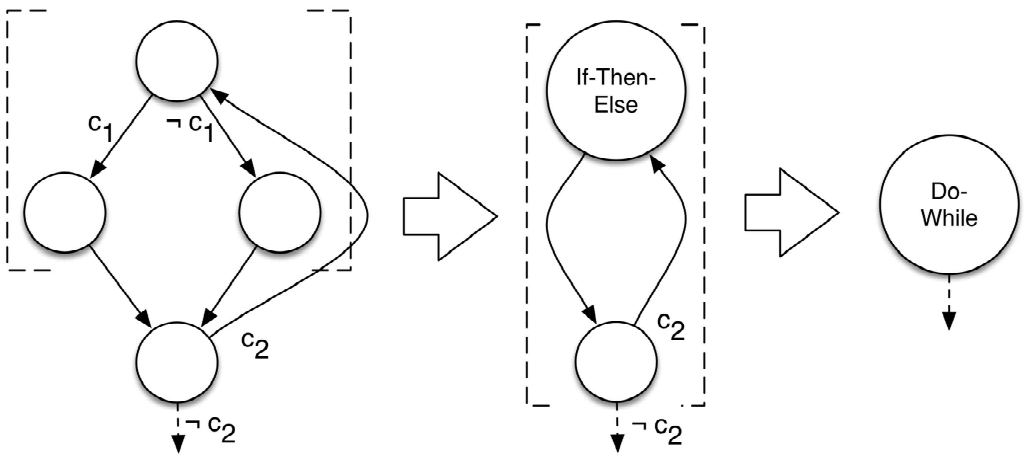


국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 55] 함수에 대한 Cycle의 유무 파악

- 이과정에서 수행되는 structure analysis에서는 CFG의 각 노드를 분리되어 있는 Region 노드로 변경
- Region 노드는 하나의 basic block을 포함하는 BB Region을 기반으로 하여, [그림 56]과 같이 특정한 control flow 패턴을 하나의 region으로 collapse 하여 그래프를 단순히 하는 과정을 반복해 가면 수행



[그림 56] control 구조를 포함하고 있는 region의 변경 과정

○ 고수준 스마트 컨트랙트의 문법(syntax) 변환

- 역공학된 스마트 컨트랙트의 결과를 더 이해하기 쉬운 고수준 문법으로 표현하기 위하여 변환기를 개발
- 복원한 control flow 정보를 고수준 언어인 Solidity의 해당 statement로 자동 변환
- Structural analysis의 결과에 해당하는 Region의 정보를 내부 statement와 함께 재귀적으로 탐색
- 위의 그림과 같은 과정을 통해서 분석된 구조는 다음과 같은 statement 구조로 변경

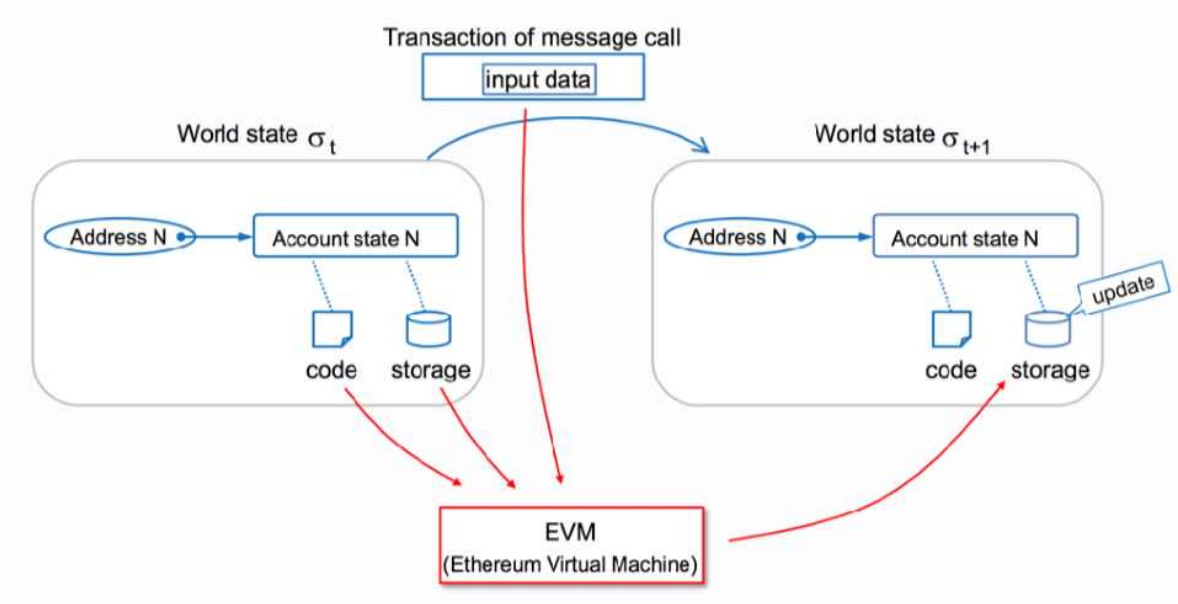
```
do {
        if (    ) { ... }
} while (   )
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 스마트 컨트랙트 취약점 분석기 개발

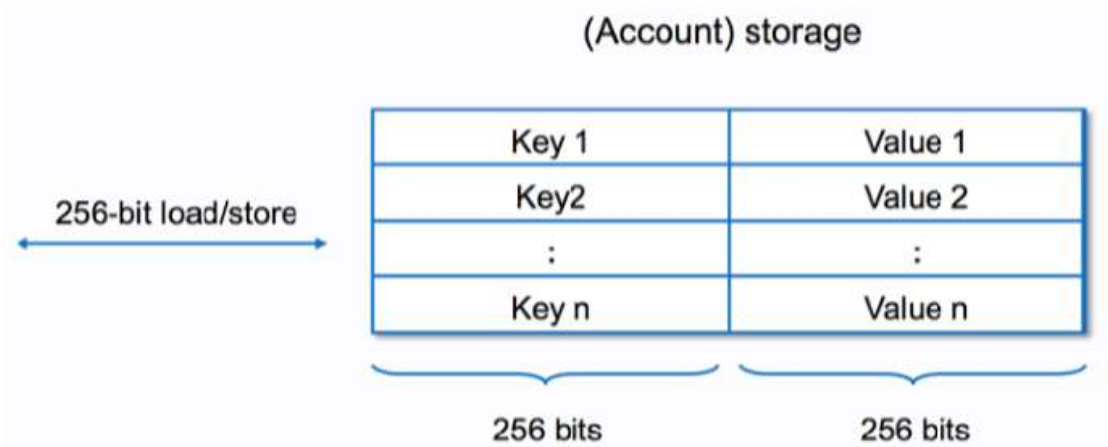
○ Storage Layout 문제점

- Ethereum의 경우 [그림 57]과 같이 storage가 존재하며 이는 데이터를 저장하는 저장 공간으로 사용됨



[그림 57] Ethereum의 state 변환 과정

- thereum의 경우 데이터를 영구적으로 저장할 수 있는 곳은 storage가 유일하며 다른 저장 공간은 임시적으로 데이터를 저장할 수 있음
- 따라서 Storage는 공격자의 타겟이 될 수 있으며, 스마트 컨트랙트는 storage를 활용하여 개발되기 때문에 storage를 정확하게 분석하는 것이 중요함
- Storage는 [그림 58]과 같이 256비트의 키에서 256 비트 값으로 맵핑된 구조를 가지고 있음



[그림 58] Storage의 구조

- Storage의 값을 읽을 시에는 SLOAD EVM 명령어를 사용하며, 값을 저장할 시에는 SSTORE EVM 명령어를 사용함
- Storage의 초기값은 0으로 셋팅
- EVM Bytecode 레벨에서 storage는 하나의 map으로 구성되어 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- [그림 59]는 Solidity와 EVM Bytecode에서 storage를 다루는 방법을 나타냄

```
pragma solidity ^0.4.25;
contract Something {
 uint var1;
 mapping(uint => uint) map1;
 mapping(uint => uint) map2;

 function func1 (uint a) public{
  map1[a] = 1;
  map2[a] = var1 + 2;
  var1 = var1 + 3;
 }
}
```

Solidity

```
pragma solidity ^0.4.25;
contract Something {

 function func1 (uint a) public{
  storage[SHA3(1 || a)] = 1;
  storage[SHA3(2 || a)] =
storage[0] + 2;
  storage[0] = storage[0] + 3;
 }
}
```

EVM Bytecode

[그림 59] high-level 언어와 low-level 언어의 storage 탐지 방법 비교

- High-level 언어인 Solidity에서는 map1 및 map2의 storage 변수가 명확히 구분
- Low-level 언어인 EVM Bytecode에서는 map1 및 map2가 모두 하나의 storage로 인식
- EVM bytecode에서 storage가 분리되어 있지 않아 분석기의 미탐 및 오탐이 발생

○ Oyente의 storage layout 처리 방법

- Oyente의 경우 storage의 index를 하나의 문자열로 지정하고, 문자열이 다른 경우 다른 storage로 인식하여 분석을 수행

```
pragma solidity ^0.4.25;
contract Something {

 function func1 (uint a, uint b) public{
  storage[SHA3(1 || a)] = 1;
  storage[SHA3(1 || b)] = storage[0] + 2;
  storage[0] = storage[0] + 3;
 }
}
```

storage[0]	3
storage["SHA3(1||a)"]	1
storage["SHA3(1||b)"]	2

[그림 60] Oyente에서의 stroage 처리 방법

- Oyente의 경우, storage index를 가리키는 "SHA3(1||a)", "SHA3(1||b)", 및 "0"을 문자열로 인식 후, 3개의 문자열이 모두 다르기 때문에 각각 다른 storage로 판단하여 분석을 수행
- [그림 60]에서 a와 b 변수의 값이 동일한 경우, 실제로 같은 storage를 가리키게 되지만, 문자열 방식으로 처리되는 Oyente의 경우 다른 storage로 판단하여 분석기에 오탐이 발생

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
pragma solidity ^0.4.25;
contract Something {

  function func1 (uint a) public{
    storage[SHA3(1 || a)] = 1;
    storage[SHA3(a || 1)] = storage[0] + 2;
    storage[0] = storage[0] + 3;
  }
}
```

storage[0]	3
storage["SHA3(1||a)"]	1
storage["SHA3(a||1)"]	2

[그림 61] Oyente에서의 오탐 발생 예제

- [그림 61]에서 a의 변수 값이 1인 경우, "SHA3(1¦¦a)" 및 "SHA3(a¦¦1)" 은 동일한 storage를 가리키나, Oyente의 경우 단순히 문자열로 비교하기 때문에 다른 위치의 storage를 가리키는 것으로 판단하여 분석을 수행하여 오탐이 발생

```
pragma solidity ^0.4.25;
contract Something {
  mapping(uint => mapping(uint => uint)) map1;

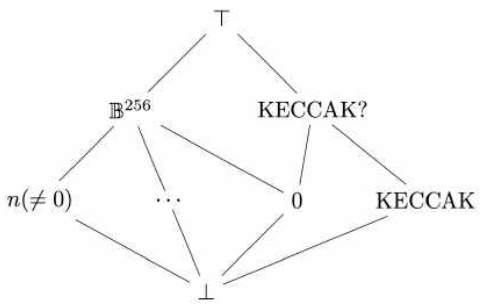
  function func1 (uint a, uint b) public{
    map1[a][0] = 1;
    map1[0][b] = 2;
    if (map1[a][0] + map1[0][b] == 4)
    {
       msg.sender.call.value(100)();
    }
  }
}
```

Cannot detects reentrancy for current version of Oyente

[그림 62] re-entrancy 취약점을 포함하고 있는 컨트랙트

- Oyente의 경우 storage 분석에 문제가 있어 [그림 62]의 컨트랙트에 존재하는 re-entrancy 취약점을 탐지하지 못하는 것을 확인

$$
\begin{array}{rcll}
\tau & = & \top & \text{Top} \\
 & | & \mathbb{B}^{256} & \text{256bit number} \\
 & | & n \in \mathbb{B}^{256} & \text{constant} \\
 & | & \text{KECCAK}[n, \tau_1, \tau_2, ...] & \text{hash consists of } \tau_1, \tau_2, \ldots \text{ plus } n \\
 & | & \text{KECCAK?}[n, \tau_1, \tau_2, ...] & \text{union of KECCAK and 0}
\end{array}
$$



$$m^{\#} := \Lambda_{(a,v)} Strong_{a,v} \vee \Lambda_{(a,v)} Weak_{a,v}$$

$Strong_{a,v} := (index = a \wedge |\gamma(index)| = 1) \Rightarrow value \sqsupseteq v,$
$Weak_{a,v} := index \sqsubseteq a \Rightarrow value \sqsupseteq v$로 정의한다.

[그림63] 요약 실행을 위한 요약 값 도메인

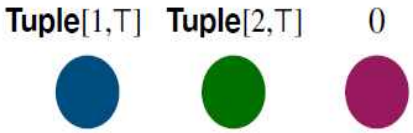
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

◦ Storage Layout 분리 기법

- 분석기의 정확도 향상을 위해 storage layout 분리 방법을 제안
- [그림 63]은 storage를 분리하여 요약 실행하기 위해 정의한 요약 값 도메인(abstract value domain)을 나타냄
- SHA256 연산의 경우 특별한 타입(KECCAK)을 만들어서 정확하게 분석할 수 있도록 정의
- 메모리 영역 및 스토리지 영역에서의 포인터 분석을 위한 도메인을 아래와 같이 정의
- Strong update의 경우, 한가지 값이 도출된 경우, 해당 값만 가져오는 것을 의미
- Weak update의 경우, index를 포함하는 모든 값을 join해서 가져오는 것을 의미

```
pragma solidity ^0.4.25;
contract Something {

 function func1 (uint a) public{
   storage[SHA3(1 || a)] = 1;
   storage[SHA3(2 || a)] = storage[0] + 2;
   storage[0] = storage[0] + 3;
 }
}
```

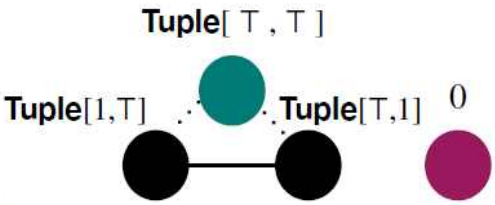


[그림 64] 교집합이 없는 경우의 분석 방법

- [그림 64]의 경우, storage 인덱스인 “SHA3(a¦¦a)”, “SHA3(2¦¦a)”, 및 “0“에 교집합이 존재하지 않아 3개의 다른 storage로 나누어 분석을 수행

```
pragma solidity ^0.4.25;
contract Something {

 function func1 (uint a) public{
   storage[SHA3(1 || a)] = 1;
   storage[SHA3(a || 1)] = storage[0] + 2;
   storage[0] = storage[0] + 3;
 }
}
```



[그림 65] 집합이 존재 하는 경우의 분석 방법(1)

- [그림 65]의 경우, “(1¦¦a)”, “(a¦¦1)“에 교집합이 존재하여 동일한 storage로 판단하여 분석을 수행

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

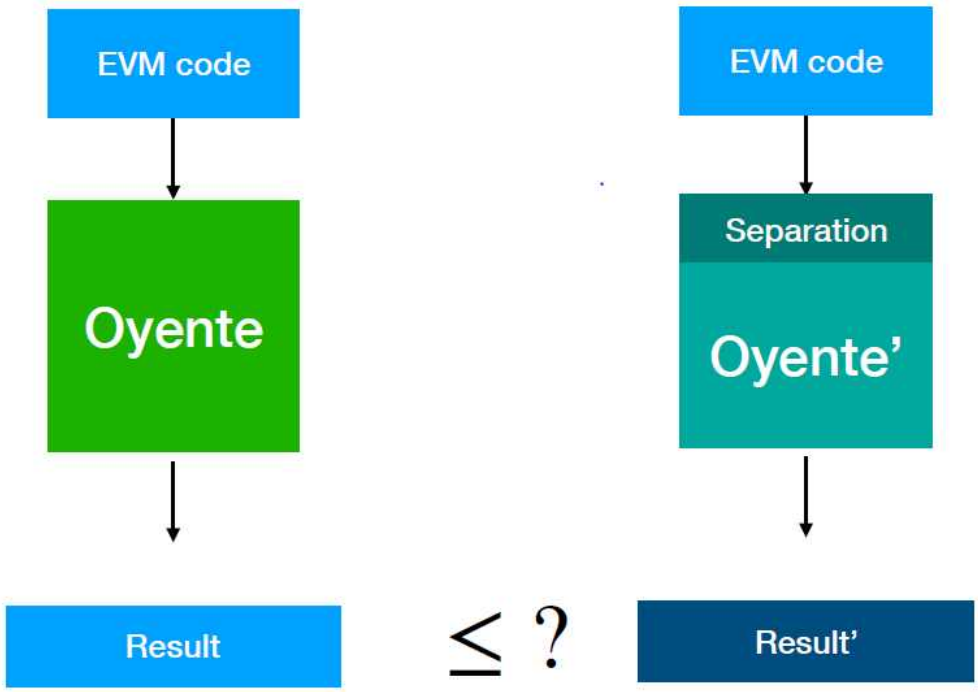
```
pragma solidity ^0.4.25;
contract Something {

  function func1 (uint a, uint b) public{
    storage[SHA3(1 || a)] = 1;
    storage[SHA3(1 || b)] = storage[0] + 2;
    storage[0] = storage[0] + 3;
  }
}
```



[그림 66] 교집합이 존재 하는 경우의 분석 방법(2)

- [그림 66]의 경우, “(1¦a)” , “(1¦b)“에 교집합이 존재하여 동일한 storage로 판단하여 분석을 수행



[그림 67] Oyente에 Speration 기능 추가

◦ Oyente 확장 개발

- Storage layout 분리 기법을 Oyente에 적용
- [그림 67]과 같이 Oyente에 Speration 기능을 추가하여 storage를 적절하게 분리하여 분석을 수행하도록 확장
- Oyente의 확장 부분에는 전 분석에서 얻어진 메모리 모양을 바탕으로 symbolic memory 분석을 추가 적용
- smtlib array model을 사용해 메모리에 접근할 때는 select, 저장할 때는 storage 함수를

사용해 symbolic 메모리 값을 읽거나 쓸 수 있도록 구현

```
#0: KECCAK(B256 || 0x1) => B256
#1: 0x0 => B256

00000: PUSH1 0x80 @ []
00002: PUSH1 0x40 @ []
…
00293: DUP3 @ [B256, 0x0, KECCAK(B256 || 0x1)]
00294: SLOAD @ [KECCAK(B256 || 0x1)] #0
00295: ADD @ [0x0, B256]
…
00300: SWAP1 @ [B256, KECCAK(B256 || 0x1)]
00301: SSTORE @ [KECCAK(B256 || 0x1), B256] #0
…
00311: SWAP1 @ [B256, 0x0]
00312: SSTORE @ [0x0, B256] #1
```

[그림 68] 분석을 통해 얻어진 메모리 모양

```
contract SimpleStorage {
    uint storedData;
    mapping (address => uint) balances;
    function set(uint x) public {
        storedData = x;
    }

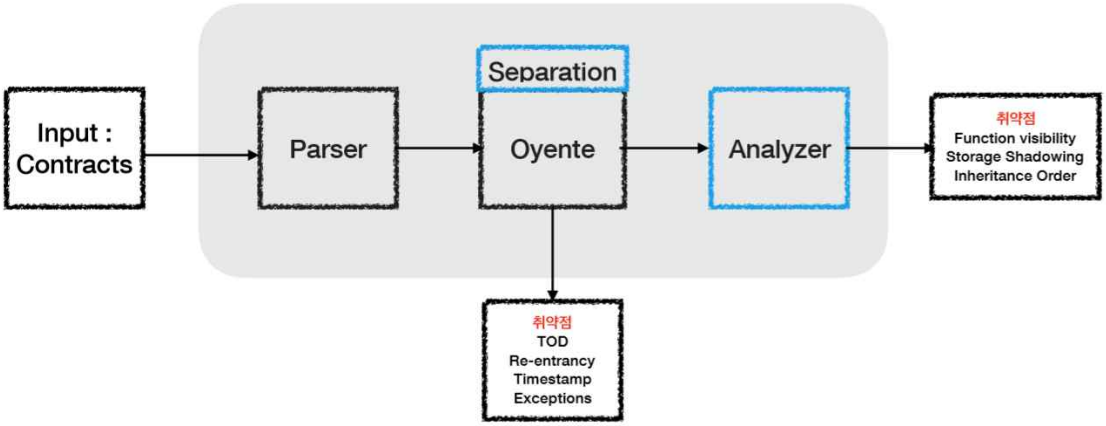
function mint(address receiver, uint amount)
public {
        balances[receiver] += amount;
    }
}
```

[그림 69] 분석을 통해 얻어진 Solidty 코드

- [그림 69]의 Solidity에서 확인할 수 있듯이 storedData 및 balances 2개의 storage 변수가 존재
- storedData는 set 함수에서 상수 x 값이 저장되며 이는 [그림 68] bytecode에서 312번째 줄에 해당하며, storage 파티션 #1번을 사용하고 있음
- balances는 mint 함수를 통해 값이 저장되며 이는 bytecode의 294 및 301번째 줄에 해당하며, storage 파티션 #0번을 사용하고 있음
- 2개의 storage 접근이 서로 다른 파티션을 사용하기 때문에 분석의 정확도가 향상
- 확장된 Oyente 결과
  - 기존 Oyente의 Code Coverage는 80.4%이며, 확장된 Oyente의 Code Coverage는 80.2%로 차이가 없음
  - 확장된 Oyente에서 기존 Oyente에서 탐지하지 못한 re-entrance 취약점을 18개의 컨트랙트에서 추가로 탐지

○ Oyente와 연동되는 분석기 개발

- Oyente의 경우 4개의 취약점(TOD, Re-entrancy, Timestamp, Mishandled Exception)을 탐지
- 4개의 취약점 외에 새로운 취약점 3개를 추가적으로 탐지하기 위한 분석기를 개발



[그림 70] Oyente와 연동되는 분석기 프레임워크

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 새로운 분석기는 [그림 70]과 같이 Oyente에서 탐지하는 취약점 외에 3가지 취약점(Function Visibility, Storage Shadowing, Inheritance Order)를 추가적으로 탐지
- 따라서 본 분석 프레임워크에서는 총 6가지의 취약점을 탐지하도록 개발
- Function Visibility 취약점은 문법적 분석을 수행하여 탐지하며, Storage Shadowing 및 Inheritance Order 취약점은 클래스 구조 분석을 통해 탐지
- Function Visibility 취약점
  · Function Visibility는 함수 접근범위를 지정
  · Solidity에서 Function Visibility는 public, external, private, internal로 설정가능
  · public 및 external로 설정된 function은 외부에서 호출 가능
  · Function Visibility를 선언하지 않은 경우 public으로 설정
  · 개발자 실수로 Function Visibility가 누락될 경우, public function이 되어 공격자의 공격 대상이 될 수 있음
  · 본 분석기는 보안에 취약한 함수가 Visibility가 누락되어 public 함수가 되는 경우를 탐지
- Storage Shadowing 취약점
  · Storage Shadowing 취약점은 부모 컨트랙트와 자식 컨트랙트에서 동일한 이름의 storage 변수가 존재할 경우 문제가 발생
  · 부모 컨트랙트에 존재하는 함수는 부모 컨트랙트에 선언된 storage를 접근
  · 자식 컨트랙트에 존재하는 함수는 자식 컨트랙트에 선언된 storage를 접근
  · 본 분석기는 동일한 이름의 storage 변수로 인해 컨트랙트의 기능이 충돌나는 경우를 탐지
- Inheritance Order 취약점
  · Inheritance Order 취약점의 경우 Solidity 문서의 오류로 인해 발생
  · 2018년 5월 16일 이전에 배포된 Solidity 문서에는 C3 Linearization 알고리즘을 사용하는 것으로 잘못 표기
  · 이후 버전의 Solidity 문서에는 Reverse of C3 Linearization 알고리즘을 사용하는 것으로 정정
  · Multiple Inheritance 상황에서 2개의 함수가 동일한 signature로 정의된 경우, C3 혹은 reverse of C3 linearization 알고리즘 적용에 따라 호출되는 함수가 다름
  · 따라서 어떤 알고리즘이 적용되는지에 따라 프로그램의 semantic이 변경
- 본 분석기는 C3 linearization 알고리즘 적용에 따라 프로그램의 semantic이 변경되는 경우를 탐지

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 분석기 결과

· 754개의 컨트랙트에서 Function visibility 취약점을 탐지

· 493개의 컨트랙트에서 Storage Shadowing 취약점을 탐지

· 475개의 컨트랙트에서 Inheritance Order 취약점을 탐지

ㅇ 스마트 컨트랙트 신규 취약점 발견

- 3개의 신규 스마트 컨트랙트 취약점 발견 후 CVE 발급

· CVE-2019-15078 : 생성자의 오타로 인한 컨트랙트 소유자 변경 취약점

· CVE-2019-15079 : 생성자의 오타로 인한 token 취득 취약점

· CVE-2019-15080 :생성자의 오타로 인한 컨트랙트 소유자 변경 취약점

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2-3. 3차년도

1) 스마트 컨트랙트 프라이버시 보호 기술 연구

□ 실행 환경에 따른 일반 산술회로를 위한 영지식 SNARK 프로토콜 최적화

○ SNARK 프로토콜의 사용자 측면의 성능의 개선을 위하여 사용자 실행환경에 특화된 트랜잭션 코드 집합을 위한 전용 SNARK 개발

- 증거를 생성하는 노드에 우선하여 검증역할을 하는 사용자가 수신하는 증거의 크기는 2단계에서 약 50% 수준으로 개선할 기법 제안하였고 3단계에서 구현
- 구현 결과의 용이한 적용을 위하여 IKOS 기법을 증거 생성자용과 증거 검증자용으로 분리하여 _IkosVariable4P와 _IkosVariabel4V로 설계
- IKOS 기법를 바탕으로 사용자의 편의가 개선된 연산검증이 가능한 ZKBoo 기법을 _OptimizedZKBoo로 구현
- 사용자 함수의 입력과 출력의 다양한 Data Type을 지원하기 위하여 C++의 template을 적용

○ 전용 SNARK 프로토콜의 시범구현 및 구현 수준 최적화

- 2단계에서 제안한 기법이 대칭키 암호기법을 적용하여 실제 실행시간의 개선에 초점이 있으나, 증거의 크기가 수백 KBytes에 이르는 문제가 있어 증거의 크기를 수십 Bytes로 개선할 수 있는 기법 제안하고 Rust 언어를 사용하여 구현

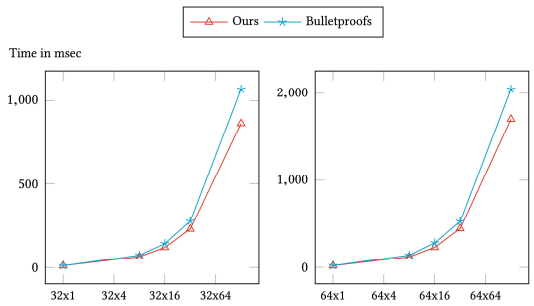


그림 71. 증거생성 시간 비교

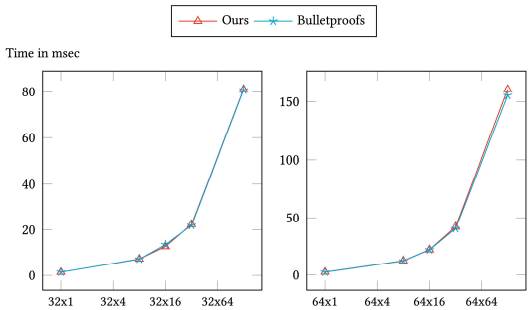


그림 72. 증거 검증 시간 비교

- 제안하는 기법을 Bulletproofs+로 명명
- 범용 SNARK 프로토콜 라이브러리화 및 시스템 통합
- 개발된 기법을 임의의 함수를 논리회로로 표현한 후 검증가능하도록 libzk4bc 형태로 개발하여 github에 등록

○ Bulletproofs 기법의 증거크기 개선[그림 71,72]

- Bulletproofs는 Bunz 등의 개발자들에 의하여 2018년 IEEE S&P [BBB+18]에 제안된 기법으로 범위증명에서 증거의 크기를 극단적으로 개선할 수 있는 방법을 제안
- 증거의 크기를 개선할 수 있는 방법을 논리회로난 산술회로의 검증에 적용할 수 있도록 확장하는 방법과 결합하여 제시
- Bunz 등의 Bulletproofs의 증거의 크기를 개선할 수 있는 아이디어의 핵심은 내적을 재귀적으로 적용한 반면, 본 연구진은 가중치 내적 (Weighted Inner Product)를 더욱 개선할 수 있는 방법을 적용한 Bulletproofs+를 제안
- Bulletproofs 기술 소개

(1) 표기법 정리

· $\mathbb{G}$는 위수가 p인 Diffie-Hellman 가정의 안전성이 보장되는 그룹이며 임의의 벡터 $\boldsymbol{a}=(a_1,\cdots,a_n)\in\mathbb{G}^n$이라 할 때 $\boldsymbol{a}_{[:\ell]}=(a_1,\cdots,a_\ell)\in\mathbb{G}^\ell$을 의미하며 $\boldsymbol{a}_{[\ell:]}=(a_{\ell+1},\cdots,a_n)$을

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

의미

- $k \in \mathbb{G}$일 때 k의 급수의 벡터는 $\boldsymbol{k^n} = (1, k, k^2, \cdots, k^{n-1})$을 의미
- 만약 $\boldsymbol{g} = (g_1, \cdots, g_n) \in \mathbb{G}^n$과 $\boldsymbol{a} \in \mathbb{Z}_p^n$가 주어질 때 $\boldsymbol{g^a} = \prod_{i=1}^{n} g_i^{a_i}$를 의미
- $c \in \mathbb{Z}_p$일 때 $\boldsymbol{b} = c_n \cdot \boldsymbol{a} = (ca_a, \cdots, ca_n)$을 의미하며 $\boldsymbol{a} \circ \boldsymbol{b}$는 Hadamard 곱을 의미한다. 마지막으로 $p(X) = \sum_{i=0} \boldsymbol{p_i} X^i \in \mathbb{Z}_p^n[X]$인 벡타다항식으로 다항식간의 내적은 다음과 같이 정의

$$\langle \boldsymbol{l}(X), \boldsymbol{r}(X) \rangle = \sum_{i=0}^{d} \sum_{j=0}^{i} \langle \boldsymbol{l_i}, \boldsymbol{r_j} \rangle X^{i+j}.$$

(2) 설정

- 다른 SANRK 기법들이 설정을 위하여 신뢰기관이 필요한 반면 Bulletproofs는 설정을 위한 신뢰기관을 요구하지 않음
- 참여자들이 전술한 위수 p의 그룹 $\mathbb{G}$를 생성하는데 동의한 후 다음 설정 알고리즘을 수행하여 $(g, h, \boldsymbol{g}, \boldsymbol{h})$를 공유함

Algorithm ComputeGenerators

Input. 공개된 문자열 σ, $g \in \mathbb{G}^n$, $n \in \mathbb{Z}$
Output. $(g, h, \boldsymbol{g}, \boldsymbol{h})$

```
Compute h = MapToGroup(σ, p)
i = 0
while i < n do
    Choose random c, d ∈ Z_p
    Compute g[i] = c · g and h[i] = d · g
    i = i + 1
return (g, h, g, h)
```

- 위 알고리즘을 수행하기 위하여 임의의 값을 그룹의 원소로 대응시키는 함수가 필요한데 다음과 같은 방법으로 수행

Algorithm MapToGroup

Input. 공개된 문자열 σ, 소수 p
Output. $h \in \mathbb{G}$

```
i = 0
while i < 2^8 do
    x = H(σ, i)                          /* H: 해시함수 */
    rhs = x^3 + 7 mod p
    if rhs is a square mod p then
        y = √rhs mod p
        if (x, y) not the point of infinity then
            return h = (x, y)
    i = i + 1
return ⊥
```

(3) 내적에 대한 효율적 증명

- 내적에 대한 증명이 Bulletproofs의 핵심으로 우선 위에서 설명한 설정과정을 통하여 $(g, h, \boldsymbol{g}, \boldsymbol{h})$를 구한 후 추가로 u = MapToGrpup(σ, p)를 계산하여 이것과 함께 $(g, h, \boldsymbol{g}, \boldsymbol{h}, u)$를 공개
- 이제 자신의 비밀 벡터값 $\boldsymbol{a}, \boldsymbol{b}$에 대하여 Pederson 위임기법을 적용하여 다음과 같이 P를 구하여 공개

$$P = \boldsymbol{g^a} \cdot \boldsymbol{h^b} \in \mathbb{G}$$

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 내적 $c = \boldsymbol{a} \cdot \boldsymbol{b}$에 대한 증명을 위하여 증거 생성자는 먼저 $x = H(\boldsymbol{g}, \boldsymbol{h}, P, c)$를 계산하고 $P' = u^{x \cdot c} P$를 이용하여 벡터 $\boldsymbol{l}, \boldsymbol{r}$을 구하여 증거를 생성하기 위한 알고리즘 ComputeProof($\boldsymbol{g}, \boldsymbol{h}, P, u^x, \boldsymbol{a}, \boldsymbol{b}, \boldsymbol{l}, \boldsymbol{r}$)을 이용하여 증거 π_{IP}를 구하여 공개
- 증거 π_{IP}는 재귀적으로 만들어지기 때문에 벡터의 길이 n에 대하여 $O(\log n)$의 크기를 갖는다. 증거 검증자는 다음 $\mathrm{Verify}_{\mathrm{IP}}$에 의하여 효율적으로 계산

Algorithm $\mathrm{Verify}_{\mathrm{IP}}$

Input. 소수 p, 위임값 P와 π_{IP}

Output. true or false

$i = 0$

while $n < \log n$ **do**

$n' = n/2$

Compute $x = H(\boldsymbol{l}[i], \boldsymbol{r}[i])$

Compute $\boldsymbol{g}' = \boldsymbol{g}_{[:n']}^{\boldsymbol{x}^{-1}} \boldsymbol{g}_{[n':]}^{\boldsymbol{x}}$ and $\boldsymbol{h}' = \boldsymbol{g}_{[:n']}^{\boldsymbol{x}} \boldsymbol{g}_{[n':]}^{\boldsymbol{x}^{-1}} \in \mathbb{G}^{n'}$

Compute $P' = L^{x^2} P R^{x^{-2}} \in \mathbb{G}$

$i = i + 1$

return true if $P = g^a h^b u^c$; false otherwise

Algorithm ComputeProof

Input. $\boldsymbol{g}, \boldsymbol{h}, P, u, \boldsymbol{a}, \boldsymbol{b}, \boldsymbol{l}, \boldsymbol{r}$

Output. π_{IP}

$x = H(\boldsymbol{g}, \boldsymbol{h}, P, c)$

Compute $P' = u^{x \cdot c} P$

if $n = 1$ **then**

return $(\boldsymbol{g}, \boldsymbol{h}, u, a, b, \boldsymbol{l}, \boldsymbol{r})$

else

$n' = n/2$

Compute $c_L = \langle \boldsymbol{a}_{[:n']}, \boldsymbol{b}_{[n':]} \rangle$ and $c_R = \langle \boldsymbol{a}_{[n':]}, \boldsymbol{b}_{[:n']} \rangle \in \mathbb{Z}_p$

Compute $L = \boldsymbol{g}^{\boldsymbol{a}_{[:n']}} \boldsymbol{h}^{\boldsymbol{b}_{[n':]}} u^{c_L}$ and $R = \boldsymbol{g}^{\boldsymbol{a}_{[n':]}} \boldsymbol{h}^{\boldsymbol{b}_{[:n']}} u^{c_R} \in \mathbb{G}$

Append L, R to $\boldsymbol{l}, \boldsymbol{r}$

$x = H(L, R)$

Compute $\boldsymbol{g}' = \boldsymbol{g}_{[:n']}^{\boldsymbol{x}^{-1}} \boldsymbol{g}_{[n':]}^{\boldsymbol{x}} \in \mathbb{G}^{n'}$ and $\boldsymbol{h}' = \boldsymbol{g}_{[:n']}^{\boldsymbol{x}} \boldsymbol{g}_{[n':]}^{\boldsymbol{x}^{-1}} \in \mathbb{G}^{n'}$

Compute $P' = L^{x^2} P R^{x^{-2}} \in \mathbb{G}$

Compute $\boldsymbol{a}' = \boldsymbol{a}_{[:n']} x + \boldsymbol{a}_{[n':]} x^{-1} \in \mathbb{Z}_p^{n'}$ and $\boldsymbol{b}' = \boldsymbol{b}_{[:n']} x^{-1} + \boldsymbol{b}_{[n':]} x \in \mathbb{Z}_p^{n'}$

ComputeProof($\boldsymbol{g}', \boldsymbol{h}', P', u, \boldsymbol{a}', \boldsymbol{b}', \boldsymbol{l}, \boldsymbol{r}$)

(4) 영지식 범위 증명

- 먼저 개념적으로 설명하면, 비밀값 v가 공개된 범위 $[0, 2^n)$에 속하는 값임을 증명하기 위한 것으로 먼저 $\boldsymbol{a_L} \in \{0,1\}^n$으로 v를 표현한 후 $\langle \boldsymbol{a_L}, \boldsymbol{2^n} \rangle = v$를 증명
- 이때 $\boldsymbol{a_R}$은 $\boldsymbol{a_L}$의 보수값을 갖는 비트벡터로 설정한다. 그러면 다음 두가지 특징을 가짐

$$\boldsymbol{a_L} \circ \boldsymbol{a_R} = \boldsymbol{0^n}$$
$$\boldsymbol{a_R} = \boldsymbol{a_L} - \boldsymbol{1^n} \pmod 2$$

- 이제 비밀값 v의 영지식 증명을 위해 임의의 난수 $y \in \mathbb{Z}_p$를 선택한 후 다음 값을 계산하면

$$\langle \boldsymbol{a_L}, \boldsymbol{a_R} \circ \boldsymbol{y^n} \rangle = 0$$
$$\langle \boldsymbol{a_L} - \boldsymbol{1^n} - \boldsymbol{a_R}, \boldsymbol{y^n} \rangle = 0$$

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

· 위 두식을 하나의 내적 증명으로 결합할 수 있는데, 임의의 난수 $z \in \mathbb{Z}_p$에 대하여 $\langle \boldsymbol{a_L} - z \cdot \mathbf{1^n}, \boldsymbol{y^n} \circ (\boldsymbol{a_R} + z \cdot \mathbf{1^n}) + z^2 \cdot \mathbf{2^n} \rangle = z^2 v + \delta(y,z)$ 임을 알 수 있다. 여기서 함수 $\delta(y,z) = (z - z^2)(\langle \mathbf{1^n}, \boldsymbol{y^n} \rangle - z^3 \langle \mathbf{1^n}, \mathbf{2^n} \rangle \in \mathbb{Z}_p$로 지정

· 문제는 증거 생성자가 벡터 $\boldsymbol{a_L}$을 전송하는 내적 증명이 영지식 증명이 아니므로 값이 드러나는 문제가 발생하여 다음과 같이 랜덤 벡터 $\boldsymbol{s_L}, \boldsymbol{s_R}$을 추가로 선택한 후 다항식 벡터에 내장하여 증명을 진행한다. 이때 사용되는 다항식 벡터는 3가지인데 차례대로 다음과 같음

$$
\begin{aligned}
l[X] &= \boldsymbol{a_L} - z \cdot \mathbf{1^n} + \boldsymbol{s_L} X \in \mathbb{Z}_p^n \\
r[X] &= \boldsymbol{y^n} \circ (\boldsymbol{a_R} + z \cdot \mathbf{1^n} + \boldsymbol{s_R} X) + z^2 \cdot \mathbf{2^n} \in \mathbb{Z}_p^n \\
t[X] &= \langle l[X], r[X] \rangle = t_0 + t_1 X + t_2 X^2
\end{aligned}
$$

여기서 계수 $t_1 = \langle \boldsymbol{a_L} - z \cdot \mathbf{1^n}, \boldsymbol{y^n} \cdot \boldsymbol{s_R} \rangle + \langle \boldsymbol{s_L}, \boldsymbol{y^n} (\boldsymbol{a_R} + z \cdot \mathbf{1^n}) \rangle$이고 $t_2 = \langle \boldsymbol{s_L}, \boldsymbol{y^n} \cdot \boldsymbol{s_R} \rangle$

· 정리하여 증거 생성자의 알고리즘 Prove_{RP}을 기술하면 다음과 같음

Algorithm Prove_{RP}

Input. 공개 파라미터 $(g, h, \boldsymbol{g}, \boldsymbol{h})$와 비밀값 v

Output. π_{RP}

Choose $\gamma \in \mathbb{Z}_p$ randomly

Compute $V = g^v h^\gamma \in \mathbb{G}$

Find $\boldsymbol{a_L} \in \{0,1\}^n$ such that $\langle \boldsymbol{a_L}, \mathbf{2^n} \rangle = v$

Find $\boldsymbol{a_R} = \boldsymbol{a_L} - \mathbf{1^n} \in \mathbb{Z}_p^n$

Choose $\alpha \in \mathbb{Z}_p$ randomly

Compute $A = h^\alpha \boldsymbol{g}^{\boldsymbol{a_L}} \boldsymbol{h}^{\boldsymbol{a_R}} \in \mathbb{G}$

Choose $\boldsymbol{s_L}, \boldsymbol{s_R} \in \mathbb{Z}_p^n$ randomly

Choose $\rho \in \mathbb{Z}_p$ randomly

Compute $S = h^\rho \boldsymbol{g}^{\boldsymbol{s_L}} \boldsymbol{h}^{\boldsymbol{s_R}} \in \mathbb{G}$

Find $y = H(A, S) \in \mathbb{Z}_p$

Find $z = H(A, S, y) \in \mathbb{Z}_p$

Choose $\tau_1, \tau_2 \in \mathbb{Z}_p$ randomly

Compute $T_1 = g^{t_1} h^{\tau_1} \in \mathbb{G}$

Compute $T_2 = g^{t_2} h^{\tau_2} \in \mathbb{G}$

Compute $x = H(T_1, T_2) \in \mathbb{Z}_p$

Compute $\boldsymbol{l} = l(X) = \boldsymbol{a_L} - z \cdot \mathbf{1^n} + \boldsymbol{s_L} X \in \mathbb{Z}_p^n$

Compute $\boldsymbol{r} = r(X) = \boldsymbol{y^n} \circ (\boldsymbol{a_R} + z \cdot \mathbf{1^n} + \boldsymbol{s_R} X) + z^2 \mathbf{2^n} \in \mathbb{Z}_p^n$

Compute $\hat{t} = \langle \boldsymbol{l}, \boldsymbol{r} \rangle \in \mathbb{Z}_p$

Compute $\tau_x = \tau_2 x^2 + \tau_1 x + z^2 \gamma \in \mathbb{Z}_p$

Compute $\mu = \alpha + \rho x \in \mathbb{Z}_p$

Compute a commitment cmt_{IP} to $(\boldsymbol{l}, \boldsymbol{r})$ using Pederson commitment

Compute π_{IP} using the Inner Product argument

Compute $\pi_{RP} = (\tau_x, \mu, \hat{t}, V, A, S, T_1, T_2, cmt_{IP}, \pi_{IP})$

return π_{RP}

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

· 증거를 생성하기 위한 과정이 복잡한 것에 비하여 증거를 검증하는 검증자 알고리즘은 상대적으로 간단하게 구현된다. Verify_{RP}는 다음과 같음

Algorithm Verify_{RP}

Input. 공개 파라미터 $(g,h,\boldsymbol{g},\boldsymbol{h})$와 π_{RP}

Output. true or false

Compute $y = H(A,S) \in \mathbb{Z}_p$

Compute $z = H(A,S,y) \in \mathbb{Z}_p$

Compute $x = H(T_1, T_2) \in \mathbb{Z}_p$

Compute $h_i = h_i^{y^{-i+1}} \in \mathbb{G}$ for all $i \in [1,n]$

Compute $P_l = Ph^{\mu}$

Compute $P_{r=A} \cdot S^x \cdot \boldsymbol{g}^{-\boldsymbol{z}} \cdot \boldsymbol{h}^{\boldsymbol{z} \cdot \boldsymbol{y^n} + z^2 \cdot \boldsymbol{2^n}} \in \mathbb{G}$

Set $b_1 = 1$ if $P_l = P_r$; otherwise $b_1 = 0$

Set $b_2 = 1$ if $g^{\hat{t}} h^{\tau_x} = V^{z^2} g^{\delta(y,z)} T_1^x T_2^{x^2}$; otherwise $b_2 = 0$

Run $b_3 = \mathrm{Verify}_{IP}(P, \pi_{IP})$

return $(b_1 \wedge b_2 \wedge b_3)$

- Bulletproofs+ 기술 소개

(1) 가중치 내적 기법

· 가중치를 갖는 내적을 사용하는 목적은 Bulletproofs의 단점 두가지를 해결하기 위한 것으로 두 가지 단점을 정리하면 다음과 같다:

i. Bulletproofs의 내적 증명은 영지식 증명이 아님

ii. 증거의 크기가 최적이 아님

· 위 두 가지 단점을 해결하기 위한 가중치내적 $\odot_{\boldsymbol{c}}$은 다음과 같이 정의된다. 먼저 $\boldsymbol{c}$를 상수 벡터라고 할 때 주어진 두 벡터 $\boldsymbol{a},\boldsymbol{b}$에 대하여

$$\odot_{\boldsymbol{c}} : \mathbb{Z}_p^n \times \mathbb{Z}_p^n \to \mathbb{Z}_p$$
$$(\boldsymbol{a},\boldsymbol{b}) \mapsto \langle \boldsymbol{a}, (\boldsymbol{c} \circ \boldsymbol{b}) \rangle$$

· 편의를 위해 $\odot_{\boldsymbol{y^n}}$ 대신 $\odot_y$를 쓰기로 하고, 가중치내적의 특징을 정리한다. 가장 먼저 $y=1$이면 가중치내적은 보통 내적과 동일하다. 만약 $y > 1$ 이며

$$\boldsymbol{a} \odot_y (\boldsymbol{b} \circ \boldsymbol{y^n}) = \boldsymbol{y}^{n+1} \cdot \langle \boldsymbol{a}, \boldsymbol{b} \rangle$$

이때 사용되는 상수값 y가 램덤하게 선택되면 내적증명을 영지식 증명으로 쉽게 변형할 수 있으며 또한 원소를 압축하여 저장하는 것이 가능해져, 증거의 길이를 개선하는 역할을 함

· 그래서 가중치 내적증명을 표현하면 다음임

$$PK\left\{(\boldsymbol{g},\boldsymbol{h} \in \mathbb{G}^n, g, h, P \in \mathbb{G}; \boldsymbol{a},\boldsymbol{b} \in \mathbb{Z}_p^n, \alpha \in \mathbb{Z}_p | P = \boldsymbol{g}^{\boldsymbol{a}} \boldsymbol{h}^{\boldsymbol{b}} g^{\boldsymbol{a} \odot_y \boldsymbol{b}} h^{\alpha}\right\}$$

· 설정과 위임기법은 Bulletproofs와 차이가 없기 때문에 생략하고 먼저 증거 생성자의 알고리즘 Prove_{WIP}는 다음과 같이 기술할 수 있다. 일반적으로 영지식증명은 비대면 방식으로 기술되나 이해의 편의를 위해 대면방식으로 작성한다. 알고리즘의 WIP는 Weighted Inner Product의 줄임말임

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

Algorithm Prove_{WIP}

Input. 공개 파라미터 $(g,h,\boldsymbol{g},\boldsymbol{h})$, 위임값 P, 비밀값 $(\boldsymbol{a},\boldsymbol{b},\alpha)$

Output. π_{WIP}

if $(n=1)$ then

Choose $r,s,\delta,\eta \in \mathbb{Z}_p$ randomly

Compute $A = \boldsymbol{g}^r \boldsymbol{h}^s g^{r \odot_y \boldsymbol{b} + s \odot_y \boldsymbol{a}} h^{\delta} \in \mathbb{G}$

Compute $B = g^{r \odot_y s} h^{\eta} \in \mathbb{G}$

Send A,B to the verifier and receive e from it

Compute $r' = r + \boldsymbol{a} \in \mathbb{Z}_p$, $s' = s + \boldsymbol{b}e \in \mathbb{Z}_p$ and $\delta' = \eta + \delta e + \alpha e^2 \in \mathbb{Z}_p$

Send (r',s',δ') to the verifier

else

Set $\hat{n} = n/2, \boldsymbol{a} = (\boldsymbol{a}_1,\boldsymbol{a}_2), \boldsymbol{b} = (\boldsymbol{b}_1,\boldsymbol{b}_2), \boldsymbol{g} = (\boldsymbol{g}_1,\boldsymbol{g}_2), \boldsymbol{h} = (\boldsymbol{h}_1,\boldsymbol{h}_2)$

Choose $d_L, d_R \in \mathbb{Z}_p$

Compute $c_L = \boldsymbol{a}_1 \odot_y \boldsymbol{b}_2 \in \mathbb{Z}_p$ and $c_R = (\boldsymbol{y}^{\hat{n}} \boldsymbol{a}_2) \odot_y \boldsymbol{b}_1 \in \mathbb{Z}_p$

Compute $L = \boldsymbol{g}_2^{y^{-\hat{n}} \boldsymbol{a}_1} \boldsymbol{h}_1^{\boldsymbol{b}_2} g^{c_L} h^{d_L} \in \mathbb{G}$ and $R = \boldsymbol{g}_1^{y^{\hat{n}} \boldsymbol{a}_2} \boldsymbol{h}_2^{\boldsymbol{b}_1} g^{c_R} h^{d_R} \in \mathbb{G}$

Send L,R and receive e from the verifier

Compute $\hat{\boldsymbol{g}} = \boldsymbol{g}_1^{e^{-1}} \circ \boldsymbol{g}_2^{e \cdot y^{-\hat{n}}} \in \mathbb{G}^{\hat{n}}$

Compute $\hat{\boldsymbol{h}} = \boldsymbol{h}_1^{e} \circ \boldsymbol{h}_2^{e^{-1}} \in \mathbb{G}^{\hat{n}}$

Compute $\hat{P} = L^{e^2} R^{e^{-2}} \in \mathbb{G}$

Compute $\hat{\boldsymbol{a}} = \boldsymbol{a}_1 \cdot e + (\boldsymbol{a}_2 \cdot y^{\hat{n}}) \cdot e^{-1} \in \mathbb{Z}_p^{\hat{n}}$

Compute $\hat{\boldsymbol{b}} = \boldsymbol{b}_1 \cdot e^{-1} + \boldsymbol{b}_2 \cdot e \in \mathbb{Z}_p^{\hat{n}}$

Compute $\hat{\alpha} = d_L \cdot e^2 + \alpha + d_R \cdot e^{-2} \in \mathbb{Z}_p$

Recursively run $\mathrm{Prove}_{WIP}(\hat{\boldsymbol{g}},\hat{\boldsymbol{h}},g,h,\hat{P},\hat{\boldsymbol{a}},\hat{\boldsymbol{b}},\hat{\alpha})$

Algorithm Verify_{WIP}

Input. 공개 파라미터 $(g,h,\boldsymbol{g},\boldsymbol{h})$, 증거값 π_{WIP}

Output. true or false

if $(n=1)$ then

Choose $e \in \mathbb{Z}_p$ randomly

Send e to the prover

return true if

$$P^{e^2} A^{e} B = \boldsymbol{g}^{r' \cdot e} \boldsymbol{h}^{s' \cdot e} g^{r' \odot_y s'} h^{\delta'} \in \mathbb{G}$$

else

Choose $e \in \mathbb{Z}_p$ randomly

Send e to the prover

Compute $\hat{\boldsymbol{g}} = \boldsymbol{g}_1^{e^{-1}} \circ \boldsymbol{g}_2^{e \cdot y^{-\hat{n}}} \in \mathbb{G}^{\hat{n}}$

Compute $\hat{\boldsymbol{h}} = \boldsymbol{h}_1^{e} \circ \boldsymbol{h}_2^{e^{-1}} \in \mathbb{G}^{\hat{n}}$

Compute $\hat{P} = L^{e^2} R^{e^{-2}} \in \mathbb{G}$

Recursively run with $\mathrm{Prove}_{WIP}(\hat{\boldsymbol{g}},\hat{\boldsymbol{h}},g,h,\hat{P},\hat{\boldsymbol{a}},\hat{\boldsymbol{b}},\hat{\alpha})$

· 증거를 생성하는 알고리즘 Prove_{WIP}는 Bulletproofs의 증거 생성 알고리즘과 동일하게 재귀적으로 호출하면서 증거를 생성하기 때문에 전체 증거의 크기는 $O(\log n)$임

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

을 알 수 있으며 상수 벡터 $\boldsymbol{y}$의 사용에 의해서 연산이 조금 증가하는 단점이 있으나, 이러한 단점은 구현의 최적화 과정과 배치 연산을 병행하여 적용함으로써 해결할 수 으며, 이제 증거 검증자의 알고리즘 Verify_{WIP}는 위와 같음

- 재귀적으로 수행되는 가중치내적 연산 $\hat{\boldsymbol{a}}\odot_y\hat{\boldsymbol{b}}$을 좀 더 자세히 살펴보면 위의 정직한 증거 생성자가 옳게 동작함을 알 수 있음

$$\begin{aligned}\hat{\boldsymbol{a}}\odot_y\hat{\boldsymbol{b}} &= \boldsymbol{a}_1\odot_y\boldsymbol{b}_2e^2+\boldsymbol{a}_1\odot_y\boldsymbol{b}_1+(y^{\hat{n}}\cdot\boldsymbol{a}_2)\odot_y\boldsymbol{b}_2+y^{\hat{n}}\boldsymbol{a}_2\odot_y\boldsymbol{b}_1e^{-2}\\ &= c_L\cdot e^2+c+c_R\cdot e^{-2}\end{aligned}$$

(2) 가중치 내적 영지식 증명기반 범위 영지식 증명

- Bulletproofs의 경우와 동일한 방식으로 가중치 내적증명을 Sub-Protocol로 사용하여 영지식 범위증명을 설계할 수 있다. 그러나 Bulletproofs에 비교할 때 범위증명을 훨씬 간단하게 구성할 수 있는데, 그 이유는 Sub-Protocol로 사용하는 가중치 내적증명이 이미 영지식을 달성하기 때문에 Bulletproofs와 범위증명에서 영지식을 달성하기 위한 별도의 연산을 요구하지 않는다. 알고리즘의 설명에 사용된 기호는 Bulletproofs와 동일함
- 또한 여러 개의 값에 대한 범위증명도 가능한데, 이러한 기능은 Bulletproofs에서도 제공된 기능이다. 단일값 v의 범위증명이 아래와 같은 반면

$$PK\{\boldsymbol{g},\boldsymbol{h}\in\mathbb{G}^n,g,h,V\in\mathbb{G};v,\gamma\in\mathbb{Z}_p|\,V=g^vh^\gamma\wedge v\in[0,2^n)\}$$

배치 범위증명은 m개의 값 $v_1,\cdots,v_m$이 모두 $[0,2^n)$의 범위에 포함된다는 것을 한번에 증명하기 위한 것으로 확장하는 방법은 Bulletproofs와 동일하기 때문에 자세한 설명은 생략

- 편의를 위하여 제안하는 범위증명 기법을 증거 생성자와 증거 검증자를 병합하여 기술한다. 여기서 $\mathbb{P}$는 증거 생성자, $\mathbb{V}$는 증거 검증자를 나타냄

Protocol $\mathrm{zk-Range}$	
Prover's Input. 공개 파라미터 $(g,h,\boldsymbol{g},\boldsymbol{h})$과 위임값 V 및 비밀값 v,γ	
Verifier's Input. 공개 파라미터 $(g,h,\boldsymbol{g},\boldsymbol{h})$	
Output. true or false	
$\mathbb{P}$	Choose $\alpha\in\mathbb{Z}_p$ randomly
	Set $\boldsymbol{a}_L$ such that $\langle\boldsymbol{a}_L,\boldsymbol{2}^n\rangle=v$ and $\boldsymbol{a}_R=\boldsymbol{a}_L-\boldsymbol{1}\in\mathbb{Z}_p^n$
	Compute $A=\boldsymbol{g}^{\boldsymbol{a}_L}\boldsymbol{h}^{\boldsymbol{a}_R}h^\alpha\in\mathbb{G}$
$\mathbb{P}\rightarrow\mathbb{V}$	Send A to the verifier $\mathbb{V}$
$\mathbb{V}$	Choose $y,z\in\mathbb{Z}_p$ randomly
$\mathbb{P}\leftarrow\mathbb{V}$	Send y,z to the prover $\mathbb{P}$
$\mathbb{P}$ & $\mathbb{V}$	Compute $\hat{A}=A\cdot\boldsymbol{g}^{-\boldsymbol{1}^n\cdot z}\boldsymbol{h}^{\boldsymbol{2}^n\circ\boldsymbol{y}^n+\boldsymbol{1}^n\cdot z}V^{y^{n+1}}g^{\langle\boldsymbol{1}^n,\boldsymbol{y}^n\rangle\cdot z-\langle\boldsymbol{1}^n,\boldsymbol{2}^n\rangle\cdot\boldsymbol{y}^{n+1}z-\langle\boldsymbol{1}^n,\boldsymbol{y}^n\rangle\cdot z^2}$
$\mathbb{P}$	Compute $\hat{\boldsymbol{a}}_L=\boldsymbol{a}_L-\boldsymbol{1}^n\cdot z\in\mathbb{Z}_p^n$
	Compute $\hat{\boldsymbol{a}}_R=\boldsymbol{a}_R+\boldsymbol{2}^n\circ\boldsymbol{y}^n+\boldsymbol{1}^n\cdot z\in\mathbb{Z}_p^n$
	Compute $\hat{\alpha}=\alpha+\gamma\cdot\boldsymbol{y}^{n+1}\in\mathbb{Z}_p$
$\mathbb{P}$	Run $\mathrm{Prover}_{WIP}(\boldsymbol{g},\boldsymbol{h},g,h,\hat{A};\hat{\boldsymbol{a}}_L,\hat{\boldsymbol{a}}_R,\hat{\alpha})$
$\mathbb{V}$	Run $\mathrm{Verifier}_{WIP}(\boldsymbol{g},\boldsymbol{h},g,h,\hat{A};\pi_{WIP})$

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(3) 구현의 최적화

- Bulletproofs와 마찬가지로 Bulletproofs+ 역시 다양한 modulus p 연산이 사용되고 있으나 특히 지수연산이 연산의 비용이 가장 비싸다. 이러한 문제를 해결하기 위한 방법으로 Multi-Exponentiation 기법이 널리 적용됨
- 임의의 벡터 $\boldsymbol{g}=(g_1,\cdots,g_n)$, $\boldsymbol{h}=(h_1,\cdots,h_n)$가 주어진 경우 다음과 같은 연산이 필요

$$g=\prod_{i=1}^{n} g_i^{s_i}\in\mathbb{G}$$
$$h=\prod_{i=1}^{n} h_i^{1/s_i}\in\mathbb{G}$$

여기에서 지수 s_i는 다시 다음과 같이 다시 작성할 수 있음:

$$s_i=\prod_{j=1}^{\log n} x_j^{b(i,j)}$$

그리고 함수 $b(i,j)$는

$$b(i,j)=\begin{cases}1 & \text{if the j}-\text{th bit of } i-1 \text{ is 1}\\ -1 & \text{otherwise}\end{cases}$$

- 그러면 검증에 다음과 같은 식을 적용하면 n^2 번의 지수연산을 수행하는 대신 $O(n\log n)$ 번의 지수연산으로 검증을 완료할 수 있음

$$\boldsymbol{g}^{a\cdot s}\boldsymbol{h}^{b\cdot s^{-1}}u^{a\cdot b}=P\cdot\prod_{j=1}^{\log n} L_j^{x_j^2}R_j^{x_j^{-2}}$$

- 동일한 기법을 Bulletproofs+에도 적용할 수 있는데, 먼저 다음과 같이 s_i, s_i'을 계산한다, 여기서 $b(i,j)$는 위와 같음

$$s_i=(1/y^{i-1})\prod_{j=1}^{\log n} e_j^{b(i,j)},\quad s_i'=\prod_{j=1}^{\log n} e_j^{-b(i,j)}$$

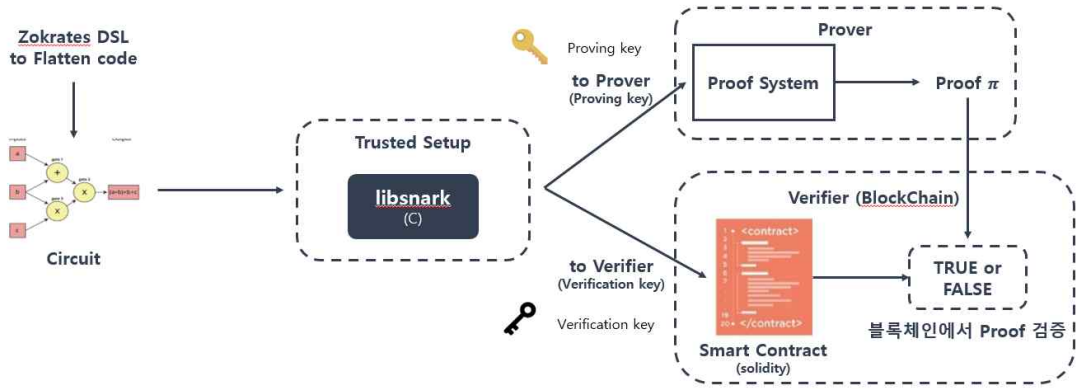
- 이렇게 표현한 후 연산으로 검증을 효율적으로 수행할 수 있음

$$\begin{aligned}\boldsymbol{g}^{r'e\cdot\boldsymbol{s}}\boldsymbol{h}^{s'e\cdot\boldsymbol{s}'}g^{r'\odot s'}h^{\delta'} &= \left(P\cdot\prod_{j=1}^{\log n} L_j^{e_j^2}R_j^{e_j^{-2}}\right)\cdot A^e\cdot B\\ &= P^{e^2}\cdot\left(\prod_{j=1}^{\log n} L_j^{e^2\cdot e_j^2}R_j^{e^2\cdot e_j^{-2}}\right)\cdot A^e\cdot B\end{aligned}$$

□ 블록체인에서 프라이버시를 지키기 위한 재작성 도구 개발

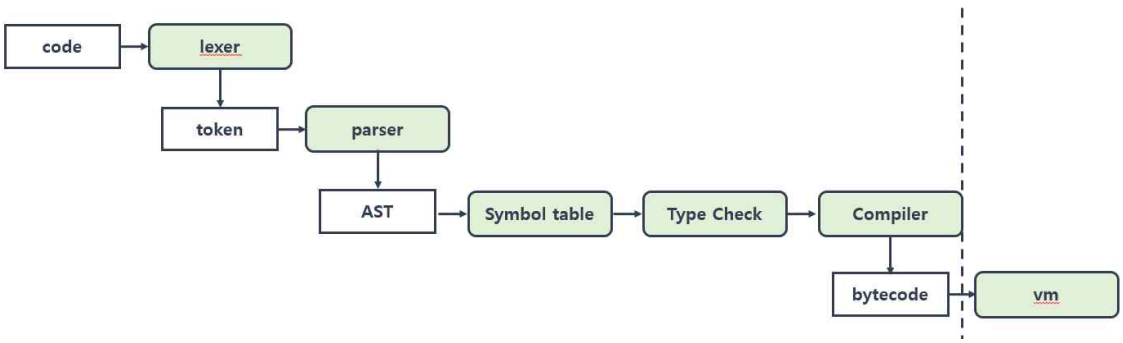
○ Solidity 코드 재작성

- 스마트 컨트랙트를 이용하는 블록체인 플랫폼 중 하나인 Ethereum에서는 Solidity라는 언어를 사용한다. 기존의 EVM을 수정하지 않고 새로운 기능을 적용하기 위해서는 Solidity의 기능을 이용해야 함
- 때문에 개발자들이 쉽게 새로운 기능을 사용할 수 있도록 기능을 지원하기 위해서는 새로운 언어 또는 프레임워크를 지원하여 Solidity로 변환해주는 재작성 도구를 이용해야 함



[그림 73] ZoKrates의 사용 방법

- ZoKrates는 [그림 73]처럼 python like Domain Specific Language(DSL)을 지원하여 작성한 코드를 영지식 증명을 적용할 수 있도록 기능을 지원
- Verifiable Computation의 영지식 증명을 통해 자신의 Local 환경에서 작업한 내용을 on-chain 상에서 직접 실행하지 않더라도 값을 검증
- 이를 통해 privacy가 필요한 값의 연산을 Local 환경에서만 실행하여 사용자의 프라이버시를 보장
- 사용자는 Local 환경에서 실행한 결과로 실행을 검증하기 위한 Proof를 생성하고 이를 검증하여 자신의 연산이 올바르게 수행되었음을 검증
- 이 검증을 위한 함수를 solidity로 구현하여 on-chain에 스마트 컨트랙트를 deploy하게 되면 블록체인 상에서 Proof을 검증할 수 있다. 이 경우 Local 환경에서 실행한 값을 on-chain에서 검증할 수 있는 시스템을 가짐
- 그러나 ZoKrates에 사용된 영지식 증명 알고리즘인 Pinocchio 알고리즘은 검증 비용과 proof 크기가 작다는 장점을 갖지만 Circuit마다 신뢰기관의 Setup이 필요하다는 Trusted Setup문제가 발생할 수 있음

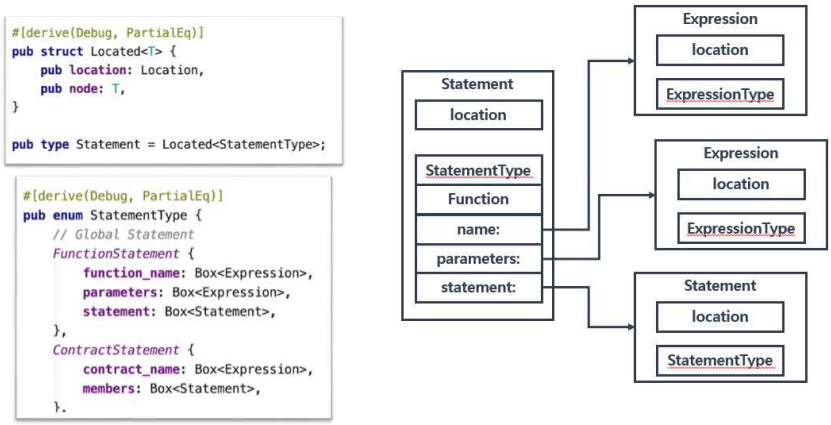


[그림 74] zoker 구현 방법

- Trusted Setup이 없을 경우 Setup을 수행한사람이 실제 올바른 연산을 수행하지 않더라도 Proof의 검증 결과가 참이 되는 값을 만들어낼 수 있음
- 이를 막기 위해 수원대에서 ZKBOO를 개선한 Trusted Setup이 필요 없는 알고리즘을

이용하여 Trusted Setup 없이 개발자가 쉽게 영지식 증명을 스마트컨트랙트에서 이용할 수 있도록 함

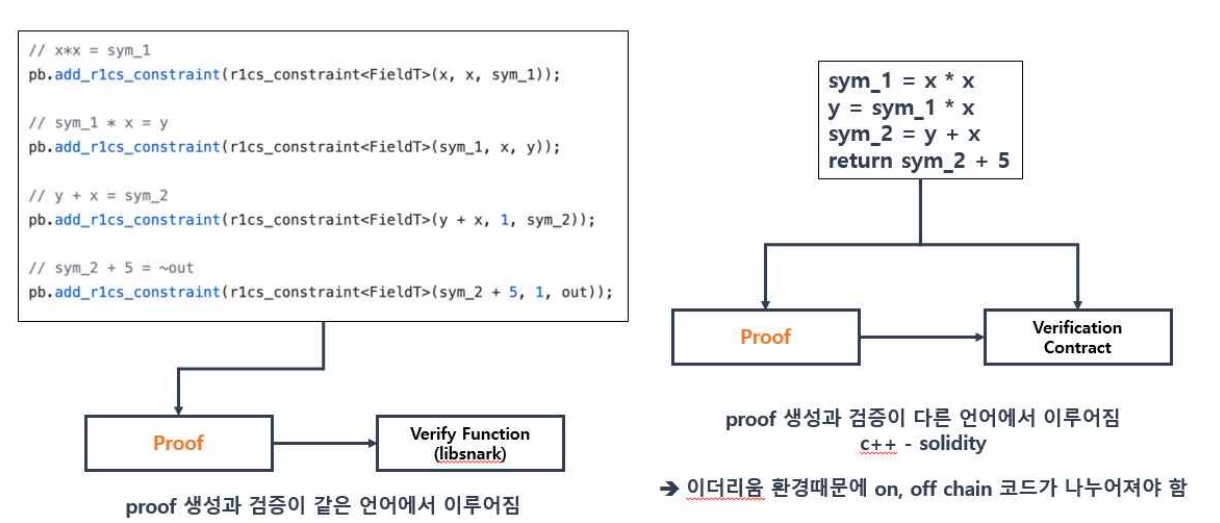
- 재작성 도구를 구현하기 위해서는 컴파일러를 구현하는 것과 비슷한 작업을 수행한다. [그림 74]와 같이 임의의 코드를 받아 Lexer를 통해 Token으로 분리하고 Token을 통해 문법을 parsing하는 parser를 구현한다. Parser에서는 Abstract Syntax Tree(AST)를 만들어내고 이 AST를 통해 Symbol Table을 만들어 Type Check, 컴파일을 하여 bytecode를 생성한 후 vm에서 이 값을 읽어 영지식 증명을 적용하는 스마트 컨트랙트를 반환
- 재작성 도구는 컴파일 속도와 proof을 생성하는 속도와 메모리 안전성을 고려하여 Rust 언어로 구현
- 컴파일에 사용하는 라이브러리는 LR Parser인 LALRPOP 라이브러리를 이용하여 컴파일하여 AST를 읽음



[그림 75] Zoker AST

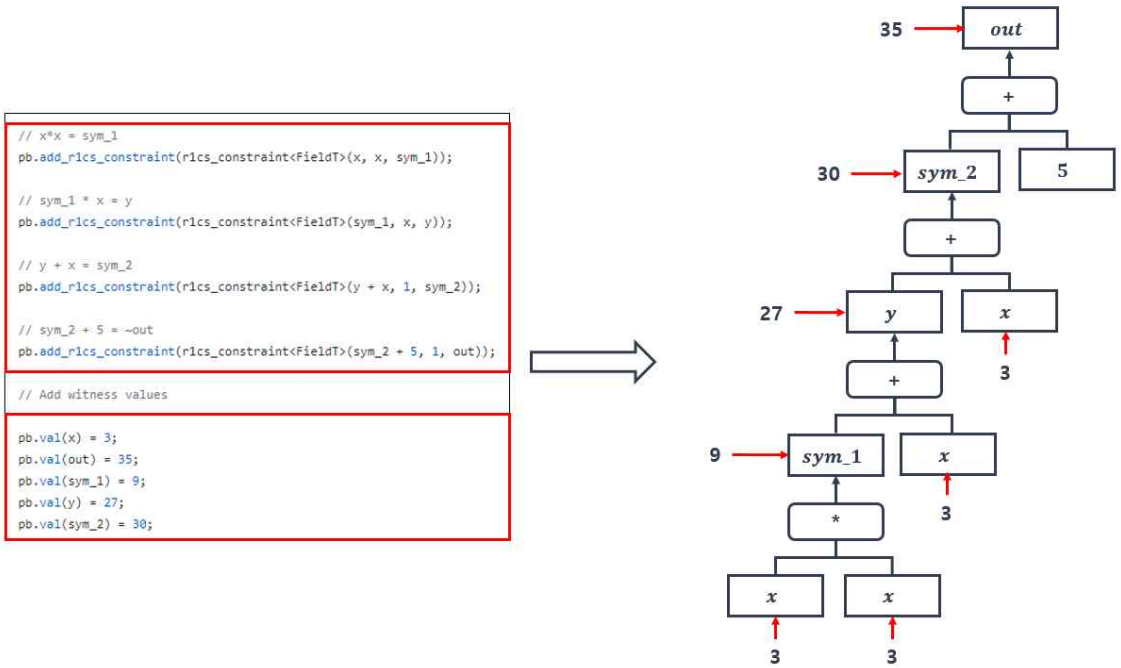
- AST는 [그림 75]에서 소개하듯이 Rust의 enum을 통해 구현하였으며 expression과 statement의 값을 내부 값으로 가지며 tree 형태로 값을 읽어나갈 수 있도록 구현
- AST에서는 Statement와 Expression으로 타입을 나누며 값을 상위 AST로 전달하는 경우 Expression, 전달하지 않는 경우 Statement로 지정
- Statement는 클래스, 함수 정의, 변수 선언 및 초기화 등이 있고, Expression으로는 변수 및 Binary 연산, Unary 연산, if 연산과 for 연산 등이 존재
- If 연산 같은 경우 If, else문에서 마지막에 semi colon을 지정하지 않은 값을 상위 AST로 전달
- 프로그래밍 언어에서 영지식 증명을 적용하기 위해 가장 중요한 것은 Jump문을 제거하는 것으로 if문과 for문을 flatten하는 작업을 진행
- if 문은 조건문 값과 연산 결과를 곱연산 하는 것으로 jump문 없이 계산할 수 있음
- for 문 또는 while loop는 flatten하는 개수마다 값이 달라지면 안되기 때문에 일정 회수의 정해진 반복만을 허용해야한다. 때문에 동적인 loop를 지원하기 위해서는 최대 반복 회수를 지정하고 그 안에서 반복할 수 있도록 해야함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 76] libSNARK과 재작성 도구의 차이점

- libSNARK는 [그림 76]와 같이 같은 언어에서 proof 생성과 검증을 함께 실행한다.
- 하지만 재작성 도구를 이용하는 경우 proof 생성은 재작성 도구를 구현한 언어 (Rust, C++, python 등)을 이용하지만 검증은 on-chain에서 사용하기 위한 solidity를 이용
- 때문에 구현할 때 검증 로직을 Solidity로 포팅하여 구현
- 영지식 증명을 적용할 때에는 libSNARK과 같은 라이브러리를 이용해 r1cs와 같은 circuit 생성을 위한 함수를 이용



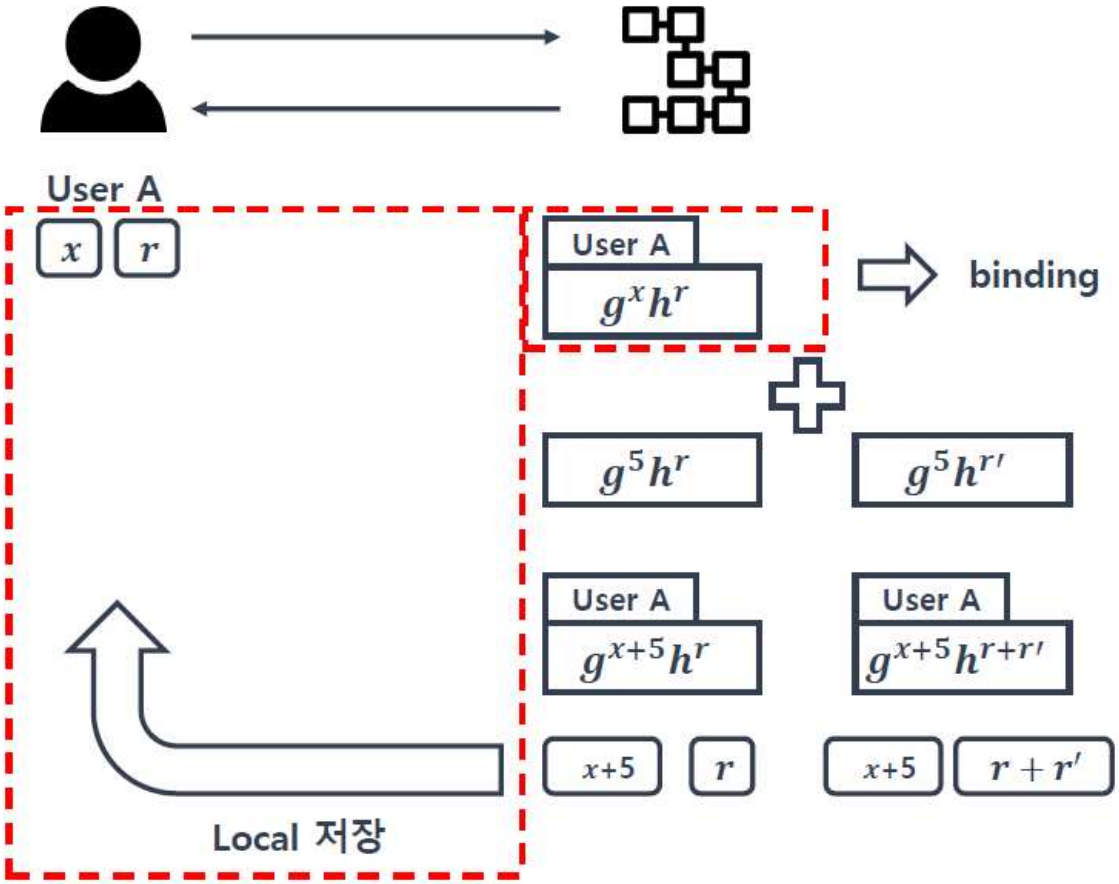
[그림 77] r1cs 적용

- R1CS(Rank 1 Constraint System)은 곱셈 연산으로 이루어진 산술회로를 영지식 증명을 적용할 때 사용하는 방법으로 프로그래밍 언어의 대부분의 구현이 산술 연산을 통해 이루어짐을 이용
- 프로그래밍 언어의 산술 연산을 [그림 77]와 같이 R1CS의 gate로 변환하여 실행하는 모

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

든 연산을 R1CS를 통해 검증되도록 만든다. 이후 R1CS에서 사용된 변수의 값을 대입하여 연산이 옳게 수행되었음을 확인하기 위한 proof을 생성

- VM(Virtual Machine)에서 컴파일 단계를 거친 후 생성된 bytecode를 읽어 연산을 R1CS를 적용하게 되면 DSL을 이용하는 개발자들에게 영지식 증명을 사용할 수 있도록 만들 수 있음



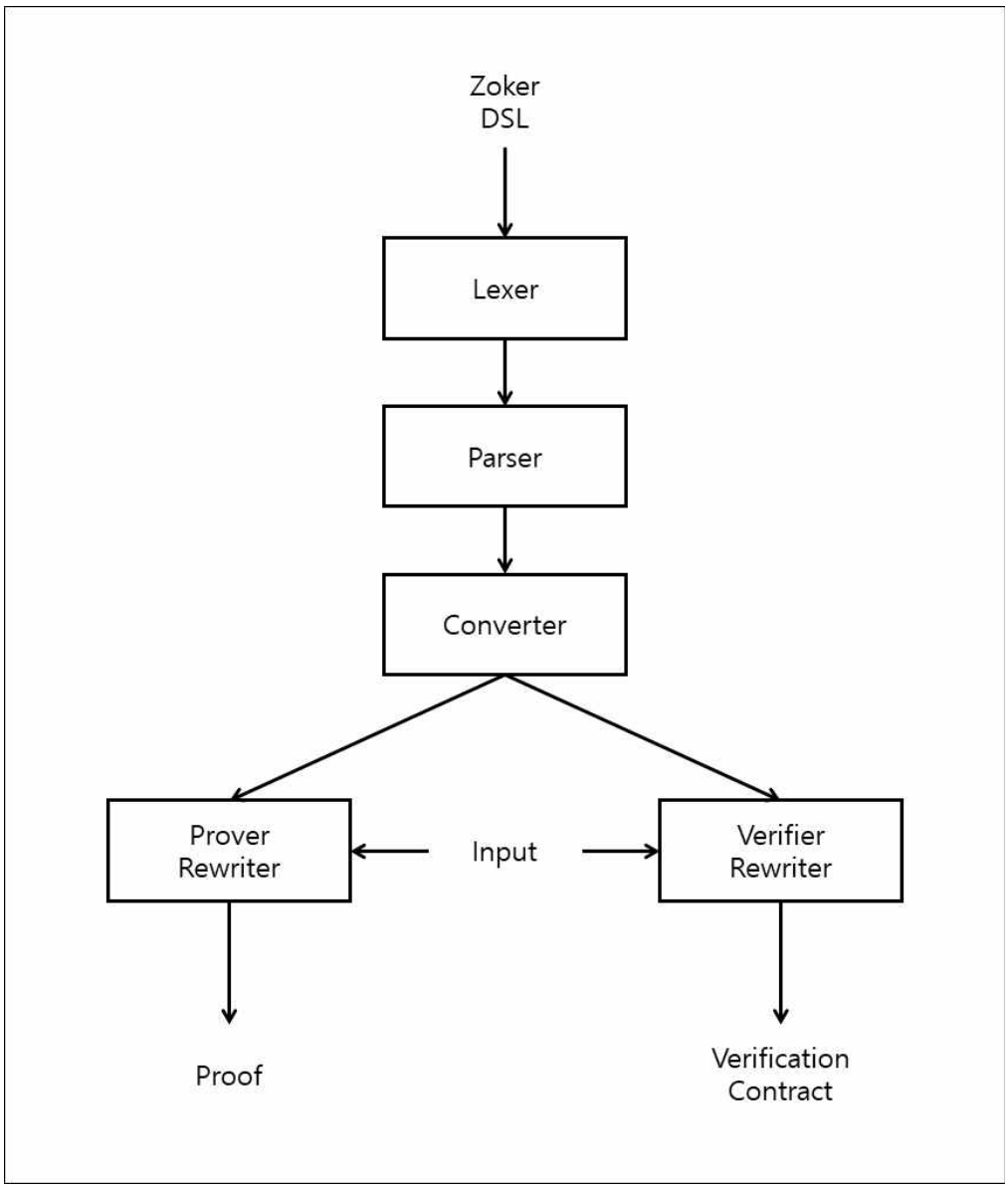
[그림 78] Pedersen Commitment

- 한편, 프라이버시를 위해 기존의 동형암호 계산처럼 하나의 generator만을 두고 지수연산을 할 경우 brute force 공격에 취약할 수 있다. 이를 막기 위해 generator를 두 값을 두고 random 값을 통해 brute force를 이용해 같은 결과를 만들어내는 값을 구할 수 있더라도 실제 x값이 무슨 값인지 추측하기 어렵게 할 수 있다. 이러한 commitment 방법을 [그림 78]과 같은 pedersen commitment라고 함
- pedersen commitment를 이용할 경우 pedersen commitment 값을 통해 실제 비밀 값 x와 random 값 r을 복호화할 수 없기 때문에 블록체인에 저장하더라도 개인정보가 드러나지 않지만 [그림 78]과 같이 x값과 r 값을 local에 저장해야 함
- 하지만 이를 이용하면 동형 암호, range proof 뿐만 아니라 영지식 증명으로 값을 검증하기 위한 commitment 역할을 할 수 있다. 이를 이용하여 블록체인에 올라가는 데이터는 검증을 위해서만 저장되고 실제 데이터는 local 환경에 저장하도록 만들 수 있도록 함 이에 우리는 Zoker를 활용하여 재작성 도구 및 pedersen commitment를 활용하여 zoker라는 도구를 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **개발된 재작성 도구 “Zoker”**

- Zoker framework의 아키텍처는 [그림 79]와 같음
- Zoker DSL으로 작성한 코드를 compiler를 이용해 재작성 하고, Lexer에서는 기존의 solidity에서 지원하는 키워드(contract, private, uint, etc)와 operator(+, -, *, etc)를 사용할 수 있도록 키워드와 연산자를 token으로 변환하는 작업을 수행
- parser에서는 token을 zoker에서 지정한 grammar 규칙에 따라 AST(Abstract Syntax Tree)를 생성한다. 이후 converter에서는 AST를 순회하면서 함수 내부의 연산을 사칙연산, 대입연산, 반복문, 조건문, 등에 대해 Operation 객체로 반환
- Operation 객체는 proof을 생성할 때에는 prover rewriter를 통해 실행한 연산에 대한 proof을 생성하고, 검증 컨트랙트를 생성할 때는 Operation 객체를 기반으로 verifier rewriter에서 circuit을 생성하도록 재작
- 검증 컨트랙트에서는 각 함수에서 circuit을 생성하는 코드와 proof의 검증을 수행하는 코드를 갖도록 하여 검증 컨트랙트에서 연산을 동적으로 검증할 수 있도록 함

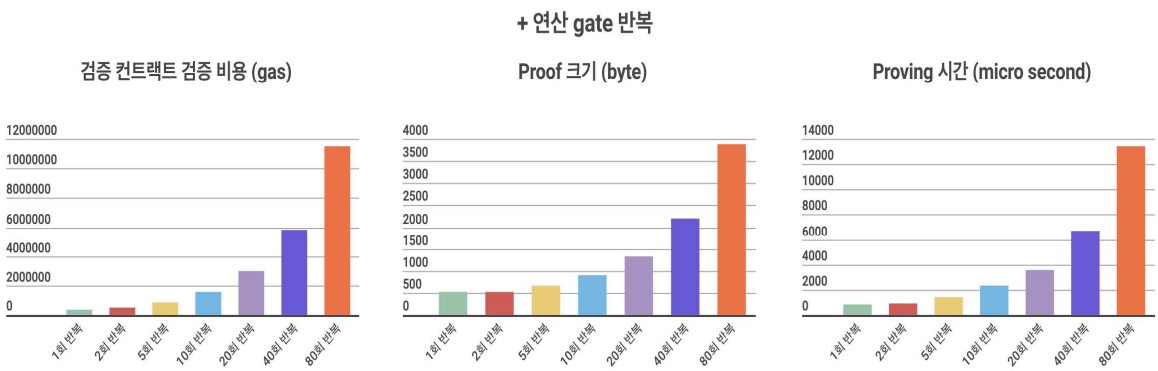


[그림 79] Zoker 아키텍처

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

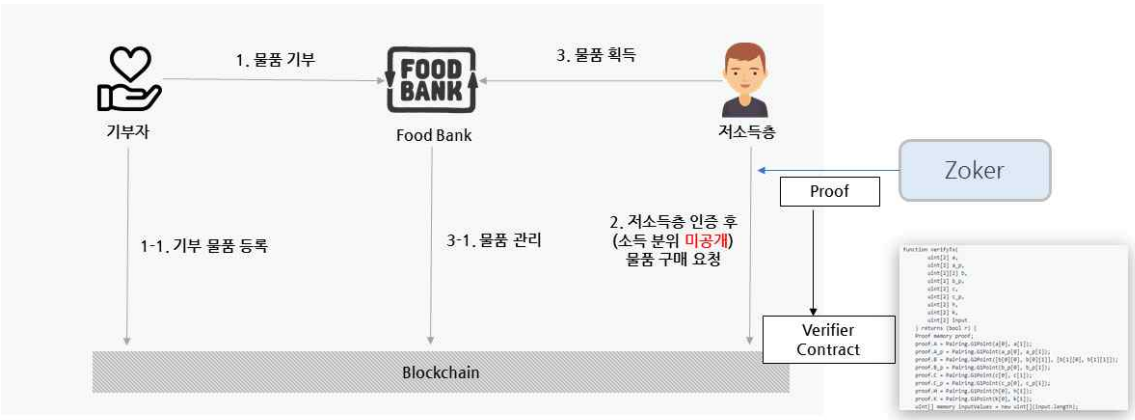
- [그림 80]과 같이 개발된 Zoker를 통해 Performance를 측한 결과, 연산 gate를 반복할수록 컨트랙트 검증 비용 및 Proving 시간이 증가되는 것을 확인

검증 컨트랙트 실행 환경	Proving 실행 환경	검증 컨트랙트 배포 비용	비용 측정 방법
Ethereum Kovan network	MAC OS 2.6 GHz I7 CPU 16GB RAM	2,451,867 gas	10회 반복 측정값 평균

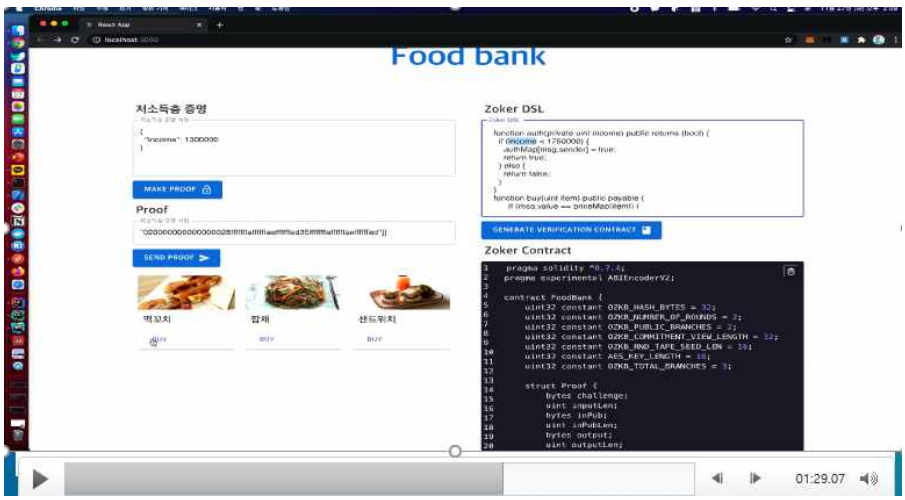


[그림 80] Zoker를 통한 perforamance 측정

- 개발된 Zoker를 통해 [그림 81,82]과 같이 후원 물품 관리 시스템을 만들어 후원 물품에 대해 투명하게 활용할 수 있도록 할 뿐 아니라, 기존 저소득층의 경우 자신의 소득을 드러내지 않고 저소득층임을 드러낼 수 있는 응용프로그램을 만들어 데모 시연



[그림 81] Zoker를 활용한 후원 물품 관리 시스템



[그림 82] 후원 관리 시스템 데모 영상(그림)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ EVM+ 개발을 위한 매커니즘 및 도구 개발

○ Call chain 식별을 위한 매커니즘 개발

- 실행 주체에 대한 정보를 모아 런타임에 스마트컨트랙트의 함수가 어떤 경로로 호출되었는지 파악하고 제어하기 위해 Solidity 언어에서 쓰이는 함수 call 매커니즘 분석을 진행
- 스마트컨트랙트 코드는 creation 부분과 runtime 부분으로 나뉘어 각각 처음 생성할 때에만 존재하거나 일반적인 런타임에 존재하게 됨
- 이러한 특성을 반영해서 함수의 실행 주체를 찾기 위한 매커니즘을 연구하여 함수 selector가 함수 실행에서 중요한 부분을 차지하는 것을 확인하고 이를 이용해 런타임 인스펙션에 적용하는 방법 고안 중

○ EVM에서 동작 가능한 런타임 스택 인스펙션도구 개발

- EVM 바이트코드의 callee-caller 주체 간의 관계를 확인하는 런타임 스택 인스펙션 도구 개발
- 실행 주체를 확인하고 의도하는 행위의 실행 가능 여부를 판단할 수 있는 도구 제시
- Go-Ethereum에서 동작하는 EVM에 적용할 수 있도록 Go언어로 작성된 call stack 개발
- Call chain을 이용해서 함수 호출관계를 파악
- [그림 83]과 같이 실행 주체를 확인하는 시기를 표시하기 위해 새로운 키워드(ForInit)와 EVM opcode(FORINITSTART)를 도입



[그림 83]추가된 opcode

- 키워드 ForInit은 컴파일 후 EVM opcode FORINITSTART로 매칭됨
- EVM 내의 interpreter에서 스마트 컨트랙트를 실행 중 추가된 키워드 Forinit을 만나면 Call stack확인

○ 런타임 스택 인스펙션도구를 이용한 취약점 보완

- 실행 중 함수 호출관계를 파악해 제어하는 것을 이용해 보완할 수 있는 취약점에 런타임 스택 인스펙션 도구를 사용 가능
- <표 17>과 같이 Parity사고의 경우 아래 표에서 볼 수 있듯이 정상적인 경우 생성자만 initWallet함수를 호출할 수 있어야하지만 공격자가 임의로 생성자를 통하지 않고 initWallet 함수를 호출해 문제가 발생

<표 17> Parity 사고 정상흐름(왼쪽)과 공격 흐름(오른쪽)

정상 흐름의 call stack

WalletLibrary.initMultiowned	User
WalletLibrary.initWallet	User
Wallet.Wallet() // 생성자	User

증가방향

공격자가 유도한 흐름의 call stack

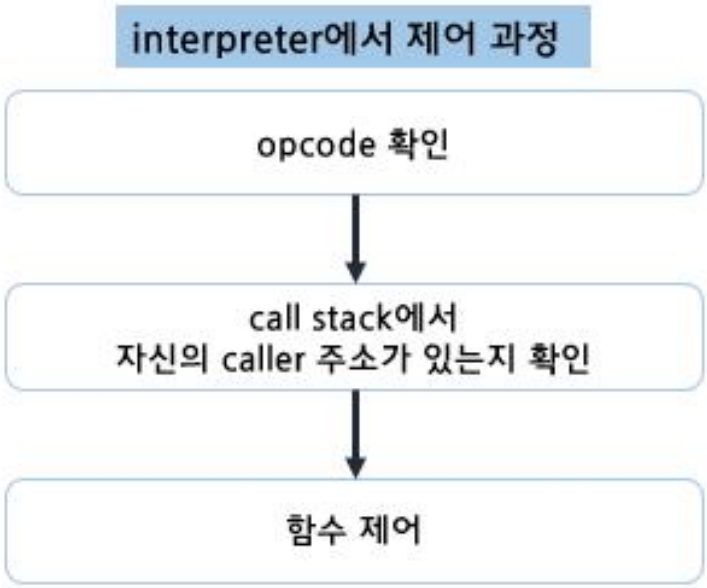
WalletLibrary.initMultiowned	User
WalletLibrary.initWallet	Attacker
Wallet. () // Fallback호출	Attacker
Wallet.initWallet() // error발생	Attacker

증가방향

- 위와 같은 취약점을 보완하는 방법으로 생성자에서만 호출될 수 있는 함수를 표시하고 실행 시에 호출관계를 파악해 호출을 제어하는 방법을 제안

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 새로 추가했던 ForInit키워드를 이용해 실행 주체를 확인하는 시기(Parity예시 에서는 initWallet함수 내부)를 표시해둔 후 interpreter에서 아래 흐름에 따라 함수 호출을 제어



[그림 84] 함수 호출 제어 흐름

- interpreter 내에서 스마트 컨트랙트 실행 중에 FORINITSTART opcode를 만나면 call stack을 조회해 주소를 비교하여 같은 컨트랙트 내에서 호출되었는지 확인한 후 함수 호출 제어 여부를 결정
- 위의 흐름을 이용해 생성자에서만 호출되어야하는 함수의 호출을 제어 가능

○ 수정된 EVM 테스팅을 위한 실행환경 및 보조 도구 개발

- EVM에서 실행 시에 실제로 동작하는지 확인하기 위해 private network를 구성하여 테스팅 환경을 구성하였으며 테스팅을 보조하기 위한 도구 개발
- 보조 도구는 컴파일 자동화 프로그램, 스마트컨트랙트 실행환경에 쓰일 스크립트 자동 생성 프로그램이 있음
- 컴파일 자동화 프로그램은 스마트 컨트랙트를 각 버전에 따라 알맞은 컴파일러 버전에 맞춰 컴파일
- 스크립트 자동 생성 프로그램은 스마트 컨트랙트 실행환경에서 컨트랙트를 실행하기 위해 필요한 일련의 명령어들을 개발자가 준 입력에 따라 자동으로 생성

○ 실시간 모니터링 기능 지원하도록 확장된 EVM (EVM+2.0) 개발

- 기존의 안전하지 않은 행위를 실행중 감지하는 방법은 프로그램 실행중 특정행위가 발생할 때 정지나 로깅 등 작업을 주입하는 방식으로 동작
- 그러나 EVM은 이러한 기능을 제공하지 않으므로 실행 중 프로그램의 행위를 모니터링하는 데에 어려움이 존재
- 때문에 모니터링을 지원하는 확장된 EVM 제안
- EVM+2.0은 다음과 같은 3가지 시점에서 기존 EVM과 다르게 동작
- 첫째, 스마트 컨트랙트가 바이트 코드 형태로 EVM 상에서 실행이 개시되는 시점으로, 바이트 코드는 모니터링 가능한 바이트코드로 변경
- 둘째, EVM에서 실행 전 검증을 하는 부분들로서 단일 시점이 아닌 여러 부분에서 실시되고 있으며, 대부분 첨부된 메타데이터를 가지고 바이트코드에 내재되어 있는 정보의 무결성을 검증

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- EVM+2.0은 재작성된 바이트코드와 원본 바이트 코드를 모두 관리하여 적절한 검증이 효과적으로 이루어지게 함
- 셋째, 의도된 모니터링 지점이 수행되는 시점으로서, 접근제어나 기타 조건 검사 및 로깅이나 정지등의 새로운 기능이 추가

○ **실시간 모니터링 기능 지원하도록 확장된 EVM (EVM+2.0) 개발을 위한 바이트 코드 변경 방법 개발**

- 제안하는 EVM+2.0은 바이트코드 실행 개시 시점에, 주어진 바이트 코드의 일부 부분에 코드를 주입
- 코드 주입은 코드 전체의 행위를 바꿀 수도 있는 사안이므로 주입 위치와 주입 내용에 대해 제한이 필요
- EVM+2.0는 함수 위치를 중심으로 코드 주입을 지원하나 코드 바이트코드로 바뀐 Solidity 컨트랙트는 함수 호출이 명확히 식별되지 않는다는 문제가 있어 본 EVM+2.0에서는 함수 선택자 위치에 코드를 주입할 수 있게 함.
- 이를 위해 EVM+2.0은 함수 선택자의 분기 대상 주소를 조작하여 다른 위치로 분기하며, 이 때 새로운 분기 대상에는 주입될 코드가 바이트코드로 변환되어 존재하고 이 코드의 수행을 마치면 원래 위치로 복귀

```
3   PUSH1 0x80 PUSH1 0x40 MSTORE
4   CALLVALUE DUP1 ISZERO
5   PUSH2 0x10 JUMPI PUSH1 0x0
6   DUP1 REVERT JUMPDEST POP
7   ...
```

[그림 85] 컨트랙트의 Opcodes 중 일부

- 예를 들어 [그림 85]와 같이 원래의 JUMPI의 대상 위치인 PUSH2의 피연산자 값(5번 줄 0x10)에 해당 바이트코드 내의 다른 위치 (주로 가장 마지막 위치)를 저장
- 바이트코드의 마지막 위치에는 각 함수마다 JUMPDEST를 삽입하고 이 위치에 코드주입이 가능하도록 한 후 원래의 JUMPI의 대상 위치인 PUSH2명령과 기존 피연산자를 추가. 이러한 과정을 각 함수에 대해 실행

○ **실시간 모니터링 기능 지원하도록 확장된 EVM (EVM+2.0) 개발을 위한 바이트 코드 검증과정 개선**

- 이런 내용이 실제로 이루어지기 위해서는 EVM에서 기존에 이루어지고 있었던 검증 과정도 개선 필요
- 예를 들어 기존의 메타데이터로 새로운 바이트코드를 검증하거나 기존 코드에 대해 JUMPDEST 위치 등의 분석결과를 확보하여 새로운 코드에 적용한다면 검증 실패로 인해 실행이 진행이 불가능
- 따라서 EVM+2.0은 기존 EVM 의 해당 부분을 수정하여 기존 코드와 새로운 바이트코드를 동시에 보유하고 관리하도록 개선
- 코드 해시 검증을 하는 부분에서는 새로 변경된 코드가 아닌 예전 코드에 대해 검증하도록 관리하며, 이 외에도 추가된 코드의 정렬(alignment) 등 여러 곳에 분포되어 있는 검증에 대해 체계적으로 관리
- 현재는 일부에 대해 구동중이며 Solidity 0.6.x 이상의 언어로 기술된 컨트랙트에 대해서는 정상 작동됨을 확인

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **가스 소모량 추정 도구**

- 이더리움은 컨트랙트가 명령을 실행할 때 마다 가스를 소비하도록 되어 있음. 컨트랙트 실행시 가용 가스량이 돈으로 환산되어 미리 정의되어 있고, 소진하게 되면 오류를 내며 컨트랙트 실행이 멈추게 되어 sender에게 손해를 끼치거나 체인 전체의 일관성을 해칠 수도 있음. 따라서 소비 예상되는 가스량을 미리 추정하는 것은 도움이 됨
- 그러나 실행 전에 가스 소비량을 예측하는 것은 매우 어려운 문제임. 특히 반복문의 종료 조건이 동적으로 결정되는 경우나 조건문이 있는 재귀적 호출에서는 실행이 없이는 예측이 힘듬. 본 연구에서는 종료 조건이 동적으로 결정되는 반복문이 있는 경우에 대한 함수단위 추정치 예측 기술을 개발함
- 각 함수마다 분석하며 코드 분석단계와 추정 단계로 나뉨. 코드 분석단계는 컨트랙트 개발자나 생성자 측에서 상위 언어 대상으로 하며 별도의 자료로 ABI와 함께 추가로 제공될 수 있을 것으로 가정하였음
- 코드 분석 단계

  1) 컨트랙트의 의미를 파악하여 노드 정보를 가지는 제어흐름그래프로 변환

  2) 반복문의 위치와 헤더, 조건, 본문을 파악하고, 반복문의 인덱스 변수 집합(IV)을 파악하고, 하나의 변수로 대체 (IV substitution)

  3) 각 경로의 심볼릭 표현식을 계산하고 각 반복문 조건에 대해 반복문 본문으로 진입하는 경로의 심볼릭 표현식을 사용한 반복 회수 계산식을 확보

  4) 각 기본 블록의 예상 가스량을 계산하고, 위 3)의 결과를 이용하여 함수 전체 예상 가스량 식 확보. 예를 들어 반복 회수가 함수의 인자 x의 식 k(x) 로 표현되면 반복문 본문의 예상 가스량과 k(x)를 곱한 식을 구하여 바이트코드와 함께 전달
- 실행 중 해당 함수 진입 직전

  1) 코드 주입을 통해 실행 중 함수 진입지점을 가로채어 아래와 같은 작업을 수행할 수 있도록 실행 환경을 수정하고 바이트코드를 변경함

  2) 바이트코드와 함께 전달된 가스 추정치에 대한 심볼릭 표현식을 분석함

  3) 호출 정보 및 스택 값을 분석하여 전달된 구체적인 인자 값을 추출함

  4) 심볼값을 구체적인 인자 값으로 대체함. 예를 들어 f(100)과 같은 호출이 이루어졌고, f의 인자가 x이고 한 반복문의 반복 회수가 2*x라면 반복문 본문의 예상 가스량에 200을 곱한 값을 전체 가스량 추정치 식에 반영하여 구체화

  5) 만일 결과가 상수로 나오는 경우 가스 추정 값을 함수 실행 전 주어진 가스와 비교하며, 위험한 경우는 함수 실행 방지

  6) 상수가 아닌 경우에도 가스량 예측이 어려우므로 취약한 경우로 분류
- 추후 함수 호출간의 분석등의 연구로 본 기법의 적용 범위를 확장 가능

□ 이종 역공학 프로그래밍 언어 지원

○ 스마트컨트랙트 역공학 Solidity와 Vyper 지원

- 바이트코드 상태의 스마트 컨트랙트를 역공학하여 중간언어를 생성하고 이를 이용하여 Solidity와 Vyper의 프로그래밍 언어로 변환
- 현재 가장 많이 사용되고 있는 스마트 컨트랙트 언어인 Solidity와 Vyper를 지원함으로써 대부분의 스마트 컨트랙트 개발자가 인식할 수 있는 정보를 제공
- 다음은 바이트코드 상태의 스마트 컨트랙트의 예 (ERC-20 token)

```
0x6080604052348015610010576000800fd5b506004361061009357600035600e01c8063313ce567116100665
78063313ce5671461022157806370a08231146102425780639d89b411461029a578063a9059cbb1461031d
578063dd62ed3e14610381576100935b806306fdde03146100985780630ea7b31461011b57806318160
ddd1461017f57806323b872dd1461019d575b600080fd5b6100a06103f9565b604051808060200182810382
528381815181526020019150805190602001908083836000[5b838110156100e0578082015181840152602081
0190506100c5565b50505050905090810190601f16801561010d578082038051600183602003610100a031
916815260200191505b509250505060405180910390f35b610167600480360....
```

- 개발된 역공학 시스템을 통하여 복원된 결과의 일부 (Solidity)

```
...
        function totalSupply() public returns (uint256)
        {
                return __totalSupply;
        }
        function balanceOf(address _owner) public returns (uint256)
        {
                require((_len(msg.data) - 4) >= 32);
                return __balanceOf[addr(_owner)];
        }
        function allowance(address _owner, address _spender) public returns (uint256)
        {
                require((_len(msg.data) - 4) >= 64);
                return __allowance[addr(_owner)][addr(_spender)];
        }
        function approve(address _spender, uint256 _value) public returns (uint256)
        {
                require((_len(msg.data) - 4) >= 64);
                __allowance[caller][addr(_spender)] = _value;
                return 1;
        }
...
```

- 개발된 역공학 시스템을 통하여 복원된 결과의 일부 (Vyper)

```
...
@public
def totalSupply ()-> uint256:
  return self.__totalSupply

@public
def balanceOf (_owner: address)-> uint256:
  util.require(_len(msg.getData()) - 4 >= 32)
  return self.__balanceOf[addr(_owner)]

@public
def allowance (_owner: address, _spender: address)-> uint256:
  util.require(_len(msg.getData()) - 4 >= 64)
  return self.__allowance[addr(_owner)][addr(_spender)]

@public
def approve (_spender: address, _value: uint256)-> uint256:
  util.require(_len(msg.getData()) - 4 >= 64)
  self.__allowance[self.caller][addr(_spender)] = _value
  return 1
...
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 스마트 컨트랙트 취약점 탐색 테스트를 위한 테스트셋 생성 기술 개발

○ 공격가능한(Exploitable) 버그 조사

- 테스트셋 생성 구현을 위해 smart contract의 공격가능한 버그들을 조사하여 다양한 데이터베이스를 구축
- Smart contract 보안에서 잘 알려져있는 5개의 버그(Re-entrancy, Integer Overflow/Underflow, Bad randomness, Self-destruct)를 기준으로 공격 가능한 버그들을 조사

∘ Re-entracny

- The Dao의 해킹 사례를 통해 알려진 취약점임
- Smart contract의 함수에 대한 의도하지않은 재진입을 통해서 Ether를 반복적으로 전송하는 버그임
- 재귀적 호출의 가능성에 대해서 고려하지않은 것이 핵심 원인임
- 연관 함수: (address).call.value() 함수

∘ Integer Overflow/Underflow

- 일반적인 컴퓨팅 환경에서는 임의 코드 실행등과 같은 치명적인 취약점이 아니지만, 컨트랙트 내에서는 해당하는 버그를 통하여 유저의 잔액 정보가 부정확해질 수 있는 치명적인 버그임
- 특정 값이 주어진 자료형에서 표현할 수 있는 크기를 벗어나는 경우 발생
- 표현 범위를 벗어나면서 최소한계(e.g., 0)을 지나는 경우에는 Underflow, 최대한계(e.g., 2^256-1)를 지나는 경우에는 Overflow로 표현

∘ Bad randomness

- 블록체인은 참여자가 같은 정보를 동일하게 서로 공유하는 특성을 가지기에 자체적으로 예측 불가능한 random number를 만들기 어려움
- 블록의 타임스탬프 등을 활용하여 random number를 생성하는 사례들이 많지만, 해당하는 예측가능한 타임스탬프 정보를 통해 random number가 추측이 가능한 취약점임
- 블록체인 내의 블록 정보(타임스탬프, 블록해시 등등)는 모든 참여자에게 공유되어지기 때문에, 블록 정보를 활용한 random number 생성은 예측 가능한 값이 됨

∘ Self-destruct

- 불충분한 access control로 인해 발생하는 취약점으로 openzepplin의 라이브러리와 같이 ownership과 관련된 modifier 라이브러리를 활용하는 것을 권장

∘ 컴파일러(Compiler) 버그 조사

- smart contract의 언어인 solidity의 컴파일러는 완결성 높은 컴파일러가 아니므로 지속하여 업그레이드를 시켜 배포하고 있음
- solidity의 컴파일러 버그로 인해 소스 코드의 Semantic이 바이트 코드에 제대로 반영

되지 못하여 실행 결과 값이 예상치 못한 값이 발생하는 경우들이 많음

- 기존의 smart contract 테스트셋 구현 과정에서 취약점 탐색의 난이도를 높이기 위해 컴파일러 버그를 활용하여 공격 가능한 버그를 탐색하기 어렵게 만듬
- 코드 조합을 위해 이전의 solidity 버전에서 발생한 다양한 컴 파일러 버그들을 조사하였음

○ ABIEncoderV2 버그

- Pragma experimental ABIEncoderV2의 0.5.7 solidity 컴파일러 버전에서 패치된 버그임
- ABIEncoderV2는 배열/구조체를 포함한 복잡한 데이터 유형을 제공하기 위해 배포된 인코더
- ABIEncoderV2는 32 바이트보다 짧은 값들에 대해 인코딩과 디코딩 과정에서 문제가 되는 경우들이 발생
- bytesNN, bool, enum과 같은 타입이 구조체나 배열의 한 부분으로 정의되었을 때 문제가 발생

○ ExponentCleanup 버그

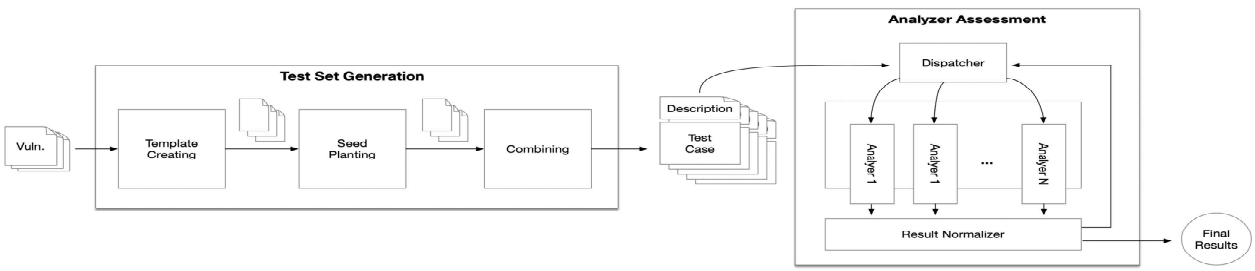
- 0.4.23 solidity 컴파일러 버전에서 패치된 버그임
- 32바이트 보다 더 짧은 지수 타입과 연산자(**)를 사용할 경우 EXP opcode가 적용되기 전에 exponent 상위 비트에 대한 clear 처리가 제대로 이루어지지않아, 예상보다 큰 값(value)이 결과 값으로 나오게되는 버그임
- 32 바이트보다 작은 단위의 Exponent 연산자의 결과 값을 담는 EVM의 slot에 대한 cleanUp 문제로 인해 해당하는 결과 값이 부정확한 값이 나오게 됨

○ HighOrderByteCleanStorage 버그

- 0.4.2 solidity 컴파일러 버전에서 패치된 버그
- 32바이트보다 짧은 type들은 32 바이트인 하나의 storage slot에 저장되지만, 몇몇 type들(e.g.,bytes8)에 대해서 상위 바이트들이 제대 로 cleanUp 처리가 적절하게 되지않아 다른 storage의 값을 덮어쓰는 경우가 발생하는 버그

○ Testset 코드 생성 과정(process)

- Smart contract의 testset 코드 생성을 위한 코드의 조합 과정에 대한 Overview는 아래와 같음
- [그림 86]와 같은 과정을 통해 취약점 탐색에 대한 성능 테스트를 위한 testset 코드들을 자동화하여 생성



[그림 86] 취약점 탐색에 대한 성능 테스트 과정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Test Set Generation 단계
  - 해당하는 단계는 공격 가능한 버그와 컴파일러 버그를 조합하여 템플릿 기반의 코드를 생성하는 단계
  - 템플릿 기반의 코드 생성을 통해 컴파일이 가능한 solidity testset 코드를 생성함
- Template Creating 단계
  - 해당하는 단계는 기존의 공격 가능한 버그와 컴파일러 버그 각각의 Code Template을 성성하는 단계임
  - Creating 단계를 통해 코드 조합을 위한 재료인 Code Template을 사전에 생성
- Seed Planting 단계
  - 해당하는 단계는 컴파일러 버그가 담겨있는 코드인 template code에 공격 가능한 버그의 seed들을 주입하는 단계
  - Seed Planting 단계를 통해 특정 버그의 seed들을 template code에 주입할 위치들을 지정
- ombining 단계
  - 해당하는 단계는 template code와 버그의 seed들을 하나의 코드로 생성하는 단계
  - combining 단계를 통해 컴파일 가능한 solidity testset 코드를 생성
- Analyzer Assessment 단계
  - Analyzer Assessment 단계는 생성된 testset을 기반으로 Dispatcher를 통해 다양한 보안 툴들에 할당하여 취약점 탐색에 대한 오탐 여부를 체크하는 단계
- Dispatcher 단계
  - Dispathcer는 Test Set Generation 단계에서 생성된 testset을 받아 각각의 Docker에 할당된 smart contract 분석 툴들에 코드를 할당
  - 각각의 분석 툴(Analyzer)들은 가장 많이 사용되는 5개(manticore, slither, mythx, securitfy, smartcheck)가 사용됨
- Result Normalizer 단계
  - smart contract 분석 툴들은 각각 output 형태가 다르므로, 해당하는 output을 하나의 표준으로 통합하여 결과들을 normalize하는 단계
  - 각각의 분석 툴들의 json 형식의 output들을 하나의 output으로 통합하고 오탐 여부를 체크

○ Testset 코드 생성의 Evaluation

- <표 18>은 Re-entrancy 버그에 대해 생성된 Testset 코드들을 기반으로 5개의 분석툴들에 할당하여 나온 evalution 결과
- 3개의 툴들(Slither, Securify, Smartcheck)은 모든 단계에서 취약점 탐색을 하는 과정에

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

서 오탐을 하는 결과들을 보임

- Manticore 툴은 3단계의 컴파일러 버그 중 2개의 버그 단계에서 취약점 탐색에 대해 오탐의 결과를 보임
- Mythx 툴은 3단계의 컴파일러 버그 중 1개의 버그 단계에서 오탐의 결과를 보임
- Smart contract 보안에서 가장 많이 알려진 공격인 Re-entrancy는 0단계에서 5개의 툴 모두 공격에 대해 탐지 가능

<표 18> Re-entracny 버그에 대한 evaluation 결과

Complexity Level		Slither		Securify		Manticore		Smart check		Mythx	
Level	Topic	FP	FN	FP	FN	FP	FN	FP	FN	FP	FN
0	-	-	X	-	×	-	×	-	×	-	×
1	if-condition	√	×	√	×	×	×	√	×	×	×
	while-loop	√	×	√	×	×	×	√	×	×	×
	for-loop	√	×	√	×	×	×	√	×	×	×
2	Stage 1	√	×	√	×	×	×	√	×	×	×
	Stage 2	√	×	√	×	×	×	√	×	×	×
	Stage 3	√	×	√	×	×	×	√	×	×	×
3	ABIEncoder V2	√	×	√	×	×	×	√	×	×	×
	Exponent CleanUp	√	×	√	×	×	×	√	×	×	×
	HigherByte Clean Storage	√	×	√	×	√	×	√	×	√	×
	Uninitialized Storage Pointer	√	×	√	×	√	×	√	×	×	×

□ 정적 분석 기법 연구

○ Formal Verification은 무엇인가?

- 정적분석의 대표적인 방식인 Formal Verification이란 P라는 프로그램과 Q라는 성질이 주어졌을 때, P가 어떠한 상황에서도 Q를 만족함을 증명하는 기법
- 해당 증명은 test scenario나 test case 없이 smart contract 안에서 취약점이 발생 가능한 모든 부분을 수학적인 방법으로 찾아냄
- 특히, integer overflow나 integer underflow와 같은 취약점을 찾아내서 smart contract를 blockchain에 배포하기 전에 계약서가 정확하고도 완전하게 동작하게끔 검증

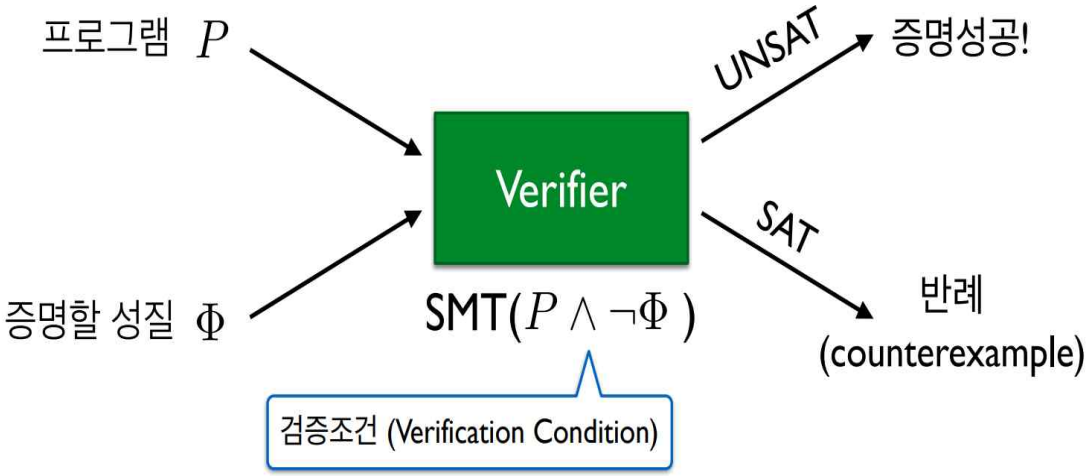
□ Formal Verification의 과정 (VeriSmart)

○ VeriSmart의 추상적인 구조

- VeriSmart는 Formal Verification을 기반으로한 자동 smart contract 안전 검증기

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

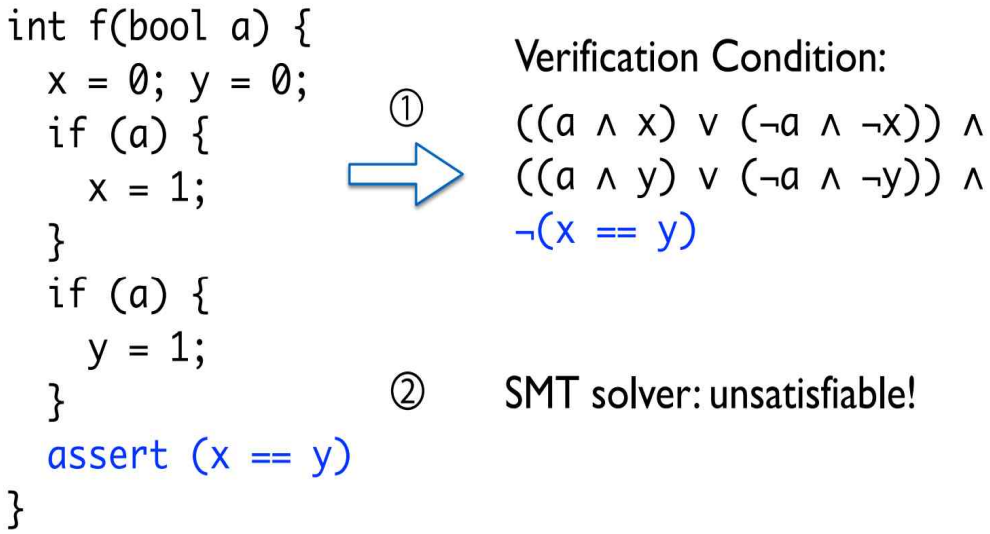
- VeriSmart에서는 트랜잭션 불변성질 (Transaction invariant)를 자동 생성하고 이것을 이용해서 프로그램의 안전성을 증명



[그림 85] 자동 smart contract 안전 검증기

- 여기서 프로그램과 불변 성질은 일차 논리식으로 표현되어 SMT Solver의 입력값으로 들어감
- 프로그램과 부정된 성질이 서로 충족되지 않는 것을 (unsatisfiable) 증명함

○ 불변 성질을 이용한 간단한 C 언어 예제



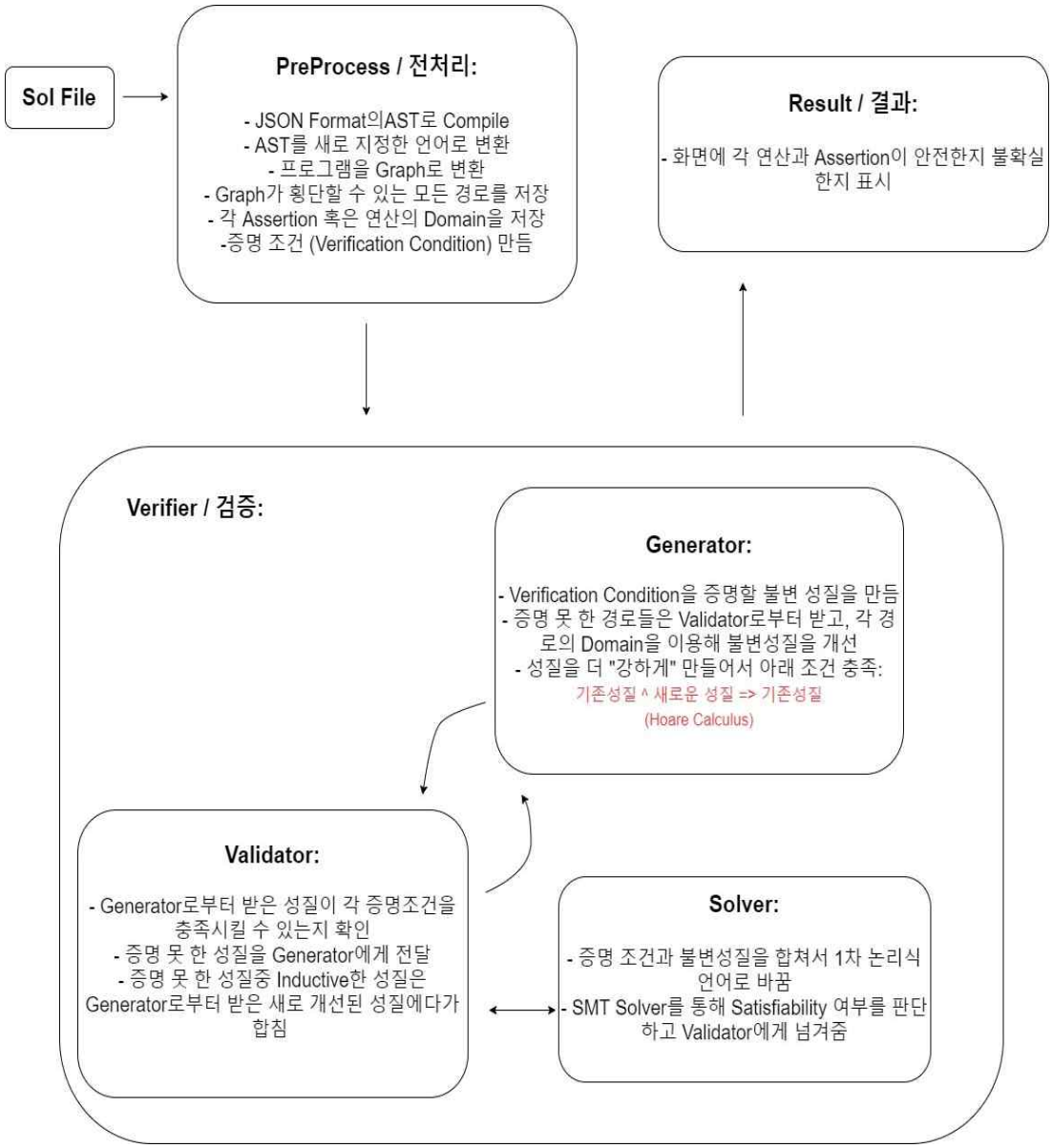
[그림 86] 불변 성질을 이용하기 위한 C 언어 예제

- SMT Solver 결과가 unsatisfiable이기 때문에 이 코드는 취약점으로부터 안전
- 위 코드는 C 언어이지만 VeriSmart는 Solidity 언어로 작성된 Ethereum smart contract에서 실제 검증

○ VeriSmart의 전체적인 flow chart

- [그림 87]은 VeriSmart 형태의 formal verification 과정을 분석하여 도식화한 그림임

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 87] VeriSmart의 전체적인 flow chart

- VeriSmart는 반복적으로 후보 불변 성질을 생성하고 이 불변 성질이 프로그램의 증명 조건을 다 증명할 수 있는지 확인
- 증명 조건을 입증 못 하면 불변 성질을 더 개선하고 다시 증명 조건을 입증한
- 실제 실험 결과, VeriSmart는 취약점 정확도가 99,5%이였고 검출률 또한 100%임
- 다른 취약점 검출기의 결과에 비해 월등한 결과임

□ Formal Verification의 Generalization 방안

○ Formal Verification의 중요한 요소

- VeriSmart와 같은 Formal Verification을 기반을 둔 안전성 검출기가 거의 완벽한 증명을 할 수 있는 이유는 트랜잭션 불변 성질 때문임
- 실제 불변성질 추론하는 과정 없이 (모든 성질을 true로 만듬) VeriSmart를 실행한 결과, 25의 smart contract 중 17개의 안정성을 증명하지 못 함
- 불변성질 추론했을 때는 하나의 계약서만 증명하지 못 함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 따라서, “강한” 트랜잭션 불변 성질을 효율적으로 생성하는 것이 안전성 검출기의 핵심

◦ **기존 기법에서의 한계점**

1. JSON format으로 저장된 Soldity AST의 모호성
   - VeriSmart에서는 JSON format으로 컴파일된 Solidity AST를 새로운 언어로 변환해서 증명 조건을 생성
   - 필요한 정보만 골라 접근하기 쉬운 곳에 저장하여 증명 조건을 더 효율적으로 만듬
   - Solidity 소스 코드 단계에서 바로 증명 조건을 만들면 Gas Calculation과 같은 코드의 내부적인 문제를 커버할 수 없음
   - JSON AST가 bin, asm, abi 등 보다 훨씬 읽기 쉬움
   - 이 과정에서 신경 써야 할 점은 바로 JSON Field의 모호성이 존재함
   - **Solidity 파일을 “--ast-compact-json” 으로 compile 했을 경우:**
     - contract/function 안에서 지역 변수의 값을 설정할 경우, JSON AST 안에 나타나는 “value” field의 타입은 string임
     - 하지만, Global 변수의 초깃값을 설정해준다면 “value” field는 새로운 record type이 되며 그 record type 안에 “value”라는 field가 또 존재하게 됨
     - 즉 같은 이름의 field의 type이 상황에 따라 다를 수 있음
2. SMT Solver를 위한 Interface
   - 각 증명 조건이 불변 성질을 충족하는지 확인하기 위해 SMT Interface가 필요함
   - 먼저, 1차 논리식으로 증면 조건을 변환한다. 이 과정에서 또 새로운 언어 정의
   - VeriSmart에서 이러한 목적으로 VLang을 사용함
   - 이 언어는 SMT Solver의 입력값의 문법과 유사해서 효율적으로 증명 조건을 변환
3. SMT Solver의 효율
   - SMT Solver는 입력값이 복잡할수록 실행 속도가 급격히 낮아짐
   - 4.8.4 버전의 Z3 Solver도 다음과 같은 증명 조건이 충족되지 않는다는걸 알아내기위해 3분 이상이 걸림

$$true \rightarrow (a-b=0) \vee (a-b \neq 0 \wedge ((a-b)*255)/(a-b) = 255).$$

   - 이러한 부분을 보완하기 위해 증명조건은 simplify 해주고 template를 생성해서 많은 조건을 SMT Solver 없이 검증할 수 있도록 해주는 것이 효율적임

◦ **VeriSmart형태의 formal verfication의 발전 방법**

- 트랜잭션 불변 성질을 개선하기 위해 새로운 불변 성질을 만들어 기존 성질에다가 결합
- 새로 추가된 불변 성질은 굉장히 단순한 선형 공식이며 다음과 같음

$$x = y,\ x \geq y,\ x = n,\ x \geq n,\ \text{and}\ x \leq n,$$

- 또한 disjunction과 quantifier 등 1차 논리식의 중요한 부분을 포함하지 않음
- Smart contract loop의 연산이 대부분 다음와 같이 단순하기 때문임

```
for(i = 0; i < x ; i++)
```

- 하지만, 다음과 같은 상황을 증명하지 못함

```
1 function unlockReward(address addr, uint value) {
2    require(totalLocked[addr] > value);
3    require(locked[addr][msg.sender] >= value);
4    if(value == 0) value = locked[addr][msg.sender];
5    totalLocked[addr] -= value; // false positive
6    locked[addr][msg.sender] -= value;
7 }
```

- 4번 줄에서 value의 값이 바뀔 수도 있어서 5번 줄에서 Integer Underflow가 일어날 것처럼 보임

하지만, 다음과 같은 성질이 충족되기에 5번 줄은 안전함

$$\forall x.\texttt{totalLocked}[x] = \sum_i \texttt{locked}[x][i]$$

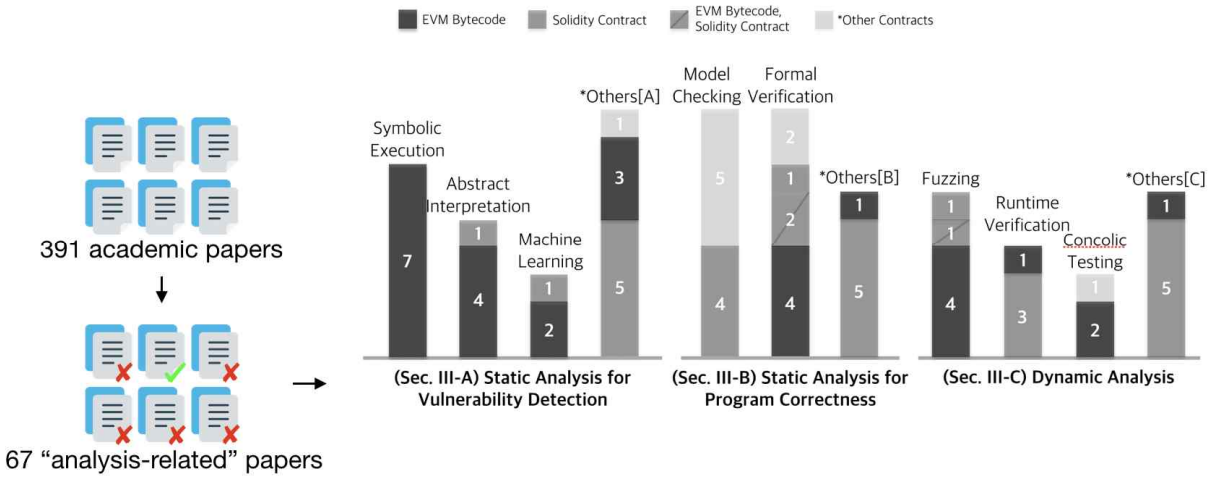
- Disjunction과 quantifier와 같은 불변 성질이 생성을 통해 더 정확한 정보를 생성할 수 있음

□ 스마트 컨트랙트 취약점 분석기 개발

○ 스마트 컨트랙트 취약점 분석기 조사

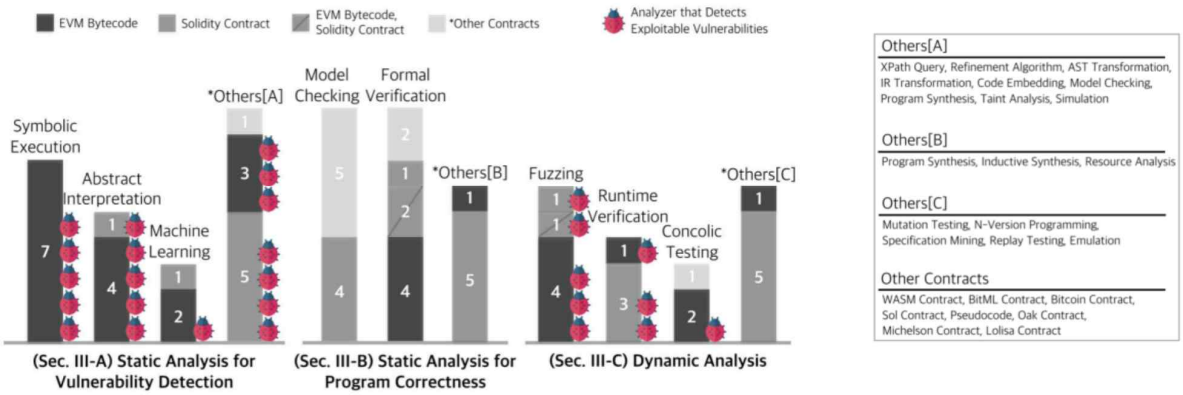
- 현재의 스마트 컨트랙트 분석기 수준 파악 및 미래 연구 주제를 파악하기 위해 Survey 진행
- “Smart Contract” 키워드를 포함한 391편의 학회 논문을 수집하였으며, 이 중 분석기 관련 논문으로 67편을 추려냄
- 67편의 논문을 분석해본 결과, 스마트 컨트랙트 분석은 취약점 탐지를 위한 정적분석, 프로그램의 정확도(correctness)를 분석하기 위한 정적분석 및 동적분석에 대한 연구가 주로 진행되었음
- 분석기의 분석 대상은 EVM Bytecode, Solidity 스마트 컨트랙트, 기타로 분류할 수 있었음
- 기타 분류에는 Bitcoin 컨트랙트, WASM 컨트랙트, BitML 컨트랙트, Oak 컨트래트, Lolisa 컨트랙트 등이 대상이 됨
- 이러한 결과를 보면 알 수 있듯이 대부분의 스마트 컨트랙트 분석기의 연구는 EVM bytecode 및 Solidity 스마트 컨트랙트가 대상이 되고 있음.
- [그림 87]은 67편의 논문이 어떤 스마트 컨트랙트를 대상으로 분석을 수행하고, 어떤 분석 기법을 사용하는지 표현하고 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 87] 스마트 컨트랙트 분석 연구

- [그림 87]에서 볼 수 있듯이, 취약점 탐지 기법으로는 기호실행(Symbolic Execution) 방법론을 가장 많이 활용함
- 이외 취약점 탐지 방법론으로 Abstract Interpretation이 많이 사용되고 있으며, 일부 machine learning을 활용하는 연구도 존재함
- 기타로는 추상구문트리 기반의 분석, Taint 분석, instrument 방법론이 활용이 되었음
- 스마트 컨트랙트 코드의 크기가 작다보니 기호실행 기법을 활용하는데 적합한 것으로 판단됨
- 프로그램의 정확도를 위해서는 Model Checking 및 Formal Verification 방법론이 많이 사용되고 있음
- 기타로는 프로그램 합성(Program Synthesis), 리소스 부석 방법론이 활용이 됨
- 동적 분석으로는 퍼징(Fuzzing)이 가장 많이 활용되고 있는 방법론이였으며 실시간 검증(Runtime Verification) 기법도 많이 사용되고 있음
- 동적분석의 기타 방법론으로는 N-version 프로그래밍, Mutation 테스팅, Replay 테스팅이 사용됨



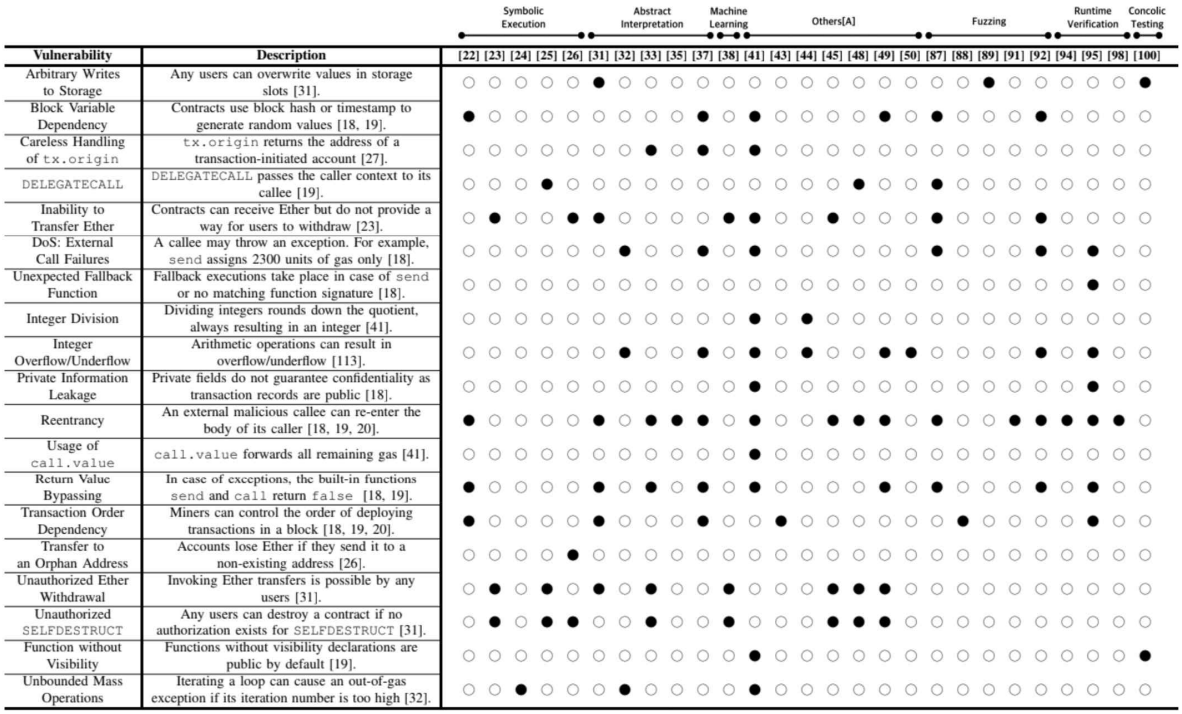
[그림 88] 스마트 컨트랙트 취약점 탐지 연구

- [그림 88]은 스마트 컨트랙트 분석 연구 중 Exploitable 취약점을 탐지하는 연구의 비율을 보여줌
- 총 27편의 연구에서 Exploitable 취약점을 탐지하는 분석 방법론을 적용함
- 기타를 제외하고는 Symbolic Execution, Abstract Interperation, Fuzzing 방법론이

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

Exploitable 취약점을 탐지하는데 활발히 사용됨

- 이러한 스마트 컨트랙트가 어떤 종류의 취약점을 탐지하는지 분석을 진행함
- 분석 결과 현재 총 19개의 취약점을 다루고 있는 것을 파악함
- [그림 89]는 스마트 컨트랙트가 탐지하는 19개의 취약점을 표시함



[그림 89] 취약점 탐지 분석

- 현재 19개의 취약점을 모두 탐지하는 스마트 컨트랙트 분석기는 존재하지 않음
- [그림 87]에서 보이듯이, 각각의 스마트 컨트랙트 분석기 별로 탐지 가능한 취약점 리스트가 다름을 알 수 있음

○ 스마트 컨트랙트 분석기 분야의 미래 연구 분야

- 67편의 연구 분석을 통해 스마트 컨트랙트의 분석을 어렵게 만드는 요인을 파악함.
- 이러한 요인은 프로그램 언어 스펙, 취약점에 대한 부정확한 Property 정의, 명확하지 않은 컨트랙트 동작
  - 프로그램 언어 스펙이 분석의 어려운 요인이 되는 이유는 다양함
  - 스마트 컨트랙트를 개발하는데 사용되는 프로그래밍 언어의 스펙이 자주 변동되며, 이로인해 언어의 구문 및 의미가 변경될 수 있고, 이로인해 분석기를 새로 개발해야 하는 이슈가 발생함
  - 프로그램 언어의 스펙이 완벽하게 정의되어 있지않은 부분이 존재하며 이러한 부분은 보안 취약점으로도 연결됨.
  - 분석기에서 정의한 취약점에 대한 부정확한 Property가 오탐율을 발생하는 것을 파악함
  - 예를 들어 TOD(Transaction Ordering Dependence) 취약점 탐지를 위해 Oyente에서는 서로 다른 2개의 Ether flow가 존재하는지만 확인함
  - TOD 취약점 탐지에 이렇게 간단한 Property를 사용하다보니 오탐이 발생함
  - 분석을 어렵게 만드는 다른 요건은 명확하지 않은 스마트 컨트랙트의 동작이 있음
  - 예를 들면 일부 스마트 컨트랙트의 동작은 프로그램의 소스코드와 상관없이 마이너의 행동에 의해 달라지는 경우가 존재함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

  - 이러한 경우 스마트 컨트랙트의 소스코드를 기반으로 분석을 수행하는 분석기는 스마트 컨트랙트의 동작을 명확하게 예측할 수 없어 분석에 어려움이 존재함
- 분석 내용을 기반으로 스마트 컨트랙트 분석기 분야의 미래 연구 방향을 도출함
- 보안상 안전한 스마트 컨트랙트를 개발하는데 사용될 수 있는 프로그래밍 언어가 존재하지 않음
- 가장 많이 사용되고 있는 프로그래밍 언어는 Solidity이나, 보안에 취약함
- 스마트 컨트랙트 실행 환경 및 어플리케이션을 고려한 보안상 안전한 프로그래밍 언어가 필요함
- 현재 머신러닝을 활용한 스마트 컨트랙트 분석 연구가 많이 진행되지 않음
- 머신러닝을 기법을 활용하는 연구가 향후 많이 진행되어질 것으로 판단됨
- 아직 기준이되는 하나의 블록체인 플랫폼 및 프로토콜이 존재하지 않음
- 또한 이러한 블록체인 플랫폼을 테스팅하는 표준 검증 방법론이 존재하지 않음
- 이러한 분야는 향후 계속 연구가 진행될 것으로 판단됨

ㅇ **3차년도 스마트 컨트랙트 신규 취약점 발견**

- 5개의 신규 스마트 컨트랙트 취약점 발견 후 CVE 발급
  - CVE-2019-18775 : 취약한 생성자로 인한 코인 탈취
  - CVE-2019-18776 : Integer Overflow 취약점
  - CVE-2019-18777 : Down-Casting 취약점으로 인한 예상치 못한 실행
  - CVE-2019-18778 : 변수 쉐도잉으로 인한 프로그램 오동작
  - CVE-2019-18779 : Race Condition 취약점

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 목표 달성도 및 관련 분야 기여도

3-1. 목표 및 달성여부

○ 과제 제안 당시 정량적 목표 대비 최종 실적 달성

- 한국정보통신기술협회(TTA)를 통한 시험결과서 첨부

* 영지식 증명 생성 및 검증의 경우, 기존 비교 대상인 HAWK에서 사용하는 알고리즘을 변경하여 알고리즘 성능에 대한 평가서 제출, 연산에 대한 성능은 자체평가를 통한 성능 비교

○ 세부사업 질적 성과 목표 또한 달성

- 특허 등록에 대한 SMART 등급 점수 자료 첨부

<정량적 목표 항목>

평가 항목 (주요성능 Spec)	단위	전체 항목에서 차지하는 비중 (%)	세계최고 수준 보유국/보유기업 (/)	연구개발 전 국내수준	1차 년도 (2018년)		2차 년도 (2019년)		3차 년도 (2020년)		평가 방법
			성능수준	성능수준	목표	실적	목표	실적	목표	실적	
1. 영지식 증명 생성	sec	30%	미국/HAWK 70.7	비교 대상 없음	80	71.9	60	55	40	8.6	블록체인 전문기관 평가 및 자체평가 9)
2. 영지식 증명 검증	sec	20%	미국/HAWK 28.8	비교 대상 없음	50	23.9	40	15	30	5.4	
3 스마트 컨트랙트 역공학 지원 프로그래밍 언어 수	개	20%	미국, UAE/ Comae 1	비교 대상 없음	1	1	2	2	2	2	
4 스마트 컨트랙트 역공학을 통한 탐지율 및 오탐률	%	20%	싱가폴 /OYENTE 탐지율 45.61% 오탐률 6.40%	비교 대상 없음	탐지율 45%	45%	55%	54%	75%	81%	
					오탐률 20%	20%	10%	10%	5%	4%	
5 스마트 컨트랙트 신규 취약성발견	개	10%	비교 대상 없음	비교 대상 없음	2		5		8		

9) 한국블록체인협회, 한국블록체인학회 등 블록체인 기술 전문기관의 회원 또는 스마트컨트랙트 보안 성관련 분야 전문가로부터 결과물에 대한 평가를 의뢰하고 평가서를 취합

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 세부사업 질적 성과 목표>

성과지표	가중치	실적 및 목표치				측정산식	자료수집 방법/출처
		구분	2018	2019	2020		
등록특허 질적평가지수 (K-PEG/SMART)	50 %	목표	-	B	B	Σ(특허별 K-PEG/ SMART 등급 점수) /전체 특허수	한국특허정보원, 한국발명진흥회 특허평가시스템
		실적	-	0	B		
논문 IF 지수 (Impact Factor)	50 %	목표	-	1.0	1.0	당해년도 논문 실적 평균	조사분석 (NTIS) 입력자료
		실적	-	4.098	2.645		
합계	100 %						

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3-2. 목표의 세부 성과 달성 현황

○ 과제 제안 당시 언급하였던 세부 성과 달성. 표준화의 경우 기고서 제출 1건에 대해 초과 달성, 기술 이전의 경우 500만원 초과 달성, 소프트웨어의 경우도 1건 초과 달성하여 목표 대비 실적을 완벽히 수행하였음

<성과달성 총괄표>

구분			목표	실적	비고
특허 출원	국내		9	9	
	국외		-	-	
	계		9	9	
특허 등록	국내		4	4	
	국외		-	-	
	계		4	4	
국제 표준화	기고서 제출		1	1	
	기고서 채택		-	-	
	표준안 채택		-	-	
국내 표준화	기고서 제출		2	5	초과달성
	기고서 채택		3	3	
	표준안 채택		-	-	
기술 이전	계약 건수		1	1	
	계약액		30(백만원)	35(백만원)	초과달성
	징수액		-	-	
등록·기탁 대상 성과물	보고서 원문		-	-	
	연구시설 · 장비		-	-	
	기술요약정보		-	-	
	생명자원	생명정보	-	-	
		생물자원	-	-	
		신품종	-	-	
	소프트웨어		4	5	초과달성
	화학물		-	-	
인력 양성	학사		00	00	
	석사		00	00	
	박사		00	00	
기타	시제품		2	2	
	성과 홍보		2	2	
	기술문서(CVE)		6	8	초과달성
	시제품		2	2	
	상용화		1/ 50(백만원)	1 / 50(백만원)	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[기술적 성과]

□ 지식재산권(특허, 실용신안, 의장, 디자인, 상표, 규격, 신품종, 프로그램)

번호	지식재산권 등 명칭 (건별 각각 기재)	국명	출원			등록			기여율	활용 여부
			출원인	출원일	출원번호	등록인	등록일	등록번호		
1	스마트 컨트랙트 실행 장치 및 그 장치에서 취약점 탐지 방법	대한민국	충남대학교 산학협력단	2018.11.05	10-2018-0134411	충남대학교 산학협력단	2020.09.11	10-2157208-0000	100	특허
2	태그 추천 모델을 이용한 자동화된 스마트 컨트랙트 태그 생성 및 추천 시스템	대한민국	㈜ 파이랩테크놀로지	2018.11.28	10-2018-0149850	㈜ 파이랩테크놀로지	2020.11.23	10-2184772-0000	100	제품화
3	블록체인의 확장성을 위한 영지식 증명의 선택적 검증 시스템 및 그 방법	대한민국	충남대학교 산학협력단	2019.11.20	10-2019-0149544	충남대학교 산학협력단	2020.04.16	10-2103177-0000	100	특허
4	스마트 컨트랙트를 활용한 Payment Channel에서의 멀티 홉 트랜잭션 라우팅 방법	대한민국	충남대학교 산학협력단	2019.01.23	10-2019-0008887	충남대학교 산학협력단	2019.01.23	10-2019-0008887	100	특허
5	이더리움 스마트계약 가상머신 바이트코드 보안성 향상 방법	대한민국	충남대학교 산학협력단	2019.12.30	10-2019-0177987	-	-	-	-	-
6	동적 연산을 통한 검증이 가능한 영지식 증명 시스템 및 증명 방법	대한민국	충남대학교 산학협력단	2021.01.11	10-2021-0003531	-	-	-	-	-
7	블록체인 프라이버시 보호와 확장성을 위한 영지식 증명의 선택적 검증	대한민국	충남대학교 산학협력단	2021.01.11	10-2021-0003532	-	-	-	-	-
8	EVM_2.0(스마트 컨트랙트 재작성기)	대한민국	충남대학교 산학협력단	2021.01.15	10-2021-0006233	-	-	-	-	-
9	블록체인에서 오라클 시스템을 보호하기 위한 스마트 계약 데이터 피드 프레임워크	대한민국	충남대학교 산학협력단	2021.01.27	10-2021-0011277	-	-	-	-	-

- 지식재산권 활용 유형

번호	제품화	방어	전용실시	통상실시	무상실시	매매/양도	상호실시	담보대출	투자	기타
	✓									

□ 신기술 지정

번호	명칭	출원일	고시일	보호 기간	지정 번호

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 저작권(소프트웨어, 서적 등)

번호	저작권명	창작일	저작자명	등록일	등록번호	저작권자명	기여율
1	함수호출 안전성을 위한 Solidity(솔리디티) 전처리기	2019.01.15	충남대학교 산학협력단	2019.10.25	C-2019-030173	충남대학교 산학협력단	100
2	스마트 컨트랙트를 이용한 블록체인 공개키 기반 구조의 탈중앙화 앱 스토어	2019.11.25	충남대학교 산학협력단	2019.12.05	C-2019-038069	충남대학교 산학협력단	100
3	스마트 컨트랙트 데이터 피드를 위한 신뢰 실행 환경 기반 블록체인 클라이언트	2019.11.25	충남대학교 산학협력단	2019.12.04	C-2019-037417	충남대학교 산학협력단	100
4	EVM+2.0(스마트 컨트랙트 재작성기)	2020.08.05	충남대학교 산학협력단	2020.09.08	C-2020-030522	충남대학교 산학협력단	100
5	Solidity(솔리디티) 컴파일 자동화 프로그램	2020.08.05	충남대학교 산학협력단	2020.08.31	C-2020-029530	충남대학교 산학협력단	100

□ 기술 및 제품 인증

번호	인증 분야	인증 기관	인증 내용		인증 획득일	국가명
			인증명	인증 번호		

□ 표준화

번호	수행 기관명	표준화 주제	표준화 기구	표준화 단계	관련 번호	제출(채택)일	국가
1	충남대학교 산학협력단	분산원장기술 전자지불 시스템 보안 위협 및 요구사항	분산원장기술 표준포럼	표준제안서 승인	DLTSF-2018-D003	2018.12.15	대한민국
2	충남대학교 산학협력단	분산원장시스템 참조구조	분산원장기술 표준포럼	표준제안서 승인	DLTSF-2018-D003	2018.12.05	대한민국
3	충남대학교 산학협력단	Blockchain and distributed ledger technologies - Terminology_KATS	분산원장기술 표준포럼	표준제안서 승인	ISO/CD 22739.2	2019.04.12	대한민국 (국제)
4	충남대학교 산학협력단	블록체인 등 분산원장기술 용어정의 사례 연구	한국정보통신 기술협회	기술보고서	TTAR.12-0044	2019.10.31	대한민국
5	충남대학교 산학협력단	분산원장기술 참조구조 사례 연구	한국정보통신 기술협회	기술보고서	TTAR-12.0043	2019.10.30	대한민국
6	충남대학교 산학협력단	분산원장기술 활용사례	한국정보통신 기술협회	기술보고서	TTAR-12.0031	2018.11.07	대한민국

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[과학적 성과]

□ 국내 및 국외 논문 게재

번호	논문명	학술지명	주저자명	호	국명	발행기관	SCI 여부 (SCI/비SCI)	게재일	등록번호
1	Homomorphic comparison for point numbers with user-controllable precision and its applications	Symmetry	김명선	12(5)	미국	MDPI	SCI (IF:2.645)	2020.5	2074-8994
2	Experimenting with Non-interactive Range Proofs based on the Strong RSA Assumption	IEEE Access	김명선	7	미국	IEEE	SCI (IF:4.098)	2019.8	10.1109/ACCESS.2019.2936210
3	ScanAT: Identification of Bytecode-Only Smart Contracts With Multiple Attribute Tags	IEEE Access	김윤태, 박도현, 이종협	7	미국	IEEE	SCI (IF:4.098)	2019.7	10.1109/ACCESS.2019.2927003
4	Analysis of Blockchain Smart Contracts: Techniques and Insights	IEEE Secure Development	김신해. 류석영	1	미국	IEEE	SCI급	2020.10	10.1109/SecDev45635.2020.00026
5	함수 호출의 안전성 향상을 돕는 스마트 계약 코드 재작성기	한국정보보호학회논문지	이수연	29	대한민국	한국정보보호학회	비SCI	2019년 2월.01	10.13089/JKIISC.2019.29.1.67

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 국내 및 국제학술회의 발표

번호	회의명칭	발표자	발표 일시	장소	국명
1	2018 한국정보보호학회 하계학술대회	이수연	2018.6.21	나주 동신대학교	대한민국
2	2018 한국정보보호학회 하계학술대회	김근영	2018.06.21	나주 동신대학교	대한민국
3	APIC-IST 2018	오원찬	2018.06.25	나뜨랑	베트남
4	CUTE 2018	박준후	2018.12.17	쿠알라룸푸르	말레이시아
5	2018 한국정보보호학회 하계학술대회	김윤태	2018.06.21	나주 동신대학교	대한민국
6	The 43rd IEEE COMSAC	이수연	2019.07.19	Marquette Univ., Milwaukee	USA
7	2019 한국정보과학회 소프트웨어종합학술대회	이수연	2019.12.20	휘닉스 평창	대한민국
8	2020 한국정보보호학회 하계학술대회	김지수	2020.07.15	온라인	대한민국
9	한국정보보호학회 충청지부 학술대회	김근영	2019.10.18	한남대학교	대한민국
10	ICOIN 2018	김혁진	2019.12.16	하노이	베트남
11	ICSE	황성재	2020.6	Virtual Conference	미국

□ 기술 요약 정보

연도	기술명	요약 내용	기술 완성도	등록 번호	활용 여부	미활용사유	연구개발기관 외 활용여부	허용방식

□ 보고서 원문

연도	보고서 구분	발간일	등록 번호

□ 생명자원(생물자원, 생명정보)/화합물

번호	생명자원(생물자원, 생명정보)/화합물명	등록/기탁 번호	등록/기탁 기관	발생 연도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[경제적 성과]

□ 기술 거래(이전) 등

번호	기술 이전 유형	기술 실시 계약명	기술 실시 대상 기관	기술 실시 발생일	기술료 (해당 연도 발생액)	누적 징수 현황
1	노하우	영지식 증명 검증이 가능한 스마트 컨트랙트 생성 기법	㈜지란지교 시큐리티	2019.11.21. ~2020.05.20	35,000,000(원)	

□ 사업화 투자 실적

번호	연구개발 투자	설비 투자	기타 투자	합계	투자 자금 성격

□ 사업화 현황

(단위: 명, 년)

번호	사업화 방식	사업화 형태	지역	사업화명	내용	업체명	매출액		매출 발생 연도	기술 수명
							국내	국외		
1	사업화를 통한 직접 매출 발생	기술보유자의 직접사업화_기존업체_상품화	국내	스마트 컨트랙트 분석 솔루션 공급계약	블록체인 서비스 분석 솔루션 공급	(주)파이랩테크놀로지	50,000,000원		2020년	5년

□ 비용 절감(누적)

순번	사업화명	발생연도	비용 절감액(천원)	산정 방법
합계				

□ 고용 창출

순번	사업화명	사업화 업체	고용창출 인원(명)		합계
			○○○○년	○○○○년	
합계					

□ 고용 효과

구분			고용 효과(명)
고용 효과	개발 전	연구인력	
		생산인력	
	개발 후	연구인력	
		생산인력	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 경제적 파급 효과

(단위: 천원/년)

구분	사업화명	수입 대체	수출 증대	매출 증대	생산성 향상	고용 창출 (인력 양성 수)	기타
해당 연도							
기대 목표							

□ 전문 연구 인력 양성

번호	분류	기준 연도	현황										
			학위별			성별			지역별				
			박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타

□ 산업 기술 인력 양성

번호	프로그램명	프로그램 내용	교육 기관	교육 개최 횟수	총 교육 시간	총 교육 인원

□ 연구시설・장비

구축기관	연구시설/ 연구장비명	규격 (모델명)	개발여부 (○/×)	연구시설・장비 종합정보시스템 * 등록여부	연구시설・장비 종합정보시스템 * 등록번호	구축일자 (YY.MM.DD)	구축비용 (천원)	비고 (설치 장소)
충남대학교	데스크톱컴퓨터	Intel Core i7	X	X	X	2018.11.19	2,450	충남대학교 공대 5호관 529호
충남대학교	데스크톱컴퓨터	Intel Core i7	X	X	X	2018.11.19	2,450	충남대학교 공대 5호관 529호
충남대학교	노트북	Intel Core i7	X	X	X	2018.11.19	1,920	충남대학교 공대 5호관 529호
충남대학교	노트북	Intel Core i7	X	X	X	2019.06.26	3,380	충남대학교 공대 5호관 529호
충남대학교	노트북	Intel Core i7	X	X	X	2019.06.26	2,260	충남대학교 공대 5호관 529호
충남대학교	태블릿컴퓨터	11’	X	X	X	2019.06.26	1,550	충남대학교 공대 5호관 529호

□ 기타성과

○ 2020 한국정보과학회 소프트웨어종합학술대회 우수논문상

- “Improve the safety of the environment running smart contracts by utilizing the call relationship of functions (함수 호출관계를 활용한 스마트 계약 실행환경 안전성 개선),“ 이수연, 목성균, 조은선, 한국정보과학회 소프트웨어종합학술대회 (KSC), Dec. 2019

○ 2018 한국정보보호학회 하계학술대회 우수논문상

- “스마트 계약 언어에서 함수 호출의 안전성 향상을 위한 코드 재작성 기법(Code rewriting techniques for improving the safety of function calls in smart contract

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

languages)“, 이수연, 정형근, 조은선, 한국정보보호학회 하계학술대회 Conference on Information Security and Cryptography (CISP-S)

○ 라인X한국정보과학회 블록체인 경진대회 우수상, “실물화폐간의 교환을 위한 탈중앙화된 TEE 기반 개인간 거래 플랫폼(DPE)” 김근영,김혁진,박준후

○ 한국정보보호학회 충청지부 학술논문발표회 장려상, “Chainlink에서 다양한 프로토콜을 지원하기 위한 스마트 컨트랙트 라이브러리 개발” 김근영,김혁진,박준후,류재철

□ 시제품

번호	시제품명	주요내용	소요비용
1	Zokrates+	영지식 증명 도구를 생성해주는 Zokrates+ 툴 개발 (기존의영지식증명엔진교체) 기존의Zokrates에비해Proof생성속도가빠르다는장점이 있으며,Solidity언어를기반으로하기때문에기존의스마트 컨트랙트개발자들이사용하기편리함	1,000(천원)
2	확장된 oyente 도구	스마트 컨트랙트 취약점 도구인 Oyente를 확장시켜, 더욱 더 많은 탐지 및 적은 오탐율을 보임 해당 제품을 깃허브에 업로드 시켜 놓았으며, 개발한 스마트 컨트랙트를 업로드 시켜 해당 컨트랙트의 취약점 유무를 파악	1,000(천원)

□ 기술문서(CVE)

번호	CVE Number	Vulnerability	Github
1	CVE-2019-15078	Misuse of constructors	https://github.com/sjmini/icse2020-Solidity
2	CVE-2019-15079	Misuse of constructors	
3	CVE-2019-15080	Misuse of constructors	
4	CVE-2019-18775	Misuse of constructors	
5	CVE-2019-18776	Uninitialize Storage pointer	
6	CVE-2019-18777	Type casting to arbirary contracts	
7	CVE-2019-18778	Storage variable shadowing confusion	
8	CVE-2019-18779	Function without visibility	

□ 성과홍보

- 박도현 파이랩 대표 “IEO로 영토확장 노크”
- 파이랩테크놀로지, 딥러닝 이용한 스마트 컨트랙트의 취약점 분석한 기술 상용화

3-3. 목표 미달성 시 원인(사유) 및 차후대책(후속연구의 필요성 등)

- 해당 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 연구개발성과의 활용 계획 등

4-1. 향후 5년간 성과활용·확산 활용방안 및 계획(활동계획)

○ **딥러닝 기반 이미지 분류 시스템 결과 검증**

- 이미지 분류를 목적으로 하는 딥러닝 기반의 뉴럴 네트워크들은 딥러닝 이전의 머신러닝 기반 모델들에 비해서 성능이 매우 뛰어남
- 딥러닝 기반 이미지 분류 모델은 스마트폰의 이미지 인식 기술이나 카메라 기술에 이미 적용되어 있으며, 구체적으로 상품이나 사물을 인식하거나 인물의 사진을 더욱 또렷하게 만들어주는 등에 사용되고 있음
- 이미지 분류 모델의 응용에 따라 다양한 연산이 사용되는데, 행렬 연산이나 Convolution 연산 또는 ReLU, sigmoid와 같은 비선형 연산들 까지 종류가 다양함

○ **데이터베이스 TPC-H[10] 쿼리 결과 검증**

- 클라우드 기반의 Database-as-a-Service (DaaS)는 저장공간의 유지비용과 회계 가치의 효율적 관리 등 경제적 관점에서 유리 (예: Google의 BigQuery, Amazon의 RDS)
- 대표적인 DaaS로는 온라인 배달 업체에서 발주량의 관리 및 기존 발주량의 비교 분석에 이용하기도 하며, 대형 유통업체나 백화점에서는 재고관리에 각 물품별 재고 내용과 발주 물건의 예측 등에 이용
- DaaS의 응용에 따라 다양한 쿼리가 필요한데, 기업에서 이용하는 대표적인 유형은 TPC-H에 제시된 목록이 이용

○ **블록체인 서비스 온체인/오프체인 자동 분석 솔루션 개발**

- 본 과제에서 개발된 스마트 컨트랙트에 대한 다양한 역공학 및 분석 기술을 활용하여 스마트 컨트랙트로 구성되어 있는 블록체인 서비스를 자동 분석 기술 솔루션 개발
- 오프체인(offchain)에서는 스마트 컨트랙트 분석을 통해 특성을 파악한다. 딥러닝을 이용하여 스마트 컨트랙트를 가장 잘 설명할 수 있는 간략화된 정보를 추출하고, 코드 분석을 통해서 사용하는 상태변수에 대한 정보를 추출
- 온체인(onchain)에서는 수집된 정보를 바탕으로 스마트 컨트랙트의 행동 변화를 모니터링한다. 핵심이 되는 상태변수의 정상범주 또는 안전 조건을 유지하는지를 실시간 확인

<연도별 주요 활동계획(요약표)>

구분	종료+1 (0000년)	종료+2 (0000년)	종료+3 (0000년)	종료+4 (0000년)	종료+5 (0000년)
영지식 증명 후속연구	산술 회로 변환 방법 설계	딥러닝 연산에 특화된 SNARK 기법 설계 연구	특별한 자료구조 기반의 쿼리 연산 검증기법의 설계 연구	기본 쿼리 연산을 위한 SNARK 연구	고급 쿼리의 연산 검증기법 설계

10) TPC (Transaction Performance Council)는 데이터베이스 성능기준을 정의하는 비영리 단체이며 TPC-H는 상업적으로 중요한 Query에 대한 성능측정의 기준 자료를 제시

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4-2. 주요 성과활용 (기대)성과목표

(1) 정성적 목표

○ 딥러닝 연산 증명 특화 SNARK

- 딥러닝 연산에 사용한 함수들을 산술회로로 변환하는 과정을 적용하여 딥러닝 연산에 특화된 SNARK 기법 설계
- 딥러닝에 많이 사용되는 딥러닝 연산, 데이터의 형태에 특화된 효율적인 SNARK 암호 원천 기술을 설계
- 구체적인 응용 시나리오에 따른 딥러닝 연산 증명을 위한 SNARK 변형을 통한 고도화
- 딥러닝의 응용 기술에 특화된 SNARK 알고리즘의 최적화 연구

○ 데이터베이스 쿼리 연산 증명 특화 SNARK

- 기본 연산을 위임하는 사용자가 지정되는 검증자 지정 SNARK 기법 설계
- 검증자 지정 SNARK을 공개검증 가능한 SNARK 기법 설계
- 검증자가 지정되는 특별한 자료구조 기반의 쿼리 연산 검증기법의 설계
- 데이터베이스 쿼리 연산 검증기법의 성능 최적화 연구

○ 온체인/오프체인의 통합 블록체인 서비스 분석 시스템 개발

- 본 과제의 연구 결과를 바탕으로 오프체인의 분석을 바탕으로 온체인에서 블록체인 서비스의 이상상태를 탐지하는 분석 시스템을 개발

(2) 정량적 목표

<연도별 성과활용 (기대)성과목표(요약표)>

구분	종료+1 (0000년)	종료+2 (0000년)	종료+3 (0000년)	종료+4 (0000년)	종료+5 (0000년)
연구논문			1		1
특허		1		1	
공개 S/W				1	1

4-3. 성과활용・관리 추진체계

○ ezone 시스템을 활용한 주기적인 성과 등록

4-4. 기타

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 1. 사업화대상기술별 기술명세서

1. 과제정보

과 제 명	국문	스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술개발				
	영문	Privacy-Preserving and Vulnerability analysis for Smart Contract				
기술분류	대분류	차세대보안	중분류	데이터 및 응용서비스보안	소분류	전자화폐·핀테크보안
주관기관	충남대학교 산학협력단		연구책임자	류재철		
연락처(책임자)	사무실 : 042-821-5443		이메일(책임자)	jcryou@cnu.ac.kr		

2 사업화대상기술 목록

순번	개발기술(Product)명 (관련 기술의 Know-how 및 특허 등)		보유특허(기술별로 중복기재 가능)		
			특허		발명의 명칭
			출원번호	등록번호	
1	국문	영지식 증명 검증이 가능한 스마트 컨트랙트 생성 기법	10-2021-0003531		동적 연산을 통한 검증이 가능한 영지식 증명 시스템 및 증명 방법
	영문	Smart Contract Generation Scheme with Zero-Knowledge Verification		10-2103177-0000	블록체인의 확장성을 위한 영지식 증명의 선택적 검증 시스템 및 그 방법
2	국문	블록체인 서비스 분석 솔루션		10-2184772	태그 추천 모델을 이용한 자동화된 스마트 컨트랙트 태그 생성 및 추천 시스템
	영문	Blockchain Service Analysis Solution			
~n	국문				
	영문				

* 보유특허 란은, 기술예고의 경우 작성하지 않고 사업화대상기술은 반드시 작성
* 사업화대상기술의 경우, 기술이전·사업화가 가능한 핵심 요소기술로 관련 특허번호가 있는 경우에만 출원·등록된 기술을 작성
* 하나의 기술에 다수의 특허 또는 여러 기술에 하나의 특허 중복 작성 가능

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 기술예고/ 사업화 대상기술

개발기술 1		
	국문	영지식 증명 검증이 가능한 스마트 컨트랙트 생성 기법
	영문	Smart Contract Generation Scheme with Zero-Knowledge Verification

공개	☑	비공개	☐ (사유)

① (예상) 결과물

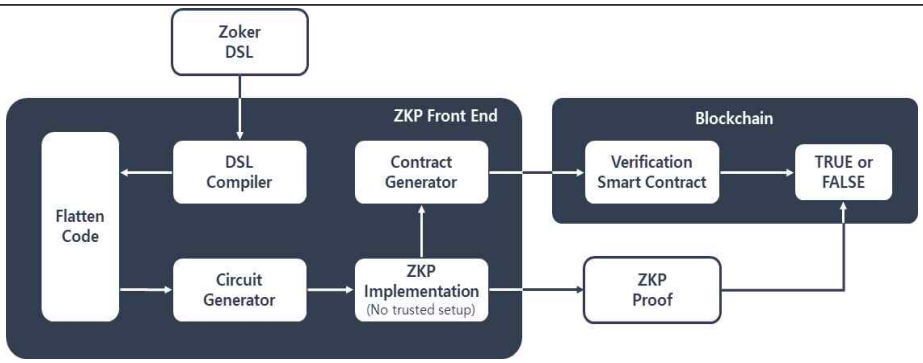
개발목표 시기	2020.12		기술성숙도 (TRL)	개발 전	개발 후
				2	5
결과물 형태	SW-Platform.		검증방법	자체검증	
색인어 (Keywords)	국문	검증 컨트랙트, 영지식 증명, 블록체인, 증명 생성 도구			
	영문	Verification Contract, Zero Knowledge Proof, Blockchain, Proof Generation Tool			
외부 기술요소	Open Source 사용 및 자체 개발 알고리즘 적용		권리성	SW, 기술이전	

② 기술의 개념(Concept) 및 구성

영문				
	Title	Smart Contract Generation Scheme with Zero-Knowledge Verification		
	Concept	Zero proof proof generation and verification contract generation tool according to user requirements		
	Service Offering	SW Tool (Open Source based)		
	Comparative Advantage	The zero-knowledge proof algorithm, which does not require Trusted Setup, is used to increase safety compared to the existing Zokrates, and the use of ZKBOO series algorithms increases the speed of zero-knowledge proof generation.		
	Patents (Domestics)	Application(√) Registration( )	Patents (International)	Application(√) Registration( )

국문	☐ (기술의 개념) - 블록체인에서 사용되는 영지식 증명은 NIZK(Non-Interactive Zero Knowledge)로 한번의 증명으로 검증자에게 증명 사실을 확신시킨다 이 때, 증명 사실을 구성하는 프로그램을 작성하여 Circuit으로 변환 후 처리된다. Circuit을 활용하기 위해서는 암호에 대한 지식이 필요하여 쉽게 사용할 수 없다. 따라서, 암호학적 지식과 상관없이 영지식 증명을 사용할 수 있게 도와주는 도구가 필요하다. 특히, Circuit을 올바르게 검증하였는지를 스마트 컨트랙트로 검증할 경우 블록체인이 검증자 역할을 수행할 수 있는 장점이 있다. <기술개념도>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[그림 1] 영지식 증명 검증 컨트랙트 생성 도구

제안하는 도구인 Zoker는 [그림 1]과 같이 증명하고자 하는 프로그램에 대한 구현을 파이썬과 비슷한 언어 양식의 Domain Specific Language(DSL)을 입력으로 받는다. 그 후 컴파일과정을 거쳐 영지식 증명의 구현에 입력으로 넣고, 실행 결과를 검증할 수 있는 증거(proof)와 검증 스마트 컨트랙트(Veirification Smart Contract) 코드를 생성한다. Zoker의 사용자는 위의 검증 스마트 컨트랙트를 이더리움에 저장하여 검증 서비스를 이용하는 사람들은 자신의 정보를 공개하지 않고 자신의 신분과 증명해야 할 값에 대한 사실을 개인정보를 공개하지 않은 상태로 증명한다.

③ 기술적 경쟁력

○ 기술의 특성 및 성능

- 도구에 사용된 영지식 증명 implementation으로는 ZKBOO에서 개선된 버전의 영지식 증명을 사용하여 proof size가 작고 proving 시간도 짧은 편인 영지식 증명으로 사용자가 증명 값을 생성하는데 시간이 너무 오래 걸리지 않게 만들어 사용자 경험을 개선할 수 있음

○ 경쟁기술/대체기술 동향 및 현황

- Zokrates : TU Berlin에서 2017년에 개발한 영지식 증명 검증 스마트 컨트랙트 생성 도구로 zk-SNARK를 지원하기 위하여 만들어진 libSNARK을 활용하여 proof와 검증 컨트랙트를 생성하나 trusted setup 과정이 필요하다는 단점이 존재

○ 우수성 및 차별성

- Zoker는 기존의 영지식 증명을 사용하는 것을 도와주는 Zokrates의 개선된 버전을 지향하는 프로그램으로 Zokrates에서 지원하는 library는 libSNARK으로 Trusted setup을 필요로 하는 시스템이었지만 zoker는 trusted setup이 필요하지 않은 영지식 증명을 사용하여 금융 시스템이나 실거래, Decentralized Identity와 같은 fake proof에 민감한 분야에서도 쉽게 적용할 수 있는 스마트 컨트랙트를 작성할 수 있음
- 또한 생성된 Implementation은 Trusted Setup이 필요 없기 때문에 신뢰기관이 필요하거나 Multi party computation이 필요하지 않고, 오버헤드가 적고 개발자 입장에서 쉽게 영지식 증명을 적용할 수 있음
- Zoker의 영지식증명 Implementation은 proving time이 짧고 proof size가 작기 때문에 블록체인 환경에 적합하게 사용할 수 있음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

④ 표준화 및 특허

No.	국가	출원.등록번호(출원.등록일)	상태	명칭
1	대한민국	10-2021-0003531(2021.01.11.)	출원	태그 추천 모델을 이용한 자동화된 스마트 컨트랙트 태그 생성 및 추천 시스템
1	대한민국	10-2103177-0000(2019.11.20)	등록	블록체인의 확장성을 위한 영지식 증명의 선택적 검증 시스템 및 그 방법

⑤ 시장성

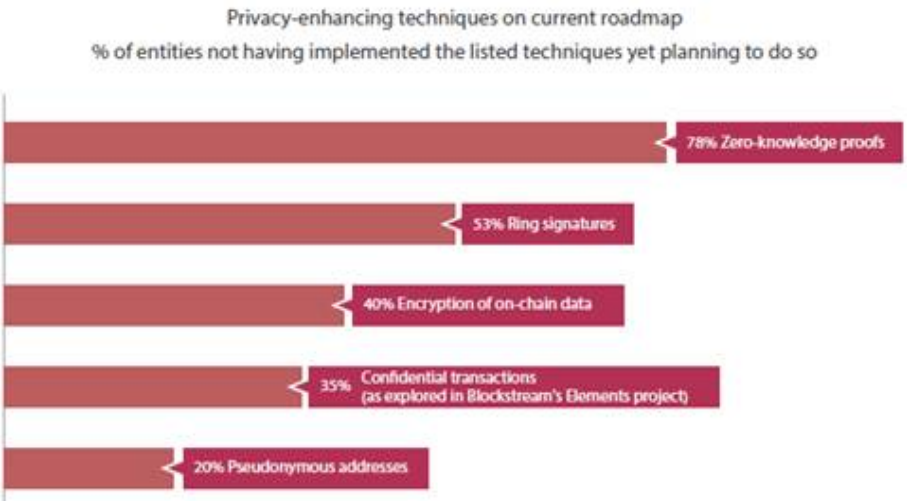
○ 국내외 시장동향 및 전망

- 글로벌 시장 조사 업체인 Market Research Future에서 제시한 스마트 컨트랙트 시장은 [그림 2]와 같이 2023년에는 연평균복합성장률이 2016년 대비 32%증가한 300억 달러에 이를 것으로 예상



[그림 2] 스마트 컨트랙트 시장 전망

○ 추가적으로 University of Cambridge Judge Business school는 49개의 국가, 200여개의 블록체인 기업을 대상으로 설문조사를 진행하였으며 [그림 3]과 같이 71%의 기업이 프라이버시 보호 문제 해결을 위해 영지식 증명 기술을 구현 혹은 도입을 계획하고 있는 것으로 확인됨



[그림 3] Global Blockchain Benchmarking Study 설문 조사 결과

○ 제품화 및 활용분야

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

활용분야(제품/서비스)		제품 및 활용분야 세부내용
1	스마트 컨트랙트 영지식 증명 검증 도구(DID)	본 도구를 활용하여 개인정보를 보호하는 여러 가지 어플리케이션에 활용할 수 있음. 예를 들어 DID 시스템에 적용할 수 있는데, 기존의 DID 방식에서는 자신이 성인임을 드러내기 위해서는 자신의 주민등록증을 공개할 수 있는데 이 때 성인임을 증명하는 것 뿐만이 아니라 정확한 나이, 주소, 얼굴에 대한 모든 정보가 공개되어야 하지만, 본 도구를 활용하여 영지식 증명을 적용한 DID에서는 자신의 주민등록증에 있는 나이에 대한 값이 19보다 큰 값이라는 것만 증명하여 proof을 제시하고 이 proof을 서비스를 운영하는 사람이 블록체인에서 검증하여 자신의 정확한 나이를 공개하지 않고도 자신이 성인임을 증명할 수 있음

○ 시장규모(국내추정치)

활용분야(제품/서비스)		관련 시장 규모(5년), 단위: 억원				
		2020	2021	2022	2023	2024
1	스마트 컨트랙트 영지식 증명 검증 도구(DID)	3150	3300	3500	3750	4000

- 국내 블록체인 어드바이저 Beeny에 따르면, DID가 유형별 인증시장(Certification, Mobile, I-Pin, OTP, ID/PW, KYC)를 대체한다는 가정 하 위와 같은 시장 규모를 추정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

⑥ 기술이전 조건 ※ **해당된 경우, 해당항목에** √하여 주시기 바랍니다. 복수 선택 가능

개발상태	아이디어 창안	특허만 신청(등록)	기술개발 진행중	기술개발 완료	시제품 단계	실용화 단계	시장개척 단계	기타
				√				
기술이전 금액	무료 -5백만원 이하	5백만원 초과 -1천만원 이하	1천만원 초과 -2천만원 이하	2천만원 초과 -3천만원 이하	3천만원 초과 -5천만원 이하	5천만원 초과 -1억원 이하	1억원 초과	기타
					√			

희망거래 유형	기술매매	라이센스	기술협력	기술지도	M&A	기타
	√					

⑦ 기대효과

○ 기술사업화로 인한 파급효과

- Trusted setup이 필요 없고 짧은 proving time과 작은 proof size를 가지고 있는 알고리즘의 특성으로 인해 블록체인 환경에서 효율적으로 사용될 것이며 검증 컨트랙트 생성 시 개발한 취약점 분석 도구를 활용하여 안전한 컨트랙트를 보장할 수 있으므로 블록체인 내에 영지식 증명을 이용할 때 많은 도움이 됨

⑧ 기술사업화 적용 및 제약사항

○ 사업화 제약사항 : 없음

○ 지원 필요사항 : 없음

⑨ 기타

기술세미나 발표 가능 여부(O/X)	O	매칭 희망 여부(O/X) (R&D 바우처 사업 등)	X

담당	구분	성명	소속	직위	회사 전화번호	회사 이메일
	판매자 (TLO 등)					
	연구자	김근영	충남대학교 산학협력단	석박통합 과정	042-821-7443	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 사업화 대상기술

※ 2. 사업화대상기술 목록 순번에 따라 세부 개발기술별로 작성, 세부기술별 작성분량은 가급적 5쪽 내외

개발기술 1	국문	블록체인 서비스 분석 솔루션
	영문	Blockchain Service Analysis Solution

공개	☑	비공개	☐ (사유)

① (예상) 결과물

개발목표 시기	2021. 12		기술성숙도 (TRL)	개발 전	개발 후
결과물 형태	SW-Server		검증방법	자체 검증	
색인어 (Keywords)	국문	불록체인, 딥러닝, 스마트 컨트랙트			
	영문	Blockchain, Deep learning, Smart contract			
외부 기술요소	Open Source사용		권리성	특허	

② 기술의 개념(Concept) 및 구성

국문

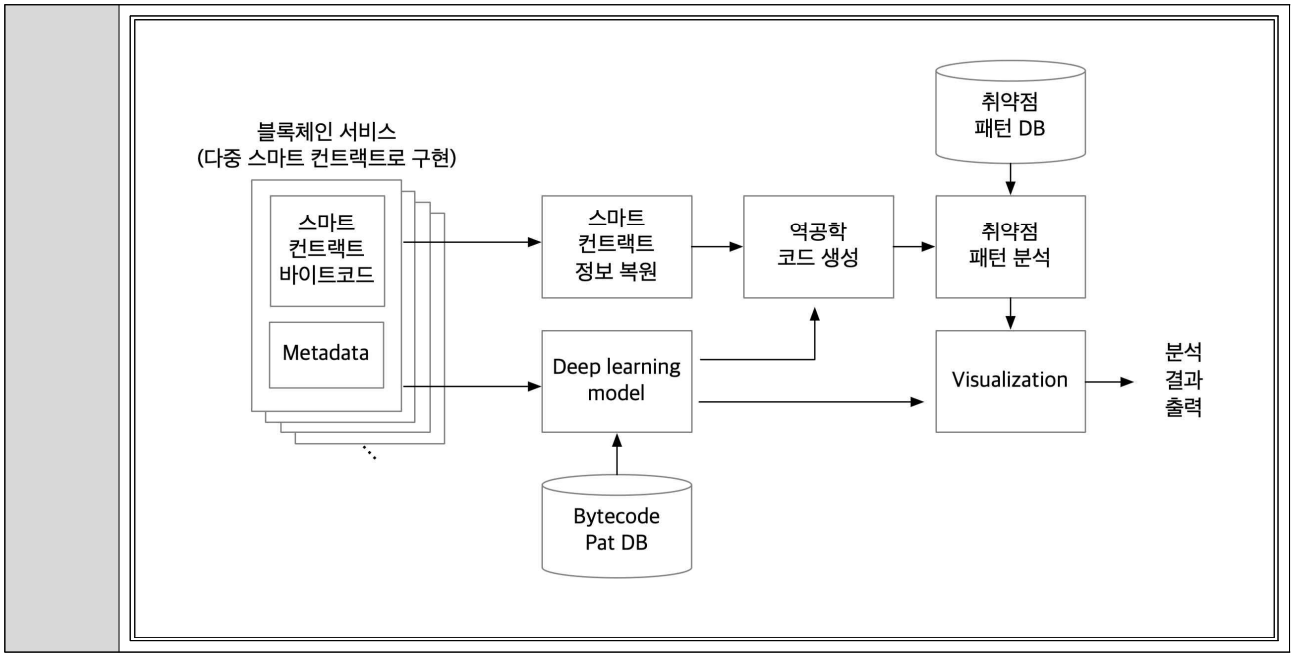
☐ (기술의 개념)

○ 블록체인 서비스를 구성하는 스마트 컨트랙트를 딥러닝 기술로 분석

- 최근 다양한 블록체인 서비스들이 실용화 되고 널리 퍼지기 시작하면서, 사용자에게는 사용의 안전성을 위해서, 투자자에게는 장래성 있는 프로젝트를 찾기 위해서, 서비스 개발자는 연동된 서비스의 신뢰성을 보장하기 위해서 블록체인 서비스의 정확한 분석에 대한 요구가 증대
- 블록체인 서비스는 블록체인 위에서 동작하는 스마트 컨트랙트가 비즈니스 로직 및 블록체인 기능의 핵심을 담당
- 스마트 컨트랙트의 실제 기능 및 구현 방식에 따라서 블록체인 서비스의 안전성이 결정
- 현재 블록체인 서비스는 복수의 스마트 컨트랙트들 사이의 연결을 통해서 구성되는 것이 일반적이므로 다양한 형태의 컨트랙트들을 분석하여 안전성에 대한 확인이 필요
- 스마트 컨트랙트 바이트 코드를 기반으로 딥러닝을 이용한 간략화된 특징을 찾고 역공학을 통해서 세부적인 정보들을 파악한다. 최종 결과는 visualization 과정을 통해 사용자에게 전달한다.

<기술개념도>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



③ 기술적 경쟁력

○ **기술의 특성 및 성능**

- 기존의 스마트 컨트랙트의 바이트코드를 대상으로 하는 분석은 취약점을 점검만을 하는 제한적인 기능을 갖추고 있으며, 전문적인 지식이 없는 경우에는 사용하기 어려움
- 전문적인 지식이 없더라도 스마트 컨트랙트의 특징을 한눈에 알 수 있는 키워드 중심의 정보를 제공하여 서비스에 대한 보편적인 이해도 향상
- 딥러닝의 학습 결과를 바탕으로 하므로, 해당 스마트 컨트랙트가 실제 사용되는 용도에 가까운 정보를 제공하여, 표면적 속임수를 이용하는 위험한 스마트 컨트랙트를 탐지 가능

○ **경쟁기술/대체기술 동향 및 현황**

- 블록체인에서 동작하고 있는 스마트 컨트랙트 바이트 코드에서 바로 키워드 중심의 분석 결과를 생성하는 경쟁기술은 없음
- 스마트 컨트랙트 바이트 코드를 대상으로 하는 취약점 점검 시스템은 전문적 취약점만을 찾는다는 점에서 완전한 대체는 불가

○ **우수성 및 차별성**

경쟁기술	본 기술의 우수성/차별성
- 스마트 컨트랙트 바이트코드를 통해 취약점만을 분석 - 분석 결과 이해를 위해 전문적 지식이 필요	- 전문적인 지식이 없어도 쉽게 이해할 수 있는 키워드 중심의 결과 출력 - 기능적 취약점만이 아닌 악의적인 목적으로 동작하는 컨트랙트도 탐지

④ 표준화 및 특허

○ **표준화**

- 블록체인과 딥러닝을 이용한 기술에 대한 표준화는 아직 정립되지 않았음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **관련 보유특허** ※ 대외 공개 가능한 특허만 작성

No.	국가	출원.등록번호(출원.등록일)	상태	명칭
1	대한민국	10-2184772 (2020-11-23)	등록	태그 추천 모델을 이용한 자동화된 스마트 컨트랙트 태그 생성 및 추천 시스템

⑤ 시장성

○ **국내외 시장동향 및 전망**

- 최근 블록체인 서비스는 실제 상용 서비스가 활발히 운영되면서 본격적인 증흥기를 앞두고 있다.
- 실사용 예로서 중앙은행화폐(Central Bank Digital Currency), 탈중앙금융(decentalized finance) 및 다양한 자산을 다루는 서비스가 중심으로 발전하고 있다.
- 블록체인 서비스의 다양한 발전과 함께 서비스를 글로벌 사용자가 쉽고 빠르게 판단할 수 있는 분석 시스템에 대한 전망이 커지고 있다.

○ **제품화 및 활용분야**

활용분야(제품/서비스)		제품 및 활용분야 세부내용
1	블록체인 서비스 분석 솔루션	블록체인 서비스에 대한 종합적 특징 및 실제 동작하는 바이트 코드를 바탕으로한 정확한 진단 서비스 제공

※ 해당기술이 적용/활용될 수 있는 소재/부품/제품/서비스 등을 최대한 기재

○ **시장규모(추정치)**

활용분야(제품/서비스)		관련 시장 규모(5년), 단위: 억원				
		2020	2021	2022	2023	2024
1	블록체인 서비스 분석 솔루션	120	160	300	550	900

※ 관련자료 등을 참고로 과제책임자가 예상치 기재

⑥ 기술이전 조건 및 고려사항 ※ **해당된 경우, 해당항목에** √하여 주시기 바랍니다. 복수 선택 가능

개발상태	아이디어 창안	특허만 신청(등록)	기술개발 진행중	기술개발 완료	시제품 단계	실용화 단계	시장개척 단계	기타
		√	√		√			
기술이전 금액	무료 -5백만원 이하	5백만원 초과 -1천만원 이하	1천만원 초과 -2천만원 이하	2천만원 초과 -3천만원 이하	3천만원 초과 -5천만원 이하	5천만원 초과 -1억원 이하	1억원 초과	기타
							√	

희망거래 유형	기술매매	라이센스	기술협력	기술지도	M&A	기타
	√		√			

○ **고려사항** ※ 기술이전시 기타 고려사항

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

⑦ 기대효과

○ 기술도입으로 인한 경제적 효과

- 안전한 블록체인 서비스에 대한 판단 및 자동 식별을 통해서 서비스 상용화 가속화 및 시장 확대 기대

○ 기술사업화로 인한 파급효과

- 최근 중요점이 강조되고 있는 프로토콜 경제에서의 핵심 기술로서 블록체인 서비스의 위상 증대
- 블록체인 기반 서비스에 대한 신뢰도 향상으로 관련 가상자산 산업의 발전 기대

⑧ 기술사업화 적용 및 제약사항

- 해당사항 없음

⑨ 기타

기술세미나 발표 가능 여부(O/X)	O	매칭 희망 여부(O/X) (R&D 바우처 사업 등)	X

담당	구분	성명	소속	직위	회사 전화번호	회사 이메일
	판매자 (TLO 등)	유창현	㈜파이랩테크놀로지	COO	02-552-3720	changhyun@pilab.co
	연구자	이종협	㈜파이랩테크놀로지	CTO	02-552-3720	jonghyup@pilab.co

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 2. 참고문헌

- [BBB+18] B. Bunz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, Bulletproofs: Short proofs for confidential transactions and more, IEEE S&P 2018: 315-334.
- [GMO16] I. Giacomelli, J. Madsen, and C. Orlandi, ZKBoo: Faster zero-knowledge for boolean circuits, Usenix security 2016: 1069-1083.
- [GKR08] S. Goldwasser, Y. Kalai, and G. Rothblum, Delegating computation: inteactive proofs for muggles: STOC 2008: 113-122.
- [Gro16] J. Groth, Efficient zero-knowledge arguments for arithmetic circuits in discrete log setting, Eurocrypt 2016: 327-357.
- [IKOS07] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, Zero knowledge from secure multiparty computation, STOC 2007: 21-30.
- [Kil92] J. Kilian, A note on efficient zero-knowledge proofs and arguments, STOC 1992: 723-732.
- [Mic94] S. Micali, CS proofs, FOCS 1994: 436-453.
- “Ethereum vulnerabilities”, Ethereum Stmart Contract Best Practices, https://consensys.github.io/smart-contract-best-practices/known_attacks/
- Gbadebo Ayoade et. al, "Smart contract Defense Through Bytecode Rewriting", Blockchain 2019
- Sukrit Kalra, Seep Goel, Mohan Dhawan and Subodh Sharma, “ZEUS: Analyzing Safety of Smart Contracts”,Network and Distributed Systems Security (NDSS) Symposium 2018,IBM Research and IIT Delhi, 2018
- Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli, “A survey of attacks on Ethereum smart contracts”, Proceedings of the 6th International Conference on Principles of Security and Trust, Page164-186, April 22-29, 2017
- Deviare API Hook https://www.nektra.com/products/deviare-api-hook-windows/
- Scott Oaks, Java Security, O’Reilly & Assiciates, Inc. 2001
- Ramnivas Laddad, Aspect J in Action:Enterprise AOP with Spring Applications, Manning, 2009
- Ethereum, https://www.ethereum.org/
- EVM, https://github.com/ethereum/wiki/wiki/Ethereum-Virtual-Machine-(EVM)-Awesome-List
- Solidity, https://solidity.readthedocs.io/en/ develop/
- N.Atzei, “A survey of attacks on Ethereum smart contracts”, In POST (LNCS), Vol. 10204. Springer, 164--186.
- Ethreum Yellow Paper , https://ethereum.github.io/yellowpaper/paper.pdf
- https://blog.openzeppelin.com/deconstructing-a-solidity-contract-part-i-introduction-832efd2d7737/
- L. Luu, D. Chu, H. Olickel, et. al, “Making Smart ContractsSmarter”, Proceedings of the 2016 ACM SIGSAC Conference onComputer and Communications Security, p.254-269, October 24 - 28, 2016
- Symbolic execution, https://en.wikipedia.org/wiki/Symbolic_execution
- T. Cook ,A. Latham and J. Lee, “DappGuard : Active Monitoring and Defense for Solidity Smart Contracts”, mit, 2017
- P. Tsankov, A. Dan, et. al., Securify: Practical security analysis of smart contracts, 2018
- Feist, J., Greico, G., Groce, “A.: Slither: a static analysis framework for smart contracts”, WETSEB 2019 : 2nd international workshop on emerging trends in software engineering

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

for blockchain ,2019

- Y. Zhou, D.Kumar ,et. al. “Erays: Reverse Engineering Ethereum’s Opaque Smart Contracts”, 27th USENIX Security Symposium, 2018
- Suiche, “Porosity: a decompiler for blockchain-based smart contracts bytecode”, DEF CON, 25 , 2017
- Viper, https://vyper.readthedocs.io/en/latest/
- Remix, https://remix.ethereum.org/
- ANTLR, https://www.antlr.org/
- Etherscan, https://etherscan.io/
- J. Birch, R. A. van Engelen, and K. A. Gallivan. Value rangeanalysis of conditionally updated variables and pointers. InIn Proceedings of Compilers for Parallel Computing (CPC’04), pages 265–276, 2004.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 3. 자체 보안관리 진단표 (연구기관 모두 동일)

구분	체크항목	결과 체크 (√표)	비고 (미실시 사유)
보안관리 체계	o 기관 내 보안관리규정을 제정/적용하고 있다	O(√), X( )	
	o 보안관리 조직이 있으며, 자체 보안점검실시 등 잘 운영되고 있다	O(√), X( )	
	o 보안교육을 정기적(1회이상/연)으로 실시하고 있다	O(√), X( )	
	o 보안사고에 대한 방지대책 및 비상시 대응계획이 준비되어 있다	O(√), X( )	
참여연구원 관리	o 참여연구원에 대하여 보안서약서를 받았다	O(√), X( )	
	o 참여연구원에게 보안관리의 중요성 등을 인식시키고 있다	O(√), X( )	
연구개발 내용/결과 관리	o 주요 연구자료 및 성과물의 무단유출 방지대책을 수립하고 있다	O(√), X( )	
	o 보안성 검토 방법 및 절차를 이행하고 있다	O(√), X( )	
	o 기술이전 관련 내부규정 및 절차를 준수하고 있다	O(√), X( )	
연구시설 관리	o 연구시설 보안관련 내부규정 또는 지침을 이행하고 있다	O(√), X( )	
	o 주요 시설에는 보안장비가 설치되어 있다	O(√), X( )	
	o 보호구역이 지정되어 있다	O(√), X( )	
정보통신망 관리	o 정보통신망 보안관련 내부규정 또는 지침이 구비되어 있다	O(√), X( )	
	o 보안관리 책임자의 승인 항목이 구분되어 있다	O(√), X( )	
	o 주요 데이터에 대해 백업을 실시하고 있다	O(√), X( )	
	o 개인용 정보통신장비(노트북, USB메모리)에 대하여 인가/관리중이다	O(√), X( )	
	o 전산망 보호를 위한 HW 및 SW 등을 도입하여 적용하고 있다	O(√), X( )	
	o 직책, 임무별 열람 권한을 차등화하여 부여하고 있다	O(√), X( )	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 5. 연구데이터 관리계획 *(연구데이터 관리계획(DMP) 과제만 작성)*

<table>
<tr><th>구분</th><th colspan="2">정보시스템 구성 항목</th><th colspan="3">입력값</th></tr>
<tr><td colspan="2">o 데이터관리계획의 유무</td><td colspan="3">유</td></tr>
<tr><td rowspan="5">과제현황</td><td>사업명</td><td colspan="3"></td></tr>
<tr><td>과제명</td><td colspan="3">스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술개발</td></tr>
<tr><td>주관기관명</td><td>충남대학교 산학협력단</td><td>총괄책임자</td><td colspan="2">류재철</td></tr>
<tr><td>총연구기간</td><td>2018.04.01 ~ 2020.12.31</td><td>당해연구기간</td><td colspan="2">2020.01.01 ~ 2020.12.31</td></tr>
<tr><td>총 출연금</td><td>1,646백만원</td><td>당해출연금</td><td colspan="2">591백만원</td></tr>
<tr><td rowspan="6">연구데이터
생산 및 관리</td><td colspan="2" rowspan="2">o 연구데이터의 유형</td><td>① 텍스트</td><td>② 이미지</td><td>③ 음향</td></tr>
<tr><td>④ 동영상</td><td>⑤ 소스코드</td><td>⑥
기타(서술식)</td></tr>
<tr><td colspan="2" rowspan="6">o 연구데이터의 생산 및 수집방법</td><td rowspan="5">생산 및
수집방법</td><td colspan="2">① 관측·실험장비</td></tr>
<tr><td colspan="2">② 시뮬레이션</td></tr>
<tr><td colspan="2">③ 관찰 및 현장조사</td></tr>
<tr><td colspan="2">④ 설문 및 인터뷰</td></tr>
<tr><td colspan="2">⑤ 기타(스마트 컨트랙트에 관해 리포트 하는 Ethersacn 웹사이트를 통해 수집하여 사용. 수집한 데이터 중에서 소스코드를 추출하는 자체 프로그램을 제작해 수집함)</td></tr>
<tr><td>설명</td><td colspan="2">(스마트 컨트랙트 실행환경은 Solidity로 작성된 스마트 컨트랙트 파일을 이용한다는 가정 하에 개발될 것. 따라서 Solidity 언어로 작성된 스마트 컨트랙트 파일을 대상으로 테스트 될 것.)</td></tr>
<tr><td colspan="2" rowspan="2">o 연구데이터의 용량, 대표적 포맷</td><td>용량</td><td colspan="2"></td></tr>
<tr><td>포맷</td><td colspan="2">Solidity(sol), Go(go)</td></tr>
<tr><td colspan="2" rowspan="2">o 연구데이터에 관한 상세정보</td><td>상세 식별자 정보
(데이터 식별자 부여 여부 포함)</td><td colspan="2"></td></tr>
<tr><td>메타데이터 표준</td><td colspan="2"></td></tr>
<tr><td>연구데이터
저장 및 보존</td><td colspan="2">o 연구수행 중 연구데이터의 보존 및 백업방안</td><td colspan="3">(서술식)스마트 컨트랙트 수집 프로그램이 수집한 데이터는 수집이후부터 보존될 것. 수집된 데이터들은 내부 저장소에 유지됨</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><td rowspan="4"></td><td rowspan="4"></td><td rowspan="3">보존장소</td><td>① 내부 시스템 등록</td></tr>
<tr><td>② 외부 시스템 기탁</td></tr>
<tr><td>③ 기타</td></tr>
<tr><td>보존기간</td><td>0000년도 00월 00일</td></tr>
<tr><td rowspan="11">연구데이터
공동 활용</td><td>o 전체 생산된 연구데이터 중 공동 활용 가능한 연구데이터의 종류와 범위</td><td colspan="2">(서술식) 소스코드,SW</td></tr>
<tr><td rowspan="3">o 연구데이터의 공동 활용 수준</td><td colspan="2">① 공개</td></tr>
<tr><td colspan="2">② 조건부공개</td></tr>
<tr><td colspan="2">③ 비공개</td></tr>
<tr><td>o 연구데이터의 공동 활용 방식</td><td colspan="2">(서술식)데이터는 연구실 내부 저장소에 보존되어 있으며, 연구가 종료된 후 3개월 이내 본 공개 SW 결과물 (SW,소스코드 등)을 오픈소스 사이트 github에 업로드 하여 공개·공유할 예정임.</td></tr>
<tr><td>o 연구데이터의 공개·공유 시점</td><td colspan="2">2021년도 03월 31일</td></tr>
<tr><td>o 데이터 이용 및 이전의 조건, 라이센싱 조건 등</td><td colspan="2">(서술식) *SW는 계속 공개되어 있을 것
* Ethersacn에서 수집한 스마트 컨트랙트 데이터는 사이트 규정에 따라 공개하지 않음</td></tr>
<tr><td rowspan="6">o 연구데이터의 공개·공유시 제한 조건</td><td rowspan="5">제한여부</td><td>① 해당없음</td></tr>
<tr><td>② 민감정보 포함</td></tr>
<tr><td>③ 지식재산권 및 영업비밀 등의 보호</td></tr>
<tr><td>④ 국가보안</td></tr>
<tr><td>⑤ 기타(서술식)</td></tr>
<tr><td>사유</td><td>(서술식)</td></tr>
<tr><td>o 데이터관리계획 원문정보 연계</td><td colspan="2">(전자문서 혹은 URL 정보) 연계</td></tr>
<tr><td rowspan="4">연구데이터
관리 책임자
(복수 가능)</td><td>소속기관명</td><td>충남대학교</td><td>성명</td><td>조은선</td></tr>
<tr><td>부서</td><td>컴퓨터공학과</td><td>직위</td><td>교수</td></tr>
<tr><td>전화</td><td>042-821-6857</td><td>e-mail</td><td>eschough@cnu.ac.kr</td></tr>
</table>

* 본 연구데이터 관리계획(DMP)을 IITP의 이지원시스템(EZOne)에 업로드 필수(별도 안내예정)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><th>구분</th><th colspan="2">정보시스템 구성 항목</th><th colspan="3">입력값</th></tr>
<tr><td colspan="2">o 데이터관리계획의 유무</td><td colspan="3">유</td></tr>
<tr><td rowspan="5">과제현황</td><td>사업명</td><td colspan="3"></td></tr>
<tr><td>과제명</td><td colspan="3">스마트 컨트랙트 프라이버시 보호 및 취약점 분석 기술개발</td></tr>
<tr><td>주관기관명</td><td>충남대학교 산학협력단</td><td>총괄책임자</td><td>류재철</td></tr>
<tr><td>총연구기간</td><td>2018.04.01 ~ 2020.12.31</td><td>당해연구기간</td><td>2020.01.01 ~ 2020.12.31</td></tr>
<tr><td>총 출연금</td><td>1,646백만원</td><td>당해출연금</td><td>591백만원</td></tr>
<tr><td rowspan="5">연구데이터
생산 및 관리</td><td colspan="2" rowspan="2">o 연구데이터의 유형</td><td>① 텍스트</td><td>② 이미지</td><td>③ 음향</td></tr>
<tr><td>④ 동영상</td><td>⑤ 소스코드</td><td>⑥
기타(서술식)</td></tr>
<tr><td colspan="2" rowspan="2">o 연구데이터의 생산 및 수집방법</td><td rowspan="5">생산 및
수집방법</td><td colspan="2">① 관측·실험장비</td></tr>
<tr><td colspan="2">② 시뮬레이션</td></tr>
<tr><td colspan="2">③ 관찰 및 현장조사</td></tr>
<tr><td colspan="2">④ 설문 및 인터뷰</td></tr>
<tr><td colspan="2">⑤ 기타(서술식)</td></tr>
<tr><td colspan="2" rowspan="2">o 연구데이터의 용량, 대표적 포맷</td><td>용량</td><td colspan="2">180000000 bytes</td></tr>
<tr><td>포맷</td><td colspan="2">scala source code : scala</td></tr>
<tr><td colspan="2">o 연구데이터에 관한 상세정보</td><td>상세 식별자 정보
(데이터 식별자 부여 여부 포함)</td><td colspan="2">Solidity 언어를 파싱하여 AST를 생성한다.
- AST 위에서 정적분석을 통해 8개의 취약점을 탐지한다.</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><td></td><td></td><td></td><td>- 소프트웨어에 존재하는 취약점을 자동으로 탐지한다.
- 총 8개의 취약점을 탐지한다.</td></tr>
<tr><td></td><td></td><td>메타데이터 표준</td><td></td></tr>
<tr><td>연구데이터 저장 및 보존</td><td>o 연구수행 중 연구데이터의 보존 및 백업방안</td><td colspan="2">- 오픈소스 git 저장소에 분석 SW 소스코드를 저장
- 데이터 백업은 Git 저장소에서 자동으로 이루어지며, 버전관리 기능을 통해 문제가 생겼을 경우 데이터 복원이 가능
- 저장소: https://github.com/sjmini/...</td></tr>
<tr><td></td><td></td><td rowspan="3">보존장소</td><td>① 내부 시스템 등록</td></tr>
<tr><td></td><td></td><td>② 외부 시스템 기탁</td></tr>
<tr><td></td><td></td><td>③ 기타</td></tr>
<tr><td></td><td></td><td>보존기간</td><td>2026년도 12월 23일</td></tr>
<tr><td rowspan="6">연구데이터 공동 활용</td><td>o 전체 생산된 연구데이터 중 공동 활용 가능한 연구데이터의 종류와 범위</td><td colspan="2">scala source code : scala</td></tr>
<tr><td rowspan="3">o 연구데이터의 공동 활용 수준</td><td colspan="2">① 공개</td></tr>
<tr><td colspan="2">② 조건부공개</td></tr>
<tr><td colspan="2">③ 비공개</td></tr>
<tr><td>o 연구데이터의 공동 활용 방식</td><td colspan="2">오픈소스 git 저장소를 통해 SW를 지속적으로 업데이트 할 예정이며, 현재 공유되어 있는 상황임.</td></tr>
<tr><td>o 연구데이터의 공개·공유 시점</td><td colspan="2">2020년도 01월 23일</td></tr>
<tr><td>o 데이터 이용 및 이전의 조건, 라이센싱 조건 등</td><td colspan="2">솔리디티 스마트 컨트랙트 분석 SW는 BSD 라이선스로 보호되는 오픈소스로써 라이선스가 지정한 제약 사항 외에는 누구나 자유롭게 활용 가능함.</td></tr>
<tr><td rowspan="2">o 연구데이터의 공개·공유시 제한 조건</td><td rowspan="5">제한여부</td><td>① 해당없음</td></tr>
<tr><td>② 민감정보 포함</td></tr>
<tr><td>③ 지식재산권 및 영업비밀 등의 보호</td></tr>
<tr><td>④ 국가보안</td></tr>
<tr><td>⑤ 기타(서술식)</td></tr>
<tr><td></td><td>사유</td><td>(서술식)</td></tr>
<tr><td></td><td>o 데이터관리계획 원문정보 연계</td><td colspan="2">(전자문서 혹은 URL 정보) 연계</td></tr>
<tr><td rowspan="3">연구데이터 관리 책임자 (복수 가능)</td><td>소속기관명</td><td>한국과학기술원</td><td>성명</td><td>류석영</td></tr>
<tr><td>부서</td><td>전산학부</td><td>직위</td><td>부교수</td></tr>
<tr><td>전화</td><td>042-350-3538</td><td>e-mail</td><td>sryu.cs@kaist.ac.kr</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주 의

1. 이 보고서는 과학기술정보통신부에서 시행한 정보통신·방송 연구개발사업의 보고서입니다.

2. 이 보고서 내용을 발표하는 때에는 반드시 과학기술정보통신부(정보통신기획평가원)에서 시행한 사업의 연구 개발 결과임을 밝혀야 합니다.

3. 국가과학기술 기밀유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 안됩니다.